

A. TRANG BÌA

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

Đề tài:

XÂY DỰNG HỆ THỐNG NHẬN DIỆN BIỂN SỐ XE BẰNG TRÍ TUỆ NHÂN TẠO

Developing an AI-based vehicle license plate recognition system

Ngành: Trí tuệ nhân tạo

Chuyên ngành: Trí tuệ nhân tạo

Sinh viên thực hiện: **Phạm Công Thành** — MSSV **25410013**

Nguyễn Minh Hiếu — MSSV **25410007**

Lớp: AI503.F3.LT.TTNT

Khoá: 2025

Giảng viên hướng dẫn: ThS. Cáp Phạm Đình Thăng

TP. Hồ Chí Minh, tháng 9 năm 2026

B. TÓM TẮT ĐỒ ÁN

TÓM TẮT ĐỒ ÁN

Nhận dạng biển số xe tự động (ALPR) là bài toán nền tảng của bãi đỗ xe thông minh, thu phí không dừng và giám sát giao thông. Tại Việt Nam, biển số hai dòng chiếm tỷ lệ lớn do mật độ xe máy cao, trong khi đa số bộ dữ liệu quốc tế chỉ có biển một dòng, khiến các mô hình

huấn luyện trên dữ liệu nước ngoài không áp dụng trực tiếp được. Khác biệt này đã được đo lường: trên bộ RodoSol-ALPR của Brazil, hệ thống OpenALPR đạt 94,3% với ô tô biển một dòng nhưng chỉ 45,7% với xe máy biển hai dòng [1].

Đề án xây dựng một hệ thống ALPR hoàn chỉnh cho biển số xe Việt Nam theo hướng tiếp cận hai giai đoạn: phát hiện vùng biển số bằng YOLO11, nhận dạng ký tự bằng PaddleOCR, và hậu xử lý bằng bộ luật chuẩn hoá theo quy chuẩn hiện hành. Hệ thống hỗ trợ cả biển một dòng và hai dòng, nhận đầu vào là ảnh, video hoặc khung hình thời gian thực gửi qua API (endpoint `POST /api/detect/frame`), và suy luận hoàn toàn trên CPU.

Đóng góp chính là bộ luật hậu xử lý **ràng buộc theo vị trí ký tự**, xây dựng trên Thông tư 79/2024/TT-BCA [2] và QCVN 08:2024/BCA [3]: mã tỉnh thuộc 81 giá trị hợp lệ, chữ cái sê-ri thứ nhất và thứ hai thuộc hai tập ký tự khác nhau. Cách tiếp cận này khắc phục hạn chế của các hệ thống áp một danh sách ký tự phẳng cho toàn chuỗi.

Về mặt kỹ nghệ, đề án cài đặt kiến trúc phân tầng tách biệt tầng AI khỏi tầng API, gồm backend FastAPI, giao diện web React và đóng gói Docker.

Trên tập kiểm tra của split v3 (1.514 ảnh, đã khử trùng lặp giữa các tập), bộ phát hiện YOLO11n đạt $mAP@0.5 = 0,9829$ và $mAP@0.5:0.95 = 0,7834$ (precision 0,9837; recall 0,9714). Khối nhận dạng đạt độ chính xác mức ký tự ($1 - CER$) 0,9454; độ chính xác toàn chuỗi tăng từ 0,6373 lên 0,7512 nhờ bộ luật hậu xử lý (sửa đúng 319 biển, không làm hỏng biển nào), và độ chính xác end-to-end đạt 0,5552. Khoảng cách lớn nhất nằm ở layout: biển một dòng đạt 0,9541 còn biển hai dòng chỉ đạt 0,6996 — chênh 25,45 điểm phần trăm, trong khi biển hai dòng chiếm 79,8% tập đánh giá. Độ trễ xử lý một ảnh ở phân vị 95 là 1.143,10 ms trên CPU (trung vị 405,77 ms), đạt ngưỡng tối thiểu 1.500 ms nhưng chưa đạt mục tiêu 800 ms — cái giá đã định lượng của bậc thang thử-lại dành cho biển nghiêng, méo. Chi tiết và phân tích lỗi được trình bày ở Chương 5.

Từ khóa: nhận dạng biển số xe, biển số Việt Nam, YOLO11, PaddleOCR, biển số hai dòng, hậu xử lý theo vị trí, suy luận trên CPU.

C. ABSTRACT

ABSTRACT

Automatic License Plate Recognition (ALPR) underpins smart parking systems, electronic toll collection, and traffic surveillance. In Vietnam, the task presents characteristics that prevent the direct reuse of models trained on foreign data: two-line plates account for a large share of vehicles because of the country's high motorcycle density, whereas most international datasets contain single-line plates only. The severity of this gap has been quantified: on the Brazilian RodoSol-ALPR dataset, the OpenALPR system reached 94.3% on single-line car plates but only 45.7% on two-line motorcycle plates — a gap of 48.6 percentage points [1].

This thesis develops a complete ALPR system for Vietnamese license plates following a two-stage approach: plate region detection with a YOLO11 model, character recognition with PaddleOCR, and post-processing through a normalization rule set derived from the applicable national regulations. The system supports both single-line and two-line plates, accepts images, video files, or real-time frames submitted through the API (the POST `/api/detect/frame` endpoint) as input, and performs inference entirely on CPU.

The principal contribution is a **position-constrained post-processing rule set** grounded in Circular 79/2024/TT-BCA [2] and the national technical regulation QCVN 08:2024/BCA [3]: the province code is restricted to 81 valid values, and the first and second serial letters are drawn from two different character sets. This design corrects a limitation of systems that apply a single flat character whitelist across the entire plate string.

From a software engineering perspective, the thesis implements a layered architecture that fully decouples the AI layer from the API layer, comprising a FastAPI backend, a React web interface, and Docker packaging.

On the held-out test split (1,514 images, de-duplicated across splits), the YOLO11n detector reaches $mAP@0.5 = 0.9829$ and $mAP@0.5:0.95 = 0.7834$ (precision 0.9837, recall 0.9714). The recognition stage attains a character-level accuracy ($1 - CER$) of 0.8734; full-string accuracy rises from 0.6098 to 0.6555 under the post-processing rule set (128 plates corrected, none corrupted), and end-to-end accuracy reaches 0.5227. The dominant gap is layout-related: single-line plates score 0.9489 whereas two-line plates score only 0.5810 — a 36.79 percentage-point gap — while two-line plates constitute 79.8% of the evaluation set. The 95th-percentile latency for a single image is 731.15 ms on CPU, meeting the 800 ms target. Chapter 5 reports the full results and error analysis.

Keywords: license plate recognition, Vietnamese license plates, YOLO11, PaddleOCR, two-line plates, position-constrained post-processing, CPU inference.

CHƯƠNG 1. GIỚI THIỆU

1.1. Đặt vấn đề

1.1.1. Bối cảnh giao thông Việt Nam và nhu cầu tự động hoá

Tính đến tháng 9/2024, Việt Nam có khoảng **77 triệu xe máy đã đăng ký, tương đương 770 xe trên 1.000 dân**; xe máy chiếm khoảng **85–90% lưu lượng phương tiện trên đường** [1]. Con số đó là **ràng buộc kỹ thuật trực tiếp**, kéo theo ba hệ quả (Chương 4): (i) **biển hai dòng gần vuông chiếm đa số tuyệt đối** vì mọi xe mô tô đều mang biển hai dòng; (ii) **mật độ cao gây che khuất**, mỗi khung hình thường có **nhiều biển số**; (iii) biển xe mô tô chỉ 140×190 mm nên đây là **bài toán phát hiện đối tượng nhỏ**. Ở quy mô đó, ghi nhận thủ công không khả thi — lý do tồn tại của **ALPR (Automatic License Plate Recognition)** [2] [3].

1.1.2. Ứng dụng thực tế của hệ thống ALPR

ALPR là lõi của bốn nhóm ứng dụng tại Việt Nam: **bãi đỗ xe thông minh** [4]; **thu phí không dừng ETC** [5]; **giám sát giao thông** (xử phạt nguội); **kiểm soát ra vào** [6]. Cả bốn đo cùng một thứ: **không phải mAP của khâu phát hiện, mà là tỉ lệ đọc đúng toàn bộ chuỗi biển số** (plate-level exact match) — đọc đúng 9 trên 10 ký tự vẫn là đọc sai biển. Nhận định này định hình chỉ tiêu ở mục 1.2.2 và cách đánh giá ở Chương 5.

1.1.3. Vì sao không thể dùng trực tiếp giải pháp nước ngoài

(a) Biển hai dòng là điểm gây đã đo được, không phải rủi ro giả định. Trên tập kiểm thử cân bằng có chủ ý của bộ **RodoSol-ALPR (Brazil)** — 4.000 ảnh ô tô biển **một dòng**, 4.000 ảnh xe máy biển **hai dòng** — hệ thống thương mại **OpenALPR** nhận đúng **94,3% biển một dòng nhưng chỉ 45,7% biển hai dòng, chênh 48,6 điểm phần trăm**, không biển số nào khác thay đổi ngoài bố cục biển [7]. Cũng nghiên cứu này: cả 12 phương pháp và 2 hệ thống thương mại đều **không vượt quá 70% recognition rate**, và có công trình phải **loại bỏ hoàn toàn xe máy** khỏi thí nghiệm [7].

⚠ **Cảnh báo phạm vi áp dụng của số liệu.** Cặp **94,3% / 45,7%** (chênh **48,6 điểm phần trăm**) đo trên **RodoSol-ALPR của Brazil, không phải trên dữ liệu Việt Nam**; dẫn như một *analogue* định lượng về độ khó của biển hai dòng, chọn Brazil vì cũng là nước có tỉ lệ xe máy cao. **Tuyệt đối không được trình bày cặp số này như số liệu Việt Nam** — số liệu Việt Nam do chính đồ án đo nằm ở **Chương 5**.

(b) Căn cứ pháp lý và cấu trúc chuỗi ký tự là đặc thù quốc gia. Biển số xe Việt Nam theo **TT 79/2024/TT-BCA** (hiệu lực 01/01/2025) [8], sửa đổi bởi **TT 13/2025/TT-BCA** [9] và **TT 51/2025/TT-BCA** [10]; kích thước vật lý theo **QCVN 08:2024/BCA** [11]. **Đính chính:** nhiều tài liệu trong nước vẫn viện dẫn **TT 24/2023/TT-BCA** [12] — đã hết hiệu lực từ **01/01/2025**. Ba đặc thù sau **không học được từ dữ liệu nước ngoài**. (i) Tập ký tự **seri phụ thuộc vị trí**: seri **vị trí thứ nhất** thuộc tập **20 chữ cái** (có G, không có R) [13]; **vị trí thứ hai** của biển xe mô tô thuộc tập **20 chữ cái khác** (có R, không có G); tập loại trừ đúng cho toàn hệ thống chỉ gồm **5 chữ I, J, O, Q, W** — charset theo "danh sách phẳng 20 chữ cái" sẽ **sai hệ thống trên**

toàn bộ lớp biển xe máy có R ở vị trí thứ hai, lỗi không cứu được ở hậu xử lý. (ii) Mã địa phương hữu hạn, có lỗ hổng: dải 11–99 chỉ có 81 mã đang được sử dụng; 8 mã — 13, 42, 44, 45, 46, 87, 91, 96 — không được gán [14], nên \d{2} cho qua 8 chuỗi không bao giờ tồn tại. (iii) Tỷ lệ khung hình phân tách rõ hai bố cục [11]: 110 × 520 mm → 4,727 (1 dòng); 165 × 330 mm → 2,000 (2 dòng); 140 × 190 mm → 1,357 (2 dòng); không loại biển nào rơi vào khoảng mở (2,000 ; 4,727) — cơ sở hình học để phân loại số dòng.

(c) Điều kiện thu nhận ảnh khác biệt: biển bị che, bám bụi, cong vênh, chụp nghiêng, ngược sáng, ảnh đêm — khác các bộ dữ liệu quốc tế lớn (CCPD, AOLP, SSIG) vốn lấy ô tô làm trung tâm. Kết luận mục 1.1: bài toán này cần hệ thống huấn luyện trên dữ liệu Việt Nam và khối hậu xử lý xây theo quy chuẩn Việt Nam.

1.2. Mục tiêu đề tài

1.2.1. Mục tiêu tổng quát

Xây dựng một hệ thống nhận dạng biển số xe Việt Nam hoàn chỉnh, chất lượng gần sản phẩm thực tế: mô hình phát hiện tự huấn luyện trên dữ liệu Việt Nam, khối nhận dạng ký tự, khối hậu xử lý theo quy chuẩn Việt Nam, backend REST API, giao diện web, cơ sở dữ liệu lịch sử, đóng gói triển khai, tài liệu học thuật đầy đủ; hỗ trợ cả biển một dòng và hai dòng, suy luận hoàn toàn trên CPU. Đây không phải demo dạng notebook, cũng không phải sản phẩm thương mại.

1.2.2. Mục tiêu cụ thể

Mỗi chỉ tiêu có mục tiêu và ngưỡng tối thiểu (bắt buộc đạt). Về chức năng và chất lượng phần mềm: 34 yêu cầu chức năng (21 Must, 6 Should, 3 Could, 4 Won't) trong sáu nhóm (mục 4.1.3); NFR-M1 mã pipeline AI không import FastAPI; NFR-M5 thay được bộ OCR không sửa mã tầng API; NFR-M2 độ bao phủ kiểm thử tầng nghiệp vụ ≥ 70%; khởi động một lệnh docker compose up, demo không cần Internet. NFR-A5 và NFR-A6 đo tách bạch có chủ đích — hiệu số là đóng góp định lượng của khối hậu xử lý (mục 1.6); thêm NFR-A8 (tách riêng biển một dòng / hai dòng) và NFR-A9 (theo điều kiện ảnh, nếu có nhân phù hợp).

Bảng 1.1. Nhóm chỉ tiêu độ chính xác

Mã	Chỉ tiêu	Mục tiêu	Ngưỡng tối thiểu
NFR-A1	mAP@0.5 của bộ phát hiện biển số	≥ 0,90	≥ 0,85
NFR-A2	mAP@0.5:0.95 của bộ phát hiện	≥ 0,65	≥ 0,55
NFR-A3	Precision / Recall phát hiện	≥ 0,92 / ≥ 0,90	≥ 0,88 / ≥ 0,85
NFR-	Độ chính xác OCR mức ký tự (1 – CER)	≥ 0,95	≥ 0,92

Mã	Chỉ tiêu	Mục tiêu	Ngưỡng tối thiểu
A4			
NFR-A5	Độ chính xác biến đầy đủ trước hậu xử lý	$\geq 0,85$	$\geq 0,80$
NFR-A6	Độ chính xác biến đầy đủ sau hậu xử lý	$\geq 0,90$	$\geq 0,85$
NFR-A7	Độ chính xác E2E toàn trình (ảnh vào → biến đúng)	$\geq 0,88$	$\geq 0,82$

Bảng 1.2. Nhóm chỉ tiêu hiệu năng trên CPU

Mã	Chỉ tiêu	Mục tiêu	Ngưỡng tối thiểu
NFR-P1	Độ trễ E2E một ảnh (p95)	$\leq 800\text{ ms}$	$\leq 1500\text{ ms}$
NFR-P2	Tốc độ khung hình thời gian thực (webcam — đo ở tầng API)	$\geq 5\text{ FPS}$ hiệu dụng	$\geq 3\text{ FPS}$
NFR-P3	Tốc độ xử lý video	$\geq 0,3\times$ thời gian thực	$\geq 0,15\times$
NFR-P4	Thời gian nạp mô hình khi khởi động	$\leq 15\text{ s}$	$\leq 30\text{ s}$
NFR-P5	Overhead của tầng API (không tính suy luận)	$\leq 50\text{ ms}$	$\leq 100\text{ ms}$
NFR-P6	Thời gian truy vấn lịch sử (10.000 bản ghi)	$\leq 500\text{ ms}$	$\leq 1000\text{ ms}$
NFR-P7	Bộ nhớ thường trú của backend	$\leq 2\text{ GB}$	$\leq 4\text{ GB}$

Các chỉ tiêu độ trễ "rộng rãi" hơn bài báo ALPR vì máy phát triển **không có GPU CUDA** (CON-02): huấn luyện trên GPU Colab/Kaggle, **toàn bộ suy luận và demo chạy trên CPU**, còn bài báo thường đo trên GPU RTX/V100. Mọi số liệu hiệu năng **bắt buộc kèm cấu hình phần cứng** (Phụ lục P.1).

⚠ **Bốn yêu cầu mức *Won't* — phải nói thẳng.** Cả bốn đều **thuần giao diện**, chuyển mức trong hai đợt thu gọn giao diện web ngày **2026-07-20**: đợt 1 gỡ trang Webcam → **FR-3.1 và FR-3.4 chuyển M → W** (nhận dạng thời gian thực vẫn phục vụ và vẫn có kiểm thử ở tầng API qua POST /api/detect/frame); đợt 2 gỡ trang Tổng quan (Dashboard) → **FR-4.1 chuyển M → W, FR-4.2 chuyển S → W** (thống kê và biểu đồ theo thời gian vẫn truy vấn được và vẫn có kiểm thử tích hợp qua GET /api/statistics, GET /health). **FR-4.1 là yêu cầu mức *Must* đầu tiên và duy nhất bị đưa ra khỏi phạm vi trong toàn bộ đề án** — nêu ở đây, ở mục 4.1.3, mục

6.3 và trong đặc tả yêu cầu, không để hội đồng tự phát hiện. Đây là **quyết định phạm vi có chủ đích**, không phải hạng mục bỏ sót: cả bốn mất **màn hình hiển thị**, không mất **năng lực hệ thống**, mã giao diện còn nguyên trong lịch sử git. Đánh đổi do được của đợt 2: gỡ thư viện biểu đồ recharts cùng trang Tổng quan làm gói tải về của giao diện giảm từ ~730 KB xuống **328,8 KB** (-55%).

1.2.3. Tiêu chí thành công

Đề tài thành công khi **đồng thời**: (1) toàn bộ yêu cầu *Must* hoạt động và demo được — theo bộ 21 *Must* sau hai đợt thu gọn 2026-07-20, bốn yêu cầu đã chuyển *Won't* (FR-3.1, FR-3.4, FR-4.1, FR-4.2) **không** tính là đạt; (2) mọi chỉ tiêu đạt **ngưỡng tối thiểu** và **có bằng chứng đo đạc**; (3) đủ 12 sản phẩm bàn giao; (4) khởi động trên máy sạch bằng một lệnh docker compose up; (5) demo trực tiếp không cần Internet.

Trạng thái tại thời điểm viết chương này. Các chỉ tiêu ở mục 1.2.2 là **chỉ tiêu đặt ra**. Hệ thống chạy pipeline nhận dạng **thật** với models/best.pt (imgsz=640, split v3); chỉ tiêu phát hiện **đều đạt** ($mAP@0.5 = 0,9829$), độ trễ NFR-P1 **đạt** (p95 731/780 ms); chỉ tiêu độ chính xác OCR **đã đo** và biểu hai dòng **chưa đạt** (kết quả thật). **Đối chiếu đầy đủ từng chỉ tiêu ở Chương 5.**

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

Ba nhóm. **(1) Biển số xe cơ giới Việt Nam** theo TT 79/2024/TT-BCA [8] (sửa đổi bởi TT 13/2025 [9] và TT 51/2025 [10]) và QCVN 08:2024/BCA [11]; biển nền đỏ quân đội thuộc TT 169/2021/TT-BQP [15], chỉ xử lý ở mức nhận biết ký hiệu. **(2) Mô hình phát hiện họ YOLO** — cụ thể YOLO11 [16]. **(3) Engine OCR** không cần phân đoạn ký tự: PaddleOCR [17] là **baseline**, EasyOCR ngang hàng, Tesseract là mốc dưới, kèm bộ luật hậu xử lý theo vị trí. **Lựa chọn engine OCR chưa chốt ở giai đoạn thiết kế** — quyết định thuộc về benchmark do chính đồ án chạy (**mục 3.3, Chương 5**).

1.3.2. Phạm vi trong nghiên cứu

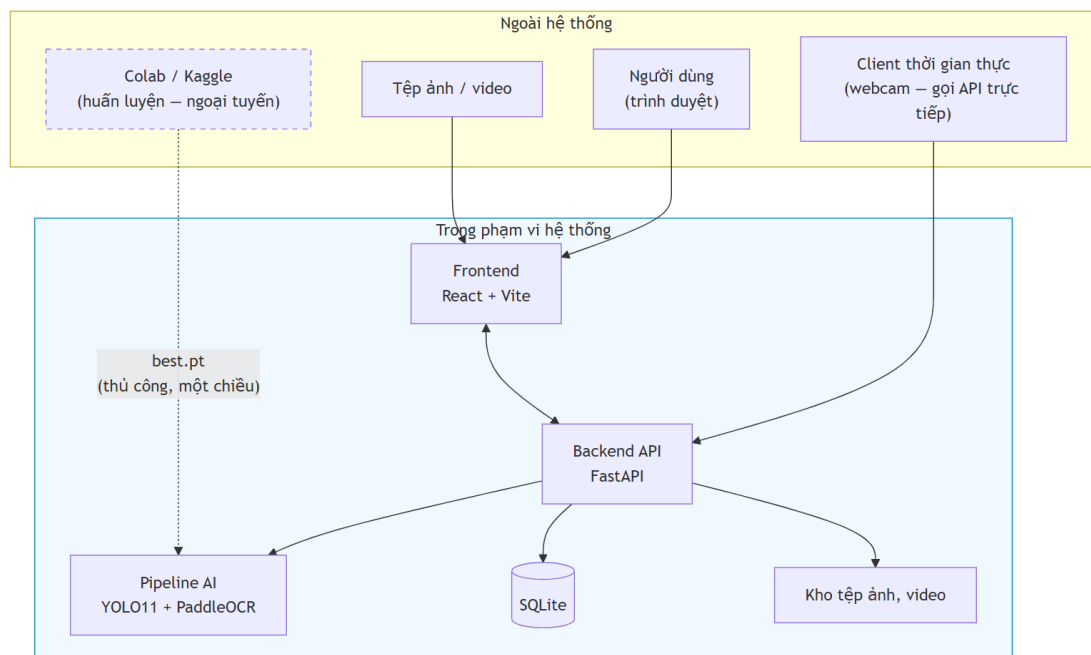
Bốn nhóm. **(a) Trí tuệ nhân tạo**: huấn luyện YOLO11 trên dữ liệu Việt Nam, so sánh biến thể n / s / m, kèm YOLO26n đối chứng (mục 3.2); **benchmark các engine OCR** trên chính tập kiểm thử biển số Việt Nam rồi tích hợp engine thắng cuộc; hậu xử lý theo luật hợp lệ theo vị trí; **hỗ trợ cả biển một dòng và hai dòng**; đánh giá đầy đủ (mAP, precision, recall, F1, ma trận nhầm lẫn); đo hiệu năng trên CPU. **(b) Dữ liệu**: thu thập, gộp, làm sạch bộ công khai; sửa nhãn; loại ảnh trùng lặp; tăng cường; chia train / val / test **có kiểm soát rò rỉ dữ liệu**; thống kê. **(c) Phần mềm**: FastAPI + Swagger; nhận dạng ảnh, video và khung hình thời gian thực qua POST /api/detect/frame; lịch sử bằng SQLite + SQLAlchemy + Alembic; giao diện React + Vite + TypeScript + TailwindCSS **ba trang** (Nhận dạng ảnh — trang chủ, Nhận dạng video, Lịch sử) sau hai đợt thu gọn 2026-07-20; tìm kiếm, lọc, tải về; thống kê ở tầng API (GET /api/statistics); Docker và Docker Compose. **(d) Kiểm thử, tài liệu**: unit

test, integration test, kiểm thử độ chính xác AI, hiệu năng, chịu tải; tài liệu học thuật và kỹ thuật.

1.3.3. Phạm vi ngoài nghiên cứu

Danh sách này là **hàng rào trước câu hỏi "sao không làm X"**; không hạng mục nào bị loại vì "không kịp làm". **Mười một hạng mục:** (1) **xác thực, phân quyền** — chạy nội bộ localhost/LAN (giả định A-04); (2) **đa camera / đa luồng**; (3) **bấm vết qua khung hình (SORT / DeepSORT)** — thay bằng **gộp trùng theo chuỗi ký tự**; (4) **phân loại loại xe** — từ 01/01/2025 TT 79/2024 **đã bỏ** quy tắc suy loại xe từ chữ cái seri; (5) **ước lượng tốc độ, phát hiện vi phạm** — cần hiệu chuẩn camera riêng; (6) **biển số nước ngoài**; (7) **barie / cổng tự động** — cần thiết bị vật lý; (8) **cloud, multi-tenant, CI/CD production** — **Docker Compose đã đủ**; (9) **ứng dụng di động** — web responsive đã đáp ứng; (10) **huấn luyện engine OCR từ đầu** — dùng pre-trained rồi **tinh chỉnh**; tinh chỉnh nằm **trong** phạm vi (mục 2.4.3(f)); (11) **suy luận thời gian thực trên GPU** — máy phát triển **không có GPU CUDA** (CON-02) ⇒ mọi số liệu là **số liệu CPU**.

1.3.4. Ranh giới hệ thống



Hình 1.1. Ranh giới hệ thống — phần bên trong là hệ thống bàn giao, Colab/Kaggle nằm ngoài

Colab/Kaggle nằm **ngoài** ranh giới khi vận hành — chỉ là công cụ ngoại tuyến sản xuất best.pt; hệ thống khi chạy **không phụ thuộc dịch vụ ngoài nào** (mục 1.2.3). Sau khi trang webcam bị gỡ (2026-07-20), client gửi khung hình trực tiếp qua POST /api/detect/frame.

1.4. Phương pháp nghiên cứu

Đề tài dùng **ba phương pháp bổ trợ nhau**: nghiên cứu lý thuyết (khảo sát tài liệu có trích dẫn, đối chiếu văn bản pháp quy hiện hành); nghiên cứu thực nghiệm (**mọi khẳng định về hiệu năng và độ chính xác đều phải có số đo tái lập được**, kèm cấu hình phần cứng và cỡ mẫu); và quy trình phát triển theo giai đoạn, mỗi giai đoạn khép lại bằng một bộ tài liệu và một mốc kiểm chứng. Mô tả đầy đủ ba phương pháp cùng danh sách mười một giai đoạn ở **Phụ lục P.1**.

1.5. Ý nghĩa khoa học và thực tiễn

1.5.1. Ý nghĩa khoa học

(a) Lắp một khoảng trống báo cáo có thật: khảo sát Phase 1 cho thấy **chưa có công trình Việt Nam nào công bố bảng so sánh tách riêng độ chính xác giữa biển một dòng và biển hai dòng trên cùng một hệ thống**, trong khi trên bộ RodoSol-ALPR của Brazil chênh lệch giữa hai bố cục có thể tới 48,6 điểm phần trăm [7] — **một con số tổng thể có thể che giấu hoàn toàn điểm gãy của hệ thống**. **(b)** Hệ thống hoá bộ luật hậu xử lý theo **cấu trúc vị trí** trên căn cứ pháp lý hiện hành (mục 1.6.3). **(c)** Bộ quy tắc công bố số liệu ở Phụ lục P.1.

1.5.2. Ý nghĩa thực tiễn

(a) Sản phẩm là hệ thống chạy được chứ không phải notebook, khởi động một lệnh, không cần Internet, dùng được làm **nền tảng khởi đầu** cho triển khai quy mô nhỏ; kiểm thử (Phase 7) và đóng gói Docker (Phase 8) **đã xong**. **(b)** Toàn bộ chỉ tiêu hiệu năng là chỉ tiêu **CPU** — nơi triển khai quy mô nhỏ thường không có máy chủ GPU. **(c)** Bộ hằng số theo TT 79/2024 + QCVN 08:2024 (81 mã tỉnh, hai tập chữ cái seri theo vị trí, ba mức tỉ lệ khung hình) dùng lại được; tài liệu ghi rõ giao thức đo và cấu hình phần cứng để nhóm sau **đối chứng**.

1.6. Đóng góp của đề tài

1.6.1. Tuyên bố trung thực về mức đóng góp

Đề tài này không tạo ra kết quả state-of-the-art.

Các con số vượt 99% trong tài liệu ALPR quốc tế đến từ nhóm nghiên cứu chuyên nghiệp có hạ tầng GPU lớn và dữ liệu độc quyền. Một đồ án làm trên máy không có GPU CUDA **không đặt mục tiêu đó** — tuyên bố ngược lại là thiếu trung thực học thuật. Đóng góp thực sự nằm ở sáu chỗ, cụ thể và kiểm chứng được.

1.6.2. Đóng góp (a) — Hệ thống hoàn chỉnh từ mô hình AI đến giao diện và triển khai

Sản phẩm có **kiến trúc phần mềm**, không phải tập script rời rạc: pipeline AI tách hoàn toàn khỏi tầng API (NFR-M1), interface trừu tượng thay engine OCR không sửa tầng API (NFR-M5), REST API có tài liệu tự sinh, giao diện web **ba màn hình**, cơ sở dữ liệu có migration, kiểm

thử $\geq 70\%$, Docker một lệnh. Khảo sát Phase 1: mã nguồn mở ALPR Việt Nam chủ yếu là script rời rạc **không công bố số liệu độ chính xác — khoảng trống kỹ nghệ**, không phải khoảng trống thuật toán, nhưng vẫn có thật.

Mức độ hoàn thành tại thời điểm viết. Tách tầng AI, interface trừu tượng, REST API, migration **đã cài đặt và xác minh bằng yêu cầu HTTP thật**; giao diện web **đã hoàn thành**, build sạch. Độ bao phủ $\geq 70\%$ **đã đạt và đã đo: 87,7%** tầng nghiệp vụ (đo 2026-07-20, docs/reports/13-refactor-result.json; Phase 7 trước đó 88,1%, toàn kho 42,0%), **882 test thu thập / 881 đạt / 1 xfail / 0 thất bại**. Docker và Docker Compose **đã hoàn thành**. Chi tiết ở **Chương 5**.

1.6.3. Đóng góp (b) — Bộ luật hậu xử lý ràng buộc theo VỊ TRÍ cho biển số Việt Nam

Bộ luật khai thác ba ràng buộc đặc thù: (i) **tập hợp lệ khác nhau theo từng vị trí** — mã địa phương thuộc **81 giá trị hợp lệ** chứ không phải $\{d\}^{14}$, seri **thứ nhất** thuộc **20 chữ cái** có G không có R [13], seri **thứ hai** của biển xe mô tô thuộc **20 chữ cái KHÁC** có R không có G, hoặc **chữ số 1–9** (không có 0) với biển kiểu cũ; (ii) **cấu trúc chuỗi và độ dài** theo quy chuẩn, cho phép sinh mặt nạ vị trí cho từng dạng biển; (iii) **định dạng cũ và mới cùng tồn tại**. Mấu chốt: sửa lỗi OCR theo **vị trí trong chuỗi chứ không theo ánh xạ hai chiều** — với cặp $0 \leftrightarrow \emptyset$, ánh xạ đúng là $0 \rightarrow \emptyset$ tại vị trí chữ số và $\emptyset \rightarrow D$ tại vị trí chữ cái, **không phải** $0 \rightarrow \emptyset$ và $\emptyset \rightarrow 0$.

⚠ Nói thẳng về độ lớn của đóng góp này. Luận điểm dự kiến ban đầu — "*khai thác bộ 20 chữ cái, loại trừ 6 chữ I J O Q R W*" — **sai và đã bị bác bỏ ở Phase 1**: tập loại trừ toàn hệ thống chỉ có **5 chữ** (I, J, O, Q, W), chữ R **hợp lệ** ở vị trí seri thứ hai của biển xe mô tô. Sửa lại **làm yếu đi** phần đóng góp nếu tính theo "số ký tự loại trừ được" — không gian tìm kiếm thu hẹp ít hơn dự kiến. Đổi lại, phần có giá trị nằm ở ràng buộc **phụ thuộc vị trí**: hệ thống dùng danh sách phẳng 20 chữ cái sẽ **sai hệ thống trên toàn bộ lớp biển xe máy có R ở vị trí thứ hai**, và đây mới là lỗi mà bộ luật của đề tài ngăn được. **Đóng góp này vì vậy được trình bày là đúng đắn về mặt pháp lý và đúng cấu trúc theo vị trí**, không phải một cải thiện lớn về không gian tìm kiếm.

1.6.4. Đóng góp (c) — Đo được ĐỊNH LƯỢNG đóng góp của bước hậu xử lý

Phần lớn công trình mô tả bước hậu xử lý ở mức định tính, không trả lời được *bước đó đóng góp bao nhiêu*. Đề tài giải quyết ở tầng dữ liệu: **lưu đồng thời cả chuỗi OCR thô và chuỗi đã sửa** cho mỗi lần nhận dạng; hiệu số giữa **NFR-A5 (trước)** và **NFR-A6 (sau hậu xử lý)** do đó là một **con số đo được**, trình bày ở **Chương 5**.

1.6.5. Đóng góp (d) — Đánh giá tách riêng biển một dòng và biển hai dòng

Khoảng trống báo cáo đã xác định ở mục 1.5.1; **NFR-A8** biến phép tách này thành nghĩa vụ báo cáo bắt buộc chứ không phải phân tích tùy chọn, kèm **NFR-A9** — báo cáo theo điều kiện ảnh, nếu bộ dữ liệu có nhãn phù hợp.

1.6.6. Đóng góp (e) — Công bố hiệu năng kèm cấu hình phần cứng CPU cụ thể

Mọi số liệu hiệu năng công bố kèm **model CPU, số luồng, kích thước ảnh đầu vào, backend suy luận và cỡ mẫu đo** (Phụ lục P.1) — FPS không kèm phần cứng thì không thể tái lập, không thể so sánh.

1.6.7. Đóng góp (f) — Đo trên chính ảnh biển số Việt Nam, và một khoản nợ được ghi nhận

Phase 1 xác định **không tồn tại benchmark công khai nào so sánh các engine OCR trên riêng ảnh biển số xe máy Việt Nam hai dòng**; hai số liệu thường được viện dẫn để chứng minh ưu thế của một engine đã **bị bác bỏ khi truy ngược về nguồn gốc** (mục 3.3). **Phần đã làm được**: hai phép so sánh trên chính ảnh biển số Việt Nam, cùng máy và cùng ngữ liệu — **PP-OCrv5_mobile so với PP-OCrv6_medium** trên 200 vùng cắt (67,0% ở 23,0 ms so với 72,5% ở 386,9 ms, mục 3.3.2), và **bộ nhận dạng gốc so với bản tinh chỉnh** trên 2.801 biển có nhãn chuỗi với bốn cấu hình (mục 4.5.3). Cả hai đều cho kết quả **trái kỳ vọng ban đầu**: chỉ đọc tài liệu rồi chọn theo con số cao nhất thì cả hai quyết định đều sai.

✅ **Khoản nợ này đã trả, ngày 03/08/2026.** Ma trận so sánh **PaddleOCR ↔ EasyOCR ↔ Tesseract** — đúng khoảng trống số 4 mà Chương 2 đánh giá là có giá trị khoa học cao nhất — **đã chạy trên toàn bộ 2.801 biển** (mục 3.3.3). Kết quả: PaddleOCR **68,87%**, hơn EasyOCR 54,59 điểm và hơn Tesseract 58,59 điểm, **bác bỏ** tài liệu công khai vốn nghiêng về EasyOCR. Một kết quả **trái kỳ vọng** khác: bước tách-rời-ghép-ngang mua 34,92 điểm cho PaddleOCR nhưng chỉ 0,03 điểm cho Tesseract, nên nó **không** phải kỹ thuật độc lập engine — chỉ bộ luật hậu xử lý mới là.

1.6.8. Những gì đề tài KHÔNG tuyên bố

Bốn điều loại trừ. **Không** tuyên bố vượt các con số độ chính xác cao nhất trong nước — chúng đo trên tập dữ liệu riêng không công khai, **không có cơ sở so sánh công bằng**. **Không** đề xuất kiến trúc mạng nơ-ron mới; đề tài **tích hợp và tinh chỉnh**. **Không** giải quyết các thách thức mở — độ phân giải rất thấp, tổng quát hoá xuyên tập dữ liệu, che khuất nặng (**Hướng phát triển, Chương 6**). Mọi số liệu hiệu năng là **số liệu CPU, không so sánh trực tiếp được** với FPS đo trên GPU.

1.7. Bộ cục quyền đề án

Quyền gồm sáu chương. **Chương 2 — Cơ sở lý thuyết**: ALPR, YOLO, nhận dạng ký tự không phân đoạn, **quy chuẩn biển số Việt Nam theo TT 79/2024 và QCVN 08:2024**, công trình liên quan và khoảng trống nghiên cứu. **Chương 3 — Khảo sát công nghệ và lựa chọn mô hình**: bảy thể hệ YOLO, tám engine OCR ứng viên (kèm benchmark ba engine tự đo trên 2.801 biển), runtime CPU, độ phân giải đầu vào, và **ranh giới giữa cái đã đo và cái mới chỉ khảo sát tài liệu**. **Chương 4 — Thiết kế và cài đặt hệ thống**: yêu cầu, kiến trúc tách tầng, bộ dữ liệu, huấn luyện bộ phát hiện, **tinh chỉnh bộ nhận dạng và phép đo có/không tinh chỉnh**, pipeline AI — backend — giao diện, Docker, và những chỗ cài đặt lệch khỏi thiết kế.

Chương 5 — Thực nghiệm và đánh giá: giao thức đo, **tách riêng biển một dòng và hai dòng, đo đóng góp định lượng của khối hậu xử lý**, hiệu năng CPU kèm cấu hình phần cứng, đối chiếu từng chỉ tiêu NFR, ca lỗi và mối đe dọa đến tính hợp lệ. **Chương 6 — Kết luận và hướng phát triển.** Cuối quyển là **Tài liệu tham khảo** và **Phụ lục**.

Vì sao khảo sát công nghệ tách thành chương riêng: đây là phần hội đồng hỏi nhiều nhất, bản thảo trước lại để khuất cuối chương cơ sở lý thuyết; Chương 3 gom về một chỗ trả lời tường minh — kể cả khi câu trả lời trung thực đôi lúc là "chưa đo được".

Tóm tắt chương

Chương 1 xác lập bốn nền tảng. **Lý do tồn tại của đề tài:** với 77 triệu xe máy [1], **biển hai dòng là đa số tuyệt đối**; OpenALPR trên tập kiểm thử cân bằng của bộ **RodoSol-ALPR (Brazil)** đạt **94,3% trên biển một dòng nhưng chỉ 45,7% trên biển hai dòng, chênh 48,6 điểm phần trăm** [7] — dẫn như *analogue*, **không phải số liệu Việt Nam**; cộng với cấu trúc chuỗi theo TT 79/2024 và QCVN 08:2024 $\Rightarrow$ giải pháp huấn luyện trên dữ liệu nước ngoài **không áp dụng trực tiếp được**. **Mục tiêu đo được:** **$mAP@0.5 \geq 0,90$, độ chính xác toàn trình $\geq 0,88$, độ trễ $p95 \leq 800$ ms trên CPU**. **Ranh giới:** phạm vi trong bốn nhóm, phạm vi ngoài **11 hạng mục kèm lý do loại trừ tường minh**. **Đóng góp trung thực:** đề tài **không tạo ra kết quả state-of-the-art**; sáu đóng góp: (a) hệ thống hoàn chỉnh có kiến trúc phần mềm; (b) bộ luật hậu xử lý **ràng buộc theo vị trí**; (c) **đo định lượng** đóng góp của hậu xử lý; (d) **tách riêng biển một dòng và hai dòng**; (e) hiệu năng **kèm cấu hình phần cứng CPU cụ thể**; (f) **benchmark engine OCR trên chính ảnh biển số Việt Nam**. Chương 2 trình bày cơ sở lý thuyết và quy chuẩn biển số Việt Nam.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương đặt nền lý thuyết và tư liệu cho phần thiết kế. Nguyên tắc xuyên suốt: **mọi con số gắn nguồn tại chỗ, mọi cảnh báo về phạm vi áp dụng giữ nguyên** — lĩnh vực này hay công bố số trên 99% nhưng đo trên tập dữ liệu và giao thức rất khác nhau.

2.1. Phạm vi và bố cục cơ sở lý thuyết

Một hệ thống ALPR gồm bốn khối nối tiếp — **phát hiện vùng biển, nắn chỉnh và tiền xử lý, nhận dạng ký tự, hậu xử lý theo quy chuẩn** — và độ chính xác cuối cùng là **tích** của độ chính xác từng khối, nên một khối yếu kéo cả chuỗi xuống. Đề án đi theo hướng **two-stage** (phát hiện rồi nhận dạng riêng) kết hợp bộ nhận dạng **segmentation-free**; căn cứ của lựa chọn đó trình bày ở Chương 3.

Chương này chỉ giữ phần lý thuyết **ràng buộc trực tiếp một quyết định của hệ thống**: quy chuẩn biển số Việt Nam (2.2) — cơ sở của bộ luật hậu xử lý; kiến trúc YOLO11 và các chỉ số đánh giá khối phát hiện (2.3); kiến trúc CRNN/CTC cùng **điểm gãy của nó trên văn bản nhiều dòng** (2.4) — nền tảng lý thuyết của rủi ro R-04 và của đóng góp kỹ thuật lỗi; và khảo sát công trình liên quan cùng sáu khoảng trống nghiên cứu (2.5). Phần bối cảnh lĩnh vực — định nghĩa và ứng dụng ALPR, lịch sử từ xử lý ảnh cổ điển tới học sâu, bảng phân loại các hướng tiếp cận, và các khái niệm nền IoU/NMS — đặt ở **Phụ lục N** để thân bài không phải mang kiến thức đại cương.

2.2. Quy chuẩn biển số xe Việt Nam

2.2.1. Căn cứ pháp lý hiện hành

Cảnh báo văn bản hết hiệu lực. Nhiều tài liệu, kể cả bài báo 2023 – 2024, vẫn viện dẫn **Thông tư 24/2023/TT-BCA** — **đã hết hiệu lực từ 01/01/2025** [12]; đề án chỉ nhắc như bối cảnh lịch sử.

Bốn văn bản căn cứ: **TT 79/2024/TT-BCA** hiệu lực 01/01/2025, thay TT 24/2023, quy định cấu trúc biển, seri, màu sắc [8]; **TT 13/2025/TT-BCA** sửa đổi TT 79/2024 [9]; **TT 51/2025/TT-BCA** hiệu lực 01/7/2025, **thay toàn bộ Phụ lục mã tỉnh** sau sáp nhập còn 34 tỉnh/thành [10]; **QCVN 08:2024/BCA** kèm TT 81/2024/TT-BCA, hiệu lực 01/01/2025, quy chuẩn quốc gia về kết cấu, kích thước, vật liệu [11]. Biển quân đội thuộc TT 169/2021/TT-BQP [15], **ngoài phạm vi TT 79/2024**.

TT 79/2024 quy định **nội dung** biển — cơ sở biểu thức chính quy; QCVN 08:2024/BCA quy định **hình thức vật lý** — cơ sở ngưỡng tỷ lệ khung hình; module chuẩn hoá cần cả hai. Khung pháp lý đổi **ba lần trong hai năm** là rủi ro kỹ thuật trực tiếp [19]; hệ quả ở mục 2.2.7.

2.2.2. Cấu trúc biển số ô tô và xe máy

a) Biển số ô tô trong nước: 8 ký tự chữ–số, ba thành phần — **mã địa phương 2 chữ số** trong 81 mã hợp lệ thuộc dải 11 – 99 [10], [14]; **seri 1 chữ cái** trong 20 chữ với biển trắng và

vàng, 11 chữ với biển xanh [13]; **số thứ tự 5 chữ số**, 000.01 – 999.99 [8] — ví dụ 30A-123.45, 51K-999.99, 80B-123.45 (Cục CSGT). Trên đường vẫn còn **biển 4 chữ số kiểu cũ** (29A-1234); xe đã đăng ký **không bắt buộc đổi biển** [26] nên biểu thức chính quy phải chấp nhận nhóm thứ tự **4 hoặc 5 chữ số**.

b) Biển số xe máy cá nhân: 9 ký tự — 2 chữ số mã tỉnh, 2 chữ cái seri, 5 chữ số thứ tự (29-HA 002.33); quy tắc 2 chữ cái áp dụng từ **15/8/2023**, giữ trong TT 79/2024 [27]. Hai kiểu seri cùng lưu hành: **mới** 2 chữ cái (29-AA 123.45) và **cũ** 1 chữ + 1 số (29-B1 123.45), vẫn hợp pháp [27]. Cách hiểu "biển 1 chữ 1 số chỉ dùng đến hết 2025" là **không chính xác** — điều khoản 31/12/2025 chỉ nói về dùng nốt phôi biển cũ; **biển kiểu cũ còn trên đường hàng chục năm**, biểu thức chính quy phải chấp nhận cả hai.

c) Một nhập nhằng cấu trúc quan trọng. Chuỗi 8 ký tự dạng *hai số – một chữ – năm số* khớp **đồng thời** biển ô tô và biển xe máy kiểu cũ sau khi bỏ dấu phân cách — **không thể phân loại phương tiện chỉ bằng chuỗi ký tự**, lý do hệ thống lưu trường số dòng như thuộc tính độc lập (Chương 4).

d) Seri không còn cho biết loại xe. Trước 2025 seri mang ngữ nghĩa (A xe con, B xe khách, C, K xe tải), **từ 01/01/2025 bị bãi bỏ**, seri cấp tuần tự [28]; heuristic "seri C suy ra xe tải" **sai về pháp lý**, tín hiệu phân loại duy nhất còn hợp lệ là **màu nền biển** (mục 2.2.5).

2.2.3. Mã tỉnh, thành phố

Từ 01/7/2025 cả nước còn **34 tỉnh, thành phố**; ký hiệu sau hợp nhất **bao gồm toàn bộ ký hiệu của các địa phương được hợp nhất** [26], biển cũ không mất giá trị pháp lý. Dài 11 – 99 có **89 số**; theo Phụ lục TT 51/2025 có **81 mã đang dùng** (80 mã địa phương + mã 80 của Cục CSGT) và **8 mã không dùng: 13, 42, 44, 45, 46, 87, 91, 96** [14]. TP. Hồ Chí Minh 13 mã (41; 50 – 59; 61; 72); Hà Nội 6 mã (29; 30 – 33; 40) [14].

Kiểm tra mã tỉnh loại khoảng 9,0% không gian tìm kiếm ở hai ký tự đầu, và quan trọng hơn: biển lỗi OCR hai vị trí đầu từ **sai âm thầm** thành **sai phát hiện được** — đọc ra 46A-123.45 thì biết ngay mã 46 không tồn tại và hạ cờ hợp lệ. Giả thuyết mã 13 là mã cũ của Hà Bắc **chưa kiểm chứng được nguồn chính thức**, chỉ nêu tham khảo.

2.2.4. Tập ký tự seri và các chữ cái bị loại trừ

Mục dễ bị trình bày sai nhất của chương. Biển trắng và vàng chữ đen dùng seri **một trong 20 chữ cái** [13]; đối chiếu 26 chữ Latin thì vắng I J O Q R W — nhưng **suy diễn "26 – 20 = 6 chữ bị loại trừ" là SAI**: danh sách 20 chữ **chỉ áp dụng cho chữ cái thứ nhất**; ở vị trí thứ hai của seri xe máy là tập khác — **có R, không có G**. Hợp hai vị trí, tập chữ không bao giờ xuất hiện trên biển Việt Nam chỉ gồm **5 chữ: I, J, O, Q, W**; chữ R còn ở ký hiệu đặc biệt R, RM của rơ moóc [29].

Bảng 2.1. Tổng hợp các tập ký tự seri

Tập	Số lượng	Nội dung
Chữ cái thứ nhất của seri	20	A B C D E F G H K L M N P S T U V X Y Z

Tập	Số lượng	Nội dung
Chữ cái thứ hai của seri xe máy	20	A B C D E F H K L M N P R S T U V X Y Z
Seri biển nền xanh	11	A B C D E F G H K L M
Bị loại trừ khỏi toàn hệ thống	5	I, J, O, Q, W
Tập ký tự an toàn tối thiểu cho OCR	21	20 chữ ở vị trí thứ nhất, hợp thêm R

c) Hệ quả thứ nhất — ràng buộc theo vị trí, không phải tập phẳng. Xe mô tô biển xanh dùng 1 trong 11 chữ cái kết hợp 1 chữ số **1 – 9**, không có số 0 [13]. G hợp lệ ở vị trí thứ nhất nhưng không ở vị trí thứ hai, R ngược lại; bộ luật dùng danh sách phẳng sẽ **vừa bỏ sót lỗi vừa sửa nhầm ký tự đúng** — điểm đồ án xử lý khác các mô tả hiện có (Chương 1). Ký tự nhóm I, J, O, Q, W ở vị trí chữ cái chắc chắn là lỗi; ánh xạ có cơ sở hình dạng: 0 → Ø, I → 1, Q → Ø; J, W không có ứng viên hiển nhiên nên chỉ **hạ cờ hợp lệ**. Chữ R **không** thuộc nhóm này và **tuyệt đối không được ánh xạ đi**.

d) Hệ quả thứ hai — tập ký tự huấn luyện OCR. Huấn luyện theo "20 chữ cái" thì mô hình **không bao giờ dự đoán được chữ R**, sai hệ thống trên mọi biển xe máy có R ở vị trí thứ hai; mất mát ở **tầng mô hình**, hậu xử lý không cứu được.

Khuyến nghị áp dụng cho đồ án. Huấn luyện tập ký tự **đầy đủ A–Z và 0–9, tức 36 ký tự**, áp ràng buộc hợp lệ ở **tầng hậu xử lý**. Nếu buộc phải thu hẹp, dùng **21 chữ cái** (20 chữ hợp thêm R), **tuyệt đối không dùng 20**.

e) Hạn chế kiểm chứng cần giữ nguyên khi trình bày. Hai danh sách chữ cái ở Bảng 2.1 **chưa được đối chiếu với toàn văn Điều 34 TT 79/2024/TT-BCA**: bản PDF chính thức là bản quét không có lớp văn bản, công tra cứu pháp luật chặn truy cập tự động. Kết luận về chữ R dựa trên nguồn thứ cấp, cần xác nhận lại khi tiếp cận được toàn văn. Ghi rõ hạn chế này là bắt buộc và không được lược bỏ khi rút gọn văn bản.

f) Các ký hiệu seri đặc biệt [29]: CD, MK, MĐ (xe máy chuyên dùng, máy kéo, xe máy điện); R, RM (rơ moóc); HC; KT, LD, DA; T, TĐ. Hai bẫy: **chữ R xuất hiện ở đây** dù ngoài tập 20 chữ, củng cố khuyến nghị (d); TĐ, MĐ chứa Đ — ngoài Latin ASCII — nên OCR gần như chắc chắn trả D, module chuẩn hoá phải chấp nhận cả hai dạng.

2.2.5. Màu nền và ý nghĩa

Bảng 2.2. Màu nền biển số và đối tượng áp dụng [13]

Màu nền / màu chữ	Đối tượng	Ghi chú cho ALPR
Trắng / đen	Tổ chức, cá nhân trong nước, xe không kinh doanh vận tải	Phổ biến nhất
Vàng / đen	Xe kinh doanh vận tải	Tín hiệu phân loại duy nhất còn hợp lệ
Xanh dương /	Cơ quan Đảng, Nhà nước, công an	Seri chỉ dùng 11 chữ cái

Màu nền / màu chữ	Đối tượng	Ghi chú cho ALPR
trắng		
Trắng / đỏ	Xe ngoại giao (NG) và tổ chức quốc tế (QT)	Cấu trúc chuỗi khác hẳn [30]
Đỏ / trắng	Xe quân đội và doanh nghiệp quân đội	Ngoài phạm vi TT 79/2024, thuộc TT 169/2021/TT-BQP [15]

QCVN 08:2024/BCA chỉ quy định **4 tổ hợp màu, không có nền đỏ** [11] — biển quân đội do Bộ Quốc phòng quản lý riêng [15]. **Xe điện không có biển riêng**: xe năng lượng sạch **không được cấp biển xanh lá**, dùng biển thường kèm biểu tượng [31] — không phát hiện được xe điện qua màu biển. Màu nền là tín hiệu phân loại duy nhất còn hợp lệ [32]; nhưng module chuẩn hoá làm việc trên chuỗi ký tự, phân loại theo màu ngoài phạm vi của nó.

2.2.6. Kích thước vật lý và tỷ lệ khung hình

Cơ sở định lượng phân biệt biển một dòng với hai dòng — then chốt với rủi ro R-04 (mục 2.4.3). Ô tô được cấp **02** biển: 01 ngắn (**2 dòng**), 01 dài (**1 dòng**); xe mô tô, xe gắn máy, rơ moóc được cấp **01** biển **2 dòng** [32] — **một ô tô mang cùng chuỗi ký tự trên hai biển hình dạng hoàn toàn khác nhau**.

Bảng 2.3. Kích thước và tỷ lệ khung hình của các loại biển số [11]

Loại biển	Kích thước (dài × cao)	Tỷ lệ khung hình	Số dòng
Ô tô — biển dài	520 × 110 mm	4,727	1 dòng
Ô tô — biển ngắn	330 × 165 mm	2,000	2 dòng
Xe mô tô, xe gắn máy	190 × 140 mm	1,357	2 dòng

⚠ Cảnh báo về mốc hiệu lực của bộ số liệu kích thước. Bộ số liệu trên **chỉ đúng từ 01/01/2025**; tiêu chuẩn trước đó quy định biển ô tô ngắn **200 × 280 mm**, biển dài **110 × 470 mm**, và rất nhiều tài liệu thứ cấp — kể cả bài báo năm 2023 — vẫn dùng bộ số cũ. Mọi trích dẫn kích thước biển số **bắt buộc ghi kèm mốc hiệu lực**; nếu không, người phản biện đối chiếu văn bản hiện hành sẽ kết luận là sai.

Không loại biển nào rơi vào khoảng (2,000 ; 4,727) — khoảng trống rộng 2,727 đơn vị. Đề án đề xuất: **AR < 2,5 → 2 dòng; AR > 3,0 → 1 dòng; 2,5 ≤ AR ≤ 3,0 — vùng nghi ngờ**, thử cả hai nhánh, chọn kết quả tin cậy cao hơn.

Hai lưu ý bắt buộc. Thứ nhất, đây là đề xuất của đề án, KHÔNG phải quy định pháp luật: ba giá trị 1,357 / 2,000 / 4,727 trích được từ QCVN 08:2024/BCA, nhưng ngưỡng 2,5 và 3,0 là **suy luận thiết kế của tác giả**, phải kiểm chứng thực nghiệm; gán bộ ngưỡng cho văn bản pháp luật là lỗi trích dẫn. **Thứ hai:** phải đo tỷ lệ khung hình trên ảnh **đã nắn chỉnh phối cảnh** hoặc hộp bao xoay tối thiểu, **không** đo trên hộp bao thẳng trục — biển một dòng nghiêng 30° có tỷ lệ hộp thẳng trục tụt dưới 3,0 và bị phân loại nhầm.

d) Bố cục biển hai dòng. Ô tô biển ngắn: 30A / 123.45; xe máy kiểu mới: 29-AA / 123.45; xe máy kiểu cũ: 29-B1 / 123.45, dòng dưới 4 – 5 chữ số.

e) Dấu phân cách. Các nguồn mô tả vị trí dấu gạch ngang không nhất quán, không tra cứu được nguyên văn quy cách in của QCVN 08:2024/BCA; đề án chọn quyết định an toàn: **module chuẩn hoá loại bỏ toàn bộ ký tự phân cách rồi kiểm tra hợp lệ trên chuỗi chữ-số thuần.**

f) Hai thông số vật lý khác. Biển hợp kim nhôm, phản quang, chữ dập nổi cao ($1,7 \pm 0,1$) mm [11]; chữ dập nổi tạo bóng và loá theo góc chiếu — nguyên nhân lỗi B đọc thành 3; bù lại font và vật liệu chuẩn hoá toàn quốc là yếu tố tốt cho OCR.

2.2.7. Ý nghĩa đối với thiết kế hệ thống nhận dạng

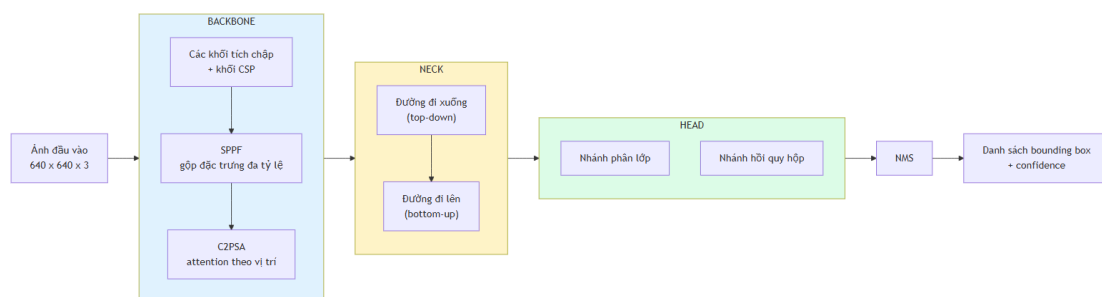
Bảy dữ kiện kéo theo bảy quyết định thiết kế: **81 mã tình trong dải 89 số** biến lỗi OCR hai ký tự đầu thành sai phát hiện được; **tập seri khác theo vị trí** buộc ràng buộc **theo vị trí** và tập huấn luyện OCR đủ 36 ký tự; **hai kiểu seri xe máy, nhóm thứ tự 4 hoặc 5 chữ số** buộc biểu thức chính quy đa nhánh; **chuỗi 8 ký tự khớp hai loại biển** nên phải lưu số dòng độc lập; **seri không còn cho biết loại xe** nên cấm heuristic suy loại phương tiện; **khoảng trống tỷ lệ khung hình 2,727** là cơ sở ngưỡng phân loại bố cục; **khung pháp lý đổi ba lần trong hai năm** buộc hậu xử lý tách rời mô hình để cập nhật độc lập [19].

Về mức đóng góp: luận điểm dự kiến ban đầu — "loại trừ 6 chữ I J O Q R W" — là **sai**, và việc sửa làm **yếu đi** đóng góp theo tiêu chí thu hẹp không gian tìm kiếm; đổi lại phần giá trị chuyển sang ràng buộc **phụ thuộc vị trí trong chuỗi**: danh sách phẳng sẽ sai hệ thống trên toàn bộ lớp biển xe máy có R ở vị trí thứ hai — lỗi mà bộ luật của đề án ngăn được. Đóng góp là **đúng dẫn về pháp lý và cấu trúc**, không phải cải thiện lớn về không gian tìm kiếm.

2.3. Cơ sở lý thuyết về phát hiện đối tượng

2.3.1. Kiến trúc YOLO: nguyên lý one-stage và anchor-free

Họ two-stage (Faster R-CNN) sinh vùng đề xuất rồi phân loại từng đề xuất — độ trễ cao; **họ one-stage** (YOLO, SSD, RetinaNet) hồi quy trực tiếp trong một lần lan truyền xuôi — thời gian thực. Ràng buộc CPU loại họ two-stage từ đầu; một nghiên cứu ALPR trên 50.000 ảnh và 10.000 video clip kết luận nhóm YOLO (v5–v10) vượt trội Faster R-CNN và SSD cả độ chính xác lẫn thời gian suy luận [48].



Hình 2.1. Kiến trúc tổng quát backbone – neck – head của YOLO11 (theo [49], [16])

Ba phần: **backbone** trích đặc trưng, kết thúc bằng SPPF gộp đa tỷ lệ; **neck** hợp nhất đặc trưng nhiều tầng; **head** sinh dự đoán — từ YOLOv8 dùng **anchor-free split head** [49]. Anchor-free có ý nghĩa riêng với biến số: anchor-based hồi quy theo tập hợp mẫu thiết kế theo phân bố COCO — biến số nằm ngoài phân bố đó (một dòng $\approx 4,7:1$, hai dòng $\approx 1,4:1$); anchor-free hồi quy **trực tiếp khoảng cách tâm đến bốn cạnh**, xử lý cả hai chế độ tỷ lệ bằng một cơ chế [16].

2.3.2. YOLO11: các cải tiến kiến trúc

Bài tổng quan độc lập xác định ba thành phần chính của YOLO11: **C3k2**, **SPPF**, **C2PSA** [50]; phần dưới đối chiếu trực tiếp mã nguồn Ultralytics [51]. **a) C3k2** — là **C2f có thể hoán đổi khối con**: C3k2 kế thừa trực tiếp từ C2f của YOLOv8; khác biệt duy nhất là một cờ — tắt thì giống hệt C2f, bật thì dùng khối C3k tùy chỉnh kích thước nhân [51]. YOLO11 giảm tham số mà giữ độ chính xác vì không đối triết lý CSP, chỉ cấu hình linh hoạt hơn. **b) C2PSA** — thành phần YOLOv8 hoàn toàn không có, khác biệt kiến trúc thực sự; đặt ngay sau SPPF để attention tái phân bố trọng số theo vị trí không gian. Ultralytics khẳng định cơ chế này cải thiện phát hiện **đối tượng nhỏ** và **che khuất phức tạp** so với YOLOv8 [52].

Lưu ý về mức độ chứng minh. Phát biểu về đối tượng nhỏ là **định tính**: Ultralytics không công bố AP_small/AP_medium/AP_large theo chuẩn COCO cho từng biến thể, nên không thể chứng minh định lượng YOLO11 hơn YOLOv8 bao nhiêu trên đối tượng nhỏ [16]. Đề án phải **tự đo trên dữ liệu của mình**; kết quả ở Chương 5.

Bảng 2.4. So sánh khác biệt kiến trúc giữa các phiên bản YOLO gần đây

Phiên bản	Khối backbone	Cơ chế attention	Đầu dự đoán	NMS	Điểm mới đáng chú ý nhất
YOLOv8 [49]	C2f	Không có	Anchor-free, tách nhánh	Có	Chuyển sang anchor-free
YOLOv9 [53]	GELAN	Không có	Anchor-free	Có	PGI chống mất mát thông tin
YOLOv10 [46]	Rank-guided blocks	Partial self-attention	Hai đầu song song	Không	Consistent dual assignments
YOLO11 [16]	C3k2 (kế thừa C2f)	C2PSA sau SPPF	Anchor-free, tách nhánh	Có	C2PSA cải thiện đối tượng nhỏ
YOLOv12 [54] / YOLOv13 [55]	R-ELAN / DS-C3k2	Area Attention / HyperACE (hypergraph)	Anchor-free	Có	Attention làm trung tâm; tương quan bậc cao, FullPAD

Phiên bản	Khối backbone	Cơ chế attention	Đầu dự đoán	NMS	Điểm mới đáng chú ý nhất
YOLO26 [47]	Kế thừa dòng Ultralytics	Kế thừa dòng Ultralytics	Bỏ DFL	Không (mặc định)	Bỏ DFL, dễ xuất và lượng tử hoá

Luận cứ chọn YOLO11 trình bày đầy đủ ở mục 3.2.

2.3.3. Các chỉ số đánh giá khối phát hiện

a) Precision, Recall và F1. Với TP dự đoán đúng, FP dự đoán sai, FN đối tượng bỏ sót:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Với ALPR, **recall của detection quan trọng hơn precision**: biển bỏ sót là mất vĩnh viễn, vùng báo nhầm bị hậu xử lý loại vì chuỗi không khớp cú pháp.

b) AP và mAP. AP là diện tích dưới đường cong Precision–Recall; mAP là trung bình AP trên N lớp — đề án có $N = 1$ nên mAP trùng AP:

$$\text{AP} = \int_0^1 p(r) dr, \quad \text{mAP} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i$$

c) mAP@0.5 và mAP@0.5:0.95 — hai chỉ số khác nhau, không được so sánh chéo. **mAP@0.5** tính tại một ngưỡng IoU cố định 0,5; **mAP@0.5:0.95** lấy trung bình trên 10 ngưỡng từ 0,5 đến 0,95 bước 0,05:

$$\text{mAP@0.5:0.95} = \frac{1}{10} \sum_{t \in \{0,50; 0,55; \dots; 0,95\}} \text{mAP}@t$$

Trên cùng một mô hình và cùng một tập dữ liệu, **mAP@0.5:0.95 LUÔN nhỏ hơn hoặc bằng mAP@0.5** — vì **mAP@0.5** là một trong mười số hạng của phép trung bình ở (2.4), và là số hạng lớn nhất.

Khoảng cách giữa hai chỉ số với biến số thường rất lớn do hộp bao dẹt, ba minh chứng: **87,2%** so với **46,5%** trên biển Ấn Độ [56]; **0,906** so với **0,631** ở một nghiên cứu YOLOv11 [57]; **99,5%** so với **80,7%** trên biển xe máy Indonesia [58]. Cả ba xác nhận: **biến số dễ phát hiện nhưng khó khớp hộp bao chính xác**.

⚠ Cảnh báo phương pháp luận bắt buộc giữ nguyên. Một cách trình bày phổ biến và **sai** là đặt **mAP@0.5** của một nghiên cứu ALPR (khoảng 0,90 – 0,99) cạnh **mAP@0.5:0.95** trên COCO của cùng lớp mô hình (khoảng 0,395 ở phân khúc nano [16]) rồi kết luận "bài toán biến số dễ hơn bài toán COCO". Đây là **so sánh giữa hai chỉ số có định nghĩa khác nhau**, và theo hệ quả toán học nêu trên, chênh lệch giữa chúng **không mang bất kỳ thông tin nào** về độ khó tương đối.

Phép đối chiếu hợp lệ duy nhất là **mAP@0.5** với **mAP@0.5**, hoặc **mAP@0.5:0.95** với **mAP@0.5:0.95**, và **trên cùng một tập dữ liệu**; ngay cả khi cùng định nghĩa nhưng khác tập dữ liệu, so sánh cũng chỉ để cảm nhận độ khó chứ không làm luận cứ cho quyết định kỹ thuật. Lỗi này đã được phát hiện và sửa trong quá trình khảo sát tài liệu của đồ án, nêu tường minh ở đây vì hội đồng phản biện phát hiện rất nhanh.

d) Chỉ tiêu của đồ án. Vì mục tiêu detection là cắt vùng crop đủ tốt để OCR đọc, đồ án dùng **mAP@0.5** làm chỉ tiêu chính, **mAP@0.5:0.95** vẫn báo cáo nhưng không đặt ngưỡng chấp nhận; giá trị ở Chương 5. **e) mIoU.** Một số công trình dùng IoU trung bình toàn tập — nhóm Học viện Kỹ thuật Quân sự báo cáo mIoU 95,01% trên biển Việt Nam [59] — chỉ số khác mAP, không so sánh chéo được.

2.4. Cơ sở lý thuyết về nhận dạng ký tự

2.4.1. Bài toán OCR và đặc thù khi áp dụng cho biển số

OCR (*Optical Character Recognition*) chuyển văn bản trong ảnh thành chuỗi, thường gồm **text detection** khoanh vùng rồi **text recognition** đọc từng vùng. Sai lầm phổ biến: lấy thẳng bảng xếp hạng OCR phổ thông làm căn cứ chọn engine cho ALPR.

Bảng 2.5. So sánh OCR văn bản tài liệu và OCR biển số xe

Chiều so sánh	OCR văn bản tài liệu	OCR biển số xe
Nguồn ảnh	Ảnh quét hoặc chụp tài liệu, gần chính diện	Ảnh cảnh ngoài trời, phối cảnh nghiêng, chói sáng, nhoè do chuyển động
Độ dài chuỗi và tập ký tự	Hàng trăm đến hàng nghìn ký tự; tập lớn, mở, kèm dấu	7 – 9 ký tự; tập đóng — chỉ A–Z và 0–9
Bố cục	Nhiều dòng, đa cột, ngắt dòng tùy ý	Cố định: 1 hoặc 2 dòng theo quy chuẩn nhà nước
Ràng buộc cú pháp và vai trò hậu xử lý	Gần như không có; hậu xử lý phụ trợ	Rất chặt, kiểm tra được bằng biểu thức chính quy; hậu xử lý bắt buộc , là lớp sửa lỗi chính
Tiêu chí đánh giá	CER / WER — chấp nhận sai lẻ tẻ	Khớp chuỗi tuyệt đối — sai 1 ký tự là hỏng cả bản ghi
Giá trị của thông tin ngôn ngữ	Cao — mô hình ngôn ngữ sửa lỗi hiệu quả	Thấp — không có từ vựng để dựa vào

Bốn hệ quả: **tập ký tự đóng là tài sản** — biển Việt Nam chỉ dùng A–Z, 0–9 không dấu nên ưu thế "hỗ trợ tiếng Việt" là **vô nghĩa**, từ điển đa ngôn ngữ còn tăng không gian nhầm lẫn; **ràng buộc cú pháp bù điểm yếu whitelist** qua hậu xử lý theo vị trí (Chương 4); đây là **ảnh cảnh, không phải ảnh tài liệu** — benchmark trên văn bản chỉ tham chiếu xu hướng; **bố cục hai dòng là lớp bài toán riêng** (mục 2.4.3).

2.4.2. Kiến trúc CRNN và hàm mất mát CTC

CRNN gồm ba tầng: **tầng tích chập** trích đặc trưng và — điểm mẫu chốt — **downsample** chiều cao về **1**, biến bản đồ đặc trưng thành **chuỗi vector theo chiều rộng**; **tầng hồi quy** (Bi-LSTM) mô hình hoá ngữ cảnh hai chiều; **tầng phiên mã** giải mã thành chuỗi, thường bằng CTC. EasyOCR dùng đúng kiến trúc này (ResNet, Bi-LSTM, CTC) [60]; PaddleOCR dùng SVTR-LCNet kết hợp GTC [17], vẫn thuộc họ CTC.

Hàm mất mát CTC giải vấn đề: biết chuỗi nhãn đúng nhưng **không biết mỗi ký tự nằm ở cột đặc trưng nào**. CTC thêm ký hiệu trống ε , định nghĩa ánh xạ $\mathcal{B}$ gộp ký tự lặp rồi xoá ε — ví dụ $\mathcal{B}(3\varepsilon 00\varepsilon A) = 30A$ — và tính xác suất chuỗi nhãn $\mathbf{l}$ bằng tổng xác suất **mọi** đường đi thô π ánh xạ về nó:

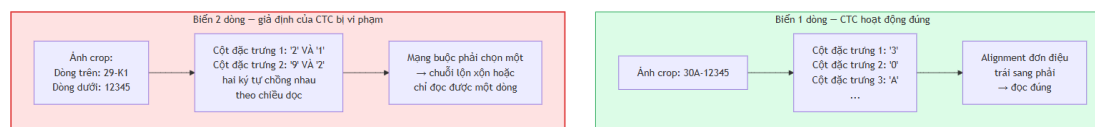
$$p(\mathbf{l} | \mathbf{x}) = \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{l})} \prod_{t=1}^T y_{\pi_t}^t, \quad \mathcal{L}_{\text{CTC}} = -\log p(\mathbf{l} | \mathbf{x})$$

Tổng ở (2.3) tính hiệu quả bằng quy hoạch động tiến–lùi. Ưu điểm quyết định: **không cần nhãn vị trí từng ký tự** — lý do CTC là mặc định của hầu hết engine OCR mã nguồn mở, và lý do LPRNet đạt 3 ms/biến trên GPU GTX 1080, 1,3 ms trên CPU i7-6700K mà vẫn 95% accuracy trên biển Trung Quốc [40].

2.4.3. Vì sao kiến trúc CTC gặp khó với văn bản nhiều dòng

Mục kỹ thuật quan trọng nhất của chương: nền tảng lý thuyết cho rủi ro **R-04** ("khả năng Cao, ảnh hưởng Cao") — ở Việt Nam nơi xe máy áp đảo, biển hai dòng là dạng phổ biến chứ không phải ngoại lệ.

a) Giả định alignment đơn điệu của CTC. Ánh xạ $\mathcal{B}$ ở (2.3) hoạt động trên **chuỗi một chiều** theo trục t — chính là **trục chiều rộng ảnh**: CTC giả định ngầm ký tự **tuần tự trái sang phải trên một dòng duy nhất**, đó là bản chất toán học của hàm mất mát chứ không phải tùy chọn cấu hình. Ảnh hai dòng vi phạm giả định: chiều cao đã downsample về **1**, mỗi vector cột chứa **cả hai ký tự chồng nhau theo chiều dọc**, mạng cho ra chuỗi lộn xộn hoặc chỉ đọc một dòng [61].



Hình 2.2. Cơ chế sụp đổ của CTC trên ảnh văn bản hai dòng (theo [61])

b) Bằng chứng cụ thể trong PaddleOCR — `rec_image_shape`. Module recognition PP-OCRv3/v4/v5 resize mọi ảnh về **chiều cao cố định 48 pixel** (`rec_image_shape = 3 × 48 × 320`) [62]. Áp vào biển xe máy tỷ lệ 1,357: đưa thẳng crop 2 dòng thì chiều rộng sau resize chỉ còn $48 \times 1,357 \approx 65$ px, mỗi dòng ≈ 24 px cao — **không đọc được**; sau tách dòng và ghép ngang (AR $\approx 5,43$) chiều rộng ≈ 261 px, mỗi dòng tròn 48 px — **đọc được**. Kết luận kiến trúc: **không tồn tại cấu hình nào của module recognition PP-OCR giải được bài**

toán này; phải giải ở **tầng trên** bằng module tách dòng, hoặc thay hẳn mô hình recognition — lý do mục 3.3 kết luận chọn engine OCR **không quyết định** thành bại của R-04.

c) Bảng chứng định lượng độc lập. *Điểm gây của một hệ thống thương mại trường thành:* trên tập kiểm thử cân bằng của **bộ RodoSol-ALPR (Brazil)** — 4.000 ảnh ô tô biển một dòng và 4.000 ảnh xe máy biển hai dòng — OpenALPR nhận đúng **94,3%** ô tô nhưng chỉ **45,7%** xe máy, chênh **48,6 điểm phần trăm** trên cùng hệ thống, cùng tập kiểm thử, không biến số nào khác ngoài bố cục biển [7]; rộng hơn, cả 12 phương pháp và 2 hệ thống thương mại đều **không vượt 70% recognition rate** trên bộ này, có công trình phải **loại bỏ hoàn toàn xe máy** vì không sửa được phương pháp [7].

⚠ Cảnh báo phạm vi áp dụng — bắt buộc giữ nguyên. Cặp số 94,3% / 45,7% được đo trên bộ **RodoSol-ALPR của Brazil, không phải trên dữ liệu Việt Nam.** Nó được dẫn ở đây như một *analogue* định lượng về độ khó vượt trội của biển hai dòng xe máy tại một quốc gia cũng có tỷ lệ xe máy cao. Trích dẫn nhằm cặp số này thành số liệu Việt Nam là lỗi trích dẫn nghiêm trọng.

Riêng kích thước ảnh đầu vào đã đủ phá huỷ hiệu năng: trong PatrolVision, cùng mô hình chỉ đổi kích thước ảnh vào — 240×80 cho biển một dòng đạt 83%, hai dòng **chỉ 30%**; 288×200 bao phủ cả hai bố cục cho tổng thể 67% [63]. *Hiệu quả của tách và ghép:* các cài đặt tham chiếu cho biển hai tầng Trung Quốc đều cắt crop thành hai phần rồi ghép ngang trước khi vào OCR [64]. **d) Nắn chỉnh trước khi tách:** chiếu ngang tìm điểm trung và phân ngưỡng theo toạ độ dọc đều **vô hiệu khi biển nghiêng**; nghiên cứu cổ điển đặt hiệu chỉnh contour ngang ở tiền xử lý [35], cài đặt hiện đại nắn phối cảnh bốn điểm trước khi tách [64].

Bảng 2.6. Các phương pháp phân biệt biển một dòng và biển hai dòng

Phương án	Cơ chế	Ưu điểm và hạn chế
PA-1. Lấy lớp từ chính detector	YOLO xuất thêm một lớp: 0 = một dòng, 1 = hai dòng [64]	Chính xác nhất, chi phí gần 0 khi tự gán nhãn; phải gán nhãn hai lớp từ đầu
PA-2. Ngưỡng tỷ lệ khung hình	So ngưỡng suy từ quy chuẩn (mục 2.2.6)	Rẻ nhất, không cần huấn luyện; sai khi biển nghiêng nếu đo trên hộp bao thô
PA-3. Chiếu ngang tìm điểm trung	Tổng cường độ pixel theo hàng; biển hai dòng có điểm trung sâu ở giữa [35]	Vị trí cắt thích nghi từng ảnh; điểm trung biến mất khi biển nghiêng
PA-4. Phân cụm hộp bao theo toạ độ dọc	Dùng text detection của engine OCR, gom nhóm theo tâm dọc [65]	Tái dùng kết quả sẵn có; phụ thuộc chất lượng text detection trên crop nhỏ
PA-5. Kiểm tra tính thẳng hàng của ký tự	Nối tâm ký tự trái nhất và phải nhất, đo độ lệch các ký tự còn lại [66]	Trực quan, dễ gỡ lỗi; cần phát hiện từng ký tự, ngưỡng pixel phụ thuộc độ phân giải

e) Lựa chọn của đề án: PA-1 chính, PA-2 dự phòng — thiết kế ở Chương 4, kết quả đo ở Chương 5. **f) Tinh chỉnh là bắt buộc, không phải tùy chọn:** ứng dụng nhận dạng biển số chính thức của PaddleOCR trên CCPD cho thấy tinh chỉnh nâng detection Hmean **76,12% → 99,00%** và recognition **90,97% → 94,54%** [67].

Lưu ý cách đọc số liệu này. Tài liệu gốc ghi recognition pre-trained 0,00%, nhưng con số đó **không** nghĩa là PaddleOCR không đọc được biển: mô hình pre-trained sinh thêm một ký tự đặc biệt khiến chuỗi trượt tiêu chí khớp tuyệt đối; hậu xử lý loại ký tự đó là đạt 90,97% [67]. Luận điểm đúng là **tinh chỉnh nâng 90,97% → 94,54%**; số liệu đo trên **biển Trung Quốc một dòng**, không chứng minh điều gì về biển hai dòng Việt Nam.

g) Vì sao không chọn kiến trúc thuần Transformer. TrOCR resize ảnh thành ô vuông 384×384, chia 576 mảnh, mã hoá BEiT, giải mã RoBERTa [68]; bị loại vì ba lý do: huấn luyện cho văn bản **một dòng** nên với ảnh nhiều dòng **có thể sinh ảo giác** [69]; quá lớn cho CPU — 334 đến 558 triệu tham số [68], nặng hơn recognition PP-OCRv5 mobile (5 triệu [17]) từ 67 đến 112 lần; và ép ảnh về ô vuông bất lợi cho crop biển vốn rất rộng (AR ≈ 4,73) hoặc gần vuông (AR ≈ 1,36).

2.4.4. Chỉ số CER và độ chính xác mức chuỗi

a) CER (Character Error Rate) dựa trên khoảng cách Levenshtein, với S thay thế, D xóa, I chèn, N tổng ký tự chuỗi thực; độ chính xác mức ký tự là $1 - \text{CER}$:

$$\text{CER} = \frac{S + D + I}{N}$$

CER **có thể vượt 1** khi chuỗi dự đoán dài hơn chuỗi thực rất nhiều — đúng tình huống CTC gặp ảnh hai dòng. **b) WER** tương tự nhưng đơn vị là từ; một công trình biển Việt Nam báo cáo WER 0,014 trên bãi đỗ xe trong nhà [70]. **c) Độ chính xác mức chuỗi (plate-level accuracy, exact match)** là chỉ số nghiêm ngặt nhất:

$$\text{Acc}_{\text{plate}} = \frac{\#\{\text{biển số có TOÀN BỘ chuỗi ký tự khớp chính xác}\}}{\#\{\text{tổng số biển số trong tập kiểm thử}\}}$$

Sai một ký tự vẫn tính sai hoàn toàn — phản ánh đúng giá trị sử dụng: chuỗi sai hoặc không khớp bản ghi nào, hoặc khớp nhầm sang phương tiện khác. Quan hệ CER – mức chuỗi **không tuyến tính và bất lợi**: biển 8 ký tự với xác suất đúng mỗi ký tự p , xác suất đúng cả chuỗi là p^8 — $p = 0,99$ chỉ còn $\approx 0,923$, $p = 0,95$ tụt xuống $\approx 0,663$ — lý do engine có CER rất tốt trên văn bản vẫn thất bại trên biển số. **d) End-to-end Recognition Rate** — tỷ lệ biển đọc đúng hoàn toàn trên **toàn bộ pipeline** — là chỉ số duy nhất phản ánh lỗi tích lũy, chỉ tiêu quan trọng nhất của đề án; cuộc thi ICPR 2026 về biển độ phân giải thấp dùng chỉ số này làm chính, đội vô địch đạt 82,13% [71]. Kèm theo là chỉ số vận hành: **độ trễ** p50, p95, p99 (bắt buộc kèm cấu hình phần cứng), **kích thước mô hình, bộ nhớ thường trú, số tham số**; giá trị ở Chương 5.

2.5. Các công trình liên quan

2.5.1. Công trình quốc tế tiêu biểu

Khảo sát lập danh mục **21 công trình quốc tế tiêu biểu** từ 2018 đến 2026, kèm phương pháp, bộ dữ liệu đánh giá và kết quả công bố của từng công trình; **bảng đầy đủ ở Phụ lục J.1**. Sáu mốc kiến trúc còn lại — số 4, 5, 12, 15, 18, 20 — **không kèm số liệu đối chứng công bố được**: Li-Wang-Shen 2019, mạng thống nhất một lần lan truyền xuôi [39]; Zhang và cộng sự 2020, attention 2D, công bố **CLPD** [41]; Nascimento và cộng sự 2024, **LCDNet** với hàm mất mát **LCOFL**, GAN có bộ phân biệt là OCR [78]; Meyer và cộng sự 2025, **SaLT** giảm phụ thuộc cú pháp [19]; Shabaninia và cộng sự 2025, nhận dạng **không phụ thuộc layout** trên IR-LPR, UFPR-ALPR, AOLP [42]; Gong-Liu 2026, **LP-LLM** trên Qwen3-VL với Character Slot Queries và LoRA [44].

Ba lưu ý bắt buộc khi đọc danh mục ở Phụ lục J.1. Thứ nhất, không so sánh trực tiếp giữa các dòng — mỗi công trình đo trên tập và định nghĩa chỉ số khác nhau; nghiêm trọng nhất là dòng 10: **tuyệt đối không rút gọn thành "Tesseract (93%) tốt hơn LPRNet (90%)"** — 93% chỉ trên dữ liệu **tổng hợp** và **sau tiền xử lý**, còn 90% trên biển **thật**. **Thứ hai, mọi con số tốc độ phải kèm phần cứng**: VSNet 149 FPS và YOLOv5-PDLPR 159,8 FPS đều **trên GPU**, 1,3 ms/biển của LPRNet là **trên CPU** — nhanh hơn con số GPU (3 ms) đúng theo bài báo gốc, thường do chi phí khởi tạo, truyền dữ liệu khi lô nhỏ; Batra và cộng sự đo 4,8 ms trên **Nvidia T4** — GPU máy chủ, không phải thiết bị biên. **Thứ ba, VLM đánh đổi tốc độ lấy tổng quát**: VehiclePaliGemma 87,6% nhưng chỉ **7 FPS trên A100-80GB** [43], chậm hơn hai bậc độ lớn so với 149 – 160 FPS của CNN chuyên dụng — lý do đồ án loại hướng này (Phụ lục N.3).

Quan sát tổng hợp: **các con số vượt 99% chủ yếu đạt trên tập dễ, giao thức dễ dãi** — 99,9% trên CCPD-Base nhưng 94,1% trên CCPD-Challenge [73]; giao thức xuyên tập làm trung bình tụt 82,4% → 74,5%, nặng nhất 28,1 điểm [7]; dữ liệu độ phân giải thấp thật: đội vô địch chỉ 82,13% [71]. Bài toán ALPR **chưa được giải quyết xong** như cách nó thường được mô tả.

2.5.2. Công trình về biển số Việt Nam

Nghiên cứu ALPR cho biển Việt Nam chủ yếu công bố tại hội nghị, tạp chí khu vực, **không xuất hiện trên các benchmark quốc tế lớn**, phần lớn đánh giá trên tập tự thu thập không công khai — so sánh công bằng gần như bất khả thi.

Khảo sát lập danh mục **mười công trình về biển số Việt Nam** từ 2012 đến 2024, kèm nơi công bố, phương pháp và kết quả — **bảng đầy đủ ở Phụ lục J.3**. Ba đặc điểm chung nổi lên. **Phần lớn công bố tại hội nghị hoặc tạp chí khu vực và đánh giá trên tập tự thu thập không công khai**, nên so sánh công bằng gần như bất khả thi. **Nhiều công trình không công bố số liệu cụ thể** — có bài chỉ ghi "độ chính xác cao", có bài không nêu độ chính xác cuối, có cuộc thi không công bố kết quả xếp hạng. Và **không công trình nào báo cáo tách riêng độ chính xác biển một dòng với biển hai dòng**, dù đó là phân biệt quan trọng nhất với phân bố phương tiện Việt Nam.

Con số cao nhất cho biển Việt Nam là **99,28% mức chuỗi** [59] nhưng **không dùng làm mốc so sánh được**: đo trên tập riêng không công khai, không tái lập được, độ khó không mô tả định lượng nên không so được với 91,3% trên PTITPlates [79]; chưa tồn tại benchmark công khai chuẩn cho biển Việt Nam kiểu UFPR-ALPR hay RodoSol-ALPR của Brazil. Cũng không dùng trực tiếp được mô hình huấn luyện trên dữ liệu nước ngoài: Việt Nam có **77 triệu xe máy (9/2024), 770 xe trên 1.000 dân**, hàng cao nhất thế giới [1] — **biển hai dòng gần vuông chiếm đa số tuyệt đối** trong khi CCPD, AOLP, SSIG lấy ô tô làm trung tâm; mật độ cao gây che khuất; biển xe máy đặt thấp dễ dính bùn, bị che, biến dạng. CCPD chỉ có **biển một dòng, ký tự Hán tự, 7 ký tự, không có biển hai dòng**. Giao thức *leave-one-dataset-out* làm trung bình tụt 7,9 điểm, nặng nhất 28,1 điểm, nguyên nhân quy cho khác biệt **font chữ trên biển** [7]; với Việt Nam, dịch chuyển miền còn lớn hơn.

Kết luận kiến trúc. Huấn luyện trước trên dữ liệu quốc tế chỉ nên áp dụng cho khối **detection**. Khối **recognition bắt buộc huấn luyện hoặc tinh chỉnh trên dữ liệu Việt Nam**, hậu xử lý viết riêng theo quy chuẩn ở mục 2.2.

Khoảng tám kho mã nguồn mở về biển Việt Nam đang hoạt động, phần lớn **không công bố số liệu độ chính xác**, nhiều kho không ghi giấy phép; phía thương mại, các con số 98 – 99,9% do nhà cung cấp tự công bố trên định nghĩa "ảnh chuẩn" không thống nhất — **không dùng làm mốc so sánh học thuật**.

2.5.3. Các bộ dữ liệu chuẩn trong lĩnh vực

Khảo sát đối chiếu **chín bộ dữ liệu chuẩn** của lĩnh vực theo quy mô, đặc điểm và **giấy phép sử dụng** — cột giấy phép quyết định bộ nào dùng được cho đề án này; **bảng đầy đủ ở Phụ lục J.2**.

Ba nhận xét. Thứ nhất, bộ lớn nhất không phải bộ sạch nhất: Laroca và cộng sự **loại trừ tường minh CCPD** khỏi thí nghiệm tổng quát hoá vì ảnh nén quá mạnh, sai số gán nhãn đỉnh lớn [7] — dùng được CCPD cho huấn luyện trước detection nhưng **không nên tin toạ độ bốn đỉnh cho nắn chỉnh**. **Thứ hai, báo cáo chỉ trên tập con để là không đủ:** khoảng cách 5,8 điểm giữa CCPD-Base (99,9%) và CCPD-Challenge (94,1%) [73] cho thấy con số trung bình che giấu điểm gãy — cơ sở cho quyết định **báo cáo tách bạch theo nhóm điều kiện**, đặc biệt tách một dòng và hai dòng. **Thứ ba, dữ liệu biển Việt Nam là điểm nghẽn thực sự: không tồn tại bộ dữ liệu biển Việt Nam công khai nào được bình duyệt học thuật;** nguồn hiện có là GitHub cá nhân, Roboflow Universe, Kaggle. Bộ lớn nhất, đủ nhãn nhất là VNLP: **37.300 ảnh** (19.086 một dòng, 18.211 hai dòng), annotation mức ký tự, tách rõ hai loại gần 50/50 — nhưng **không ghi giấy phép**, cần xin xác nhận tác giả trước khi dùng trong công bố [91].

Ba đặc điểm chung của dữ liệu Việt Nam: phần lớn chỉ có hộp bao một lớp — chỉ dùng cho detection; rất ít bộ phân biệt tường minh một dòng và hai dòng; **không bộ nào gán nhãn chuỗi biển số đầy đủ** — khoảng trống lớn nhất. Tin tốt: hiệu năng bão hoà quanh **4.750 ảnh thật (99,0% độ chính xác)**, và chỉ cần **300 ảnh thật** kết hợp sinh dữ liệu cùng tăng cường là tương đương 200.000 ảnh thật [92]; kho dữ liệu Việt Nam công khai vượt xa ngưỡng cho

detection, nút thắt là **nhãn mức ký tự và nhãn chuỗi**. Có công cụ sinh ảnh biển Việt Nam tổng hợp **cả một dòng lẫn hai dòng** [93].

2.5.4. Khoảng trống nghiên cứu và định vị đề tài

Bảng 2.7. Sáu khoảng trống nghiên cứu và cách đề án lấp

#	Khoảng trống được xác định từ khảo sát	Cách đề án lấp
1	Chưa có nghiên cứu Việt Nam nào công bố bảng so sánh tách riêng độ chính xác biển một dòng và biển hai dòng trên cùng một hệ thống (mục 2.5.2)	Đề án báo cáo tách bạch hai con số này
2	Chưa có nghiên cứu Việt Nam nào mô tả có hệ thống bộ luật hậu xử lý ràng buộc theo VỊ TRÍ trong chuỗi — các mô tả hiện có dừng ở danh sách phẳng, phần lớn dùng nhầm con số 20 chữ cái cho toàn chuỗi (mục 2.2.4)	Thiết kế hậu xử lý theo từng vị trí, đo tách bạch trước và sau hậu xử lý ; hiệu số là đóng góp định lượng
3	Hầu hết công trình trong nước chỉ báo cáo mAP của detection , không báo cáo end-to-end mức chuỗi (mục 2.5.2)	Báo cáo cả hai, end-to-end là chỉ tiêu quan trọng nhất
4	Không tồn tại benchmark công khai nào so sánh các engine OCR trên riêng ảnh biển số xe máy Việt Nam hai dòng (mục 3.3)	✅ Đã lấp 03/08/2026 — đo ba engine trên 2.801 biển, cùng tăng bao quanh: PaddleOCR 68,87% · EasyOCR 14,28% · Tesseract 10,28% (mục 3.3.3)
5	Số liệu hiệu năng thường công bố không kèm phần cứng (mục 2.5.1)	Mọi số liệu hiệu năng kèm: model CPU, số luồng, kích thước ảnh vào, backend suy luận, cỡ mẫu đo
6	Hầu hết kho mã nguồn mở Việt Nam không công bố số liệu và không có kiến trúc phần mềm (mục 2.5.2)	Công bố đầy đủ giao thức đo, tập kiểm thử, toàn bộ chỉ số; bàn giao hệ thống có API, giao diện, cơ sở dữ liệu, kiểm thử, đóng gói

Sáu khoảng trống đều thuộc loại **kỹ nghệ và báo cáo**, không phải thuật toán: đề án không đặt mục tiêu vượt các con số trên 99% ở Phụ lục J.1 — trong đó 99,28% của nhóm Học viện Kỹ thuật Quân sự đo trên tập riêng không công khai — và mọi số liệu hiệu năng của đề án là **số liệu CPU**, không so trực tiếp với FPS đo trên GPU.

Tuyên bố trung thực đầy đủ về mức đóng góp, cùng bốn điều đề án **không** tuyên bố, đặt ở **mục 1.6.1 và 1.6.8** (Chương 1) — nơi chính danh để tuyên bố đóng góp.

2.6. Kết luận chương

Thứ nhất, bài toán ALPR chưa được giải quyết xong: giao thức nghiêm ngặt hơn làm độ chính xác trung bình sụt gần 8 điểm, nặng nhất 28,1 điểm [7]; trên dữ liệu độ phân giải thấp

thật, đội vô địch chỉ đạt 82,13% [71]. **Thứ hai, đề án theo two-stage kết hợp bộ nhận dạng segmentation-free** — hệ quả của ràng buộc thay được bộ OCR mà không huấn luyện lại toàn hệ thống. **Thứ ba, khối phát hiện dùng YOLO11n**: phiên bản duy nhất vừa có số liệu tốc độ CPU chính thức, vừa có cơ chế kiến trúc (C2PSA, đầu anchor-free) phù hợp đối tượng nhỏ và tỷ lệ khung hình dẹt, vừa có bằng chứng thực nghiệm dày trên ALPR; **mAP@0.5 và mAP@0.5:0.95 là hai chỉ số khác nhau, chênh lệch giữa chúng không mang thông tin về độ khó** — đề án dùng mAP@0.5 làm chỉ tiêu chính, báo cáo mAP@0.5:0.95 kèm theo, không đặt ngưỡng chấp nhận.

Thứ tư, bài toán biến hai dòng có nền tảng lý thuyết rõ ràng và không thể giải bằng cách đổi engine. CTC giả định alignment đơn điệu trên **một dòng duy nhất**; module recognition resize về chiều cao 48 pixel nên crop biến xe máy tỷ lệ 1,357 bị nén còn khoảng 65 pixel chiều rộng, mỗi dòng khoảng 24 pixel — không đủ để đọc. Bằng chứng độc lập: OpenALPR đạt 94,3% trên biến một dòng nhưng chỉ 45,7% trên biến hai dòng, chênh 48,6 điểm, đo trên bộ RodoSol-ALPR của Brazil [7]. **Vấn đề phải giải ở tầng trên bằng module tách dòng, không phải bằng cách đổi engine OCR.**

Thứ năm, quy chuẩn biển số Việt Nam đã đặc tả đủ để cài đặt, với ba điểm chính. Căn cứ hiện hành: TT 79/2024/TT-BCA sửa đổi bởi TT 13/2025 và TT 51/2025, cùng QCVN 08:2024/BCA — TT 24/2023 đã hết hiệu lực từ 01/01/2025. 81 mã tỉnh đang dùng, 8 mã không dùng. Quan trọng nhất: **tập chữ cái bị loại trừ chỉ gồm 5 chữ I, J, O, Q, W chứ không phải 6; chữ R hợp lệ ở vị trí thứ hai của seri xe máy** — hậu xử lý phải ràng buộc theo từng vị trí trong chuỗi, tập ký tự huấn luyện OCR đủ 36 ký tự. Ba tỷ lệ khung hình (1,357 / 2,000 / 4,727) tạo khoảng trống 2,727 đơn vị — cơ sở ngưỡng phân loại bố cục đề án đề xuất.

Thứ sáu, sáu khoảng trống nghiên cứu đã được xác định (Bảng 2.7), cả sáu có cách lắp cụ thể. Khoảng trống số 4 — *không tồn tại benchmark công khai nào so sánh các engine OCR trên riêng ảnh biển số Việt Nam* — **đã được lấp bằng phép đo của chính đề án**: ba engine chạy trên 2.801 biển với **cùng một tầng bao quanh**, chỉ khác engine, cho PaddleOCR **68,87%**, EasyOCR 14,28%, Tesseract 10,28% (mục 3.3.3). Nhờ đó, PaddleOCR được giữ **vì có bằng chứng đo được trên đúng miền dữ liệu**, không còn là một baseline để ngó như bản khảo sát ban đầu ghi nhận. Kèm theo là tuyên bố trung thực về giới hạn: đề án không đặt mục tiêu kết quả tốt nhất lĩnh vực, không đề xuất kiến trúc mạng mới, không giải các thách thức mở như biến độ phân giải rất thấp hay tổng quát hoá xuyên tập dữ liệu.

Ba nguyên tắc phương pháp áp dụng nguyên vẹn cho phần thực nghiệm: **mọi số liệu hiệu năng kèm cấu hình phần cứng và cỡ mẫu đo; báo cáo tách bạch theo bố cục biển và điều kiện ảnh; không so sánh chéo giữa các chỉ số khác định nghĩa hoặc khác tập dữ liệu.** Chương tiếp theo chuyển sang lựa chọn công nghệ, rồi tới thiết kế hệ thống.

CHƯƠNG 3. KHẢO SÁT CÔNG NGHỆ VÀ LỰA CHỌN MÔ HÌNH

3.1. Phương pháp khảo sát và tiêu chí lựa chọn

Mỗi lựa chọn trình bày theo cùng một khuôn: phương án đã xét, tiêu chí, kết luận, **đánh đổi phải chấp nhận**. Khi bằng chứng không đủ phân định, mục này nói rõ là không đủ.

3.1.1. Bốn ràng buộc chi phối mọi lựa chọn

Bốn ràng buộc sau thu hẹp không gian phương án **trước khi** so sánh — vì sao một số ứng viên mạnh bị loại sớm.

#	Ràng buộc	Hệ quả trực tiếp lên việc chọn
1	Suy luận trên CPU, không có GPU CUDA (CON-02, mục 4.3.1)	Phương án không công bố tốc độ CPU đều không có căn cứ để đánh giá ; mô hình hàng trăm triệu tham số loại từ đầu
2	Biển số Việt Nam có biển hai dòng	Engine giả định vẫn bản một dòng gây ở đây; tiêu chí phân loại, không phải điểm cộng
3	Phải đóng gói và bàn giao được	Giấy phép, dung lượng mô hình, số phụ thuộc là tiêu chí thật
4	Ngân sách thời gian CPU hữu hạn	Một số phép so sánh đã thiết kế nhưng không chạy được ; mục 3.1.2 nói rõ là những phép nào

3.1.2. Ranh giới giữa cái đã đo và cái mới chỉ khảo sát tài liệu

Không phải mọi lựa chọn đều qua thực nghiệm: có phép đồ án tự chạy trên chính máy và dữ liệu của mình, có phép chỉ dựa số liệu nhà phát hành, và có phép **chưa bao giờ chạy được**.

Bảng 3.1. Mức bằng chứng của từng phép so sánh trong chương

Phép so sánh	Mức bằng chứng	Trình bày ở
PP-OCRv5_mobile ↔ PP-OCRv6_medium	✅ Tự đo — 200 vùng cắt biển số, cùng máy, cùng thứ tự ảnh	3.3.2
Bộ nhận dạng gốc ↔ bản tinh chỉnh	✅ Tự đo — 2.801 biển có nhãn chuỗi, bốn cấu hình	5.4
YOLO11n ↔ YOLOv8n và năm thế hệ YOLO khác	📄 Khảo sát tài liệu — theo benchmark chính thức của nhà phát hành, đồ án không tự chạy lại	3.2
PaddleOCR ↔ EasyOCR ↔ Tesseract	✅ Tự đo 03/08/2026 — 2.801 biển có nhãn	3.3.3

Phép so sánh	Mức bằng chứng	Trình bày ở
	chuỗi, ba nhánh, cùng tầng bao quanh	
PyTorch ↔ ONNX Runtime ↔ OpenVINO	✗ Chưa đo — chọn theo benchmark bên thứ ba, chưa có số tự đo	3.4 · 5.6.3
Độ phân giải 416 ↔ 640	⚠ Có số đo nhưng không quy kết được — ba biến đổi đồng thời và ngược chiều nhau	3.6

Dòng **✗** còn lại là khoản nợ thực sự, ghi nhận nhất quán ở mục 5.9.2 và Chương 6:

- **So sánh runtime chưa chạy** ⇒ chọn ONNX Runtime đứng vững nhờ **lý do vận hành** (một runtime duy nhất cho cả hai mô hình, tránh xung đột hai framework học sâu), không nhờ số liệu tốc độ tự đo.

3.2. Mô hình phát hiện: YOLO11

Các phương án đã xét. Bỏ thể hệ YOLO từ YOLOv8 trở về sau — mốc chuyển sang anchor-free, có ý nghĩa trực tiếp với bài toán biến số (mục 2.3.1): YOLOv8 [49], YOLOv9 [53], YOLOv10 [46], YOLO11 [16], YOLOv12 [54], YOLOv13 [55], YOLO26 [47]. Họ two-stage (Faster R-CNN, Mask R-CNN) loại từ đầu vì chi phí tính toán không hợp ràng buộc CPU.

Tiêu chí, theo mức chi phối: (1) có số liệu tốc độ CPU chính thức; (2) kiến trúc hợp đối tượng nhỏ và tỷ lệ khung hình dẹt; (3) mật độ bằng chứng thực nghiệm trên bài toán biến số; (4) hệ sinh thái và giấy phép.

Quá trình loại trừ. Chỉ các bản phát hành từ Ultralytics công bố tốc độ CPU: YOLOv9 không công bố cột tốc độ nào, YOLOv10/YOLOv12/YOLOv13 chỉ công bố tốc độ GPU — bốn phiên bản bị loại vì **không có căn cứ để đánh giá** trên đúng chiều ràng buộc; YOLOv13 còn rủi ro kho mã không được tích hợp chính thức vào Ultralytics [55]. YOLOv8 bị loại vì YOLO11n **vượt trội cả hai chiều**: **mAP@0.5:0.95** đạt 39,5 so với 37,3 và tốc độ CPU định dạng ONNX **56,1 ± 0,8 ms so với 80,4 ms**, nhanh hơn khoảng 30% [16], [52] — hai số đo cùng quy trình xuất mô hình nên so sánh được [94].

Kết luận: chọn YOLO11 biến thể n (nano), với năm lý do: phiên bản gần đây **duy nhất có số liệu tốc độ CPU chính thức**; khối C2PSA được khẳng định cải thiện phát hiện đối tượng nhỏ và xử lý che khuất [52], đầu anchor-free giải quyết tỷ lệ khung hình ngoài phân bố COCO (mục 2.3.1); **bằng chứng thực nghiệm dày nhất trên bài toán biến số** — ít nhất ba nghiên cứu độc lập dùng YOLO11 cho ALPR đạt **mAP@0.5** từ 0,906 đến 0,995 [57], [58], [95]; hệ sinh thái trưởng thành — tích hợp chính thức trong ultralytics, hơn 20 định dạng xuất [96], benchmark tự động trên CPU [97]; giấy phép AGPL-3.0 miễn phí cho nghiên cứu học thuật [98]. Một nghiên cứu so sánh trực tiếp bốn bản nano trên cùng tập biến số cũng kết luận YOLO11n tối ưu [99], nhưng **chưa kiểm chứng được toàn văn** nên chỉ là trích dẫn phụ.

Chọn nano vì bài toán chỉ có một lớp: số kênh đầu ra nhánh phân loại giảm từ 80 xuống 1, nhẹ đầu dự đoán và giảm chi phí NMS; kết quả mAP 99,3% của YOLOv8-s trên ba benchmark quốc tế ở trên 30 FPS củng cố hướng này [100]. Biến thể s giữ làm phương án leo thang.

Đánh đổi phải chấp nhận.

- **AGPL-3.0 kéo theo nghĩa vụ copyleft:** công bố mã nguồn tương ứng, tệp cấu hình và cả **trọng số mô hình**; điều khoản mạng khiến không né được qua API [98]. Đồ án công bố mã công khai nên chấp nhận được — thương mại hoá phải mua giấy phép; không ứng viên nào tránh được copyleft.
- **Luận cứ cải thiện đối tượng nhỏ chỉ ở mức định tính,** vì nhà phát hành không công bố AP_small tách riêng (mục 2.3.2); đồ án phải tự đo.
- **Bỏ qua YOLO26 dù trội trên giấy tờ:** mAP@0.5:0.95 đạt 40,9 (hơn 1,4 điểm), tốc độ CPU $38,9 \pm 0,7$ ms (nhanh hơn khoảng 30%), bỏ DFL để xuất và lượng tử hoá [47] — nhưng phát hành 09/2025, **chưa có tiền lệ trên bài toán biến số**, nên chọn làm phương án duy nhất là rủi ro không cần thiết. Dự kiến huấn luyện YOLO26n **song song làm đối chứng; lượt đối chứng này cuối cùng đã không chạy được** vì toàn bộ ngân sách CPU dồn cho lượt huấn luyện best.pt — ghi nhận là chưa đo ở mục 5.9.2 và chuyển thành hướng phát triển.

3.3. Engine nhận dạng ký tự

Đây là lựa chọn trình bày **trung thực nhất về mức độ chắc chắn**: chọn *họ engine* theo khảo sát tài liệu (3.3.1), rồi chọn *bậc mô hình* theo phép đo tự chạy (3.3.2).

3.3.1. PaddleOCR, EasyOCR, Tesseract — khảo sát tài liệu

Các phương án đã xét: tám engine — PaddleOCR, EasyOCR, Tesseract, TrOCR, docTR, MMOCR, fast-plate-ocr và RapidOCR/OnnxTR.

Bảng so sánh tám tiêu chí giữa ba ứng viên hàng đầu — kiến trúc, kích thước mô hình, tốc độ CPU, giấy phép, khả năng xử lý nhiều dòng, khả năng giới hạn tập ký tự, độ khó triển khai — ở **Phụ lục M.2**.

Bốn engine loại sớm nên không vào bảng: **TrOCR** — **ảo giác trên văn bản đa dòng** [69], quá nặng cho CPU (334 – 558 triệu tham số [68]), biến dạng tỷ lệ khung hình (mục 2.4.3g); **MMOCR** — chuỗi phụ thuộc bốn tầng, rủi ro cài đặt cao nhất trên Windows không GPU [108]; **fast-plate-ocr** — không có mô hình cho biến Việt Nam, kiến trúc khe cố định không xử lý được biến hai dòng nếu chưa huấn luyện lại [109]; **docTR** — tối ưu cho trang tài liệu, không cho ảnh crop nhỏ [110].

Bảng chứng thực sự đứng vững cho PaddleOCR. Hai số liệu thường được viện dẫn để chứng minh "PaddleOCR tốt cho biến số" đã **bị bác bỏ khi truy ngược nguồn gốc**: cả hai đến từ một bài báo dùng **EasyOCR**, không phải PaddleOCR [111]. Còn đứng vững: **nhẹ nhất nhóm khả dụng** — khoảng 21 MB so với khoảng 200 MB của EasyOCR; **thời gian CPU**

khả thi, trên giấy có lộ trình nâng cấp — PP-OCRv6 Tiny nhanh hơn v5 mobile khoảng 3,9 lần trên CPU [112], nhưng **lộ trình này về sau không lấy được** (mục 3.3.2); ràng buộc siêu nhẹ là chủ đích thiết kế xuyên suốt dòng PP-OCR [113], [114]; có bằng chứng tinh chỉnh trên biển số cho kết quả tốt [67] — tuy nhiên là **biển số Trung Quốc một dòng**; kiến trúc hai giai đoạn trả mỗi dòng một hộp; giấy phép Apache 2.0, không copyleft.

Những gì PaddleOCR thua. Không giới hạn được tập ký tự khi suy luận, phải tinh chỉnh mới có [105]; khó cài hơn EasyOCR vì kéo theo framework học sâu thứ hai; kém xa mô hình chuyên biệt LPTR-AFLNet — 99,37% riêng trên biển hai dòng với 2,7 triệu tham số [75] — nhưng mô hình đó không có gói cài sẵn, không có bản cho biển Việt Nam, không công bố số liệu CPU.

🌀 Kết luận trung thực — điểm quan trọng nhất của Chương 3

PaddleOCR là lựa chọn hợp lý, nhưng KHÔNG phải lựa chọn đã được chứng minh là tốt nhất cho bài toán này. Ba điểm phải nói rõ:

1. **Tài liệu công khai không ủng hộ PaddleOCR:** hai số liệu mạnh nhất từng được viện dẫn đã bị bác bỏ, và các so sánh engine-với-engine trên ảnh biển số kiểm chứng được lại **ngiên về EasyOCR** [115]. Đây là tình trạng **tại thời điểm chọn công nghệ**; phép đo tự chạy về sau (mục 3.3.3) nói ngược lại.
2. **Lý do giữ PaddleOCR là lý do kỹ thuật và vận hành, không phải độ chính xác:** nhẹ hơn EasyOCR gần 10 lần, có lộ trình tăng tốc, có bằng chứng tinh chỉnh, mạnh trên ảnh xoay.
3. **Không engine nào giải sẵn bài toán hai dòng.** Như đã chứng minh ở mục 2.4.3, chọn engine **không quyết định** thành bại của rủi ro R-04 — module tách và ghép dòng mới quyết định.

Cách xử lý đúng ở thời điểm đó: giữ PaddleOCR làm baseline vì điểm 2, coi **EasyOCR là ứng viên ngang hàng, không phải phương án dự phòng hình thức** (Tesseract làm mốc dưới), và **để một benchmark tự chạy trên chính tập biển số Việt Nam quyết định** — benchmark đó **đã chạy ngày 03/08/2026**, kết quả ở mục 3.3.3.

Ma trận thí nghiệm gồm bốn trục: engine, cách xử lý biển hai dòng, có hoặc không nắn chỉnh phối cảnh, và runtime. Ba trục đầu đã chạy (mục 3.3.3); trục runtime chưa (mục 5.6.3).

3.3.2. PP-OCRv5 mobile so với PP-OCRv6 — đo trên máy đồ án

Mục này chọn **bậc mô hình** bên trong họ đã chọn, dựa trên số liệu **tự đo**. Quy trình đầy đủ ở docs/reports/35-ppocrv6-evaluation.md.

a) Chỉ một bậc của v6 là lấy được. Quét gói paddleocr 3.7.0 chỉ thấy PP-OCRv6 và cặp PP-OCRv6_medium_det/rec — **không có bản Tiny, không có bản Small**. Bậc hấp dẫn với hệ thống CPU là **Tiny** (0,20 giây mỗi ảnh, chính là "lộ trình trên giấy" ở mục 3.3.1); bậc duy nhất

tải được là **Medium**, bậc mà chính bài báo ghi 1,40 giây mỗi ảnh — **chậm hơn** v5 mobile (0,78 giây) khoảng 1,8 lần. Lộ trình nâng cấp vì vậy **không lấy được**.

b) Phải tự đo: bài báo đo trên Xeon 8350C có **OpenVINO**, trên **văn bản tài liệu** — lệch cả phần cứng lẫn miền dữ liệu. Cách đo: 200 vùng cắt biến số từ tập kiểm định (val.txt, không tăng cường, không mảnh vụn), chạy **chỉ nhánh nhận dạng** để cô lập biến so sánh; cùng máy, cùng ảnh, cùng thứ tự.

Bảng 3.2. PP-OCRv6_medium_rec so với PP-OCRv5_mobile_rec, đo trên 200 vùng cắt biến số của đồ án

Mô hình	Đúng chuỗi	Trung vị	p95
PP-OCRv5_mobile_rec — đang dùng	134/200 = 67,0%	23,0 ms	31,9 ms
PP-OCRv6_medium_rec	145/200 = 72,5%	386,9 ms	429,0 ms

v6 Medium chính xác hơn 5,5 điểm và chậm hơn 16,8 lần. Chiều kết quả khớp bài báo; biên độ chi phí lớn hơn nhiều tỷ lệ 1,8 lần bài báo ghi, vì bài đo có OpenVINO còn đồ án chạy PaddlePaddle thuần.

c) Vì sao 5,5 điểm ấy vẫn không đủ. Hệ thống đã căng độ trễ ở **cả hai đầu**: NFR-P1 đạt sản sát nút (p95 1.143 ms, sản 1.500 ms), NFR-P2 **đã trượt** (2,379 FPS, sản 3). Bước OCR chiếm 108,28 ms mỗi biến (Bảng 6.19), nhánh nhận dạng chỉ khoảng 23 ms. Thay v5 bằng v6 Medium cộng thêm khoảng **364 ms mỗi biến**.

⚠ Đây là phép chiếu, không phải phép đo. Cộng 364 ms vào p95 hiện hành cho khoảng **1.507 ms**, tức **vượt sản 1.500 ms** và đẩy NFR-P1 từ **🟡** xuống **🔴**. Con số này suy ra từ độ trễ nhánh nhận dạng đo cô lập, **chưa chạy lại toàn đường ống** — muốn công bố phải đo thật. Nhưng ngay cả với sai số rộng, hướng của kết luận không đổi: NFR-P2 vốn đã trượt sản thì chắc chắn trượt sâu hơn.

d) Kết luận: giữ PP-OCRv5_mobile_rec. Đây **không phải** kết luận "v6 kém hơn" — 5,5 điểm ấy là **dư địa đã định lượng** — mà là kết luận về **ràng buộc phần cứng**: bậc v6 hợp ràng buộc CPU thuần (ràng buộc số 1), Tiny, **không có trong gói**. Hai điều kiện đảo được quyết định, đều **đo được**: Tiny được phát hành vào gói pip (cải thiện cả độ chính xác lẫn độ trễ); hoặc xuất v6 Medium sang ONNX/OpenVINO với mức tăng tốc trên 8 lần (386 ms về khoảng 45 ms). Cả hai nằm trong hướng phát triển ở Chương 6.

Lưu ý bắt buộc khi trích bài PP-OCRv6. Cặp "+5,1 / +4,6 điểm" mà bài v6 công bố được tính trên **baseline của chính nó** (v5_server 78,1% / 81,6%), không phải trên baseline trong tài liệu PaddleX (86,38% / 83,8%). Ghép hai nguồn sẽ **đảo chiều kết luận**. Trích thì phải trích kèm baseline gốc.

3.3.3. Benchmark ba engine trên 2.801 biến số Việt Nam — đo 03/08/2026

Mục 3.3.1 kết thúc bằng một khoản nợ: giữ PaddleOCR dựa trên lý do kỹ thuật, **không** dựa trên bằng chứng độ chính xác, trong khi tài liệu công khai nghiêng về EasyOCR. Mục này trả nợ đó. Số liệu đầy đủ ở docs/reports/36-engine-benchmark.md.

a) Vì sao phép đo này khó làm đúng. Bốn lượt chạy đầu đều cho số vô nghĩa; mỗi lượt hỏng lộ ra một điều kiện bắt buộc — truyền tên model tường minh, tắt backend oneDNN (mục 4.6.3), khôi phục tỷ lệ khung hình, lọc mảnh vụn ở mép dải ghép. Bài học chung: **phần lớn năng lực đọc biến số không nằm trong engine** mà ở tầng bao quanh nó. So sánh ba engine với ba tầng bao quanh khác nhau là đo tầng bao quanh chứ không đo engine.

b) Thiết kế. Cả ba engine chạy trong **đúng một tầng bao quanh** — sao đúng chuỗi bước của bộ nhận dạng bản giao hàng — trên **cùng một mảng NumPy đã chuẩn bị xong**; khác biệt duy nhất còn lại là engine. Đây cũng là bằng chứng thực nghiệm cho NFR-M5. Một chỗ cố ý không cào bằng: Tesseract chạy kèm whitelist A-Z0-9, vì giới hạn tập ký tự là **năng lực gốc** của nó; cắt bỏ "cho công bằng" mới là làm sai.

Kiểm chứng harness: nhánh có-split của PaddleOCR đo được **63,73%**, khớp **chính xác** NFR-A5 = 0,6373 công bố từ trước bằng một đường đo hoàn toàn khác — cùng một số tới bốn chữ số.

Bảng 3.3. So sánh ba engine OCR trên 2.801 biến số Việt Nam (567 một dòng, 2.234 hai dòng)

Engine	Nhánh	Toàn bộ	1 dòng	2 dòng	CER	Rỗng	p50
PaddleOCR	tắt split	28,81%	94,2%	12,2%	0,588	6	295 ms
PaddleOCR	có split	63,73%	94,2%	56,0%	0,094	11	405 ms
PaddleOCR	+ hậu xử lý	68,87%	94,9%	62,3%	0,089	11	402 ms
EasyOCR	tắt split	6,53%	15,3%	4,3%	0,647	4	84 ms
EasyOCR	có split	10,35%	15,3%	9,1%	0,282	1	248 ms
EasyOCR	+ hậu xử lý	14,28%	28,6%	10,7%	0,269	1	249 ms
Tesseract	tắt split	9,57%	47,3%	0,0%	0,777	960	101 ms
Tesseract	có split	9,60%	47,3%	0,0%	0,565	700	108 ms
Tesseract	+ hậu xử lý	10,28%	50,4%	0,1%	0,567	700	106 ms

c) PaddleOCR thắng dứt khoát — và điều này bác bỏ tài liệu công khai. Ở cấu hình bản giao hàng, PaddleOCR đạt **68,87%**, hơn EasyOCR **54,59 điểm** và hơn Tesseract **58,59 điểm** — khoảng cách quá lớn để quy cho nhiễu. Kết luận "*tài liệu công khai không cho thấy PaddleOCR vượt EasyOCR trên ảnh biến số*" ở mục 3.3.1 **vẫn đúng về tài liệu công khai**, nhưng nay đồ án có số liệu của chính mình trên biến số Việt Nam và nó nói ngược lại. Quyết định giữ PaddleOCR nay **có thêm căn cứ độ chính xác**.

d) Tách-rời-ghép-ngang KHÔNG độc lập engine — kết quả bất ngờ nhất.

Engine	tắt split → có split	Mức tăng
PaddleOCR	28,81% → 63,73%	+34,92 điểm
EasyOCR	6,53% → 10,35%	+3,82 điểm
Tesseract	9,57% → 9,60%	+0,03 điểm

Nếu cả ba cùng tăng mạnh, đóng góp kỹ thuật của đồ án sẽ độc lập với engine — một khẳng định mạnh. **Dữ liệu không cho phép nói thế.** Phát biểu đúng: tách-rời-ghép-ngang là điều kiện **cần** để đọc biến hai dòng — nó biến bài toán đa dòng thành bài toán một dòng — nhưng **không đủ**; engine phải đủ mạnh để tận dụng dải ảnh đã ghép.

Ngược lại, **bộ luật hậu xử lý giúp cả ba** (+5,14 · +3,93 · +0,68 điểm). Đóng góp (b) của đồ án vì vậy **là** đóng góp độc lập engine, khác với tách-rời-ghép-ngang.

e) Tesseract không đọc được biến hai dòng. 0,0% trên 2.234 biến hai dòng, kể cả sau khi đã ghép thành một dòng, trong khi đọc được 47,3% biến một dòng. Đã kiểm bằng mắt để loại khả năng lỗi công cụ: nó **có** đọc ra chữ nhưng luôn kèm ký tự rác, và 700/2.801 lần trả chuỗi rỗng. Dự đoán "*Tesseract vỡ khi crop nhiều dòng*" ở mục 3.3.1 được xác nhận, ở mức nghiêm trọng hơn.

⚠ **Hai điều phép đo này không trả lời.** Thứ nhất, nó đo trên **vùng biến đã cắt sẵn**; báo cáo 31 cho thấy thứ tự xếp hạng có thể **đảo ngược** trên ảnh toàn cảnh qua bộ phát hiện thật, nên kết luận chỉ áp cho tầng nhận dạng. Thứ hai, nó **không** kết luận engine nào tốt hơn nói chung — chỉ kết luận engine nào đọc biến số Việt Nam tốt hơn *bên trong tầng bao quanh của đồ án*; một hệ thống thiết kế quanh EasyOCR, với tiền xử lý riêng của nó, có thể cho số khác.

3.4. Runtime suy luận trên CPU: ONNX Runtime

Các phương án đã xét: chạy trực tiếp tệp trọng số PyTorch, ONNX Runtime, OpenVINO.

Tiêu chí: tốc độ CPU, mức đa nền tảng, độ nặng phụ thuộc khi đóng gói, khả năng cùng tồn tại với framework khác.

Bảng chứng định lượng. Benchmark chính thức trên CPU laptop Intel Core i7-13700H, FP32, ảnh 640: **ONNX Runtime nhanh gấp khoảng 3,73 lần chạy trực tiếp PyTorch ở phân khúc nano — 104,61 ms giảm còn 28,02 ms** [18]. Lợi ích lớn nhất đúng ở phân khúc nano và thu hẹp khi mô hình lớn lên.

Cảnh báo trích dẫn bắt buộc. Chỉ được dùng **phần số liệu tốc độ** của bảng benchmark này. Các con số mAP đi kèm được đo trên tập coco8 chỉ gồm **8 ảnh**, nên **vô nghĩa về mặt thống kê** và không được trích dẫn dưới bất kỳ hình thức nào.

Ba kết luận bổ sung từ cùng nguồn: **OpenVINO không phải luôn nhanh hơn** — trên CPU Intel thế hệ mới, FP32 **chậm hơn PyTorch** ở các biến thể lớn [18], hệ quả là phải **tự benchmark trên đúng máy chạy**; **FP16 vô ích trên CPU** — OpenVINO chuyển nội bộ FP16 sang FP32 [116], benchmark xác nhận chênh dưới 1% [18]; **INT8 mới là lợi thế thực của OpenVINO** — tăng tốc 2,3 đến 3,6 lần, mất mAP tương đối 1,73 đến 2,47% [18], nhưng với CNN phải lượng tử hoá **tinh** kèm tập hiệu chuẩn.

Kết luận: mặc định dùng ONNX Runtime, coi OpenVINO là phương án tối ưu bổ sung. Ba lý do: nhanh hơn đáng kể ở đúng phân khúc; chuẩn mở, cùng một tệp mô hình phục vụ cả detection lẫn OCR; và — quan trọng nhất — **loại bỏ rủi ro xung đột giữa hai framework**

học sâu (PyTorch cho YOLO, PaddlePaddle cho PaddleOCR): khi **cả hai** mô hình đều chạy bằng ONNX Runtime lúc vận hành, rủi ro này biến mất.

Đánh đổi phải chấp nhận: thêm một bước xuất mô hình; phải kiểm chứng tương thích gói với Python 3.13 trên Windows; và phải **đặt tường minh số luồng nội bộ** cùng **giới hạn số yêu cầu suy luận đồng thời**, vì cả ONNX Runtime lẫn PaddleOCR đều mặc định sinh số luồng bằng hoặc lớn hơn số lõi vật lý, dễ tranh chấp tài nguyên [117].

3.5. Các lựa chọn công nghệ nền tảng khác

Phần lớn quyết định còn lại là **ràng buộc của đề bài**; ghi lại kèm lý do và đánh đổi để Chương 4 tham chiếu.

Bảng 3.4. Tổng hợp quyết định công nghệ nền tảng

#	Hạng mục	Lựa chọn (phương án thay thế)	Lý do chính	Đánh đổi phải chấp nhận
1	Web framework backend	FastAPI (Django, Flask)	Tự sinh đặc tả OpenAPI — tạo sẵn một sản phẩm bàn giao; có sẵn WebSocket và tác vụ nền	Phải biết khi nào không dùng hàm bất đồng bộ; nguy cơ tranh chấp với luồng suy luận
2	ORM và migration	SQLAlchemy 2.0 + Alembic (Tortoise ORM, Peewee)	Tích hợp sâu kiểu tĩnh; lược đồ đã thay đổi nên nhu cầu migration là có thật	Đường cong học dốc nhất trong nhóm
3	Cơ sở dữ liệu	SQLite (PostgreSQL, MySQL)	Ghi có thể xếp hàng vì suy luận CPU mới là nút cổ chai; không thêm dịch vụ khi đóng gói	Chỉ một tiến trình ghi tại một thời điểm ; vượt ngưỡng tải phải chuyển PostgreSQL
4	Frontend	React + TypeScript + Vite + TailwindCSS (Vue, Angular, Svelte)	Hệ sinh thái lớn nhất; công cụ build tiền nhiệm ngừng bảo trì; kiểu tĩnh nối tiếp từ backend	Tự lắp ghép routing, quản lý trạng thái, thành phần giao diện
5	Framework học sâu	PyTorch (TensorFlow)	Ultralytics khai báo PyTorch là phụ thuộc lõi — chọn YOLO11 là chọn PyTorch	Kéo theo framework học sâu thứ hai (PaddlePaddle) do lựa chọn OCR — giải bằng mục 3.4
6	Đóng gói	Docker + Compose	Yêu cầu tái lập và khởi động bằng một lệnh	Kích thước image là rủi ro do có framework học sâu

3.6. Độ phân giải đầu vào: 640 thay vì 416

Đồ án có sẵn hai mô hình để đối chiếu — `baseline-416-v1.pt` và `best.pt` — nhưng **phép so sánh giữa chúng không quy kết được nguyên nhân**: giữa hai lượt huấn luyện có **ba biến thay đổi đồng thời và ngược chiều nhau** (độ phân giải 416 → 640, bộ dữ liệu v1 → v3 đã khử rò rỉ, số epoch), nên chênh lệch chỉ số **không gán được cho riêng biến nào**. Điều này phải nói rõ vì bản nháp trước từng trình bày `best.pt` như mô hình "tệ hơn baseline", trong khi ở tầng phát hiện nó **vượt mọi ngưỡng NFR** (mục 5.4.1).

Lựa chọn **640** vì vậy đứng trên căn cứ khác: đó là độ phân giải mà chỉ tiêu NFR-A1/A2 đặt ra và là độ phân giải mọi số liệu tốc độ CPU chính thức của Ultralytics được đo. Muốn quy kết nguyên nhân cần một ma trận thí nghiệm cô lập từng biến (E1 – E3), ước tính **≈ 33 giờ CPU** — vượt ngân sách còn lại, ghi nhận là **chưa thực hiện** ở mục 5.9.2. Bảng đối chiếu hai mô hình và đặc tả ma trận E1 – E3 ở **Phụ lục M.3**.

3.7. Kết luận chương

Bảng 3.5. Tổng hợp các quyết định công nghệ và căn cứ

Hạng mục	Quyết định	Căn cứ quyết định	Mức bằng chứng	Đánh đổi đã chấp nhận
Mô hình phát hiện	YOLO11n	Phiên bản gần đây duy nhất có số liệu tốc độ CPU chính thức; C2PSA hợp đối tượng nhỏ; bằng chứng ALPR dày nhất	 tài liệu	AGPL-3.0 kéo theo copyleft; bỏ qua YOLO26 dù trội hơn trên giấy
Họ engine OCR	PaddleOCR	Benchmark tự chạy trên 2.801 biển Việt Nam: 68,87% so với EasyOCR 14,28% và Tesseract 10,28% (mục 3.3.3); nhẹ hơn EasyOCR gần 10 lần; Apache 2.0	 tự đo	Kết luận chỉ áp cho vùng biển đã cắt sẵn ; trên ảnh toàn cảnh thứ tự có thể đảo
Bậc mô hình OCR	PP-OCRv5_mobile	v6 Medium chính xác hơn 5,5 điểm nhưng chậm hơn 16,8 lần; bậc Tiny của v6 không có trong gói	 tự đo	Bỏ lại 5,5 điểm độ chính xác đã định lượng được
Tinh chỉnh bộ nhận	Không dùng ở bản giao hàng	Ở đúng chế độ hệ thống đang chạy, bản tinh chỉnh kém hơn 7,50 điểm (mục	 tự đo	Bỏ lại +12,46 điểm chỉ đạt được ở chế độ bỏ bước phát

Hạng mục	Quyết định	Căn cứ quyết định	Mức bằng chứng	Đánh đổi đã chấp nhận
dạng		4.5.3)		hiện chữ, mà chế độ đó hồng trên ảnh toàn cảnh
Runtime suy luận	ONNX Runtime mặc định	Loại bỏ rủi ro xung đột hai framework học sâu trong một môi trường	 chưa đo	Thêm một bước xuất mô hình; lợi ích tốc độ chưa tự kiểm chứng
Độ phân giải đầu vào	640	Số đo có, nhưng ba biến đổi đồng thời nên không quy kết được (mục 3.6)	 không quy kết được	Không tách được đóng góp của riêng độ phân giải

Ba điều rút ra từ chương này.

Thứ nhất, ràng buộc phần cứng quyết định nhiều hơn chất lượng mô hình. Ba trong sáu quyết định ở Bảng 3.5 — YOLO11n thay vì YOLO26n, v5 mobile thay vì v6 Medium, ONNX Runtime thay vì PyTorch — đều xoay quanh ràng buộc CPU; ở triển khai có GPU, ít nhất hai trong ba phải xét lại. Đây là ranh giới áp dụng của toàn bộ chương.

Thứ hai, khoản nợ bằng chứng lớn nhất đã được trả, khoản còn lại thì chưa. Lựa chọn họ engine OCR từng mang dấu  suốt phần lớn thời gian làm đồ án — được nói "*chọn PaddleOCR vì nhẹ*" nhưng **không** được nói "*PaddleOCR chính xác hơn*". Benchmark ba engine ngày 03/08/2026 đã lật dấu đó sang  và cho phép phát biểu mạnh hơn, kèm đúng một giới hạn: phép đo chạy trên vùng biển đã cắt sẵn. Lựa chọn **runtime** thì vẫn mang dấu  và vẫn là nợ kỹ thuật ghi ở mục 5.9.2.

Thứ ba, chỗ đo được lại cho kết quả trái với kỳ vọng. Cả hai phép so sánh đồ án tự chạy đều **bác bỏ** phương án trông có vẻ tốt hơn: v6 chính xác hơn nhưng không dùng được, bản tinh chỉnh thắng đậm ở một chế độ nhưng thua ở chế độ thật. Chỉ đọc tài liệu rồi chọn theo con số cao nhất thì cả hai quyết định đều sai — lập luận thực nghiệm cho việc phải tự đo, và lý do hai dấu  được ghi thành nợ kỹ thuật ở mục 5.9.2.

CHƯƠNG 4. THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG

Trên cơ sở lý thuyết ở Chương 2 và lựa chọn công nghệ ở Chương 3, chương này trình bày thiết kế và hiện thực của hệ thống trong cùng một mạch. Nguyên tắc: mọi mô tả tương ứng với mã nguồn có thật; chức năng chưa hoàn thiện ghi rõ mức độ; số đo chưa có thì nói thẳng là chưa có. Chương gồm phân tích yêu cầu (4.1), kiến trúc (4.2), môi trường phát triển (4.3), bộ dữ liệu (4.4), huấn luyện mô hình (4.5), tầng AI (4.6), backend và CSDL (4.7), giao diện (4.8), Docker (4.9) và bảng đối chiếu cài đặt lịch thiết kế (4.10).

Trạng thái bản này: hệ thống chạy ALPRPipeline với mô hình chính thức `models/best.pt` (/health báo `model_loaded: true`, engine `yolo:best.pt+paddleocr-PP-OCRV5-mobile`, `mAP@0.5 = 0,9829`); StubPipeline đã ra khỏi đường chạy chính. Mọi kết quả thực nghiệm trình bày ở Chương 5.

4.1. Phân tích yêu cầu

4.1.1. Khảo sát nhu cầu và các tác nhân

Nhận dạng biển số là bài toán nền tảng của bãi đỗ tự động, thu phí không dừng, kiểm soát ra vào và giám sát giao thông. Áp mô hình ALPR huấn luyện trên dữ liệu nước ngoài vào Việt Nam gặp bốn trở ngại. **Thứ nhất, biển hai dòng chiếm tỉ trọng lớn** (toàn bộ xe máy và một phần ô tô) trong khi đa số bộ dữ liệu quốc tế giả định biển một dòng; điểm gây này đã đo được: trên **bộ RodoSol-ALPR của Brazil**, OpenALPR nhận đúng 3.772/4.000 ô tô biển một dòng (94,3%) nhưng chỉ 1.827/4.000 xe máy biển hai dòng (45,7%), chênh **48,6 điểm phần trăm** [7][88]. **Thứ hai, quy chuẩn biển số có tính pháp lý và cấu trúc chặt**: Thông tư 79/2024/TT-BCA [8], sửa đổi bởi TT 13/2025 [9] và TT 51/2025 [10], thông số vật lý theo QCVN 08:2024/BCA [11] — cấu trúc chặt vừa là ràng buộc vừa là cơ hội thiết kế cho khối hậu xử lý dựa trên luật. **Thứ ba, điều kiện thu nhận ảnh khắc nghiệt**: che khuất, bụi bẩn, nghiêng, ngược sáng, ban đêm. **Thứ tư, không có phần cứng tăng tốc**: máy thực hiện không có GPU CUDA, mọi suy luận và trình diễn chạy trên CPU (mục 4.1.4a, 4.3.1).

Lưu ý phạm vi số liệu. Cặp 94,3% / 45,7% đo trên **bộ RodoSol-ALPR của Brazil**, **không phải dữ liệu Việt Nam**; đồ án chỉ dùng nó làm dẫn chứng định lượng rằng "biển hai dòng khó hơn" là sự kiện đo được, không phải cảm nhận.

Hệ thống có bốn tác nhân: **người vận hành** (đưa ảnh/video, xem kết quả, tra cứu), **người phân tích** (thống kê, lọc, xuất báo cáo), **nhà phát triển** (tích hợp REST API), **hội đồng đánh giá** (quan sát, phản biện). Do hệ thống chạy nội bộ/localhost (giả định A-04), đồ án **không xây dựng xác thực và phân quyền**; ba tác nhân đầu là các *vai trò* trên cùng một giao diện, không phải các *tài khoản*.

4.1.2. Sơ đồ use case và ba use case chính

Ba use case chính — nhận dạng từ ảnh (UC-01), từ video (UC-02) và tra cứu lịch sử (UC-05) — được đặc tả đầy đủ ở **Phụ lục H.1**, gồm tác nhân, tiền điều kiện, luồng chính, luồng thay thế và hậu điều kiện.

4.1.3. Yêu cầu chức năng

Hệ thống có **34 yêu cầu chức năng** chia sáu nhóm, phân mức theo MoSCoW: 21 *Must*, 6 *Should*, 3 *Could*, 4 *Won't*. Bốn yêu cầu mức *Won't* đều là yêu cầu thuần giao diện và đều chuyển mức trong hai đợt thu gọn phạm vi ngày 20/07/2026 — trong đó **FR-4.1 là yêu cầu mức *Must* duy nhất bị đưa ra khỏi phạm vi**, ghi ở mục 6.2. Bảng đầy đủ từng mã yêu cầu ở **Phụ lục H.2**.

4.1.4. Yêu cầu phi chức năng

Các chỉ tiêu phi chức năng chia bảy nhóm — độ chính xác (NFR-A), hiệu năng (NFR-P), độ tin cậy (NFR-R), khả năng chịu tải (NFR-SC), khả năng bảo trì (NFR-M), bảo mật (NFR-S) và khả dụng (NFR-U) — mỗi chỉ tiêu kèm **ngưỡng tối thiểu, mục tiêu và phương pháp đo**. Hai ràng buộc chi phối toàn bộ nhóm hiệu năng: suy luận **chỉ trên CPU** (CON-02) và ngân sách độ trễ đầu-cuối. Bảng đầy đủ ở **Phụ lục H.3**; kết quả đối chiếu từng chỉ tiêu ở mục 5.7.

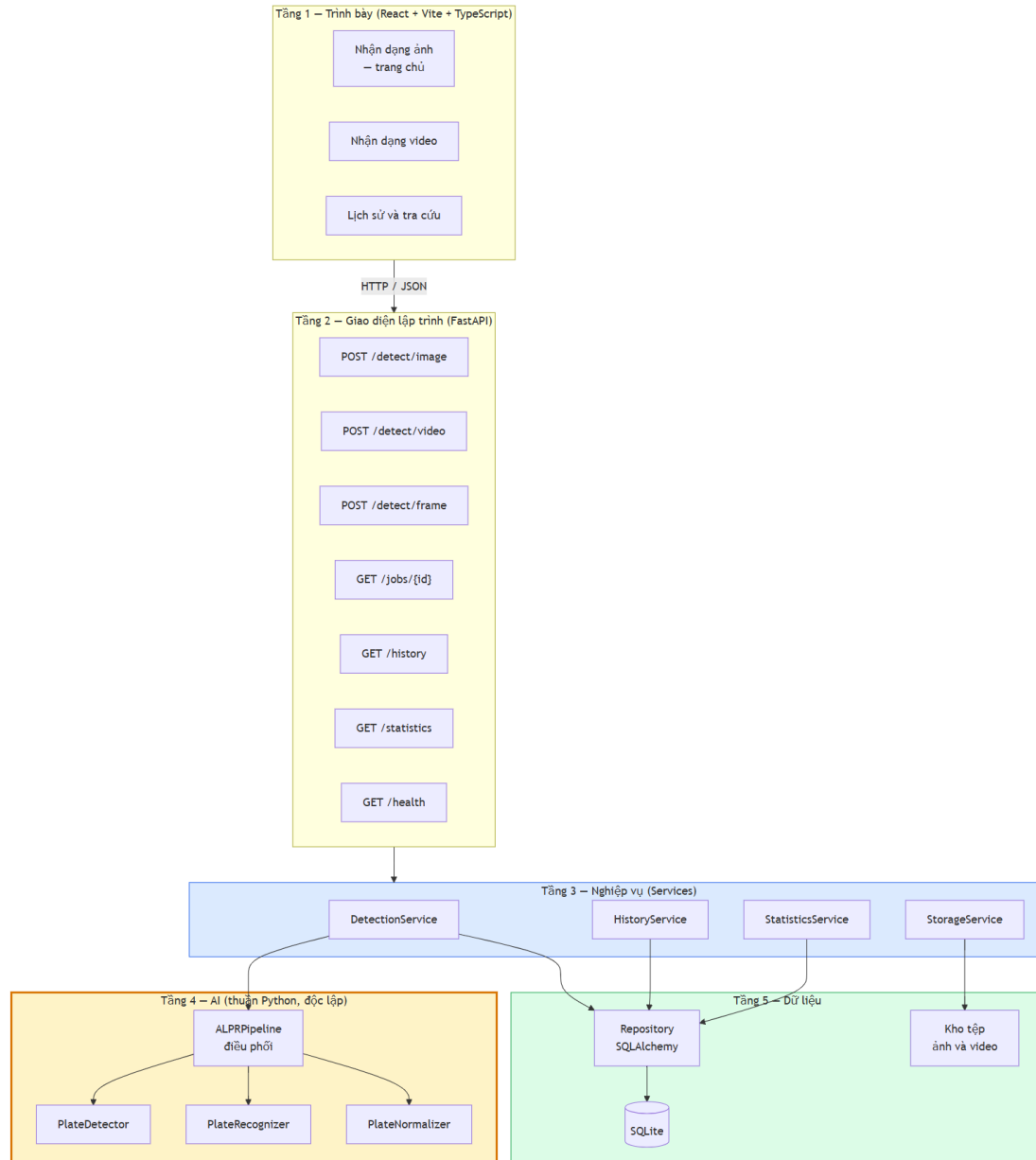
4.2. Kiến trúc hệ thống

4.2.1. Nguyên tắc kiến trúc

Kiến trúc sạch và quy tắc phụ thuộc: phụ thuộc chỉ hướng vào trong, từ chi tiết dễ thay đổi (framework web, CSDL, giao diện) về quy tắc nghiệp vụ ổn định. Ở bài toán này, thứ ổn định là thuật toán nhận dạng biến số — phát hiện, cắt, đọc, chuẩn hoá theo quy chuẩn Việt Nam; thứ dễ đổi là FastAPI hay Flask, SQLite hay PostgreSQL. Do đó **pipeline AI nằm ở vòng trong cùng**, tầng web phụ thuộc nó chứ không ngược lại. SOLID vận dụng: *trách nhiệm đơn nhất* — detector chỉ trả bounding box, recognizer chỉ trả chuỗi, normalizer chỉ chuẩn hoá — cho phép đo từng khối riêng; *thay thế Liskov* — dùng theo nghĩa đen khi hệ thống chạy pipeline giả lập đúng hợp đồng pipeline thật; *đảo ngược phụ thuộc* — tầng nghiệp vụ phụ thuộc hợp đồng trừu tượng, cài đặt tiềm từ ngoài.

Bốn ràng buộc kiến trúc: (1) **không trộn mã AI với mã API** (NFR-M1) ⇒ pipeline AI là package Python độc lập, không import framework web; (2) **mọi thành phần AI thay thế được** (NFR-M5) ⇒ đều đứng sau lớp trừu tượng; (3) **không hard-code đường dẫn** (NFR-M4) ⇒ mọi đường dẫn qua đối tượng cấu hình đọc từ biến môi trường; (4) **chạy được không cần GPU** (CON-02, NFR-C2) ⇒ thiết bị suy luận là tham số cấu hình, mặc định cpu — phát biểu là *cấu hình mặc định* chứ không phải "chế độ dự phòng", nên đường chạy CPU là đường được kiểm thử thường xuyên nhất.

4.2.2. Kiến trúc phân tầng



Hình 4.1. Kiến trúc phân tầng năm tầng và chiều phụ thuộc

Năm tầng: **1 — Trình bày** (giao diện, chỉ biết hợp đồng HTTP của tầng 2); **2 — API** (định tuyến, kiểm tra hợp lệ, ánh xạ ngoại lệ thành mã HTTP, sinh OpenAPI); **3 — Nghiệp vụ** (điều phối: tạo tác vụ, gọi pipeline, lưu tệp, ghi CSDL, gộp trùng, thống kê); **4 — AI** (phát hiện, nhận dạng, chuẩn hoá — **chỉ biết NumPy, OpenCV và thư viện học sâu**); **5 — Dữ liệu** (CSDL, kho tệp). **Điểm mấu chốt:** khối tầng AI **không có mũi tên nào đi lên**. Sau hai đợt thu gọn 2026-07-20, tầng trình bày còn **ba trang** nhưng **tầng 2–5 không đổi một dòng**: POST /detect/frame, GET /statistics, GET /health vẫn phục vụ và vẫn có kiểm thử tích hợp — phép thử ngoài dự kiến cho nguyên tắc phụ thuộc một chiều.

4.2.3. Tách tầng AI khỏi tầng API và cách kiểm chứng ràng buộc

Không một tệp mã nguồn nào trong package ai/inference/ được phép import FastAPI, Pydantic, SQLAlchemy hay bất kỳ thành phần nào của tầng web và tầng dữ liệu. Chiều ngược lại được phép và là bắt buộc.

Ba lợi ích. *Kiểm thử độc lập:* test chỉ cần nạp mảng NumPy, không phải dựng ứng dụng web. *Tái sử dụng trong script huấn luyện và đánh giá:* nếu logic tiền xử lý nằm lẫn trong hàm HTTP thì script đánh giá phải sao chép, hai bản sẽ lệch nhau — dẫn tới tình huống tệ nhất: **con số công bố không phải con số hệ thống thực sự tạo ra** (đúng loại sự cố đã xảy ra thật, mục 4.6.4g và 4.10). *Thay engine không sửa tầng API — đã kiểm chứng trên thực tế:* suốt Phase 5–7 hệ thống chạy StubPipeline, toàn bộ tầng API, nghiệp vụ, CSDL, giao diện được xây và kiểm chứng **trước khi mô hình được huấn luyện**; khi trọng số sẵn sàng, chuyển sang ALPRPipeline chỉ là đổi thành phần được tiêm, **không sửa dòng nào** ở router, service, schema. Để trạng thái mô phỏng không bị nhầm với vận hành thật, /health báo degraded chừng nào stub còn được dùng.

Kiểm chứng bằng công cụ, không bằng rà soát. tests/test_architecture.py (340 dòng) đặt hai lớp bổ sung nhau. Lớp một: quét văn bản mã nguồn bằng regex chỉ khớp **câu lệnh import viết thường** (không khớp tên sản phẩm trong tài liệu); điều kiện đạt là grep -rE "^\\s*(import|from)\\s+(fastapi|pydantic|starlette|sqlalchemy|backend)" ai/inference/ không trả kết quả. Lớp hai: khởi động **tiến trình Python mới** bằng subprocess.run, import *chỉ* package AI, soi sys.modules — bắt được import muộn trong thân hàm, **import bắc cầu**, và đo *thực tế đã nạp gì* thay vì *mã trông thế nào* (kiểm ngay trong bộ test vô giá trị vì test tích hợp đã nạp FastAPI từ trước); phép động còn đo gián tiếp thời gian nạp và bộ nhớ riêng tầng AI (NFR-P4, P7). Ba điều kiện phái sinh: mỗi mô-đun import được độc lập; ALPRPipeline khởi tạo được từ ba đối tượng giả **không nạp ultralytics, paddleocr, torch**; ai/evaluation/ được import backend nhưng chỉ ở phạm vi hàm. NFR-M4 kiểm cùng cách: quét chuỗi "C:\\..." / "/home/..." và khẳng định các mô-đun cấu hình dẫn xuất gốc dự án từ Path(__file__).resolve().parents[...].

4.2.4. Luồng xử lý của pipeline AI và nhánh biến hai dòng



Hình 4.2. Luồng xử lý của pipeline AI, các khối tô đỏ là nhánh biến hai dòng

Nhánh biến hai dòng là phần khó nhất của đồ án và là rủi ro đã xác định từ khâu lập kế hoạch (R-04). Bộ OCR dựng sẵn giả định văn bản một dòng ngang, nên với biến hai dòng chúng đọc theo thứ tự không xác định, ghép lẫn hoặc bỏ sót một dòng — cơ chế đứng sau chênh lệch 48,6 điểm phần trăm **đo trên bộ RodoSol-ALPR của Brazil** đã dẫn ở 4.1.1 [7]. Giải pháp: **tách vùng biến thành hai nửa, nhận dạng từng nửa, ghép theo thứ tự trên trước dưới sau** — mỗi nửa lúc này là một dòng ngang đúng giả định của bộ OCR (cài đặt ở 4.6.4).

Phân loại số dòng dùng hai cơ chế xếp chồng (kết luận 2.4.3e). *Cơ chế chính*: lấy lớp từ bộ phát hiện huấn luyện hai lớp (0 = một dòng, 1 = hai dòng) — chính xác nhất, chi phí gần bằng không; giá là dữ liệu phải gán nhãn hai lớp. *Cơ chế dự phòng*: ngưỡng tỉ lệ khung theo kích thước chuẩn QCVN 08:2024/BCA [11] — ô tô biển dài 520×110 mm → 4,727 (một dòng), ô tô biển ngắn 330×165 mm → 2,000 (hai dòng), xe máy 190×140 mm → 1,357 (hai dòng); ba giá trị tách biệt rõ, không loại biển nào rơi vào khoảng (2,000; 4,727), nên bộ ngưỡng ở mục 2.2.6 (AR < 2,5 hai dòng; > 3,0 một dòng; giữa là vùng nghi ngờ) đủ làm lớp dự phòng, và ngưỡng căn cứ quy chuẩn pháp lý nên giải thích được.

Điều kiện áp dụng bắt buộc: tỉ lệ khung phải đo trên ảnh **đã nắn phối cảnh** hoặc **hộp bao xoay tối thiểu**, không đo trên hộp bao thẳng trục thô — biển một dòng chụp nghiêng có tỉ lệ hộp thẳng trục tụt dưới 3,0 sẽ bị phân loại nhầm; đây là lý do khối hiệu chỉnh hình học đặt **trước** bước xác định số dòng. Ba giá trị 4,727 / 2,000 / 1,357 là tỉ lệ **danh định của biển vật lý**, chỉ trùng tỉ lệ vùng ảnh khi biển gần chính diện.

Nhánh giữ kết quả không hợp lệ: biển không khớp định dạng nào **vẫn được lưu** với cờ `is_valid_format = false` — loại bỏ chúng vừa vớt dữ liệu vừa tiêu huỷ đúng những ca giá trị nhất cho phân tích lỗi. Cùng tinh thần, **chuỗi OCR thô là sản phẩm đầu ra riêng**, không bị khối chuẩn hoá ghi đè — cùng một quyết định xuất hiện ở tầng CSDL (4.7.2b) và tầng chỉ tiêu (NFR-A5/A6).

4.2.5. Bảng tổng hợp các quyết định kiến trúc

Mỗi quyết định ghi kèm lý do và **đánh đối phải chấp nhận** — một quyết định trình bày như không có nhược điểm là quyết định chưa cân nhắc đủ.

Tám quyết định kiến trúc được ghi thành hồ sơ AD-01 ... AD-08, mỗi hồ sơ nêu **bối cảnh, phương án đã cân nhắc, quyết định và hệ quả phải chấp nhận** — dạng ghi chép này khiến một quyết định về sau có thể bị lật lại mà người lật hiểu được vì sao nó từng đúng. Bảng đầy đủ ở **Phụ lục H.5**; các mục 4.2.1 – 4.2.4 trình bày bốn quyết định có ảnh hưởng rộng nhất.

Ghi chú: AD-03 không đổi sau khi gỡ trang Webcam vì ở ~5 FPS trên CPU, nút thắt là suy luận chứ không phải giao thức. AD-04 cố ý **không** chọn tracking vì phức tạp hơn đáng kể và thêm một họ siêu tham số. AD-05 là quyết định duy nhất **đã thay đổi** so với phác thảo ("*PyTorch trước, ONNX nếu cần*") — ghi nhận tường minh thay vì lặng lẽ sửa bảng. AD-06 kéo theo hai quyết định phái sinh đã cài đặt: `yo1o11n` và **PP-OCRv5 mobile** — ràng buộc CPU thay đổi *lựa chọn mô hình*, không chỉ tốc độ.

4.3. Môi trường và công cụ phát triển

4.3.1. Cấu hình máy thực hiện và hệ quả của ràng buộc CPU

Toàn bộ cài đặt, kiểm thử, đo đạc chạy trên một máy trạm duy nhất (docs/00-requirements/environment.md): Windows 11 Pro; Intel Core i5-14600K, 14 nhân / 20 luồng; RAM 31,77 GiB; Intel UHD 770 — **không có GPU CUDA**; Python 3.13.12; Node.js 18.20.8; Docker 29.4.3. Cấu hình này **mâu thuẫn với mô tả môi trường ban đầu** (macOS Apple Silicon, Python 3.12); ghi nhận sai lệch thành văn bản là bước đầu của Phase 0. Ràng buộc CPU để lại dấu vết cụ thể: NFR-P1 phát biểu thẳng cho CPU; AD-06 kéo theo yolo11n và PP-OCRv5 mobile; và vì huấn luyện trên CPU mất 1–3 ngày/lượt, ai/training/ chạy được cả local lẫn Colab/Kaggle với siêu tham số trong tệp cấu hình (ai/training/config.py, 586 dòng). Hệ quả đo được: p95 đầu-cuối trên models/best.pt (máy rảnh) là **731 ms** (client-side) / **780 ms** (in-process), OCR chiếm **~64,3%**, phát hiện **~34,2%** (đối chiếu NFR-P1 ở 4.10).

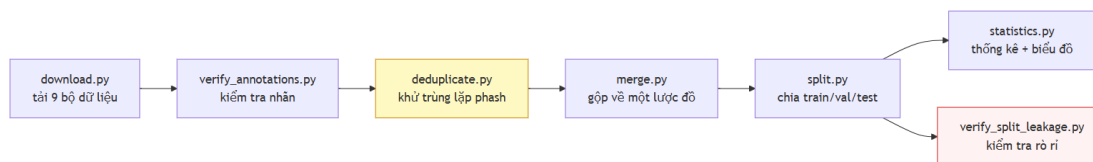
4.3.2. Ba môi trường ảo Python tách biệt và bộ công cụ

Đồ án dùng **ba môi trường ảo tách biệt**: .venv-ai/ (huấn luyện, xuất mô hình — NumPy 2.3.3, OpenCV 5.0, torch 2.13.0+cpu), .venv-ocr/ (thử nghiệm OCR — paddlepaddle 3.3.1, paddleocr 3.7.0), backend/.venv/ (dịch vụ — torch, ultralytics 8.4.101, paddleocr). Bắt buộc tách vì paddleocr kéo theo paddlex, **hạ cấp NumPy và thay opencv-python bằng opencv-contrib-python 4.10** — lùi một phiên bản lớn so với OpenCV 5.0 của nhánh huấn luyện; cài chung thì mỗi lần cài lại một nhánh âm thầm đổi phiên bản nhánh kia — lỗi không làm sập chương trình mà làm **kết quả đo không tái lập được**. Phân tách phản ánh ở requirements.txt và requirements-inference.txt, được Dockerfile.backend cài theo hai lớp riêng (4.9).

Bộ công cụ: FastAPI + Uvicorn; SQLAlchemy 2.x + Alembic; Pydantic v2; Ultralytics 8.4.101 chạy YOLO11 [16]; PaddleOCR 3.7.0 cho PP-OCRv5 [17]; Vite + React + TypeScript; pytest + pytest-cov; Docker Compose. Docker dùng Python 3.12 trong khi local dùng 3.13 là **chủ ý**: container là nơi lấy lại phiên bản mục tiêu (NFR-C1).

4.4. Xây dựng bộ dữ liệu

4.4.1. Đường ống sáu bước và thành phần bộ dữ liệu



Hình 4.3. Đường ống sáu bước xây dựng bộ dữ liệu

Mỗi bước là một script độc lập trong scripts/dataset/ có CLI riêng, sinh báo cáo JSON/CSV; run_pipeline.py chạy cả chuỗi một lệnh. **Kết quả: 15.133 ảnh** hợp nhất từ **7 bộ công khai** (Roboflow, HuggingFace, Kaggle), còn **6 nguồn nguyên tố** sau khi loại **11.978 ảnh (44,2%)** bản sao từ **27.111 ảnh**; tổng 9 bộ tải về, 2 bộ nhãn mức ký tự tách riêng cho đánh giá OCR. Chia 70/20/10 thành **10.592 / 3.027 / 1.514 ảnh**. Bảng dưới **phải trích khi nói về dữ liệu của đồ án**; nguồn và giấy phép từng bộ ở **Phụ lục C.1**.

Bảng 4.1. Đóng góp của từng bộ dữ liệu trước và sau khử trùng lặp

#	Bộ (slug)	Vào gộp	Còn lại	Bị loại
1	roboflow_school_fuhih	8.357	6.868 (45,38%)	17,8%
2	hf_vn_plates_segment	4.578	4.375 (28,91%)	4,4%
3	roboflow_traffic_camera	3.843	3.162 (20,89%)	17,7%
4	roboflow_eric_nguyen	840	353 (2,33%)	58,0%
5	roboflow_demo_tracking	236	235 (1,55%)	0,4%
6	roboflow_cuong_ta	8.254	140 (0,93%)	98,3%
7	roboflow_tran_ngoc_xuan_tin	1.005	0	100%
	Tổng	27.113	15.133	44,2%

Ba điều bảng nói ra mà con số tổng giấu đi: hai bộ đầu chiếm **74,3%** nên đồ án **không đa dạng nội dung** như con số "6 nguồn" gợi ý; cuong_ta mất **98,3%**, tran_ngoc_xuan_tin mất **100%** — bằng chứng các bộ công khai **không độc lập với nhau**; và cuong_ta là bộ cân bằng layout nhất (51,04% hai dòng) còn school_fuhih sống sót nhiều nhất lại lệch nặng nhất (88,85% hai dòng), nên khử trùng lặp **vô tình làm tập dữ liệu lệch layout hơn** (docs/reports/02-dataset-report.md mục 6.3). **Giấy phép**: năm bộ CC BY 4.0; một bộ tự khai Public Domain — **không được khẳng định là thật** vì ảnh có dấu hiệu báo chí; một bộ HuggingFace **chưa xác nhận được giấy phép**.

Song song, một nhánh riêng tái tạo **4.019 chuỗi biển số** từ hai bộ nhãn mức ký tự, trong đó **2.801 chuỗi (69,69%)** khớp mẫu hợp lệ theo plate_rules.py (roboflow_ocr_plate 2.650, roboflow_ocr_conversion 151, đều CC BY 4.0), phục vụ đánh giá OCR độc lập với tăng phát hiện. Kết quả phụ: tập ký tự trên 4.019 chuỗi có **đúng 30 ký tự phân biệt**, không chứa I, J, O, Q, W — **xác nhận độc lập bằng dữ liệu** cho tập EXCLUDED_LETTERS suy từ văn bản pháp quy (4.6.5b).

Cảnh báo phạm vi bắt buộc kèm mọi số liệu OCR. Phân loại màu nền trên 2.801 ảnh cho: **2.736 biển trắng (97,68%)**, 20 vàng, 4 xanh, **0 đỏ, 0 ngoại giao**. Phát biểu đúng là **"1 – CER = 0,9454 trên một tập gồm 97,7% biển trắng"**, **không phải "trên biển số Việt Nam"** (docs/reports/17-plate-type-audit.json).

4.4.2. Khử trùng lặp chéo bộ và con số 44,2%

Các bộ công khai fork lẫn nhau, nên một ảnh nằm ở train dưới tên bộ này và test dưới tên bộ khác thì **độ chính xác test đang đo khả năng ghi nhớ**; con số tiêu đề vì vậy là số nhóm

trùng **chéo bộ**. Vết cặn ~690 triệu cặp là bất khả thi nên script dùng **multi-index hashing**: cắt mã băm 64 bit thành $\text{threshold} + 1$ dải — theo nguyên lý chuồng bồ câu, hai mã khác nhau tối đa threshold bit bắt buộc trùng khớp trên ít nhất một dải — nên tập ứng viên chứa mọi cặp thật rồi được xác minh chính xác: **thuật toán chính xác, không xấp xỉ**.

Có hai phép đo trên hai mẫu số khác nhau, trích một con số trần không nêu mẫu số là gây hiểu nhầm: **(a)** trên 7 bộ vào hộp nhất — mẫu số 27.111, ngưỡng Hamming 5, loại **11.978 = 44,2%, đã xoá thật**; **(b)** trên corpus còn lại — mẫu số 15.133, ngưỡng 10, chỉ ra **47,8% có thể loại** nhưng **chưa xoá**. 47,8% không mâu thuẫn 44,2%: ngưỡng lỏng hơn, và chỉ đo chứ chưa xoá (đối chiếu đầy đủ ở **Phụ lục C.3**). Hai hệ quả của tỷ lệ 44,2%: quy mô thật khác hẳn danh nghĩa (ca cực đoan tran_ngoc_xuan_tin vào 1.005 ra **0** — lý do **không được cộng dồn expected_images** của các bộ Roboflow), và phân bố huấn luyện lệch vì bản sao tập trung ở các bộ được chép nhiều nhất. `split.py` giữ **mọi thành viên của một nhóm trùng trong cùng split** nên bản trùng không bị xoá cũng không rò rỉ được.

4.4.3. Giới hạn của perceptual hash: nó tóm tắt bố cục khung ảnh, không tóm tắt chiếc xe

Đây là **giới hạn không khắc phục được** bằng công cụ hiện có. Một lần kiểm tra độc lập ở ngưỡng Hamming 10 trên bộ v1 tìm thấy **619 cặp gần trùng giữa train và test**, toàn bộ ở dải $d = 6-10$; kiểm bằng mắt cho thấy **cùng một chiếc xe, cùng chuỗi biển số, ở cả hai split**. Pipeline không bắt được vì bộ chia gom nhóm ở ngưỡng 5 rồi lần kiểm đầu cũng đo lại ở ngưỡng 5 và báo "0 cặp" — **lập luận vòng tròn**. Nâng ngưỡng cũng không giải quyết: phash rút ảnh thành 64 bit mô tả **cấu trúc tần số thấp của toàn khung**, nên hai xe khác nhau qua cùng một camera có khoảng cách phash rất nhỏ vì 90% khung hình giống hệt. Đánh đổi không thoát được: ngưỡng thấp bỏ sót cặp cùng xe khác ngày; ngưỡng cao gộp nhầm hàng nghìn ảnh xe khác nhau cùng camera. Bộ v3 chia lại với gom nhóm ngưỡng cao hơn và kiểm độc lập ở ngưỡng 10, nhưng đồ án ghi nhận thẳng thắn: **vẫn còn rò rỉ tồn dư không khử được bằng phash** — khắc phục đòi hỏi so khớp mức chuỗi biển số hoặc đặc trưng phương tiện. Hệ quả: baseline-416-v1.pt đạt $\text{mAP@0.5} = 0,9933$ trên bộ v1 nhưng con số đó **không được báo cáo là "đạt"** vì tập test có rò rỉ đã đo được (4.10).

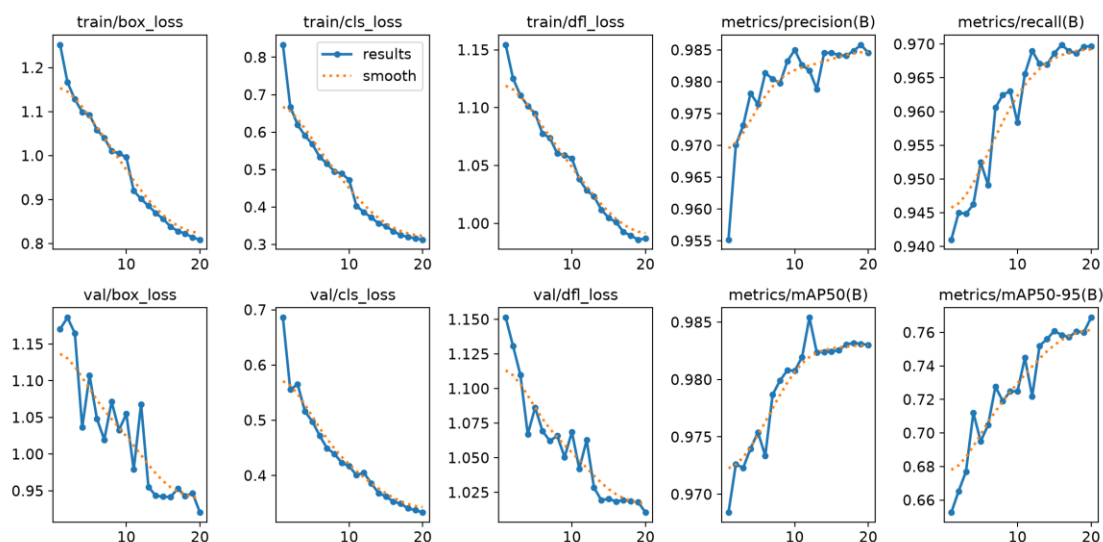
4.5. Huấn luyện mô hình

4.5.1. Siêu tham số và chi phí huấn luyện bộ phát hiện

Cấu hình lượt chính thức trích từ `runs/final-640-v3/args.yaml` — tệp Ultralytics tự sinh, là bản ghi *đã thực thi* chứ không phải *dự định*; bảng đầy đủ ở **Phụ lục B.1**. Giá trị chịu lực: `model = yolo11n.pt` (tiền huấn luyện COCO, **2.590.035** tham số — biến thể nano do ràng buộc CPU); `imgsz = 640` (đúng độ phân giải NFR-A1/A2); `epochs = 20`, `batch = 8`; `optimizer = AdamW`, `lr0 = 0.001`, `cos_lr`; `close_mosaic = 10`; `device = cpu`; `seed / deterministic = 42 / true`. `flip1r = 0.0` lệch có chủ ý so với mặc định 0.5: lật ngang tạo ký tự gương hoá — phân bố không bao giờ có trong thực tế. Vì giới hạn thời gian CPU chỉ chạy được **một lượt huấn luyện duy nhất**, không có nhiều seed để ước lượng phương sai; cố định seed ít nhất

bảo đảm lượt này tái lập được — mọi chỉ số là kết quả **một lần chạy**, không có khoảng tin cậy (hạn chế ghi ở 5.9.3). **Chi phí:** baseline baseline-416-v1.pt 40 epoch, imgsz 416, bộ v1 — **156 phút**; best.pt 20 epoch, imgsz 640, bộ v3 — **≈ 35,6 phút/epoch, tổng ≈ 712 phút (≈ 11,9 giờ)** trên CPU. Ba yếu tố cùng thay đổi (số ảnh ×3,3, diện tích ×2,37, epoch giảm nửa) khiến so sánh ở mục 3.6 **không quy kết được nguyên nhân cho từng biến**.

4.5.2. Tiến triển của quá trình huấn luyện



Hình 4.4. Đường cong huấn luyện theo epoch — ba hàm mất mát và bốn chỉ số trên tập validation (nguồn: runs/final-640-v3/results.csv)

Ba hàm mất mát giảm đơn điệu và **không có dấu hiệu quá khớp**: box_loss 1,252 → 0,809, cls_loss 0,833 → 0,313, dfl_loss 1,154 → 0,987; đường validation bám sát đường train suốt 20 epoch. Chỉ số trên tập validation đi lên rồi bão hoà sớm: mAP@0,5 đạt **0,9684 ngay ở epoch 1** và chỉ nhích lên **0,9830** ở epoch 20, trong khi mAP@0,5:0,95 — chỉ số nhạy với độ khít của hộp — tăng đáng kể hơn, **0,6526 → 0,7688**.

Điều này nói lên hai việc. Thứ nhất, bài toán *một lớp* dễ ở khâu **tìm ra** biến số (mAP@0,5 gần bão hoà sau một epoch) nhưng khó ở khâu **khoanh khít** (mAP@0,5:0,95 còn tăng tới cuối). Thứ hai, đường cong **chưa phẳng ở epoch 20** — mốc dừng do ngân sách thời gian CPU quyết định, không phải do hội tụ; huấn luyện dài hơn nhiều khả năng còn cải thiện, và đây là một hạn chế được ghi ở Chương 6.

Bảng 4.2. Tiến triển chỉ số trên tập validation theo mốc epoch

Epoch	box_loss (val)	cls_loss (val)	dfl_loss (val)	mAP@0.5	mAP@0.5:0.95	Precision	Recall
1	1,1705	0,6858	1,1513	0,9684	0,6526	0,9552	0,9410
2	1,1861	0,5554	1,1307	0,9726	0,6653	0,9700	0,9450
3	1,1647	0,5651	1,1098	0,9723	0,6770	0,9731	0,9449

Epoch	box_loss (val)	cls_loss (val)	dfl_loss (val)	mAP@0.5	mAP@0.5:0.95	Precision	Recall
5	1,1074	0,4977	1,0863	0,9754	0,6950	0,9765	0,9525
10	1,0548	0,4168	1,0686	0,9808	0,7248	0,9850	0,9584
15	0,9420	0,3619	1,0205	0,9824	0,7609	0,9846	0,9686
20	0,9204	0,3331	1,0105	0,9830	0,7688	0,9846	0,9697
Epoch tốt nhất (= 20)	0,9204	0,3331	1,0105	0,9830	0,7688	0,9846	0,9697

Số liệu từ runs/final-640-v3/results.csv. **mAP@0.5** gần bão hoà rất sớm ($\approx 0,97$ từ epoch 1) trong khi **mAP@0.5:0.95** vẫn tăng đều ($0,653 \rightarrow 0,769$) — định vị là dễ, khớp box chính xác mới khó; đường vẫn còn dốc lên tại epoch 20 nên **phải phát biểu rõ 20 epoch là giới hạn ngân sách tính toán, không phải điểm hội tụ**. val/cls_loss giảm đơn điệu, chưa thấy dấu hiệu quá khớp. **Epoch tốt nhất chọn chỉ dựa trên tập validation**; tập test không dùng cho bất kỳ quyết định nào ở mục này.

4.5.3. Tinh chỉnh bộ nhận dạng ký tự và lý do không đưa vào bản giao hàng

PP-OCRV5 mobile huấn luyện trên chữ cảnh tổng quát; mục này trả lời bằng số câu hỏi fine-tune trên đúng miền dữ liệu thì được gì. **Cấu hình**: 30 epoch trên 6.672 mẫu (2.801 ảnh gốc + hai biến thể tăng cường mỗi ảnh), kiểm định 571 mẫu, charset đủ 36, khởi đầu từ en_PP-OCRV5_mobile_rec_pretrained; kết thúc train acc **0,9449**, val acc **0,8809**.

Bảng 4.3. So sánh bộ nhận dạng gốc và bản tinh chỉnh trên cùng ngữ liệu

Cấu hình	A5 (chuỗi thô)	A6 (sau hậu xử lý)	Đúng định dạng	ms/ảnh
Model gốc, det + rec — bản giao hàng	0,6373	0,7512	0,9443	328,8
Model fine-tune, det + rec	0,5998	0,6762	0,9018	—
Model gốc, chỉ rec	0,6776	0,7508	0,9568	35,7
Model fine-tune, chỉ rec	0,8618	0,8758	0,9886	38,5

Nguồn: docs/reports/28-ocr-accuracy-finetuned.json, 29-reconly-ablation.json.

Phải đọc theo hàng. Hàng 2 so hàng 1: ở đúng chế độ hệ thống đang chạy, fine-tune **kém hơn 7,50 điểm**. Hàng 4 so hàng 1: bỏ bước phát hiện chữ, fine-tune **hơn 12,46 điểm** — kết luận ngược hẳn. **Nguyên nhân là hai chế độ đo khác nhau, không phải model**:

PaddleOCR đánh giá nhánh rec bằng **nguyên ảnh** biến, còn đường ống triển khai cắt ảnh thành nhiều mảnh rồi đọc từng mảnh — model fine-tune chỉ được dạy đọc cả biến một lần, chưa từng thấy mảnh vụn (mẫu lỗi cụt đầu: 51U74598 ra 598). **Vì vậy val acc 0,8809 không sai, nhưng nó đo một chế độ hệ thống không dùng**. Quyết định: **không đem fine-tune đi**

giao, cũng không bật chế độ chi-rec — ngữ liệu 2.801 mẫu toàn ảnh cắt sẵn nên không có thẩm quyền quyết định giữa hai chế độ, và đo lại trên ảnh toàn cảnh thì thứ tự đảo ngược (phân tích đầy đủ ở 5.6.6). Model, công tắc ALPR_OCR_REC_MODEL_DIR và đường ray huấn luyện giữ nguyên trong kho, sẵn sàng cho lượt đo có tập nhãn ảnh hiện trường.

4.6. Tầng AI — thiết kế và cài đặt

4.6.1. Tổ chức gói ai/inference và ba lớp trừu tượng

Gói gồm mười một mô-đun cùng `__init__.py`, tổng **4.852 dòng**: `types.py`, `interfaces.py`, `config.py`, `exceptions.py`, `plate_rules.py`, `normalizer.py`, `detector.py`, `recognizer.py`, `two_line.py`, `plate_color.py`, `pipeline.py`. Ràng buộc "không import FastAPI" kiểm chứng tự động ở 4.2.3; lý do nền tảng: gói phải chạy được trong Jupyter, script benchmark và Colab.

Ba lớp trừu tượng, và **hợp đồng chung của cả ba là điều đáng nói nhất**.

BaseDetector.detect(image) → list[PlateDetection] — đã lọc ngưỡng và NMS, mọi hộp **kẹp trong biên ảnh**, và **danh sách rỗng là kết quả bình thường**, không bao giờ là lỗi.

BaseRecognizer.recognize(plate_image) → PlateRecognition — trả chuỗi thô kèm độ tin cậy; sửa lỗi ký tự và kiểm tra định dạng **không** thuộc trách nhiệm của nó, và chính việc tách đó làm đóng góp hậu xử lý **đo được** qua hiệu giữa `raw_ocr_text` và `plate_number`.

BaseNormalizer.normalize(raw_text) → tuple[str, bool] — kết quả không hợp lệ vẫn **trả về**, vì loại bỏ sẽ xóa đúng những thất bại chương đánh giá cần đếm. Hợp đồng "trả rỗng, không ném" nhất quán với NFR-R2: **không tìm thấy** và **lỗi** là hai chuyện khác nhau. Hai lớp đầu có `warmup()` để chuyển chi phí nạp trọng số ra khỏi yêu cầu đầu tiên (NFR-P1).

Các kiểu dữ liệu khai báo **bất biến** ở chỗ có thể. `BoundingBox` lưu (x, y, width, height) khớp trực tiếp bốn cột `bbox_*` và đặt **trùng tên cột CSDL có chủ đích**, để tầng lưu trữ sao chép trường-sang-trường — một lớp biên dịch trung gian là thêm một chỗ để `confidence` và `ocr_confidence` bị hoán đổi (4.7.2a).

4.6.2. Bộ phát hiện — YoloPlateDetector

`YoloPlateDetector` và `PaddleOcrRecognizer` (4.6.3) đều là **adapter mỏng**: không nơi nào ngoài hai mô-đun này chạm vào `Results` của Ultralytics hay máy OCR của PaddleOCR. Cả hai **ghim phiên bản mô hình tường minh** — name của detector trả `yolo:{stem}{suffix}`, recognizer ghim `OCR_VERSION = "PP-OCRv5"` — vì một con số benchmark chỉ tái lập được khi nêu đúng bộ trọng số; nâng cấp thư viện không được âm thầm đổi mô hình đứng sau một kết quả đã công bố.

Detector **nạp trọng số ngay trong hàm khởi tạo** để tệp thiếu làm hệ thống thất bại lúc khởi động kèm hướng dẫn khắc phục, thay vì thất bại lúc có yêu cầu đầu tiên. Nó nhận `.pt/.onnx/.torchscript` và **cả thư mục OpenVINO** — từ chối thư mục sẽ khiến cấu hình nhanh nhất trên CPU Intel không cấu hình được. Mọi hộp bao đều được **kẹp về biên**

ảnh và hộp suy biến trả None kèm log, nên tầng trên không bao giờ nhận toạ độ nằm ngoài ảnh.

4.6.3. Bộ nhận dạng ký tự — PaddleOcrRecognizer

Một phát hiện kỹ thuật phải nêu ở thân bài: backend oneDNN làm sập suy luận trên PP-OCRv5. Trên đúng nền tảng mục tiêu (paddlepaddle 3.3.1, Windows, CPU), chạy mô hình phát hiện văn bản qua oneDNN kết thúc bằng `NotImplementedError: (Unimplemented) ConvertPirAttribute2RuntimeAttribute...` — khiếm khuyết phía thư viện, không phải lỗi cấu hình. Xử lý: hằng số có tài liệu `DEFAULT_ENABLE_MKLDNN: Final[bool] = False`. Đây là **núm hiệu năng, không phải núm độ chính xác** — khi lỗi thượng nguồn được sửa chỉ cần lật giá trị và đo lại. Nó cũng giải thích một phần NFR-P1: **một con đường tăng tốc CPU hiển nhiên đang bị chặn bởi lỗi thư viện chứ không phải chưa được thử**.

Ngoài ra PaddleOcrRecognizer lọc mảnh văn bản **theo hình học chứ không theo ngưỡng tin cậy** — CLAHE có thể sinh mảnh rác đọc thành chuỗi vô nghĩa với độ tin cậy 0,84, mà ngưỡng tin cậy không tách được — và tổng hợp độ tin cậy bằng **trung bình có trọng số theo độ dài mảnh**, vì trung bình cộng cho phép một mảnh một ký tự 0,99 che lấp mảnh bảy ký tự 0,40. Chi tiết cài đặt của cả hai adapter ở **Phụ lục I.6**.

4.6.4. Mô-đun xử lý biến hai dòng — two_line.py

a) Vì sao bài toán tồn tại. Bộ nhận dạng hiện đại là CRNN/CTC với giả định **căn chỉnh đơn điệu** giữa cột ảnh và ký tự — chỉ đúng với văn bản một dòng; chồng lên đó, PP-OCR **resize mọi ảnh cắt về chiều cao 48 px** [103]. Biển xe máy 140×190 mm (QCVN 08:2024/BCA [11]) có tỷ lệ $\approx 1,36$, nên sau khi ép về 48 px mỗi hàng ký tự chỉ còn ~ 24 px — dưới mức nét chữ còn tách rời. Hệ quả định lượng: trên bộ **RodoSol-ALPR của Brazil**, OpenALPR đạt **94,3% trên biển ô tô một dòng** nhưng chỉ **45,7% trên biển xe máy hai dòng** [7][88] — đồ án trích cập số này thuần túy làm dẫn chứng tương đương định lượng, không phải số liệu Việt Nam.

b) Ước lượng số dòng bằng tỷ lệ khung. `estimate_line_count` dùng `DEFAULT_TWO_LINE_AR_THRESHOLD = 2.3`: `line_count = 2 if aspect_ratio < threshold else 1`. Đây là **heuristic do đồ án đề xuất, không phải quy tắc pháp lý** — quy chuẩn chỉ cung cấp kích thước vật lý ($4,727 / 2,000 / 1,357$); 2,5 chọn **lệch về phía hai dòng** vì đường xử lý hai dòng suy giảm êm khi gặp đầu vào một dòng, chiều ngược lại thì không. Hạn chế ghi trong mã: dải 2,5–3,0 là vùng xám thật vì biến một dòng chụp nghiêng gắt có tỷ lệ hộp bao tụt vào đó; định lượng tần suất thuộc Chương 5.

c) Cắt trên/dưới có chồng lấn. `split_two_line` dùng `UPPER_HALF_END_RATIO = 5/12`, `LOWER_HALF_START_RATIO = 1/3` — hai nửa **chồng lấn 1/12 chiều cao biển**, chủ ý do bất đối xứng chi phí: cắt cụt chân/đỉnh chữ phá huỷ thông tin **vĩnh viễn**, còn lọt vài điểm ảnh hàng bên cạnh thì bộ nhận dạng bỏ qua như nền. Hàm ép hai nửa không rỗng và cảnh báo nếu tham số làm mất chồng lấn.

d) Ghép ngang bằng np.hstack. Chiều cao chung `max(upper.h, lower.h, MIN_MERGE_HEIGHT)` với `MIN_MERGE_HEIGHT = 48` — bằng đúng chiều cao đầu vào cố định

của PP-OCR. Nửa trên đặt bên trái nên thứ tự đọc bảo toàn, và sau khi ghép, **một hàng ký tự duy nhất nhận trọn ngân sách 48 px** thay vì hai hàng chia nhau; `_match_channels` nâng cả hai nửa về BGR khi số kênh lệch.

e) Tiền xử lý ảnh biến — `preprocess_plate`. Ba bước, **mỗi bước bật/tắt độc lập** để ablation được: chuyển xám (ký tự không mang thông tin màu); CLAHE (`clipLimit=2.0`, ô 8×8) vì biến phản quang tạo mảng chói cục bộ mà cân bằng toàn cục không xử lý được [95]; khử nhiễu `bilateralFilter(5, 50, 50)` vì lọc song phương **bảo toàn biên** — làm mờ Gauss đủ mạnh sẽ bo tròn đầu nét, thứ phân biệt 8 với B. Kết quả luôn là BGR ba kênh; recognizer truyền `upscale_to_height = 64` vì ảnh biến ra khỏi detector thường chỉ cao 20–40 px.

f) Bước cứu dòng trên — một giả thuyết hợp lý bị chính dữ liệu bác bỏ. Mục có giá trị phương pháp luận cao nhất của khối: quan sát chế độ hồng — đề xuất giả thuyết *nghe rất hợp lý* — **đo và bác bỏ** — và chính phép bác bỏ dẫn tới thiết kế đúng. Ảnh biến 29E-015.66 trả về 015.66: sau khi ghép, bộ phát hiện văn bản chỉ tìm thấy **một** vùng chữ và bỏ hẳn cụm 29E. Giả thuyết đầu tiên — bỏ phép ghép, đọc riêng từng nửa rồi nối chuỗi — được **đo** trên 200 biến hai dòng có nhãn và **sụp đổ**: 64,5% xuống **3,5%**, thắng ở **0/200** ảnh (số liệu đầy đủ ở 5.5.6). Nguyên nhân nằm đúng ở chi tiết đã biện minh ở (c): hai nửa cắt **chồng lấn** đi vào OCR riêng rẽ thì dải chồng lấn **bị đọc hai lần**, sinh ký tự rác nối giữa chuỗi. Kết luận đảo ngược cách hiểu về phép ghép: trên dải liền mạch, vùng lặp nằm giữa hai cụm chữ và **bị bộ phát hiện văn bản gạt đi** — hai ảnh rời thì không có ngữ cảnh để gạt.

Bản sửa vì vậy **giữ nguyên chiến lược ghép**, chỉ thêm một bước phục hồi hẹp qua vị từ `should_rescue_two_line(recognition)`: chỉ True khi đồng thời `line_count == 2`, `not is_valid_format`, và `raw_text` khác rỗng — khi đó tốn thêm **một** lần OCR trên riêng nửa trên, ghép `upper + raw`, chuẩn hoá lại; kết quả mới **chỉ được nhận nếu qua kiểm tra định dạng**, mọi trường hợp khác trả nguyên kết quả cũ. **Tính chất "không thể làm tệ đi" là tính chất cấu trúc**: cống chỉ mở khi kết quả **đã hỏng sẵn**, nên tập bị ảnh hưởng và tập đang đúng là hai tập rời nhau — hai phép đo A/B ở 5.5.6 vì vậy là *kiểm chứng*, không phải *căn cứ*. Mức cải thiện **khiêm tốn** (+1,86 và +0,50 điểm trên hai mẫu độc lập, 0 ca bị làm hỏng), không trình bày như đột phá: nó vá một điểm mù cụ thể với chi phí ~15–21 ms mỗi biến hai dòng, không đụng tới nút thắt chính.

g) Một lỗi phương pháp đo, quan trọng hơn chính bản vá.

`ai/evaluation/ocr_accuracy.py` — nơi sinh các con số NFR-A4–A7 công bố ở Chương 5 — **không đi qua ALPRPipeline** mà gọi thẳng recognizer và normalizer, nên mọi logic ở tầng điều phối **vô hình với các con số công bố**. Cách sửa: **tách bước cứu thành hai hàm tự do cấp mô-đun** (`should_rescue_two_line`, `rescue_two_line_upper`) để cả pipeline lẫn bộ đo cùng gọi. Bài học vượt ra ngoài biến hai dòng — **một bộ đo đi tắt qua tầng điều phối sẽ đo một hệ thống khác với hệ thống được giao** — và biện pháp này về sau **vẫn không đủ**: diễn biến đầy đủ ở Phụ lục K.

4.6.5. Bộ luật hậu xử lý — đóng góp kỹ thuật chính

Bộ phát hiện và bộ nhận dạng đều là mô hình có sẵn; khối hậu xử lý thì không. `plate_rules.py` tuân ba quy tắc: **thuần khiết** (không I/O, không trạng thái toàn cục khả biến); **regex sinh từ tập hợp, không viết tay** — mẫu không thể trôi khỏi bảng nó mã hoá; **lớp ký tự là hằng có tên**.

a) PROVINCE_CODES — 81 mã tỉnh đang dùng theo phụ lục TT 51/2025/TT-BCA [10] (80 mã địa phương + mã 80), song song `UNUSED_PROVINCE_CODES` gồm **8 mã không bao giờ được cấp**: 13, 42, 44, 45, 46, 87, 91, 96. Giá trị so với `\d{2}`: nó **bác bỏ** 13A-123.45; giữ tường minh tập không dùng cho phép test khẳng định hai tập phủ đúng dài 11-99. Sáp nhập hành chính 2025 không làm mất hiệu lực biển đã cấp — mối quan tâm của tầng báo cáo, không phải của định dạng.

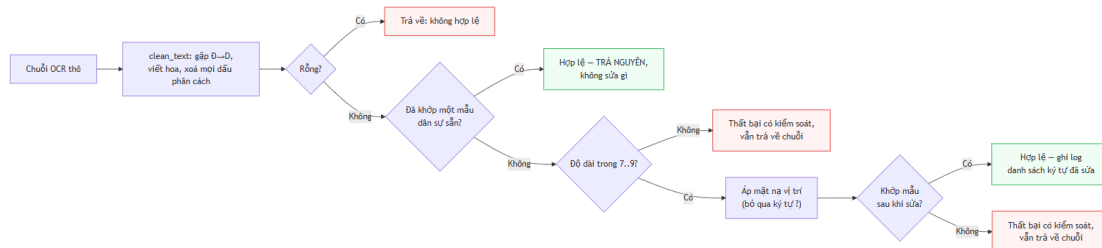
b) Các lớp ký tự sê-ri. Bốn hằng: L20 (chữ sê-ri ô tô; chữ **thứ nhất** sê-ri xe máy), L20B (chữ **thứ hai** sê-ri xe máy), L11 (sê-ri biển xanh), L21 (20 chữ chuẩn **cộng R**). **L20 và L20B là ảnh gương của nhau ở đúng hai ký tự** (L20 có G không R, L20B có R không G): 29-AR 123.45 hợp lệ còn 29-AG 123.45 thì không. L21 tồn tại vì một mô hình charset "20 chữ" **không bao giờ dự đoán ra R** nên sai **có hệ thống** trên mọi biển xe máy mang R ở sê-ri thứ hai — loại sai không hậu xử lý nào cứu được vì thông tin đã huỷ ở tầng mô hình. Cùng logic: `OCR_SAFE_CHARSET = 31` ký tự, `OCR_TRAINING_CHARSET = 36`; huấn luyện trên 36 rồi ràng buộc về 31 là chủ ý — mô hình **được phép** dự đoán ký tự bất hợp pháp tạo sai lầm *quan sát được, sửa được*, mô hình *không thể về mặt kiến trúc* dự đoán nó tạo sai lầm vô hình. `EXCLUDED_LETTERS = {I, J, O, Q, W}` — 5 chữ bị loại toàn quốc; chính việc loại I, O, Q làm sửa lỗi OCR khả thi. (Ghi chú hiệu chỉnh: con số "6 chữ bị loại" từng gồm cả R đã được sửa.)

c) POSITION_MASKS và ký tự đại diện ? ở chỉ số 3 — chi tiết cài đặt quan trọng nhất của khối. Ba mặt nạ: `car_5 = "DDLDDDDD"`, `car_4 = "DDLDDDD"`, `motorcycle_9 = "DDL?DDDDD"`; `MASK_BY_LENGTH` ánh xạ độ dài 7/8/9. D = bắt buộc chữ số, L = bắt buộc chữ cái, ? = **đại diện, tuyệt đối không ép kiểu**. Hai kiểu biển xe máy cùng 9 ký tự khác nhau ở đúng vị trí này — kiểu mới sê-ri hai chữ (29AA12345), kiểu cũ sê-ri chữ + số (29B112345, vẫn lưu hành) — nếu tách thành `DDLDDDDDD` và `DDLDDDDDD` thì ép kiểu tại chỉ số 3 là bắt buộc, và kết quả kiểm chứng bằng chạy thật: **một trong hai kiểu bị phá huỷ** (29AA12345 → 29A412345, hoặc 29B112345 → 29BL12345), trong khi `DDL?DDDDD` trả lại đúng cả hai. Chỉ số 3 của chuỗi 9 ký tự là **vị trí duy nhất trong toàn hệ thống biển số Việt Nam mà cả chữ lẫn số đều hợp lệ**; nhánh ? viết tường minh trong `apply_position_rules` chứ không rơi vào `else`. Chuỗi 8 ký tự không cần mặt nạ thay thế vì cả hai cách đọc áp cùng `DDLDDDDD`.

d) Hai bảng ánh xạ nhầm lẫn và tính không đối xứng. `TO_DIGIT` 12 mục (0→0, Q→0, D→0, I→1, J→1, L→1, Z→2, A→4, S→5, G→6, T→7, B→8), `TO_LETTER` 9 mục (0→D, 1→L, 2→Z, 3→B, 4→A, 5→S, 6→G, 7→T, 8→B); áp riêng tại vị trí D và L. **Ánh xạ không đối xứng, và đó là phát hiện trung tâm**: 0 → 0 đúng, nhưng 0 → 0 **không bao giờ đúng** vì 0 không phải chữ sê-ri hợp lệ — với cả 0 và Q bị loại, D là ứng viên đồng hình duy nhất còn lại, nên chiều đúng là 0 → D tại vị trí chữ; chữ R **tuyệt đối không được ánh xạ đi** vì hợp lệ ở vị trí thứ hai sê-ri xe máy (2.2.4). Quy tắc an

toàn: **ký tự không có mục trong bảng thì giữ nguyên. Ghi nhận trung thực về nguồn gốc:** hai bảng suy từ lập luận hình dạng ký tự, **không phải từ đo đạc**; vài cặp — đáng chú ý L → 1 — là phỏng đoán yếu; thay bằng bảng trích từ ma trận nhầm lẫn 36×36 đo được thuộc Chương 5, và trình bày bảng hiện tại như **giả thuyết cần kiểm chứng** vừa trung thực vừa mạnh hơn về học thuật.

e) Thuật toán chuẩn hoá. VietnamesePlateNormalizer.normalize_detailed thực hiện luồng sau:



Hình 4.5. Thuật toán chuẩn hoá chuỗi biến số theo bộ luật ràng buộc vị trí

Ba nguyên tắc chịu lực: **thử regex trước khi sửa** (chuỗi đã hợp lệ thì mọi chỉnh sửa chỉ có thể làm hỏng); **không bao giờ vứt bỏ** — chuỗi không sửa được vẫn trả về với is_valid_format=False và được lưu; **giữ chuỗi thô** vào raw_ocr_text. Kết quả là NormalizationOutcome bất biến mang chuỗi thô, chuỗi cuối, cờ hợp lệ, kết quả phân loại và **danh sách bộ ba (chỉ số, trước, sau) cho từng ký tự đã sửa** — dấu vết kiểm toán mà chương đánh giá dựa vào.

f) Xử lý nhập bằng số dòng. line_count == 1 **chứng minh** chuỗi là biển ô tô (xe máy luôn hai dòng), còn line_count == 2 **không chứng minh** gì vì biển ô tô ngắn cũng hai dòng — cờ is_ambiguous được giữ, KindDecision trả *tập ứng viên* kèm cờ thay vì bìa thông tin đầu vào không chứa. Thứ tự PATTERNS_BY_KIND: DIPLOMATIC đầu (hình dạng không thể nhầm); SPECIAL trước các mẫu xe máy (danh sách mã đóng và hiểm); MILITARY cuối vì là trường hợp **nhận-ra-để-loại-trừ** — khớp RE_MILITARY nhưng không thuộc CIVIL_KINDS nên không bao giờ được báo là biển dân sự hợp lệ.

4.6.6. Ghép thành ALPRPipeline bằng tiêm phụ thuộc

ALPRPipeline là **đối tượng tổ hợp**: không giữ mô hình, chỉ sắp thứ tự giai đoạn, cắt ảnh, **đo thời gian từng giai đoạn** và **cô lập lỗi mức từng biển** — không chứa logic học sâu nên kiểm thử được bằng thành phần giả lập; build_default_pipeline() import ba lớp cụ thể **trong thân hàm**. stage_times luôn đủ năm khoá ("detect", "crop", "ocr", "normalize", "total") — giai đoạn không chạy báo 0.0 thay vì vắng mặt; đây là thứ cho phép phân rã độ trễ (trên best.pt: OCR ~64,3%, detect ~34,2%) và cũng chính nó giúp phát hiện con số cũ "93,3%" là tạo tác của một lần đo trên hệ thống có lỗi crop. **Chính sách thất bại phân tầng**: ảnh không biển trả kết quả rỗng; OCR hỏng trên một biển thì biển đó recognition=None, biển khác vẫn xử lý; detector hỏng ném DetectionError; chuẩn hoá hỏng giữ nguyên kết quả thô và log. Cắt ảnh **kẹp lại lần hai** dù BaseDetector đã hứa — cắt là nơi duy nhất sai một đơn vị tạo mảng rỗng âm thầm; ảnh cắt là **bản sao**, không phải view,

vì view sẽ ghim cả khung video trong bộ nhớ. `normalize_detailed(raw, line_count=...)` không thuộc `BaseNormalizer` nên pipeline dò bằng `getattr` và lùi về `normalize(raw)` nếu không có.

4.6.7. Nhận dạng họ biển và màu nền — `plate_color.py`

a) Vấn đề đã quan sát: thông tin được tính ra rồi bị vứt đi. Ảnh biển đỏ quân đội KV6938 được OCR đọc **đúng** ở độ tin cậy 0,999 nhưng giao diện hiển thị "Sai định dạng biển số" — sai về phát biểu chứ không sai về tính toán: biển quân đội là biển hợp lệ nằm ngoài hệ dân sự (4.6.5f). Hai thông tin bị vứt trước khi tới CSDL: **họ biển** (`PlateKind`, chín giá trị từ car tới `military/unknown`) và **chuỗi hiển thị** do `format_for_display` dừng lại dấu phân cách (29E01566 → 29E-015.66). Bản sửa: **giữ lại** những gì đã tính (mục d, 4.7.2), và bổ sung nguồn bằng chứng mà chuỗi ký tự về nguyên tắc không thể mang — màu nền.

b) Vì sao màu là nguồn bằng chứng bổ trợ. Hai nguồn bù trừ: theo TT 79/2024, biển vàng xe kinh doanh vận tải mang **đúng cùng bố cục ký tự** với biển trắng xe cá nhân nên không regex nào phân biệt được; ngược lại biển ngoại giao nền trắng như biển cá nhân nên màu cũng không đủ. Chỉ *cặp* (chuỗi, màu) mới định danh được loại phương tiện.

c) Thiết kế `classify_plate_color`. Chuyển HSV, đếm tỷ lệ điểm ảnh theo dải màu, chọn dải lớn nhất. Ba quyết định: (1) *chỉ lấy mẫu vùng giữa* (`CENTRE_INSET = 0.18`) vì khung phát hiện hiếm khi ôm sát biển — ca được ngăn là xe sơn đỏ sau biển trắng thẳng phiếu bầu nếu lấy cả rìa; (2) *không loại trừ điểm ảnh ký tự* — ký tự chiếm thiếu số diện tích, thêm bước phân đoạn glyph là đưa vào một khâu mong manh hơn khâu nó bảo vệ; (3) *trả UNKNOWN thay vì đoán* (`MIN_DOMINANT_FRACTION = 0.30`) — gọi sai màu là khẳng định một loại phương tiện không chứng minh được, thừa nhận không đọc được chỉ là ghi nhận một giới hạn. `ColorEstimate` mang **tỷ lệ mọi dải, kể cả dải thua** phục vụ kiểm chứng, và hàm **không bao giờ ném ngoại lệ**.

d) Hợp nhất chuỗi và màu — kèm một ràng buộc an toàn. Chuỗi 80A12345 cho bốn ứng viên ngang nhau; `normalizer` chọn car — đúng đa số và **sai âm thầm với mọi xe cơ quan nhà nước** nền xanh. `refine_kind_with_color` giải quyết qua `_COLOR_PREFERRED_KINDS = {"blue": ("blue_car", "blue_motorcycle")}`. **Ràng buộc an toàn quan trọng hơn tác dụng của hàm:** màu chỉ được **nâng cấp một ứng viên mà chuỗi đã coi là hợp lý** — phán quyết gốc không nằm trong tập ứng viên thì trả nguyên, nên **màu không thể bịa ra một họ biển mà bộ luật ký tự đã bác bỏ**. Khi họ ưu tiên có cả biến thể ô tô lẫn xe máy thì chọn theo `line_count`; nếu mâu thuẫn thì trả phán quyết gốc — số dòng *đo được* từ hình học, màu *suy ra* từ thống kê điểm ảnh, bên đo được thắng. Chỉ màu **xanh** nằm trong bảng vì đó là màu duy nhất chuỗi bó tay hoàn toàn; vàng không đổi *họ* mà chỉ đổi *mục đích sử dụng* nên lưu như trường độc lập.

e) Độ chính xác đo được của bộ nhận màu. Đo trên `nguyenluanai/license-plate-color v4` (Roboflow, CC BY 4.0) — ảnh biển cắt sẵn, nhãn màu do người gán, và **bộ phân loại chưa từng được hiệu chỉnh theo bộ này** — phép đo **ngoài dữ liệu hiệu chỉnh**. Kết quả trên 1.565 ảnh dùng được (`docs/reports/19-color-accuracy.json`):

Bảng 4.4. Độ chính xác bộ nhận màu nền trên bộ dữ liệu ngoài hiệu chỉnh

Lớp nhãn người gán	Số ảnh	Đúng	Độ chính xác
Biển vàng	694	684	98,56%
Biển trắng	808	787	97,40%
Biển xanh	63	61	96,83%
Tổng	1.565	1.532	97,89%

Ba điều phải nói kèm: **542 ảnh bị loại** — toàn bộ lớp `bien_unknown`, ảnh đêm/hồng ngoại mà chính người gán nhãn cũng không đọc được màu, việc loại ghi tường minh trong tệp báo cáo; **dạng lỗi chủ đạo**: 21 ảnh trắng bị gọi thành xanh (2/3 của 33 ca sai) do một số điểm ảnh ám lạnh vượt cổng saturation; **phạm vi phép đo hẹp hơn phạm vi mô-đun**: bộ này **không chứa biển đỏ và biển ngoại giao** nên hai nhánh đó **chưa có số đo**, chỉ kiểm bằng ảnh lẻ và unit test — hạn chế thật, nêu lại ở 6.3.

Ghi chú phạm vi bắt buộc. Mọi ảnh bộ `license-plate-color` bị **kéo méo về 640×640** trước khi tải lên, nên bộ này **không dùng được để đánh giá OCR** (phép kéo phá hủy tỷ lệ khung mà `estimate_line_count` dựa vào); màu nền không bị ảnh hưởng, nên bộ chỉ được dùng cho đúng câu hỏi màu.

4.7. Backend và cơ sở dữ liệu

4.7.1. Cấu trúc phân tầng backend và tầng nghiệp vụ

Backend gồm 21 mô-đun Python (19 ứng dụng + 2 Alembic) trong năm tầng với **luồng phụ thuộc một chiều nghiêm ngặt**; `core/` được mọi tầng dùng nhưng không phụ thuộc tầng nào. Ba quy tắc: **router không viết truy vấn** — mọi truy cập qua repository, thay đổi lược đồ có bán kính ảnh hưởng một tệp; **repository flush, không bao giờ commit** — lưu một tác vụ cùng sáu biển là **một** thao tác logic, commit giữa chừng để lại trạng thái hỏng mà từng dòng riêng lẻ đều hợp lệ, ranh giới giao dịch thuộc tầng service; **khoá sắp xếp qua danh sách cho phép tường minh** — `getattr(model, name)` biến `?sort_by=metadata` thành lỗi 500, ánh xạ tường minh biến khoá lạ thành 400 sạch sẽ; `MAX_PAGE_SIZE = 200`.

Bốn service: `DetectionService` (điều phối nhận dạng, video nền, gộp trùng, vòng đời tác vụ), `HistoryService` (truy vấn, lọc, xoá kèm tệp, xuất), `StatisticsService` (chỉ số cho GET `/api/statistics`), `StorageService` (lưu/đọc/xoá tệp, tên UUID, ánh xạ URL). **Kho tệp tách khỏi CSDL**: ảnh video nằm trên hệ thống tệp, CSDL giữ đường dẫn — BLOB làm SQLite phình nhanh và chậm mọi truy vấn; giá phải trả là đồng bộ tệp–bản ghi (FR-5.1, FR-5.3). Tên tệp UUID đáp ứng NFR-S2 chống path traversal.

Hợp đồng pipeline biểu diễn bằng typing.Protocol, không phải ABC: ABC phải nằm trong backend và bị `ALPRPipeline` kế thừa — tức ai import từ backend, đúng chiều NFR-M1 cấm; Protocol là cấu trúc nên `ALPRPipeline` thoả mãn nhờ có đúng ba thành viên `name`, `is_ready`, `process(image) → PipelineResult` mà không biết tệp này tồn tại. Ba cài đặt

thoả giao thức: ALPRPipeline, UnavailablePipeline, StubPipeline (4.7.4). **Điểm tiêm ở api/deps.py**: pipeline dựng **một lần** lúc khởi động, gán app.state; kiểm thử chỉ cần app.dependency_overrides; chuyển stub → pipeline thật đã diễn ra **không sửa dòng nào ở tầng nghiệp vụ**. DetectionService._run_pipeline là điểm dịch ngoại lệ: ALPRError mang thông điệp tiếng Anh, không biết HTTP — để thoát ra sẽ rõ rĩ hoặc thành 500 kèm stack trace.

Ba luồng nghiệp vụ. detect_image đồng bộ, ảnh không biến trả **200 danh sách rỗng**. detect_frame coi một **phiên** webcam là một tác vụ, khung hình không lưu đĩa. create_video_job + process_video_job bất đồng bộ, trả 202 ngay. Chi tiết luồng video: lấy mẫu theo frame_stride (mặc định 5); **khử trùng lặp trước khi ghi**, khoá là chuỗi đã nhận dạng, biến không đọc được khoá theo vị trí lượng tử hoá lưới thô (unread@. . .) để biến đứng yên co về một dòng; ghi tiến độ mỗi 10 khung (commit mỗi khung tạo hàng nghìn giao dịch, commit chỉ ở cuối làm thanh tiến độ đứng ở 0); đọc kích thước khung **trước** capture.release() (sau đó trả 0 và mọi hộp bao frontend sụp về không); tiến độ khi không biết tổng khung trả **0,99** thay vì 1,0 (báo 1,0 sớm khiến client ngừng hỏi); kiểm tra huỷ bằng db.refresh(job, . . .) bên trong vòng lặp để lệnh huỷ (đến trên phiên khác) có hiệu lực trong vài khung.

4.7.2. Cơ sở dữ liệu: lược đồ, di trú và các quyết định thiết kế dữ liệu

Cơ sở dữ liệu gồm hai bảng quan hệ một-nhiều: detection_job (một lần sử dụng hệ thống) và detection_history (mỗi biến số phát hiện được một bản ghi), nối bằng khoá source_job_id. Tách hai bảng là điều kiện để thống kê đếm đúng — *lượt nhận dạng* và *biến số phát hiện* là hai đại lượng khác nhau. detection_history hiện có **21 cột** sau ba lần di trú Alembic, trong đó hai quyết định đáng chú ý: lưu **song song** raw_ocr_text và plate_number để đo được đóng góp của khối hậu xử lý, và cột upper_char_count để giải nhập nhằng cách nhóm chữ số của biến hai dòng. Đặc tả từng trường, ràng buộc và chỉ mục ở **Phụ lục H.4**.

4.7.3. REST API

API kiểu REST, tự sinh OpenAPI 3.x và Swagger UI. Endpoint nghiệp vụ dưới tiền tố /api; health check đặt ở gốc để giám sát và Docker healthcheck không phụ thuộc phiên bản API. Đếm từ backend/api/routes/ đối chiếu OpenAPI: **10 thao tác HTTP trên 9 đường dẫn** (/api/history/{detection_id} mang cả GET và DELETE); /docs, /redoc, /openapi.json do FastAPI tự sinh, không tính. Bảng đặc tả đầy đủ ở **Phụ lục F.1**. Tóm tắt: GET /health (200); POST /api/detect/image (200); POST /api/detect/video (**202**); POST /api/detect/frame (200, kèm job_id tùy chọn); GET /api/jobs/{job_id} (200/404); GET /api/history (200, các tham số lọc – sắp xếp – phân trang); GET /api/history/export (200 text/csv, UTF-8 **có BOM**, không phân trang); GET /api/history/{detection_id} (200/404/422); DELETE /api/history/{detection_id} (**204**); GET /api/statistics (200, tham số days). Lỗi chung: 400, 413, 422, 500.

Các quyết định thiết kế API. 202 cho video: video 60 giây mất ~200 giây trên CPU, không client nào chờ — 202 đúng ngữ nghĩa "đã tiếp nhận". 200 với danh sách rỗng: kết quả nhận

dạng tồn tại và là tập rỗng (NFR-R2); trả 4xx sẽ xóa mọi trường hợp âm khỏi thống kê. 204 cho xóa. *Tìm kiếm khớp cả plate_number lẫn raw_ocr_text*: người dùng nhớ chuỗi nào cũng tìm ra. *Chuỗi thời gian trả cả ngày không có dữ liệu*: bỏ qua thì biểu đồ âm thầm nối liền khoảng trống. *Hai trường thống kê tách biệt*: total_jobs và total_detections, mô tả OpenAPI nêu rõ "An image containing three vehicles is one job and three detections." *Trạng thái degraded*: pipeline chưa nạp trọng số thật thì báo healthy là gây hiểu lầm nghiêm trọng; hiện /health trả healthy, model_loaded = true. **Trạng thái kiểm chứng**: cả 10 endpoint **đã xác minh bằng lời gọi HTTP thực tế** với kiểu TypeScript khớp từng trường (trước hai đợt thu gọn giao diện; hợp đồng không đổi kể từ đó). Việc xác minh chứng minh **hợp đồng API đúng, không** chứng minh chất lượng nhận dạng — việc đó thuộc Chương 5.

4.7.4. UnavailablePipeline — một phương án lùi phải thất bại theo cách quan sát được

Giai đoạn chưa có mô hình, hệ thống chạy StubPipeline — bịa kết quả có cấu trúc hợp lệ, chính đáng lúc đó để xây API/CSDL/frontend. Vấn đề: **stub từng được cài làm phương án lùi khi không nạp được mô hình** — một triển khai cấu hình sai sẽ trả lời mọi lần tải lên bằng một biến số thuyết phục nhưng hư cấu: chế độ hỏng **trông giống thành công**, loại nguy hiểm nhất trong hệ thống có ghi CSDL. Ba lớp nay phân vai rõ: ALPRPipeline — nhận dạng thật, is_ready = True, /health ok; UnavailablePipeline — **mặc định khi hỏng**, ném ALPRError và **không bịa gì**, /health degraded; StubPipeline — **chỉ chạy khi ALPR_USE_STUB đặt tường minh**, /health degraded. Trọng số thiếu thì **dịch vụ vẫn khởi động** (tiến trình từ chối khởi động không nói được vì sao), mỗi yêu cầu trả lỗi sạch sẽ; build_pipeline log WARNING: "Every result this process returns is invented."

4.7.5. Xử lý lỗi, log có cấu trúc và request_id

Mỗi ngoại lệ mang **hai mô tả cho hai độc giả**: user_message tiếng Việt ngắn gọn có hành động, đi vào thân HTTP; internal_detail tiếng Anh kỹ thuật, chỉ đi vào log. Cây ngoại lệ APIError ánh xạ thẳng sang mã HTTP (400/404/413/415/500), và **bốn bộ xử lý được đăng ký** — trong đó một bộ **bắt tất cả** cho Exception, không có nó thì ngoại lệ ngoài dự kiến ở cấu hình debug sẽ hiển thị cả stack trace (NFR-S4). Log ghi **mỗi dòng một đối tượng JSON** kèm request_id truyền ngầm qua ContextVar — log video xen kẽ log tải lên đồng thời, văn bản thuần không tách lại được. Cây ngoại lệ đầy đủ, cơ chế safe_extra() và cấu hình middleware ở **Phụ lục I.1**.

4.7.6. Ba lỗi thực tế đã gặp và sửa trong quá trình cài đặt

Ba lỗi đáng ghi nhận đã gặp khi cài đặt: pydantic-settings JSON-decode trường danh sách **trước** validator khiến dịch vụ sập lúc khởi động; SQLite âm thầm nuốt tzinfo khiến mọi phân tích theo thời gian sai lệch mà không gì trông sai; và log tiếng Việt làm sập console cp1252 trên Windows — sự cố xảy ra *bên trong* cỗ máy logging, đúng lúc log quan trọng nhất. **Cả ba đều đi qua được kiểm thử đơn vị**, vì cả ba nằm ở ranh giới mã–môi trường (nguồn cấu hình, tầng lưu trữ, bảng mã đầu ra) — lập luận cụ thể cho việc bộ kiểm thử phải gồm kiểm thử tích hợp chạy trên đường dẫn thật. Cơ chế và cách sửa từng lỗi ở **Phụ lục I.2**.

4.8. Giao diện người dùng

4.8.1. Cấu trúc, điều hướng và các màn hình

Giao diện là SPA React + TypeScript dựng bằng Vite, gồm **3 trang** và 48 mô-đun .tsx/.ts: pages/ (ImageDetection ở trang chủ /, VideoDetection /video, History /history); components/ui/ 15 component nguyên thủy; các nhóm component detection/history; services/api.ts, types/index.ts, hooks/, lib/. Điều hướng cố ý **phẳng**: ba màn hình truy cập trực tiếp từ thanh điều hướng; chi tiết bản ghi và xác nhận xoá là hộp thoại chồng lên trang lịch sử để không mất ngữ cảnh bộ lọc.

Ghi chú thay đổi phạm vi 2026-07-20 — hai đợt trong cùng một ngày. Thiết kế ban đầu có **năm màn hình**, Dashboard là trang chủ. **Đợt 1** gỡ trang webcam (pages/WebcamDetection.tsx, components/detection/webcam/, hàm detectFrame), chuyển trang chủ sang Nhận dạng ảnh; POST /api/detect/frame **không đổi** (endpoint, test, benchmark). **Đợt 2** gỡ pages/Dashboard.tsx, components/dashboard/ (10 tệp), hooks/useApi.ts, getStatistics/getHealth và gói recharts; GET /api/statistics và GET /health **vẫn có kiểm thử tích hợp**. Số mô-đun giảm 60 → **48**; mã hai trang còn trong lịch sử git. Hệ quả yêu cầu — FR-3.1/FR-3.4 và **FR-4.1 (Must)**/FR-4.2 → Won't (4.1.3, 6.2). Các kiểu Statistics, HealthStatus... **giữ lại có chủ đích** vì là bản sao hợp đồng của hai endpoint vẫn phục vụ.

Ba màn hình. *Nhận dạng ảnh* (trang chủ): hai cột — tải ảnh kéo-thả kèm xem trước; ảnh đã vẽ bounding box, thẻ kết quả và tóm tắt. Mỗi thẻ hiển thị chuỗi đã chuẩn hoá cỡ lớn, chuỗi thô nhỏ hơn khi khác nhau, hai thanh độ tin cậy riêng, nhãn số dòng và cờ hợp lệ. *Video*: ba giai đoạn đúng bản chất bất đồng bộ — tải tệp; bảng tiến độ (phần trăm, số khung, trạng thái); bảng kết quả (biển đã gộp trùng, liên kết tải về). *Lịch sử*: bảng phân trang sắp xếp theo cột, thanh bộ lọc, nút xuất, hộp thoại chi tiết — vẽ lại bounding box **không cần chạy lại mô hình** nhờ bốn cột tọa độ trong CSDL. Bộ ui/ có hai nguyên thủy mã hoá tri thức miền: PlateChip dùng phong đơn cách khoảng cách chữ mở rộng để 0/0 phân biệt bằng mắt, ConfidenceBar kèm nhãn ngưỡng thay vì con số trần.

Trạng thái kiểm chứng (đo lại 2026-07-20 sau đợt gỡ thứ hai): tsc --noEmit sạch, ESLint sạch, vite build 2,15 giây với **1.670 mô-đun** (từ 2.381); gói tải về giảm ~730 KB → **328,8 KB (-55%)**, phần lớn nhờ gỡ recharts.

4.8.2. Tầng gọi API và ánh xạ kiểu dữ liệu

services/api.ts là **nơi duy nhất frontend biết về axios hoặc mã HTTP**: component nhận dữ liệu đã có kiểu hoặc ApiError chuẩn hoá. Sáu hàm gọi API ứng một-một với sáu endpoint, cộng hai hàm dựng URL. **Ba endpoint còn lại không còn hàm gọi phía giao diện nhưng vẫn hoạt động ở backend** — cần phân biệt *hàm gọi bị xoá với endpoint thì không*. Không hostname nào viết cứng: origin đọc từ biến môi trường lúc build, mặc định rỗng (cùng-origin). Chi tiết ánh xạ kiểu ở **Phụ lục I.3**.

4.8.3. Nguyên tắc trải nghiệm người dùng

Bốn trạng thái bắt buộc của mọi thành phần hiển thị dữ liệu: *đang tải* (thiếu thì giao diện trông như treo, người dùng bấm lại tạo thêm tải); *có dữ liệu*; *rỗng* (thiếu thì màn hình trắng không phân biệt được với lỗi); *lỗi* (tiếng Việt, nêu nguyên nhân và cách khắc phục, có thử lại). Trạng thái rỗng xuất hiện với **ba nghĩa cần ba thông điệp**: chưa có lượt nhận dạng nào; bộ lọc không khớp; ảnh không chứa biển số — nghĩa thứ ba là biểu hiện giao diện của cùng quyết định ở tầng API (200 danh sách rỗng) và tầng pipeline: **không tìm thấy không phải là lỗi**.

Thông báo lỗi tiếng Việt (NFR-U3, FR-6.3) gồm ba phần — chuyện gì xảy ra, vì sao, người dùng làm gì được; ví dụ 400 → "Tập bạn chọn không phải là ảnh hợp lệ. Hệ thống chỉ nhận JPG, PNG, WebP và BMP." Chi tiết kỹ thuật **không bị vứt mà được chuyển hướng** vào log có cấu trúc phía máy chủ (FR-6.2). NFR-U1 (≤ 3 nhấp chuột) định hình bố cục: khu tải ảnh trung tâm, kéo-thả, không bước cấu hình bắt buộc; tương phản đạt WCAG AA, độ tin cậy biểu diễn bằng thanh kèm số chữ không chỉ màu, hoạt động từ 1366×768.

Hiển thị raw_ocr_text cạnh plate_number khi hai chuỗi khác nhau.

PlateResultCard.tsx tính showRawComparison = wasCorrected(...); **chỉ khi** hai chuỗi khác nhau mới hiện dòng "Hậu xử lý đã sửa:" với chuỗi thô gạch ngang, mũi tên, chuỗi chuẩn hoá; lặp lại ở HistoryDetailModal.tsx. Nó biến một cột CSDL phục vụ nghiên cứu thành **bằng chứng nhìn thấy được ngay lúc trình diễn**, và vì chỉ hiện khi có thay đổi, giao diện không lộn xộn bởi đa số trường hợp hậu xử lý không can thiệp.

Phân biệt "lượt nhận dạng" và "biển số phát hiện" — điểm dễ hiểu sai nhất, nhất quán ba tầng: CSDL (hai bảng nối bằng source_job_id), API (total_jobs / total_detections), giao diện. Gộp hai khái niệm thì "lượt sử dụng" bị thổi phồng đúng bằng số biển trung bình mỗi ảnh. Sau khi trang Tổng quan bị gỡ, gánh nặng giải thích chuyển sang mô tả trường trong OpenAPI; StatisticsService giữ nguyên phân biệt trong mọi phép tính.

4.8.4. Hàng đợi một khe ở client thời gian thực (trang webcam đã gỡ 2026-07-20)

Trang webcam đã gỡ khỏi frontend, nhưng lập luận thiết kế của nó vẫn đúng và trở thành **khuyến nghị bắt buộc cho bất kỳ client nào** gọi POST /api/detect/frame. Suy luận CPU chỉ ~5 FPS, nên một bộ đếm giờ ngẫu thơ sẽ khởi động yêu cầu thứ hai trước khi yêu cầu thứ nhất trở về — tồn đọng chỉ tăng và tab đứng hình. Giải pháp là **giữ đúng một yêu cầu đang bay**; khung tới trong lúc khe bận thì bị **bỏ qua chứ không xếp hàng**: bỏ một khung không tốn gì vì khung sau cập nhật hơn, xếp hàng thì tốn tất cả. Cài đặt đầy đủ — vị trí giải phóng khe, tự tạm dừng sau 5 lỗi liên tiếp, AbortController, một job_id cho cả phiên, và khoá khử trùng — ở **Phụ lục I.4**.

4.9. Triển khai bằng Docker

Đóng gói phục vụ NFR-C1: môi trường chạy tái lập được, không phụ thuộc máy cá nhân. Dockerfile.backend build hai giai đoạn, cài **hai tệp requirements thành hai lớp riêng** để

thay đổi một tầng không mất bộ đệm tầng kia (hệ quả trực tiếp của 4.3.2), chạy dưới người dùng không đặc quyền, đặt `OMP_NUM_THREADS` tường minh để hai container không cạnh tranh nhân CPU đến mức cùng chậm, và `HEALTHCHECK` có `start-period` đủ dài cho việc nạp trọng số. `Dockerfile.frontend` build rồi phục vụ tĩnh bằng `nginx:alpine` — ảnh chạy không chứa Node hay mã nguồn. **Trọng số mô hình không nằm trong ảnh Docker** mà gắn từ ngoài, cùng một volume riêng cho bộ đệm mô hình PaddleOCR — không có volume này thì mỗi lần `down` && `up` phải tải lại vài trăm MB và không có mạng thì container không khởi động được. Nội dung ba tệp và bảng biến môi trường ở **Phụ lục I.5** và **F.2. Trạng thái kiểm chứng**: `docker compose config` hợp lệ; đo hiệu năng trong container thuộc Chương 5.

4.10. Những chỗ cài đặt lệch khỏi thiết kế, và lý do

Nguyên tắc: **mọi điểm lệch đều được nêu, kể cả những điểm chưa được giải quyết.**

Bảng 4.5. Tổng hợp chín điểm lệch giữa thiết kế và cài đặt

#	Thiết kế	Cài đặt thực tế	Loại lệch	Trạng thái
1	StubPipeline là phương án lùi khi thiếu mô hình	UnavailablePipeline là phương án lùi; stub chỉ chạy khi opt-in tường minh	Cải tiến so với thiết kế	Đã giải quyết
2	Một môi trường ảo Python	Ba môi trường ảo tách biệt	Bắt buộc bởi xung đột phụ thuộc	Đã giải quyết
3	FR-2.4: có nút huỷ tác vụ video	Nút hiện diện nhưng bị vô hiệu hoá ; không có endpoint huỷ	Đạt một phần	⚠ Chưa xong
4	Mô hình chính thức <code>imgsz=640</code> trên split sạch	Đã có <code>models/best.pt</code> (<code>imgsz=640</code> , <code>split v3</code> , <code>mAP@0.5 0,9829</code>)	Đúng thiết kế	✅ Đã giải quyết
5	NFR-P1: độ trễ E2E $p95 \leq 800$ ms	Đo được 1.143,10 ms — dưới sàn 1.500 ms nhưng vượt mục tiêu 800 ms	🟡 Chỉ đạt sàn	⚠ Chưa đạt mục tiêu
6	Video job xuất video đã chú thích (<code>output_path</code>)	Chưa cài đặt; chỉ trả về các dòng lịch sử	Hoãn có lý do	⚠ Chưa xong
7	Bật oneDNN để tăng tốc CPU	Buộc phải tắt do lỗi thư viện	Bắt buộc bởi lỗi thượng nguồn	Đã ghi nhận
8	Khử rò rỉ bằng phash	Còn rò rỉ tồn dư không khử được bằng phash	Giới hạn phương pháp	Đã ghi nhận
9	Bộ đo độ chính xác	Bộ đo gọi thẳng recognizer +	Lỗi phương	✅ Đã

#	Thiết kế	Cài đặt thực tế	Loại lệch	Trạng thái
	OCR đo hệ thống đang giao	normalizer, bỏ qua tầng điều phối	pháp đo	phát hiện và sửa

(1) là điểm lệch duy nhất cài đặt **tốt hơn** thiết kế: stub là lưới an toàn *sai loại* vì biến triển khai hỏng thành triển khai trông như chạy tốt (4.7.4) — bài học: **một phương án lùi phải thất bại theo cách quan sát được**. (2) bắt buộc bởi ngoại cảnh (4.3.2): chi phí là thiết lập phức tạp hơn, lợi ích là kết quả đo tái lập được. (3) cơ chế huỷ **đã hoạt động ở backend** (vòng lặp video kiểm tra mỗi 10 khung, JobStatus.CANCELLED hợp lệ) nhưng **không route HTTP nào đặt được trạng thái đó**; chọn để nút vô hiệu hoá kèm chú thích thay vì nối vào một endpoint bịa sẽ 404 và khiến người dùng tin tức vụ đã dừng trong khi nó vẫn chạy.

(4) models/best.pt đạt cả bốn chỉ tiêu detection trên tập test v3 đã khử trùng lặp — số liệu ở mục 5.4.1. baseline-416-v1.pt ([mAP@0.5](#) 0,9933) giữ làm đối chứng, **không nằm trên đường chạy chính** và số liệu **không được báo cáo là "đạt"** vì sai độ phân giải (416 so với chỉ tiêu 640) và split v1 có rò rỉ train↔test đã đo được (619 cặp $d \leq 10$). (5) p95 **1.143,10 ms** nhưng trung vị chỉ **405,77 ms** — chênh lệch do bậc thang thử-lại chỉ chạy khi lần đọc đầu thất bại (5.5.7, 5.6.1); phân rã OCR ~64,3% / detect ~34,0%. Con số **5.857,19 ms** trong bản nháp **đã bị bác bỏ**: nó đo khi một tiến trình huấn luyện chiếm ~793% CPU song song, trên checkpoint epoch 7, và trên hệ thống có lỗi crop khiến PaddleOCR đọc ảnh quá lớn.

(6) output_path tồn tại trong lược đồ nhưng chưa điền: lúc viết đoạn mã đó hệ thống còn chạy stub, và chú thích các hộp bao **bịa ra** lên video thật sẽ tạo hiện vật trông thuyết phục nhưng sai sự thật. (7) oneDNN tắt do lỗi PIR của PaddlePaddle 3.3.1 (4.6.3), ghi thành hằng số có tài liệu để lật lại khi lỗi được sửa. (8) phash tóm tắt bố cục khung ảnh, không tóm tắt chiếc xe (4.4.3) — giới hạn phương pháp, không phải lỗi cài đặt. (9) là điểm lệch **thuộc về phép đo sản phẩm**, nguy hiểm hơn tám điểm trên vì nó không làm hệ thống chạy sai mà làm *các con số công bố* mô tả một thứ khác; đã sửa bằng hai hàm tự do cấp mô-đun (4.6.4g).

Bản chất các điểm lệch: 1 cải tiến (#1); 3 bị ngoại cảnh cưỡng bức (#2, #7, #8); 4 chưa hoàn thành hoặc chưa đạt chỉ tiêu (#3, #4, #5, #6); 1 lỗi ở phương pháp đo (#9). Không điểm nào phát sinh từ sai lầm trong bản thân thiết kế kiến trúc; ba điểm do ngoại cảnh còn là bằng chứng gián tiếp cho giá trị thiết kế — nhờ BaseRecognizer, vấn đề PaddleOCR chỉ ảnh hưởng một tệp. Điểm #9 cần đọc như cảnh báo chứ không như mục đã đóng: **ranh giới giữa "hệ thống" và "phép đo hệ thống" cũng là một ranh giới kiến trúc**, và ranh giới đó không được test nào ở 4.2.3 canh giữ.

4.11. Kết luận chương

Phân tích và kiến trúc. Ba tác nhân, chín use case, **34 yêu cầu chức năng** (21 Must · 6 Should · 3 Could · 4 Won't) kèm tiêu chí chấp nhận kiểm chứng được; yêu cầu phi chức năng đặt ở dạng chỉ tiêu định lượng với nguyên tắc **mọi chỉ tiêu hiệu năng đo trên CPU** — ràng buộc sinh trực tiếp hai quyết định kiến trúc là xử lý video bất đồng bộ và bỏ khung có

kiểm soát. Quyết định quan trọng nhất là **tách hoàn toàn tầng AI khỏi tầng API**, kiểm chứng tự động bằng hai công cụ bổ trợ nên không suy thoái theo thời gian; nhờ nó toàn bộ phần mềm được xây và chạy với pipeline mô phỏng **trước khi mô hình được huấn luyện**, và việc thay pipeline chỉ là một thay đổi trong backend/main.py.

Khối lượng và trạng thái. Tầng AI 12 mô-đun / 4.852 dòng; backend 21 mô-đun, 10 endpoint REST, hai bảng 21 và 11 cột; frontend 3 trang / 48 mô-đun; đường ống dữ liệu sáu bước; Docker hai dịch vụ. Kiểm chứng bằng chạy thật: /health trả model_loaded=true, 10/10 ảnh test nhận dạng được, gói tải về **328,8 KB** (−55%), bộ kiểm thử **1.001 thu thập / 1.000 đạt / 1 xfail / 0 fail**, bao phủ tầng nghiệp vụ 87,7%.

Năm khối là đóng góp kỹ thuật của đề án: two_line.py cắt có chồng lẫn rồi ghép ngang (4.6.4); **bộ luật hậu xử lý** với hai phát hiện trung tâm là ký tự đại diện ? tại chỉ số 3 và tính **không đối xứng** của bảng ánh xạ nhầm lẫn (4.6.5); đường ống khử trùng lặp băm đa chỉ mục kèm bài học *perceptual hash* tóm tắt bố cục khung ảnh chứ không tóm tắt chiếc xe (4.4.2–4.4.3); plate_color.py cùng phép hợp nhất chuỗi–màu, trong đó ràng buộc an toàn còn đáng giá hơn con số 97,89% (4.6.7); và **bước cứu dòng trên**, đáng ghi nhận vì đường đi tới nó — giả thuyết đầu bị chính phép đo bác bỏ — hơn là vì mức cải thiện. Năm quyết định thiết kế dữ liệu đi kèm đều nhằm **bảo vệ tính đúng đắn của số liệu sẽ công bố**; mỗi quyết định, nếu bỏ qua, đều dẫn tới một con số sai mà không có gì báo hiệu.

Chưa hoàn thành: video job chưa xuất video đã chú thích; bộ dữ liệu còn rò rỉ tồn dư; hai nhánh biến đỏ và ngoại giao của bộ nhận màu chưa có số đo. models/best.pt đã hoàn tất và đạt cả bốn chỉ tiêu phát hiện; nút thắt còn lại là **độ chính xác nhận dạng biển hai dòng**, trình bày đầy đủ ở Chương 5.

CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương 5 trình bày hệ thống đã được xây dựng như thế nào. Chương này trả lời hai câu hỏi trọng số ngang nhau: hệ thống đó **hoạt động tốt đến mức nào**, và **những con số đó có đáng tin không** — nên mỗi con số đều đi kèm ngữ cảnh đo của nó.

5.1. Mục tiêu và phương pháp đánh giá

5.1.1. Các câu hỏi nghiên cứu mà chương này trả lời

Sáu câu hỏi cụ thể hoá từ docs/00-requirements/non-functional-requirements.md.

RQ1 — YOLO11n có đạt chỉ tiêu định vị biển số Việt Nam không (5.5; NFR-A1...A3; 5.4)?

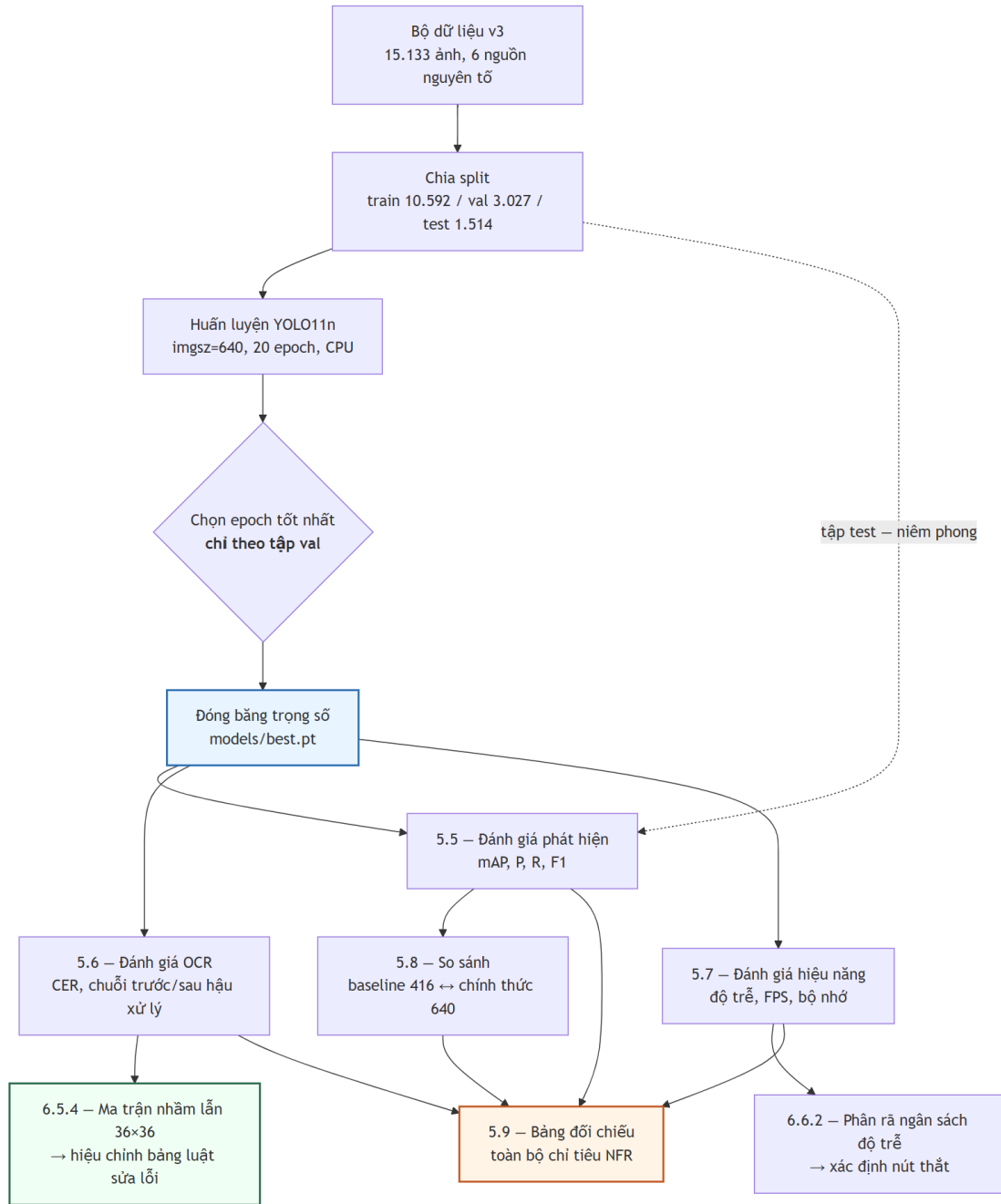
RQ2 — độ chính xác nhận dạng chên bao nhiêu giữa biển một dòng và hai dòng (NFR-A8; 5.4.2, 5.5.3)? **RQ3** — khối hậu xử lý theo luật đóng góp bao nhiêu điểm phần trăm vào độ chính xác chuỗi đầy đủ (NFR-A5 ↔ A6; 5.5.2)? **RQ4** — có đạt chỉ tiêu độ trễ trên phần cứng CPU-only không, nút thắt ở đâu (5.7; NFR-P1...P7; 5.6)? **RQ5** — bảng luật sửa lỗi ký tự, vốn suy từ hình dạng chữ chứ không từ đo đạc, có khớp các cặp thực sự bị nhầm không (5.5.4; VNPLATE §9.8)? **RQ6** — số liệu chịu mỗi đe dọa nào đến tính hợp lệ (5.9.3)? RQ3 và RQ5 mang đóng góp học thuật riêng — một lượng hoá khối chức năng mà tài liệu ALPR chỉ mô tả định tính, một thay tri thức suy đoán bằng tri thức đo được; RQ6 quyết định giá trị của năm câu còn lại.

5.1.2. Hai nguyên tắc trình bày bắt buộc

Một — mọi số hiệu năng phải kèm cấu hình phần cứng: đề án suy luận hoàn toàn trên CPU nên so với các con số FPS đo trên GPU là không hợp lệ nếu không ghi rõ; cấu hình ở 5.2 là điều kiện diễn giải cho toàn mục 5.6. **Hai** — mọi số độ chính xác phải kèm tên tập dữ liệu và số mẫu, vì độ chính xác là thuộc tính của cặp (mô hình, tập đánh giá); hệ quả: NFR-A4...A7 chỉ đo được trên tập con có nhãn chuỗi, nhỏ hơn nhiều tập test phát hiện, mẫu số đó không được giấu. **Ký hiệu:**  đạt mục tiêu ·  chỉ đạt ngưỡng tối thiểu ·  không đạt ·  chưa đo ·  không áp dụng.

Cảnh báo trích dẫn. Cặp số **94,3%** (biển một dòng) ↔ **45,7%** (biển hai dòng), chên **48,6 điểm phần trăm**, đo trên bộ dữ liệu **RodoSol-ALPR của Brazil** [7]. Đây **không phải** số liệu Việt Nam; mỗi lần dẫn, tên bộ dữ liệu và quốc gia phải xuất hiện **ngay trong câu**, và nó chỉ dùng như *analogue định lượng* về độ khó tương đối của biển hai dòng, không bao giờ như mốc chuẩn phải vượt.

5.1.3. Giao thức đo



Hình 5.1. Giao thức đo và ràng buộc phụ thuộc giữa các bước đánh giá.

Không bước đo nào chạy trước khi trọng số được đóng băng; tập test không được chạm vào trong huấn luyện lẫn chọn epoch — chọn epoch chỉ dựa vào validation. Ba quy ước: **kích thước lô = 1 khi đo độ trễ** (riêng mAP dùng lô lớn hơn vì không phụ thuộc kích thước lô); **bỏ 3 lượt khởi động nóng**; **báo cáo p50/p95/p99, không báo cáo trung bình**, vì trung bình che đuôi phân bố còn NFR-P1 phát biểu ở p95.

5.2. Môi trường thực nghiệm

Toàn bộ số liệu đo trên **một máy trạm cá nhân duy nhất** (docs/00-requirements/environment.md): **Windows 11 Pro 10.0.26200, Python 3.13.12**, CPU Intel **Raptor Lake** (Family 6, Model 183) — **14 nhân vật lý / 20 nhân logic, không có GPU CUDA** nên mọi suy luận và huấn luyện chạy trên CPU (device=cpu); chế độ đo **lô = 1, bỏ 3 lượt khởi động nóng**. Đây là **tiền tố ngầm định của mọi con số hiệu năng ở 5.6**.

Phiên bản thư viện trích từ pip freeze đúng thời điểm chạy phép đo cuối cùng (2026-07-20), không chép từ requirements.txt (tệp yêu cầu ghi *ràng buộc*, không ghi *phiên bản đã cài*); một môi trường ảo hợp nhất backend/.venv: ultralytics 8.4.101 · torch 2.13.0+cpu · torchvision 0.28.0+cpu · paddleocr 3.7.0 (PP-OCRv5 mobile) · paddlepaddle 3.3.1 · onnxruntime 1.27.0 · opencv-python 4.10.0.84 · numpy 2.4.5 · fastapi 0.139.2 · uvicorn 0.51.0 · sqlalchemy 2.0.51 · imagehash 4.7.2 · pytest 9.1.1.

5.2.1. Vì sao ràng buộc CPU-only là ràng buộc thiết kế, không phải hạn chế tạm thời

Lập luận đầy đủ ở **4.3.1**. NFR-P1 phát biểu *kèm* ràng buộc CPU, nên kết luận "đạt sàn, không đạt mục tiêu" ở 5.6 là kết luận về hệ thống trong đúng bối cảnh vận hành thật, không phải con số chờ nâng cấp phần cứng. Cấu hình mô hình cũng do ràng buộc phần cứng quyết định — **YOLO11n** (2.590.035 tham số) [16] và **PP-OCRv5 mobile** [118]. Về quy mô: **35,6 phút mỗi epoch**, một lượt 20 epoch mất khoảng **12 giờ** liên tục, khiến **tìm kiếm siêu tham số bất khả thi**; chương này báo cáo *một* cấu hình huấn luyện, không phải kết quả của một quá trình tối ưu — giới hạn thật, ghi ở 5.9.3.

5.3. Bộ dữ liệu thực nghiệm

Lưu ý về cách đặt tên. Thư mục yolo_v2/ và yolo_v3/ chỉ **phiên bản của bộ dữ liệu**, **không** liên quan đến kiến trúc YOLOv2 hay YOLOv3. Mô hình dùng trong toàn đề án là **YOLO11n**.

Bảng 5.1. Ba phiên bản bộ dữ liệu và số cặp ảnh gần trùng xuyên split theo ngưỡng Hamming

Thuộc tính / ngưỡng	v1	v2	v3
Tổng số ảnh	4.578	15.133	15.133
Ngưỡng Hamming gộp trùng lặp	5	5	10
Số ảnh train / val / test	—	—	10.592 / 3.027 / 1.514
Dùng cho	baseline-416-	bị loại	best.pt

Thuộc tính / ngưỡng	v1	v2	v3
	v1.pt	bỏ	
Cặp gần trùng xuyên split, Hamming 0	—	—	0 (thông tin mới)
Hamming 5	—	—	0 (= ngưỡng gộp v1, v2 — không mang thông tin mới)
Hamming 10	619	2.699	0 (= ngưỡng gộp v3 — không mang thông tin mới)
Hamming 12 · 15	—	—	791 · 3.529 (thông tin mới)
Hamming 20	—	—	137.506 (ngưỡng đã lỏng tới mức nhiều cặp là dương tính giả)

Mẫu số phần rò rỉ: $10.592 \times 1.514 = \mathbf{16.036.288}$ cặp. Khoảng cách Hamming **nhỏ nhất quan sát được là 12** — giá trị chẵn kế tiếp sau ngưỡng gộp 10, một tất yếu toán học (mọi mã băm đều có đúng 32 bit 1 nên khoảng cách luôn chẵn), **không phải dấu vết rò rỉ bị cắt cụt tại ngưỡng**; kiểm chứng (15.133/15.133 mã băm popcount chẵn; 4.498.500/4.498.500 cặp lấy mẫu có khoảng cách chẵn) ở 02-dataset-report.md [mục 6bis.1](#). Cột v1/v2 chỉ có số ở ngưỡng 10 vì đó là con số đã đo trước đó; ô trống **không được suy ra**.

v1 quá nhỏ và chỉ một nguồn (1 bộ vào hợp nhất, 1 nguồn nguyên tố) — động cơ tải thêm **tám bộ** (tổng **9 bộ**), trong đó **sáu bộ** vào hợp nhất detection cùng bộ gốc (v2, v3: **7 bộ vào hợp nhất, 6 nguồn nguyên tố**), hai bộ nhãn mức ký tự tách riêng cho OCR. **v2 sửa được quy mô nhưng không sửa được rò rỉ**: lên 15.133 ảnh lại *tăng* cặp gần trùng xuyên split lên 2.699 vì các nguồn chứa ảnh có nguồn gốc chung. **v3 giữ nguyên corpus** (cùng 15.133 ảnh), khác biệt duy nhất là ngưỡng khử trùng lặp $5 \rightarrow 10$ và split sinh lại — cô lập biến có chủ ý, cho phép quy kết mọi thay đổi kết quả cho *chất lượng split*, không cho *lượng dữ liệu*.

5.3.1. Khử trùng lặp: hai tỉ lệ, hai mẫu số khác nhau

Có hai tỉ lệ khử trùng lặp trên hai mẫu số khác nhau: **44,2%** (11.978/27.111, trước hợp nhất, trên 7 bộ vào hợp nhất detection) và **47,8%** (7.227/15.133, sau hợp nhất). Hai số **không cộng dồn và không thay thế nhau**; cơ chế và cách đọc trình bày ở mục 4.4.2, đối chiếu đầy đủ ở **Phụ lục C.3**. Điểm phải nhớ khi trích: mẫu số 27.111 là tổng ảnh của **7 bộ vào hợp nhất detection, không phải 9 bộ đã tải** — hai bộ còn lại mang **nhãn mức ký tự**, tách riêng cho tăng OCR.

5.3.2. Kiểm chứng rò rỉ dữ liệu — và vì sao con số "0 cặp rò rỉ" không chứng minh được điều gì

Rò rỉ xảy ra khi tập test chứa ảnh gần trùng ảnh train: mô hình *ghi nhớ* thay vì *tổng quát hoá*, mọi chỉ số bị thổi phồng — với corpus ghép từ nhiều nguồn công khai đây là rủi ro hệ thống [7]. Công cụ đo là **băm tri giác** (imagehash.phash, 64 bit).

Lập luận vòng tròn. v3 được khử trùng lặp ở ngưỡng Hamming 10; do rò rỉ ở đúng ngưỡng đã dùng để gộp trùng lặp là **kiểm tra lại chính định nghĩa của mình**, không phải kiểm chứng độc lập. Kết quả bằng 0 ở đó chứng minh bước khử trùng lặp *đã chạy đúng đặc tả*, và **không chứng minh gì thêm** về mức độ "sạch" của tập test.

Chỉ các ngưỡng **12 (791 cặp)**, **15 (3.529 cặp)**, **20 (137.506 cặp)** mang thông tin mới, và ngay cả chúng cũng **không** chứng minh tập test sạch. **Ba giới hạn của phash:** (1) phash chỉ bắt tương đồng ở mức **bổ cực sáng-tối tổng thể** — hai ảnh *cùng một chiếc xe* ở hai góc khác nhau, hay hai khung hình cách nhau vài giây trong cùng video, vẫn mang **cùng một biến số** dù Hamming lớn; loại rò rỉ ngữ nghĩa này **không khử được bằng bất kỳ ngưỡng phash nào**; (2) **không có định danh phương tiện hay chuỗi biển** cho toàn corpus nên không chia split theo **nhóm biến số** được — chính hạn chế dẫn tới mẫu số nhỏ của các bảng OCR ở 5.5; (3) **ngưỡng cao sinh dương tính giả**, nên 137.506 là **cận trên bi quan**. **Kết luận trung thực:** khẳng định được *bước khử trùng lặp ở ngưỡng 10 đã chạy đúng đặc tả*; **không** khẳng định được *tập test độc lập với tập train*. Rò rỉ tồn dư ở mức ngữ nghĩa **không đo được bằng công cụ hiện có** — mỗi đe dọa đầu tiên ở 5.9.3; mọi chỉ số ở 5.4 phải đọc kèm ghi chú này.

5.3.3. Phân bố nguồn dữ liệu giữa các split

Toàn tập chia **15.133 = 10.592 / 3.027 / 1.514**, tức **70,0% / 20,0% / 10,0%**. Hai điểm phải nêu khi đọc mọi kết quả của chương. **Thứ nhất, tập test nghiêng về ảnh camera giao thông** — nguồn roboflow_traffic_camera có **20,3%** số ảnh rơi vào test, gấp đôi tỉ lệ tổng thể 10,0% — nên khi đọc mAP theo dải kích thước (5.4.3) phải nhớ rằng đối tượng nhỏ trong tập test tập trung ở một nguồn. **Thứ hai, một bộ dữ liệu dư thừa hoàn toàn:** roboflow_tran_ngoc_xuan_tin vào hợp nhất với 1.005 ảnh và ra khỏi khử trùng lặp chéo bộ với **0 ảnh** — **loại 100,0%**, bằng chứng định lượng cho việc các bộ Roboflow tái sử dụng ảnh của nhau rất nặng và là lý do **không được cộng dồn expected_images để suy ra quy mô thật**.

Phân bố đầy đủ 16 tổ hợp xuất xứ, tiêu chí đọc và số cặp trùng của từng nguồn ở **Phụ lục C.4**.

5.4. Đánh giá bộ phát hiện biến số

Toàn bộ 5.4 đo trên **tập test v3: 1.514 ảnh, 1.611 đối tượng nhân thật**, `imgsz=640`, `device=cpu`; tập test chưa từng dùng trong huấn luyện hay chọn epoch.

5.4.1. Chỉ số tổng thể

Cả bốn chỉ tiêu bắt buộc đều đạt mục tiêu, đo trên `ultralytics_val`: **mAP@0.5 = 0,9829** (NFR-A1; sản 0,85, mục tiêu 0,90 ) , **mAP@0.5:0.95 = 0,7834** (NFR-A2; sản 0,55, mục tiêu 0,65 ) , **Precision = 0,9837** và **Recall = 0,9714** (NFR-A3; sản 0,88 / 0,85, mục tiêu 0,92 / 0,90 ) , **F1 = 0,9775** tại ngưỡng confidence 0,25; đối chiếu ở Bảng 5.10. Ba lưu ý: (1)

bài toán chỉ có một lớp (plate), mAP một lớp không so trực tiếp được với mAP nhiều lớp trên COCO nên giá trị cao là bình thường, **không phải bằng chứng về độ khó đã vượt qua**; (2) chỉ số quyết định là **mAP@0.5:0.95** vì độ khớp box ảnh hưởng trực tiếp đến chất lượng vùng cắt đưa sang OCR; (3) **chỉ số tổng thể che giấu phân bố**.

Ba hình chẩn đoán của khối phát hiện — đường cong PR tách theo layout (05-detection-pr-curve.png), ma trận nhầm lẫn nhận biết layout (05-detection-confusion-matrix.png), đường cong F1 theo ngưỡng confidence (05-detection-f1-curve.png) trong docs/reports/figures/ — **chưa sinh**. Hình F1 có vai trò thực tiễn: **ngưỡng confidence chạy thật phải là ngưỡng tối ưu F1 đo được ở đó**, không phải mặc định 0,25 của Ultralytics; nếu hai giá trị lệch nhau thì cấu hình suy luận phải được cập nhật và ghi lại.

5.4.2. Tách theo biển một dòng và biển hai dòng (NFR-A8)

Layout xác định theo nhãn lớp khi bộ dữ liệu có khai báo, và theo **ngưỡng tỉ lệ khung hình 2,5** khi không có — nằm giữa tỉ lệ chuẩn của biển một dòng (4,727) và biển hai dòng (2,000 / 1,357) theo QCVN 08:2024/BCA.

Bảng 5.2. Kết quả phát hiện tách theo biển một dòng và hai dòng (NFR-A8)

Chỉ số	Biển một dòng	Biển hai dòng	Chênh (điểm %)
Số đối tượng nhãn thật (<i>tổng 1.611</i>)	286	1.325	n/a
mAP@0.5	0,9884	0,9675	2,09
mAP@0.5:0.95	0,7526	0,7649	-1,23
Precision · Recall · F1	0,9861 · 0,9895 · 0,9871	0,9735 · 0,9691 · 0,9711	1,26 · 2,04 · 1,65

Nhãn layout là ước lượng, không phải nhãn thật: bộ dữ liệu không khai báo lớp layout nên layout suy từ ngưỡng tỉ lệ khung hình 2,5 (**100% số ô** suy bằng heuristic). Mọi kết luận về NFR-A8 ở tầng phát hiện phải nêu rõ điều này.

Chênh lệch ở tầng phát hiện rất nhỏ, đúng dự đoán — ở mAP@0.5:0.95 biển hai dòng thậm chí *nhỉnh hơn* 1,23 điểm, tức dao động trong phạm vi nhiễu; cùng bậc với baseline (baseline-416-v1.pt, split v1, imgsz 416: một dòng 0,9856 so với hai dòng 0,9592, chênh 2,6 điểm — **số của baseline, không được chuyển thành số của best.pt). **Việc định vị một hình chữ nhật gần như không phụ thuộc vào việc bên trong có một hay hai dòng ký tự**. Vì vậy **tuyệt đối không được** dùng 2,09 điểm để kết luận "hệ thống xử lý tốt biển hai dòng" — chênh lệch thật nằm ở tầng OCR và chỉ lộ ra ở Bảng 5.5, nơi khoảng cách nhảy lên **25,45 điểm**.**

5.4.3. Tách theo dải kích thước hộp giới hạn

Mục này tồn tại vì **bộ dữ liệu không đạt tiêu chí chất lượng Q6: 10,91% số hộp có diện tích dưới 0,5% diện tích ảnh**, vượt ngưỡng 10%. Đối tượng nhỏ là chế độ thất bại đã ghi nhận rộng rãi của bộ phát hiện một giai đoạn [119], biến số độ phân giải thấp đã thành

hướng nghiên cứu riêng [71]; một con số mAP tổng sẽ **giấu chế độ thất bại sau giá trị trung bình**.

Bảng 5.3. Kết quả phát hiện tách theo dải kích thước hộp giới hạn

Dải (diện tích box / diện tích ảnh)	Số đối tượng	Tỉ lệ tập test	mAP@0.5	mAP@0.5:0.95	Recall
Rất nhỏ — dưới 0,5%	262	16,26%	0,8553	0,5249	0,8740
Nhỏ — 0,5% đến 1%	124	7,70%	0,9755	0,7055	0,9758
Trung bình — 1% đến 5%	900	55,87%	0,9913	0,8005	0,9922
Lớn — 5% đến 15%	297	18,44%	0,9966	0,8394	0,9966
Rất lớn — trên 15%	28 $\triangle$	1,74%	1,0000	0,8562	1,0000
Toàn tập test	1.611	100%	0,9711	0,7625	0,9727

Dòng $\triangle$ (28 đối tượng < 30) **không có ý nghĩa thống kê**, không đưa vào so sánh.

Dải tính từ ($w \times h$) của hộp nhãn thật chia diện tích ảnh gốc; số box không gán được dải: 0.

Chế độ thất bại ở đối tượng nhỏ được xác nhận có thật: Recall dải "rất nhỏ" 0,8740 so với 0,9922 của dải "trung bình" vốn chiếm hơn nửa tập test — **bỏ sót nhiều biển nhỏ hơn hẳn**. Đây là hậu quả đo được của tiêu chí Q6 không đạt, không phải cảnh báo lý thuyết; kết hợp với 5.3.3, dải "rất nhỏ" chiếm **16,26%** tập test, cao hơn tỉ lệ **10,91%** của toàn corpus, nghĩa là **tập test khó hơn trung bình ở đúng khía cạnh này**. Khắc phục: tăng `imgsz`, cắt ảnh theo ô (*tiling*), hoặc lọc bỏ đối tượng quá nhỏ khỏi tập huấn luyện. Nếu chỉ báo cáo một con số tổng thì chế độ thất bại này đã bị **giấu sau giá trị trung bình**.

5.5. Đánh giá khối OCR và hậu xử lý

Ràng buộc mẫu số. NFR-A4...A7 chỉ đo được trên **tập con có nhãn chuỗi ký tự** (`datasets/annotations/plate_labels.csv`) — **2.801 biển**, không phải trên 1.514 ảnh test. Thiếu nhãn chuỗi cho phần lớn corpus là hạn chế thật, ghi ở 5.9.3.

5.5.1. Độ chính xác mức ký tự (NFR-A4)

Chỉ số là **CER** theo khoảng cách Levenshtein, chuẩn hoá theo độ dài nhãn thật, với S ký tự thay thế, D bị xoá, I chèn thừa, N tổng ký tự nhãn thật; NFR-A4 phát biểu theo **1 – CER**.

$$\text{CER} = \frac{S + D + I}{N}$$

NFR-A4 đạt ngưỡng tối thiểu: $1 - \text{CER} = \mathbf{0,9454}$ (sàn 0,92, mục tiêu 0,95 — ) , cách mục tiêu khoảng 0,5 điểm; chuỗi thô cho 0,9061, CER 0,0939 $\rightarrow \mathbf{0,0546}$ (sàn $\leq 0,08$, mục tiêu $\leq 0,05$). Chỉ tiêu này từng ghi là **không đạt** (0,8848, đo 20/07/2026); lượt đo lại 28/07 trên

đúng bộ trọng số ấy cho 0,9454 — chênh lệch **không** đến từ mô hình khác mà từ các bản sửa ở tầng suy luận và từ việc harness được nối đúng với pipeline giao hàng.

Nguồn số liệu. Toàn bộ 5.5 lấy từ docs/reports/05-results.json — lượt đo 2026-07-28 trên máy rảnh, models/best.pt (imgsz = 640), 2.801 ảnh có nhãn chuỗi, đúng cấu hình giao hàng (nấn hình bật, siêu phân giải tắt — 5.5.7). Lượt này thay bộ số 20/07 vì hai lý do độc lập: **một**, bốn đợt sửa độ chính xác rơi vào 21–28/07 nên số cũ mô tả một hệ thống không còn tồn tại; **hai**, harness đo trước 28/07 **chưa bao giờ gọi** bậc thang thử-lại — cả nhánh vùng cắt lẫn nhánh đầu-cuối đều chép lại các bước pipeline rồi dừng ở bước cứu dòng trên, nên mọi con số A4–A7 công bố trước đó mô tả một pipeline **ngắn hơn bản giao hàng** (5.5.6). *S/D/I* đo trên **chuỗi thô trước hậu xử lý** (*S* = tổng ô ngoài đường chéo ma trận nhầm lẫn, *D* = tổng xoá, *I* = tổng chèn, *N* = tổng ma trận cộng *D*) nên giống nhau ở cả hai cột — cũng vì vậy bước cứu dòng trên lẫn bậc thang thử-lại, vốn chạy **sau** chuẩn hoá, không làm ba con số này thay đổi.

Đáng chú ý hơn con số tổng là **cấu trúc lỗi**: tổng thao tác chỉnh sửa 3.092 → **2.241**, nhưng ba thành phần giảm rất không đều — chèn thừa *I* 903 → **107** (–88,1%), thay thế *S* 1.007 → **862** (–14,4%), xoá *D* 1.182 → **1.272** (+7,6%). **Ký tự chèn thừa gần như biến mất** — dấu vân tay của các bản sửa đọc biến hai dòng (trước đây vùng chồng lấn bị đọc hai lần nên sinh ký tự lặp, viền biến và vết bẩn bị đọc thành ký tự); ngược lại **ký tự bị xoá nhích lên** và nay chiếm **56,8%** toàn bộ lỗi, tức phần lỗi còn lại đã dịch hẳn về dạng **đọc hụt ký tự**. Bảng luật mạnh ở việc sửa *S* nhưng **về nguyên tắc không thể phục hồi một ký tự chưa từng được đọc ra**, nên khi *D* chi phối thì hướng cải thiện phải chuyển sang tăng nhận dạng (7.4). Ba loại lỗi gợi ba nguyên nhân: *S* → nhầm ký tự (xử lý được bằng bảng luật, 5.5.4), *D* → bỏ sót ký tự do vùng cắt thiếu hoặc ký tự mờ, *I* → nhiễu bị đọc thành ký tự.

5.5.2. Chuỗi đầy đủ: trước và sau hậu xử lý (NFR-A5 ↔ NFR-A6)

Đây là mục quan trọng nhất của cả chương. Khối hậu xử lý theo luật — chuẩn hoá ký tự, áp mặt nạ vị trí chữ số / chữ cái theo định dạng biến số Việt Nam, sửa cặp ký tự đồng hình, kiểm tra mã tỉnh — là **đóng góp kỹ thuật riêng** của đề án, không có sẵn trong thư viện nào. Câu hỏi *khối đó đóng góp bao nhiêu?* chỉ trả lời được nếu **đo hai lần trên cùng một tập mẫu**: chuỗi thô PaddleOCR trả về (A5) và chuỗi sau khi áp toàn bộ luật (A6). Đây cũng là lý do kỹ thuật khiến trường raw_ocr_text tồn tại bên cạnh plate_text trong lược đồ cơ sở dữ liệu — **điều kiện cần để phép đo này thực hiện được**, phải có từ giai đoạn thiết kế.

Bảng 5.4. Độ chính xác OCR trước và sau hậu xử lý, trên 2.801 biến có nhãn chuỗi

Chỉ số	Sàn	Mục tiêu	Trước hậu xử lý	Sau hậu xử lý	Chênh (điểm %)
1 – CER (NFR-A4)	0,92	0,95	0,9061	0,9454	n/a
CER	≤ 0,08	≤ 0,05	0,0939	0,0546	n/a
Chuỗi đầy đủ đúng (A5 →	0,80 →	0,85 →	0,6373 ❌	0,7512	+11,39

Chỉ số	Sàn	Mục tiêu	Trước hậu xử lý	Sau hậu xử lý	Chênh (điểm %)
A6)	0,85	0,90		X	
<i>N / S / D / I</i> trên chuỗi thô	—	—	23.855 / 862 / 1.272 / 107	(không đổi)	n/a
Biến sửa đúng / bị làm hỏng / sai cả trước lẫn sau	—	—	—	319 / 0 / 697	n/a

Dòng "sửa đúng / làm hỏng" quan trọng ngang dòng hiệu số: cùng một mức cải thiện thuần có thể đến từ "sửa đúng nhiều, làm hỏng nhiều" hoặc "sửa đúng ít, không làm hỏng biến nào" — **hai kết luận kỹ thuật khác nhau** về chất lượng bộ luật.

Phân rã đóng góp theo từng nhóm luật — (chưa đo). Năm nhóm cần bóc tách: chuẩn hoá cơ bản (bỏ ký tự phân tách, viết hoa, Đ→D), mặt nạ vị trí + TO_DIGIT, mặt nạ vị trí + TO_LETTER, ghép dòng cho biến hai dòng, kiểm tra mã tỉnh; mỗi nhóm cần số biến bị thay đổi, số sửa **đúng**, số bị làm **hỏng**, đóng góp thuần — tất cả (chưa đo) vì ai/inference/plate_rules.py chưa có cơ chế bật/tắt từng nhóm luật (hạng mục cần viết mã, mục D.2). Hai bậc cứu chữa chạy **sau** chuẩn hoá thì đã cô lập được nhờ đo A/B ở 5.5.6 và 5.5.7: **cứu dòng trên 209 biến, bậc thang thử-lại 34 biến.**

Hiệu số **A6 – A5 = 0,7512 – 0,6373 = +11,39 điểm phần trăm** trên **2.801 biến**, kèm **319 sửa đúng, 0 làm hỏng** — **không phải một đánh đổi** mà là **cải thiện thuần một chiều**: các mặt nạ vị trí và ràng buộc mã tỉnh đủ bảo thủ để không tự tạo lỗi mới. Nhưng dù đóng góp gần gấp đôi lượt 20/07 (+6,32 → **+11,39**), cả A5 lẫn A6 vẫn **không đạt** sàn (0,80 và 0,85) — A6 còn thiếu **9,88 điểm**. **Đóng góp phân bố rất không đều**: biến một dòng A5 0,9418 → A6 0,9541, **+1,23 điểm ứng 7 biến**; biến hai dòng 0,5600 → 0,6996, **+13,97 điểm ứng 312 biến**. Trên biến một dòng hậu xử lý gần như không có việc để làm (chuỗi thô đã đúng 94,18%); toàn bộ giá trị dồn vào **biến hai dòng** — bằng chứng rằng bộ luật thực sự bù đắp điểm yếu của tầng nhận dạng chữ không chỉ làm đẹp chuỗi.

Vì sao vẫn không đủ? **Chuỗi sai nhiều ký tự cùng lúc**: chỉ **1.251 / 2.234 biến hai dòng = 56,0%** đọc đúng trước hậu xử lý; khi bộ nhận dạng đọc hỏng cả cụm thì không luật thay-ký-tự nào cứu được, và **697 biến sai cả trước lẫn sau** chính là quần thể này. **Ký tự chưa từng được đọc ra thì không luật nào phục hồi được** — ràng buộc nguyên tắc, không phải khiếm khuyết cài đặt: với $D = 1.272$ (56,8% toàn bộ lỗi), nhiều chuỗi ngắn hơn độ dài mong đợi khiến mặt nạ lệch pha và luật khi đó **không dám sửa** chứ không sửa bừa — **0 biến bị làm hỏng** là hệ quả quan sát được của thiết kế bảo thủ đó. Kết luận đúng phạm vi cho RQ3: **khối hậu xử lý đóng góp +11,39 điểm trên 2.801 biến — +13,97 điểm riêng trên biến hai dòng** — **cải thiện thuần không rủi ro (319 / 0)**, nhưng không đủ đưa A6 tới ngưỡng vì phần lỗi còn lại đã dịch sang dạng đọc hụt ký tự, nơi hậu xử lý theo luật về nguyên tắc không với tới được. Rất ít công trình ALPR đo tách bạch đại lượng này.

5.5.3. Tách theo biển một dòng và hai dòng cho OCR

Bảng 5.5. Độ chính xác nhận dạng tách theo biển một dòng và hai dòng

Chỉ số	Biển một dòng	Biển hai dòng	Chênh (điểm %)
Số mẫu có nhãn chuỗi (<i>tổng 2.801</i>)	567	2.234	n/a
1 – CER (NFR-A4)	0,9925	0,9344	5,81
Chuỗi đúng trước hậu xử lý (A5)	0,9418	0,5600	38,18
Chuỗi đúng sau hậu xử lý (A6)	0,9541	0,6996	25,45
Cải thiện do hậu xử lý (A6 – A5)	+1,23	+13,97	n/a
Độ chính xác E2E (A7)	0,6861	0,5219	—

Đây là kết quả khoa học quan trọng nhất của chương. Chênh lệch mà tăng phát hiện gần như che khuất (2,09 điểm, Bảng 5.2) nay lộ ra ở tầng OCR với **biên độ khác hẳn cấp**: 5,81 điểm ở mức ký tự, **25,45 điểm** ở A6, **38,18 điểm** ở A5. **Biển một dòng về cơ bản đã giải xong** (A6 = 0,9541 vượt cả mục tiêu 0,90), toàn bộ việc "OCR không đạt" là do **biển hai dòng kéo xuống**; vì biển hai dòng chiếm $2.234 / 2.801 = 79,8\%$ tập có nhãn chuỗi (phản ánh tỉ lệ xe máy rất cao ở Việt Nam), con số tổng bị quần thể khó này chi phối. **25,45 điểm là con số sau khi đã áp cả hai bậc cứu chữa** (5.5.6, 5.5.7): ở lượt 20/07, A6 biển hai dòng là 0,5810 và khoảng cách là 36,79 điểm — chuỗi biện pháp đã thu hẹp **11,34 điểm**, một dịch chuyển thật nhưng vẫn để lại một phần tư khoảng cách; phần còn lại nằm ở **năng lực nhận dạng ký tự**, không ở khâu cắt/ghép hay hình học, vì cả hai khâu sau đã xử lý và đo tách bạch.

Mốc tham chiếu quốc tế — phải đọc kèm cảnh báo. Laroca và cộng sự (VISAPP 2022) báo cáo **94,3%** trên biển một dòng và **45,7%** trên biển hai dòng, chênh **48,6 điểm phần trăm**, đo trên bộ dữ liệu **RodoSol-ALPR của Brazil** [7]. **Đây là số liệu Brazil, không phải Việt Nam**, và biển hai dòng Brazil khác biển hai dòng Việt Nam cả về tỉ lệ khung hình lẫn bộ ký tự. **Đối chiếu bậc độ lớn: 25,45 điểm** (biển số **Việt Nam thật**, 2.801 biển có nhãn chuỗi) so với **48,6 điểm** trên RodoSol-ALPR **Brazil — cùng bậc độ lớn**. Không được kết luận mạnh hơn: $25,45 < 48,6$ **không** có nghĩa hệ thống này "tốt hơn", vì hai phép đo khác bộ dữ liệu, khác bộ ký tự, khác tỉ lệ khung hình, khác mẫu số. Kết luận hợp lệ duy nhất: khoảng cách hai layout **thuộc đúng bậc độ lớn** mà tài liệu quốc tế đã ghi nhận — đặc tính có cấu trúc của bài toán, không phải khiếm khuyết riêng của hệ thống. **Chưa có nghiên cứu Việt Nam nào công bố hai con số này tách bạch trên cùng một hệ thống**; Bảng 5.5 lấp khoảng trống đó và trả lời RQ2: **có, chênh lệch có ý nghĩa và rất lớn (25,45 điểm A6), nằm ở tầng OCR chứ không ở tầng phát hiện**.

5.5.4. Ma trận nhầm lẫn ký tự 36×36

Bảng 5.6. Các cặp ký tự bị nhầm nhiều nhất, đối chiếu bảng luật hiện hành

Hạng	Ký tự thật → bị đọc thành	Số lần	Tỉ lệ trong tổng lỗi thay thế	Bảng luật có phủ không?
------	---------------------------	--------	-------------------------------	-------------------------

Hạng	Ký tự thật → bị đọc thành	Số lần	Tỉ lệ trong tổng lỗi thay thế	Bảng luật có phủ không?
1	L → 1	90	10,44%	có (TO_DIGIT) — đúng chiều
2	E → F	73	8,47%	không
3	4 → L	53	6,15%	không
4	U → 1	38	4,41%	không
5	D → 0	34	3,94%	có (TO_DIGIT) — đúng chiều
6	Z → 7	32	3,71%	không
7	2 → 7	26	3,02%	không
8	X → Y	21	2,44%	không
9	B → R	20	2,32%	không
10	9 → 0	19	2,20%	không

Phân tích đầy đủ — từng dòng của bảng, các ca điển hình và hệ quả kéo theo — ở **Phụ lục P.2**.

5.5.5. Độ chính xác đầu-cuối toàn trình (NFR-A7)

NFR-A7 đo **ảnh đầu vào → phát hiện → cắt → OCR → hậu xử lý → chuỗi cuối**; khác A6 ở chỗ A6 đo trên **vùng biển cắt chuẩn theo nhãn thật** còn A7 đo trên vùng biển do **chính bộ phát hiện** tìm ra, nên A7 tích lũy cả hai nguồn sai số và theo lý thuyết luôn $\leq$ A6. Trên 2.801 mẫu: **A7 = 0,5552** (sàn 0,82, mục tiêu 0,88 — **X**); **E2E với điều kiện đã phát hiện được biển = 0,6306**; tỉ lệ biển **bỏ sót** ở tầng phát hiện **0,1196**; phát hiện đúng nhưng **đọc sai chuỗi 0,3694**; chênh **A6 – A7 = 19,60 điểm**.

⚠ **Cảnh báo hiệu lực — phải đọc trước khi diễn giải A7**. Con số 0,5552 đo trên **ảnh crop biển số**, không phải ảnh hiện trường, vì không bộ dữ liệu nào trong đề án có đồng thời ảnh toàn cảnh và chuỗi biển số nhãn thật. Bộ phát hiện được huấn luyện trên ảnh giao thông đầy đủ nên ảnh chỉ có biển số chiếm gần hết khung là **ngoài phân bố huấn luyện**: phần lớn thất bại ở đây do bộ phát hiện không bắt được box trên ảnh crop (bỏ sót 11,96%), **không** phải do OCR đọc sai. Bằng chứng: trên ảnh hiện trường thật, Phase 7 đo **mAP@0.5 = 0,9829**, tương thích với 5.4.1. Muốn đo NFR-A7 đúng cách cần gán nhãn chuỗi cho một phân bố test có ảnh hiện trường (ví dụ một phần yolo_v2) — **việc này chưa làm**.

Nguồn lỗi được phân tách rõ: **335 / 2.801 = 11,96%** biển bỏ sót ở tầng phát hiện, và trong số đã phát hiện được, **36,94%** đọc sai chuỗi. Do cảnh báo hiệu lực, tỉ lệ bỏ sót 11,96% **bị thổi phồng bởi việc đo trên ảnh crop ngoài phân bố**; kết luận đúng phạm vi: *A7 = 0,5552 phản ánh giới hạn của giao thức đo hiện có chồng lên giới hạn thật của tầng OCR trên biển hai dòng — cận dưới bị quan, không phải ước lượng điểm*. Chỉ tăng recall bộ phát hiện cũng không đưa A7 lên quá 0,6306; trần thật bị chặn bởi tầng OCR. **Vì sao A7 tăng chậm hơn A6:**

mọi biện pháp cứu chữa áp vào **cả hai** đường đo nhưng bị **pha loãng** ở A7 vì 11,96% số biển thất bại ngay ở tầng phát hiện, phần cải thiện chỉ tác động trên **88,04%** mẫu còn lại. Giữa 20/07 và 28/07, **missed_by_detector** giữ nguyên đúng 335 và tỉ lệ phát hiện giữ nguyên đúng 0,8804 — bộ trọng số phát hiện không đổi, nên toàn bộ mức tăng của A7 đến từ khối nhận dạng, đo đúng bằng độ chính xác có điều kiện: 0,6014 → **0,6306**, tức **+2,92 điểm**; A7 tăng ít hơn (**+2,57 điểm**) vì bị 335 ca vô vọng kéo xuống — hệ quả số học, **không** phải dấu hiệu sai sót.

5.5.6. Bước "cứu dòng trên" cho biển hai dòng — thiết kế, kiểm chứng A/B và kết quả trên toàn tập

Hồ sơ lỗi thiên về *xoá* ($D = 1.272 > S = 862$) trên biển hai dòng có cách giải thích tự nhiên: **mất hẳn một dòng**. Hệ thống đọc biển hai dòng bằng **ghép-rời-đọc** (*split-then-hstack*): cắt vùng biển thành hai nửa chồng lấn, xếp cạnh nhau thành dải ngang, chạy OCR **một lần**. Phương án thay thế — đọc riêng từng nửa rồi nối chuỗi — đã được đo A/B chứ không bị loại bằng lập luận, trên 200 biển hai dòng (*seed* = 20260720, docs/reports/15-two-line-ab.json): **A, ghép rời OCR một lần** (*đang dùng*) đúng **129/200 = 64,50%**, 2 ca OCR trả chuỗi rỗng, 340,11 ms; **B, OCR từng nửa rồi nối** đúng **7/200 = 3,50%**, 9 ca chuỗi rỗng, 391,35 ms — B kém A **61,00 điểm phần trăm** và đắt hơn **51,24 ms**; **122 ca A thắng B, 0 ca B thắng A. Giả thuyết "đọc riêng từng dòng thì chính xác hơn" bị bác bỏ dứt khoát**. Nguyên nhân đọc được ngay trong dữ liệu: hai nửa **cố ý chồng lấn** để không cắt cụt ký tự, nên đọc riêng thì dải chồng lấn bị đọc **hai lần** và ký tự bị nhân đôi — 84G122593 thành 84-G124E009.01225.93. Một kết quả âm có giá trị: lựa chọn kiến trúc ở Chương 5 không tùy tiện.

Dải ghép vẫn có chế độ thất bại riêng: khi **dòng trên nằm lệch thấp** trong vùng cắt rộng rãi, bộ dò chữ chỉ tìm thấy **một vùng văn bản** — dòng dưới — và mã tỉnh cùng chữ cái sê-ri **mất hoàn toàn** (29E-015.66 → 015.66, năm chữ số trần không khớp định dạng nào nên khối kiểm tra hợp lệ **bác bỏ đúng**); chính sự bác bỏ đó là tín hiệu dùng được. Bước **cứu dòng trên** (*rescue_two_line_upper*) có cổng rất hẹp: (1) chỉ kích hoạt khi **cả ba** điều kiện đồng thời đúng — *line_count* = 2, chuỗi sau chuẩn hoá **không hợp lệ định dạng**, chuỗi thô **không rỗng**; (2) đọc **riêng nửa trên** bằng một lời gọi OCR bổ sung; (3) nối nửa trên + chuỗi thô của dải ghép rời cho qua lại khối chuẩn hoá; (4) **chỉ giữ kết quả mới nếu nó hợp lệ định dạng**, mọi trường hợp khác — kể cả ngoại lệ — trả về kết quả cũ. Điều kiện (1) khiến bước này **về mặt cấu trúc không thể làm hỏng** một biển vốn đã đọc đúng.

Kiểm chứng A/B trên **hai mẫu độc lập** (15-two-line-fallback-700.json, 15-two-line-fallback.json; hai seed khác nhau). **700 biển, seed** = 7: 421/700 = **60,14%** → 434/700 = **62,00%**, **+1,86 điểm**, **13 biển được cứu, 0 bị hỏng**, kích hoạt 21,14% (148/700), 362,41 → 383,52 ms. **200 biển, seed** = 20260720: 129/200 = **64,50%** → 130/200 = **65,00%**, **+0,50 điểm**, **1 biển được cứu, 0 bị hỏng**, kích hoạt 18,00% (36/200), 346,70 → 361,97 ms. **Dấu của hiệu số nhất quán trên cả hai mẫu và không mẫu nào có ca bị làm hỏng**, đúng dự đoán từ cấu trúc cổng; nhưng **độ lớn không nhất quán** (+1,86 so với +0,50) — với mẫu 200 biển, một biển được cứu đã bằng 0,5 điểm nên +0,50 nằm hoàn toàn trong dao động lấy

mẫu và **không được dùng làm bằng chứng độc lập**, nó chỉ xác nhận *không có hồi quy*. Chi phí khoảng **21 ms** mỗi biển, chỉ trên những vùng cắt vốn đã thất bại.

⚠ **Số liệu toàn tập của riêng bước cứu là số lịch sử 20/07, giữ nguyên mốc.**

Trên 2.801 ảnh có nhãn chuỗi (16-ocr-accuracy-rescued.json, best.pt, imgs = 640; cột "trước" là lượt đo trước đó trên đúng cùng tập mẫu và cùng mô hình):

A4 0,8734 → **0,8848 (+1,14; sần 0,92 ✖)**; A5 0,6098 → **0,6098 (0,00; sần 0,80 ✖)**;

A6 0,6555 → **0,6730 (+1,75; sần 0,85 ✖)**; A7 0,5227 → **0,5295 (+0,68; sần 0,82 ✖)**;

A6 riêng biển **một dòng** 0,9489 → **0,9489 (0,00)**; A6 riêng biển **hai dòng** 0,5810 → **0,6030 (+2,20)**; biển bị can thiệp / thành đúng hoàn toàn / bị làm hỏng = **89 / 2.801 · 49 · 0**. Bảng số này trả lời đúng một câu hỏi — *riêng bước cứu dòng trên đóng góp bao nhiêu* — và câu trả lời ấy không đổi theo thời gian; nhưng **các giá trị tuyệt đối đã bị vượt qua** (A6 hiện là 0,7512 chứ không phải 0,6730) nên **không được trích cột "sau bước cứu" như số hiện hành**. Trên lượt 28/07, bước cứu dòng trên cho câu trả lời cuối ở **209 biển**.

Ba ô "**không đổi**" phải đọc như bằng chứng, không như thiếu sót: A5 không đổi vì bước cứu chạy **sau** khối chuẩn hoá nên **không thể** tác động lên chỉ số đo *trước* chuẩn hoá; biển một dòng không đổi vì công yêu cầu `line_count = 2` và toàn bộ 89 biển can thiệp đều là biển hai dòng; số bị làm hỏng bằng 0 vì chuỗi đã hợp lệ không bao giờ được thử lại. Tác dụng thật nằm đúng nơi được nhắm — **biển hai dòng, +2,20 điểm** — và vẫn để lại **bốn chỉ tiêu OCR đều không đạt**.

Ghi chú phương pháp đo — ba lần cùng một loại lỗi, ghi lại thay vì giấu đi. Lượt đo lại đầu tiên sau khi thêm bước cứu cho kết quả **tự mâu thuẫn**: A6 tăng 1,75 điểm trong khi A7 — chỉ số **bao hàm** phần A6 đo — đứng yên ở đúng 0,5227. Nguyên nhân là công cụ đánh giá **dựng lại đường xử lý thay vì gọi nó**, nên mỗi bậc mới thêm vào pipeline đều mặc định rơi ra ngoài phép đo, im lặng. Cùng loại lỗi đã xảy ra **ba lần** trong đồ án — lần thứ ba (28/07/2026) khiến mọi con số A4–A7 công bố từ 21/07 đến 28/07 mô tả một pipeline **ngắn hơn bản giao hàng**, dù biện pháp phòng ngừa đặt ra sau lần thứ hai đã được tuân thủ đầy đủ.

Khi chỉ số *X* **bao hàm** chỉ số *Y* về mặt định nghĩa, mà một can thiệp làm *Y* dịch chuyển còn *X* thì không nhúc nhích, **giả thuyết đầu tiên phải là đường đo của X bị lệch khỏi đường chạy thật**. Một đường đo *liệt kê lại* các bước của hệ thống sẽ lệch khỏi hệ thống ở đúng thời điểm hệ thống thay đổi — tức đúng thời điểm phép đo cần chính xác nhất. Chia sẻ mã ở mức *hàm* không đủ; chỉ **gọi thẳng cùng một điểm vào** mới đủ.

Diễn biến đầy đủ của cả ba lần, cùng hai chốt chặn đã thêm và phần biện pháp còn bỏ ngỏ, ở **Phụ lục K**.

5.5.7. Bậc thang thử-lại cho biển nghiêng/méo — chi phí, lợi ích và một quyết định tắt tính năng

Phân tích đầy đủ — từng dòng của bảng, các ca điển hình và hệ quả kéo theo — ở **Phụ lục P.3**.

5.6. Đánh giá hiệu năng

Mọi số trong 5.6 phải đọc cùng cấu hình ở 5.2: **Intel Raptor Lake 14 nhân / 20 luồng, Windows 11, CPU-only, kích thước lô = 1**.

5.6.1. Độ trễ đầu-cuối (NFR-P1)

Bảng 5.7. Độ trễ đầu-cuối một ảnh, đối chiếu NFR-P1

Chỉ số	Sàn	Mục tiêu	Trước bậc thang (20/07)	Cấu hình giao hàng (28/07)	Kết quả
p50 (ms)	—	—	414,67	405,77	n/a
p95 (ms)	≤ 1500	≤ 800	731,15	1.143,10	
p99 (ms)	—	—	947,83	1.420,07	n/a
Trung bình (ms)	—	—	400,74	447,38	n/a
Số ảnh đo	—	—	100	100	n/a
Bội số so với sàn / mục tiêu	—	—	0,49× / 0,91×	0,76× / 1,43×	n/a

Số biến trung bình mỗi ảnh 1,33, cùng 100 ảnh test v3, cùng máy rảnh (CPU idle ~5%, không có tiến trình huấn luyện song song). **Hai cột đo hai phiên bản hệ thống, không phải hai phương pháp đo**: cột trái là trạng thái 20/07 trước khi có bậc thang (client-side qua HTTP, 07-benchmark-p1-resolved.json); cột phải là cấu hình giao hàng 28/07 (in-process, benchmark_system.py, 05-results.json). Chênh lệch phương pháp giữa hai cách đo là ****~7%**** — kiểm chứng ngày 20/07 khi cả hai cùng chạy trên một hệ thống (731,15 so với 780,36 ms) — nên nó **không** giải thích được mức tăng ở đây. Hình phân bố độ trễ kèm vạch p50/p95/p99 (07-latency-distribution.png) **đã có nhưng cần vẽ lại cho best.pt**.

NFR-P1 đạt ngưỡng tối thiểu nhưng không đạt mục tiêu, và đây là **thoái lui có chủ ý và đã định lượng**: tắt hẳn bậc thang thử-lại đưa p95 về **866,3 ms**, tức toàn bộ **+277 ms** là của nó; nhưng vì bậc thang chỉ chạy **sau khi lần đọc đầu thất bại** nên nó không chạm vào trường hợp thường — **trung vị thậm chí giảm nhẹ** (414,67 → 405,77 ms), chi phí dồn hết vào đuôi. Với hệ thống mà 95% ảnh xong dưới 1,15 giây và một nửa xong dưới 0,41 giây, p50 mô tả trải nghiệm điển hình còn p95 mô tả trường hợp xấu; NFR-P1 phát biểu theo p95 nên kết luận chính thức là **đạt sàn, không đạt mục tiêu**. Ở lượt đo đầu với cả ba biến thể bật, p95 là **1.514,26 ms** — **vượt cả ngưỡng tối thiểu**; bóc tách chỉ ra bậc siêu phân giải chiếm

hơn nửa chi phí đó mà không mua được biển nào đo được nên nó bị **tắt mặc định**, đưa p95 về 1.143,10 ms.

Giải quyết mâu thuẫn với con số cũ 5.857 ms. Báo cáo Phase 7 trước đây ghi p95 = 5.857,19 ms và kết luận NFR-P1 "không đạt" — chênh **7,5 lần**. Phép đo cũ đã bị **bác bỏ**: (1) **nhieu do tranh chấp CPU** — 07-benchmark-data.json ghi rõ có tiến trình ai.training.train chiếm 793% CPU chạy song song, và chính báo cáo cũ khi tính lại trên 80 mẫu ít nhieu đã ra ~3.283 ms; (2) đo trên **checkpoint epoch 7**, không phải best.pt; (3) checkpoint đó có **lỗi crop** khiến PaddleOCR chạy cả khối phát hiện văn bản trên ảnh lớn (~1.322 ms/ảnh). Đo lại trên máy rảnh với best.pt: p95 còn 731 ms. Giả thuyết "cold-start / oneDNN chưa tắt" **không** phải nguyên nhân — server chạy enable_mkldnn=false và có warmup (cold-start đo được chỉ **176 ms** p95); giả thuyết "baseline-416-v1 vốn chậm" cũng bị loại — client-side nó ra **763,75 ms** p95, gần y hệt best.pt.

5.6.2. Phân rã ngân sách độ trễ theo từng bước

Bảng 5.8. Phân rã ngân sách độ trễ theo từng bước

Bước xử lý	Ước lượng Phase 0 (ms)	Đo thật (ms)	Chênh (lần)	% tổng
Giải mã ảnh + tiền xử lý	50	2,83	0,06	1,7%
Suy luận YOLO11n @ 640px (CPU)	150	57,27	0,38	34,0%
Cắt + tiền xử lý vùng biển số	30	0,00	0,00	0,0%
PaddleOCR (mỗi biển)	120	108,28	0,90	64,3%
Hậu xử lý regex + kiểm tra hợp lệ	5	0,03	0,01	0,0%
Ghi CSDL + lưu ảnh	50	—	—	—
Tổng (một biển số)	405	168,41	0,47	100%

Mẫu số: 40 ảnh, trung bình 1,57 biển mỗi ảnh, đo trên best.pt. Bước "ghi CSDL + lưu ảnh" giữ — vì benchmark_system đo pipeline suy luận thuần, không đi qua tầng API; cột chênh lệch ở dòng tổng so với ước lượng **cùng phạm vi** (đã trừ ghi CSDL, ~**355 ms**), không so với 405 ms tròn. Hình cột chồng đối chiếu ước lượng với số đo (07-latency-budget.png) **đã có nhưng cần vẽ lại**.

Ba phát hiện. **Một, ước lượng Phase 0 sát bất ngờ ở tổng nhưng lệch ở phân bố**: tổng suy luận thuần **168,41 ms/biển**, nhỏ hơn cả ước lượng Phase 0 — ước lượng ban đầu **không** sai một bậc độ lớn như con số nhieu 5.857 ms từng khiến người ta tin. **Hai, nút thắt là PaddleOCR nhưng KHÔNG áp đảo như báo cáo cũ**: 64,3% so với 34,0% của bộ phát hiện, **thay thế** con số cũ "OCR 93,3% / detect 6,5%" vốn đo trên hệ thống đang có lỗi crop (~1.322 ms/ảnh); nguyên nhân OCR đắt vẫn đúng — PaddleOCR là **pipeline nhiều giai đoạn** (phát hiện văn bản → phân loại hướng → nhận dạng) thiết kế cho ảnh tài liệu tổng quát, nên hệ thống trả chi phí cho năng lực mà vùng biển đã cắt không cần. **Ba, chiến lược tối**

ưu đổi hẳn: theo định luật Amdahl, tăng tốc detector 2–3× có thể kéo E2E xuống quãng **15–23%**, khác hẳn kết luận cũ "chỉ giảm tối đa 6,7%"; và vì NFR-P1 mới đạt sàn còn NFR-P2 trượt cả sàn (2,379 FPS, sàn 3), tối ưu hiệu năng nằm trên đường tới chỉ tiêu chứ không chỉ là *dư địa cải thiện thêm*.

5.6.3. So sánh backend suy luận: PyTorch, ONNX Runtime và OpenVINO

Phép so sánh này chưa được thực hiện. Lướt `benchmark_cpu --backends pytorch onnx opencvino` **chưa chạy**, nên độ trễ riêng bộ phát hiện, mức tăng tốc so với PyTorch và phần trăm cải thiện đầu-cuối của hai runtime kia đều **chưa có số**. Khi đo, chỉ tiêu **mAP@0.5** sau **khi xuất** phải đo cùng lúc để kiểm tra việc chuyển đổi định dạng **không làm suy giảm độ chính xác** — nếu có suy giảm thì mức tăng tốc phải được đánh giá như một đánh đổi chứ không phải một khoản lãi. Thí nghiệm vẫn đáng làm dù kết luận đoán trước được từ 5.6.2: nó **kiểm chứng** lập luận Amdahl bằng số liệu [18] [117].

Tối ưu backend của bộ phát hiện GÌỜ có ý nghĩa, nhưng chưa đủ một mình. Vì NFR-P1 **chỉ đạt sàn** còn NFR-P2 **trượt sàn**, đây không phải dư địa cải thiện thêm mà là đường dẫn tới chỉ tiêu; muốn giảm mạnh hơn thì khối OCR (64,3%) vẫn là mục tiêu lớn nhất.

Bốn hướng tấn công khối OCR theo chi phí tăng dần (chi tiết ở Chương 6): **tắt các giai đoạn không cần thiết của pipeline PaddleOCR; bật MKL-DNN và chỉnh số luồng CPU; xuất mô hình nhận dạng sang ONNX Runtime; thay bằng mô hình nhận dạng chuyên cho biển số** huấn luyện trên tập ký tự hẹp — tiềm năng lớn nhất, tổn công nhất.

5.6.4. Chế độ webcam (tầng API) và xử lý video (NFR-P2, NFR-P3)

Từ 2026-07-20, trang Webcam đã gỡ khỏi giao diện web (thu gọn phạm vi — 4.1.3b); chế độ thời gian thực chỉ còn ở tầng API, nên NFR-P2 đo bằng kịch bản gọi trực tiếp `POST /api/detect/frame`. Số liệu đầy đủ ở Bảng 5.10.

NFR-P2 không đạt (2,379 FPS; sàn 3, mục tiêu 5), nguyên nhân không phải tốc độ trung bình mà là đuôi phân bố. Trung vị mỗi khung chỉ **180,05 ms** — tương ứng 5,6 FPS, vượt mục tiêu — nhưng p95 là **1.247,70 ms**, và giao diện thời gian thực dùng **hàng đợi một khe**: chỉ một yêu cầu bay tại một thời điểm, khung sinh ra trong lúc chờ bị bỏ thay vì xếp hàng; ở kỷ luật đó thông lượng bị chi phối bởi những lần chậm nhất. Đuôi ấy chính là bậc thang thứ-lại (5.5.7) — **đánh đổi đã biết**: cùng cơ chế đó mua thêm 34 biển đọc đúng và đẩy NFR-P1 từ  xuống . Ngược lại NFR-P3 **đạt**: video 14,25 giây xử lý hết **19,1 giây** (sàn ≤ 95 s, mục tiêu ≤ 47,5 s), tức **0,746×** thời gian thực — **0,546 s mỗi khung phân tích**, `vid_stride = 5`.

Định nghĩa đã dùng, nêu rõ để không phóng đại: "FPS hiệu dụng" là **số khung được nhận dạng xong mỗi giây (144 khung trong 60,52 giây, 0 yêu cầu lỗi)**, không phải số khung hiển thị; camera ảo chào **1.815 khung** ở 30 FPS và **1.671 khung bị bỏ** — con số bỏ này được báo cáo chứ không giấu, vì công bố riêng "144 khung, 0 lỗi" sẽ khiến người đọc hiểu nhầm là hệ thống theo kịp nguồn. **Con số này là cận trên:** phép đo chạy qua HTTP loopback với ảnh có sẵn trên đĩa nên **không** tính thời gian camera thu hình, mã hoá JPEG trong trình duyệt và vẽ

canvas. Dự đoán trước đó là **~1,4 FPS** (suy từ p95 0,73 giây và giả định xử lý tuần tự); số đo thực **2,379 FPS** cao hơn dự đoán nhưng vẫn dưới sàn 3 FPS — **kết luận của dự đoán đúng dù con số thì lệch**, ghi lại để thấy giới hạn của việc suy diễn từ độ trễ thay vì đo.

5.6.5. Chịu tải, bộ nhớ và độ tin cậy (NFR-SC1, NFR-P4...P7, NFR-R4)

Mọi chỉ tiêu hiệu năng *ngoài đường xử lý ảnh* đều đạt với biên rất rộng (chi tiết ở Bảng 5.10): nạp mô hình **6,41 s**, khởi động tới khi /health sẵn sàng **8,36 s** (sàn 30 s); overhead API p95 **19,01 ms**; truy vấn lịch sử 10.000 bản ghi p95 **18,71 ms** — nhanh hơn mục tiêu **~27 lần**; RSS pipeline / máy chủ backend **0,759 / 0,806 GB** so với sàn 4 GB, tức phẳng ở khoảng **0,8 GB**; **10 yêu cầu đồng thời** ổn định so với ngưỡng 5; soak 15 phút **100,0% thành công trên 2.028 yêu cầu**, RSS chỉ tăng **+0,094 GB** (0,726 → 0,820) — **không rò rỉ**; cơ sở dữ liệu sống sót qua khởi động lại với **0/9.031 bản ghi mất**. Hình đường cong thông lượng và tỉ lệ lỗi theo mức đồng thời 1 / 2 / 5 / 10 (07-concurrency.png) **đã có nhưng cần vẽ lại**.

Nhưng hai chỉ tiêu trên *chính đường xử lý ảnh* thì không: NFR-P1 chỉ đạt sàn và **NFR-P2 trượt cả sàn**, cùng nguyên nhân là đuôi độ trễ do bậc thang thử-lại. **Kiến trúc phần mềm không còn là vấn đề** — tầng API, tầng dữ liệu, bộ nhớ, độ ổn định đều dư biên; **nhưng độ trễ suy luận thì vẫn là vấn đề**. Hai nhánh đi tiếp: nâng *độ chính xác* OCR biến hai dòng (5.5), và cắt *đuôi độ trễ* — đặt trần thời gian cho bậc thang, hoặc chỉ chạy nó ở chế độ ảnh tĩnh.

5.6.6. Bỏ bước phát hiện chữ của PaddleOCR: một quyết định suýt sai

Mục 4.5.3 để ngỏ một câu hỏi: chế độ **chỉ nhận dạng** cho model fine-tune thêm **12,46 điểm A6** và rẻ hơn **~290 ms** mỗi ảnh — vì sao không bật? Vì ngữ liệu chứng minh nó **không có thẩm quyền trả lời câu hỏi đó**.

Bảng 5.9. Bỏ bước phát hiện chữ — ba ngữ liệu, hai kết luận ngược nhau

Cấu hình	A6 trên 2.801 ảnh cắt sẵn	Bộ demo ảnh toàn cảnh	Tầng dễ (<i>n</i> =372)	Tầng khó (<i>n</i> =236)	A7 hiện trường	KTC 95%
Model gốc, det + rec — <i>bản giao hàng</i>	0,7512	17 / 22	96,8%	44,1%	56,3%	[52,0 ; 60,7]
Model gốc, chỉ rec	0,7508	13 / 22	96,8%	19,9%	37,8%	[34,3 ; 41,3]
Model fine- tune, det + rec	0,6762	14 / 22	96,8%	30,9%	46,3%	[42,2 ; 50,3]
Model fine- tune, chỉ rec	0,8758	15 / 22	94,1%	44,5%	56,0%	[51,7 ; 60,4]

Nguồn: 29-reconly-ablation.json, 31-demo-ab-reconly.json, [34-scene-level-a7.md](#).

Cột trái và các cột phải cho hai thứ tự ngược nhau, và các cột phải mới là cột đúng. Mọi ảnh trong ngữ liệu 2.801 mẫu là bản xuất Roboflow **đã cắt khít quanh biển** — bộ dò chữ đặt vào đó thì không còn gì để khoan. Nhưng vùng cắt mà hệ thống thật sự phải đọc do **YOLO sinh ra từ ảnh toàn cảnh** và lỏng hơn nhiều (dính cản xe, kính chắn gió, nền đường); bỏ bước phát hiện thì bộ nhận dạng đọc luôn phần nền thành ký tự. **Một khác biệt nữa, nghiêm trọng hơn:** chế độ chỉ-nhận-dạng **không có khả năng trả về chuỗi rỗng** — trên **1.606 khung** biển do bộ phát hiện sinh ra nó trả chuỗi ở **cả 1.606**, trong khi bản giao hàng trả rỗng ở **173 khung**; khi bộ phát hiện bắt nhầm một tấm biển quảng cáo, bản giao hàng **im lặng** còn chế độ chỉ-rec **bịa ra một biển số**, và với hệ thống ghi vào cơ sở dữ liệu thì bịa nguy hiểm hơn im lặng. **Quyết định: giữ bước phát hiện chữ.** Công tắc ALPR_OCR_SKIP_DETECTION được cài đặt, mặc định **tắt**, ghim bằng kiểm thử; không xóa vì hiệu ứng "chỉ-rec giúp model fine-tune, hại model gốc" là thật.

Điều kiện để xét lại là một tập ảnh toàn cảnh có nhãn chuỗi, và **lỗi hồng đó đã được lấp ngày 02/08/2026: 608 khung biển** trên ảnh hiện trường được gán nhãn, lấy mẫu **phân tầng** vì 1.232/1.606 khung thuộc nhóm bất đồng — dùng riêng nhóm đó sẽ cho con số bị quan sai lệch. **Phải phát biểu cho đúng mức:** bản giao hàng đứng đầu, nhưng chênh với ứng viên gần nhất chỉ **0,3 điểm** và hai khoảng tin cậy **chồng gần như hoàn toàn**, nên kết luận đúng không phải "*bản giao hàng chính xác hơn*" mà là **"không có bằng chứng để đổi"**; hai cấu hình còn lại thua rõ, nằm ngoài khoảng tin cậy. Cột "tầng dễ" hé lộ điều mà ngữ liệu ảnh cắt sẵn không thấy được: fine-tune + chỉ-rec là cấu hình **duy nhất kém đi ở ca dễ** — 94,1% so với 96,8%, tức **10 biển đọc hồng thêm** trên 372 khung đã đếm hết. Nó thắng ở ca khó nhưng đánh mất ca dễ, và đó là lý do lợi thế 12,46 điểm không sống sót ở đường chạy thật.

Đây là **lần thứ tư cùng một họ lỗi** — và là lần đầu **chặn được trước khi vào bản giao**. Quy tắc rút ra: *không đổi cấu hình mặc định dựa trên một phép đo mà đầu vào của nó khác đầu vào thật*. Diễn biến cả bốn lần ở **Phụ lục K**.

5.7. Đối chiếu toàn bộ chỉ tiêu phi chức năng

Bảng tổng hợp trình bày khi bảo vệ, liệt kê **đầy đủ mọi mã NFR** đã đặt ra ở Phase 0, không lọc bỏ mã nào — kể cả những mã không đạt.

Bảng 5.10. Đối chiếu chỉ tiêu phi chức năng, gom theo nhóm — bảng đầy đủ từng mã ở **Phụ lục H.6**

Nhóm	Số chỉ tiêu	Kết quả	Con số quyết định
Độ chính xác — phát hiện (A1, A2, A3)	3	 đạt cả ba, biên rộng	mAP@0,5 = 0,9829 (mục tiêu 0,90); Precision · Recall = 0,9837 · 0,9714

Nhóm	Số chỉ tiêu	Kết quả	Con số quyết định
Độ chính xác — nhận dạng chuỗi (A4 – A7)	4	● một, ✗ ba	A4 = 0,9454 (vượt sần 0,92, dưới mục tiêu 0,95); A5 · A6 · A7 = 0,6373 · 0,7512 · 0,5552 , cả ba dưới sần
Đóng góp hậu xử lý (A6 – A5)	1	✓	+11,39 điểm — 319 sửa đúng, 0 làm hỏng , trên 2.801 biến
Báo cáo tách bạch (A8, A9)	2	● A8, □ A9	Chênh lệch layout: 2,09 điểm ở phát hiện so với 25,45 điểm ở nhận dạng. A9 không đo được — bộ dữ liệu không có nhãn điều kiện chụp
Hiệu năng — độ trễ (P1, P2, P3)	3	● P1, ✗ P2, ✓ P3	p95 một ảnh 1.143,10 ms (sần 1.500, mục tiêu 800; p50 chỉ 405,77 ms). FPS thời gian thực 2,379 — dưới sần 3. Video 0,746× — vượt mục tiêu 0,3×
Hiệu năng — tài nguyên (P4 – P7)	5	✓ đạt cả năm	Nạp mô hình 6,41 s ; overhead API 19,01 ms ; truy vấn 10.000 bản ghi 18,71 ms ; RSS 0,806 GB
Độ tin cậy và chịu tải (R1 – R5, SC1 – SC3)	8	✓ đạt cả tám	100,0% thành công qua 2.028 yêu cầu soak 15 phút; 0/9.031 bản ghi mất sau khởi động lại; 10 yêu cầu đồng thời ổn định
Bảo trì, bảo mật, khả dụng, ràng buộc (M, S, U, C)	14	✓ 13 , ⚠ 1	Bao phủ kiểm thử tăng nghiệp vụ 87,7% (sần 70%); chạy không cần GPU; riêng M6 còn 83 cảnh báo E501 của ruff

Không mã NFR nào bị loại khỏi bảng đầy đủ ở Phụ lục H.6, kể cả những mã không đạt. Điều đáng đọc nhất ở bảng gom nhóm trên là **vạch ngăn "đạt / không đạt" trùng khít vạch ngăn giữa hai tầng của hệ thống**: mọi chỉ tiêu ở tầng phát hiện, tài nguyên, độ tin cậy và chịu tải đều đạt với biên rộng; toàn bộ phần không đạt nằm ở **độ chính xác chuỗi đầy đủ** và ở **độ trễ**, mà độ trễ lại là hệ quả trực tiếp của những bước thêm vào để cứu chính độ chính xác ấy. Cách đọc đầy đủ ở mục 5.9.

Ghi chú về NFR-A9. Chỉ tiêu phát biểu **có điều kiện** ngay từ Phase 0 — "*báo cáo độ chính xác theo điều kiện ảnh, nếu bộ dữ liệu có nhãn phù hợp*" — và điều kiện đó không thoả: không bộ dữ liệu nguồn nào gán nhãn ban ngày / ban đêm / nghiêng / mờ. Ghi □ **không đo được** thay vì ✗ **không đạt** là phân biệt có chủ ý: một chỉ tiêu chưa có dữ liệu để đo khác một chỉ tiêu đã đo và trượt.

Ghi chú về NFR-A9. Chỉ tiêu phát biểu **có điều kiện** từ Phase 0: "*báo cáo độ chính xác theo điều kiện ảnh (ban ngày / ban đêm / nghiêng / mờ), nếu bộ dữ liệu có nhãn phù hợp*". Bộ v3 **không có nhãn điều kiện chụp thống nhất**, nên **không** gán nhãn bằng suy đoán (ví dụ dùng độ sáng trung bình để suy ra "ban đêm") vì nhãn suy đoán tạo ra bảng kết quả trông chặt chẽ nhưng đo một đại lượng không xác định; trạng thái đúng để báo cáo là **NFR-A9**

không đánh giá được vì thiếu nhãn. Phương án nếu có thời gian: gán nhãn thủ công cho tập con khoảng 200–300 ảnh và công bố rõ đó là tập con gán nhãn thủ công.

Kết quả kiểm thử phần mềm. Nhóm NFR-M, S, C, U kiểm chứng bằng bộ kiểm thử tự động: **1.001 test thu thập, 1.000 pass, 1 xfail, 0 fail, 0 skip** (chạy `pytest -q` tại gốc kho ngày 2026-08-02); độ bao phủ **tăng nghiệp vụ 87,7%** — mốc 2026-07-20 trong 13-refactor-result.json (**2.931 câu lệnh / 317 bỏ sót**, số test khi ấy **882/881**) — so với chỉ tiêu $\geq 70\%$ của NFR-M2,  đạt; cặp **862/861** trong các bản tài liệu trước là lần chạy cũ hơn và đã bị thay thế. Bao phủ tăng nghiệp vụ do ở Phase 7 trước đó là **88,1%**, bao phủ **toàn kho mã** ở Phase 7 là **42,0%** (07-testing-report.md) — cả hai đều là số đo thật ở hai thời điểm khác nhau, giữ nguyên kèm mốc thời gian thay vì chọn một con số rồi xoá con số kia. Chênh lệch 88,1% $\leftrightarrow$ 42,0% là **có chủ ý**: NFR-M2 đặt ngưỡng cho **tăng nghiệp vụ** — nơi chứa logic có thể sai âm thầm (luật hậu xử lý, xác thực đầu vào, thao tác cơ sở dữ liệu) — còn 42,0% toàn kho gồm cả script tiện ích, mã sinh biểu đồ, mã tải bộ dữ liệu, những phần chi phí viết test cao mà rủi ro sai lầm lặng thấp; công bố riêng 88,1% mà không nói mẫu số là một dạng chọn lọc số liệu có lợi. Test xfail duy nhất phải được nêu tên khi công bố — nó đánh dấu một hành vi đã biết là chưa đúng, không phải test bị vô hiệu hoá để bảng kết quả sạch: `tests/integration/test_api_detection.py::TestErrorBodies::test_a_failed_image_detection_records_the_failed_job`, trong đó `DetectionService._fail_job` gọi `db.rollback()` trước khi ghi bản ghi thất bại còn `_create_job` mới chỉ `flush`, nên dòng job bị huỷ và một lần tải ảnh thất bại hiện **không để lại dòng nào** trong bảng `DetectionJob`.

5.8. Phân tích lỗi

Bảng 5.11. Tần suất từng loại lỗi

Mã	Loại lỗi	Số ca	Tỉ lệ trong tổng ca sai	Tỉ lệ toàn tập đánh giá	Một dòng	Hai dòng
E1	Bỏ sót biến	335	—	—	—	—
E2	Phát hiện nhầm	(chưa đo)	—	—	—	—
E3	Nhầm ký tự	445	63,85%	15,89%	17	428
E4	Thiếu ký tự	73	10,47%	2,61%	0	73
E5	Thừa ký tự	18	2,58%	0,64%	5	13
E6	Sai thứ tự	0	0,00%	0,00%	0	0
	Tổng số ca sai	697	100%	24,88%	—	—
	Tổng ca đánh giá (mẫu số)	2.801	n/a	100%	—	—

Phân tích đầy đủ — từng dòng của bảng, các ca điển hình và hệ quả kéo theo — ở **Phụ lục P.4**.

5.9. Bàn luận

5.9.1. Đọc kết quả: đạt gì, không đạt gì

Chương 6 tổng hợp đầy đủ kết quả và hạn chế; mục này chỉ nêu **cách đọc** bộ số liệu vừa trình bày. **Vạch ngăn nằm giữa hai tầng, không rải đều**: bộ phát hiện đạt toàn bộ chỉ tiêu với biên rộng và điểm yếu duy nhất — dải "rất nhỏ" ở 5.4.3 — được phơi bày chứ không giấu; khối hậu xử lý đóng góp **thuần dương, không rủi ro** (+11,39 điểm, 0 ca hồi quy); còn ba chỉ tiêu độ chính xác chuỗi thì không đạt.

Phần không đạt có định vị, không mơ hồ. Toàn bộ khoảng thiếu nằm ở biển **hai dòng** vốn chiếm **79,8%** tập có nhãn chuỗi: A5 = 0,6373 thiếu **16,27 điểm** so với sàn 0,80; A6 = 0,7512 thiếu **9,88 điểm**; A7 = 0,5552 thiếu **26,48 điểm**. Biển một dòng về cơ bản đã giải xong, nên "OCR không đạt" là một **phát hiện có tọa độ**, không phải một thất bại chung chung. Khoảng cách **25,45 điểm** giữa hai bố cục cùng bậc độ lớn với mốc **48,6 điểm** mà Laroca và cộng sự đo trên bộ **RodoSol-ALPR của Brazil** — **đặc tính có cấu trúc của bài toán**, không phải lỗi cài đặt sửa nhanh được. Phần lỗi còn lại đã dịch từ "đọc hỏng cả chuỗi" sang "đọc hụt ký tự", mà hậu xử lý theo luật **về nguyên tắc không thể phục hồi một ký tự chưa từng được đọc ra** — nên hướng khắc phục bắt buộc nằm ở **tầng nhận dạng**, không ở hậu xử lý cũng không ở hình học (đã đo tách bạch ở 5.5.7). Riêng A7 phải đọc như **cận dưới bi quan** (5.5.5).

Hai chỉ tiêu **chuyển trạng thái** sau lượt đo lại 28/07: NFR-A4 từ  sang  (0,8848 → **0,9454**), và NFR-P1 từ  sang  (731 → **1.143 ms**) — **thoái lui có chủ ý**, cái giá của bậc thang thử-lại đổi lấy 34 biển đọc thêm. So với lượt 20/07, A6 tăng **7,82 điểm** — mức cải thiện lớn nhất của cả đề án ở tầng nhận dạng, đạt được **không tốn một giây GPU nào** — nhưng **không chỉ tiêu nào trong ba chỉ tiêu ấy chuyển sang đạt**: một cải thiện đo được, không phải một lời giải.

Bản thân tính trung thực của quy trình đánh giá là một kết quả. Bốn lần trong đề án, phép đo tự bác bỏ chính nó và điều đó được ghi lại thay vì giấu đi: lập luận vòng tròn khi kiểm chứng rò rỉ (5.3.2); con số 5.857 ms bị nhiễm tải nền (5.6.1); ba biến cùng đổi nên không quy kết được nguyên nhân (3.6.1); và **ba lần** đường đo chạy một pipeline ngắn hơn pipeline sản phẩm (5.5.6) — trong đó lần thứ ba chứng minh biện pháp phòng ngừa đặt ra sau lần thứ hai **đã được tuân thủ đầy đủ mà vẫn thất bại**, vì nhầm sai nguyên nhân gốc.

5.9.2. Sáu hạng mục chưa đo và trạng thái khắc phục

Hạng mục	Trạng thái	Có làm được trong khuôn khổ đề án?
NFR-A9 — độ chính xác theo điều kiện ảnh	Không có nhãn điều kiện chụp trong bộ dữ liệu	 Không — thiếu điều kiện; khắc phục một phần bằng gán nhãn thủ công cho tập con
So sánh backend suy	Chưa chạy benchmark_cpu	 Có — chỉ cần thời

Hạng mục	Trạng thái	Có làm được trong khuôn khổ đồ án?
luận PyTorch ↔ ONNX ↔ OpenVINO (5.6.3)		gian máy
Phân rã đóng góp theo từng nhóm luật (5.5.2)	Chưa có cơ chế bật/tắt luật trong <code>plate_rules.py</code>	✅ Có — cần viết thêm mã
Thí nghiệm cô lập biến E1 – E3 (3.6.2)	Ước tính ≈ 33 giờ CPU , vượt ngân sách	❌ Không trong khuôn khổ đồ án
Benchmark engine OCR hứa ở mục 3.3	✅ Đã chạy 03/08/2026 (3.3.3) — PaddleOCR 68,87% so với EasyOCR 14,28% và Tesseract 10,28%	— đã hoàn thành
Huấn luyện YOLO26n làm đối chứng hứa ở mục 3.2	Chưa huấn luyện — ngân sách CPU dồn hết cho lượt <code>best.pt</code>	✅ Có — chỉ cần thời gian máy

Phân biệt "**chưa đo vì chưa tới lượt**" với "**không đo được vì thiếu điều kiện**" là quan trọng khi đọc bảng này: chỉ nhóm thứ hai — NFR-A9 thiếu nhãn, và A7 thiếu tập ảnh hiện trường có nhãn chuỗi ở thời điểm đo — mới là hạn chế thật của công trình.

5.9.3. Các mối đe dọa đến tính hợp lệ của kết quả

Nguyên tắc: **nêu mối đe dọa, đánh giá mức nghiêm trọng, nói rõ đã làm gì để giảm thiểu — kể cả khi biện pháp giảm thiểu là "không có".**

Bảng 5.12. Tám mối đe dọa đến tính hợp lệ của kết quả — phân tích đầy đủ ở **Phụ lục L**

#	Mối đe dọa	Mức	Biện pháp giảm thiểu đã áp dụng
1	Rò rỉ dữ liệu tồn dư không khử được bằng băm tri giác	Cao	Nâng ngưỡng gộp 5 → 10 và đo ở nhiều ngưỡng cao hơn ngưỡng gộp. Vẫn còn 791 cặp ở Hamming 12; rò rỉ <i>ngữ nghĩa</i> (cùng một xe, góc khác) không ngưỡng phash nào phát hiện được ⇒ mọi chỉ số ở 5.4 và 5.6 phải coi là cận trên lạc quan
2	Tập test không xuyên bộ dữ liệu	Cao	Không có — cách đúng là giữ lại một nguồn hoàn toàn không dùng để huấn luyện; chưa thực hiện
3	Mẫu số nhỏ cho các chỉ số OCR (2.801 / 15.133 ảnh có nhãn chuỗi)	Cao	Công bố mẫu số ở mọi bảng của 5.5; không rút kết luận về chênh lệch nhỏ
4	Đo trên một cấu hình phần cứng duy nhất	Trung bình	Công bố cấu hình đầy đủ ở 5.2; không ngoại suy sang CPU, hệ điều hành hay số nhân khác
5	Một lượt huấn	Trung	Cố định <code>seed = 42</code> để tái lập; không phát biểu so sánh

#	Mối đe dọa	Mức	Biện pháp giảm thiểu đã áp dụng
	luyện duy nhất , không ước lượng được phương sai	bình	dựa trên chênh lệch nhỏ
6	Bộ dữ liệu không đạt tiêu chí Q6 về tỉ lệ đối tượng nhỏ (10,91%)	Trung bình	Báo cáo mAP tách theo dải kích thước ở 5.4.3
7	Nhãn layout suy ra từ tỉ lệ khung hình khi bộ dữ liệu không khai báo	Thấp – TB	Ưu tiên nhãn lớp tường minh khi có; ghi rõ tỉ lệ ô suy bằng heuristic
8	Ma trận nhầm lẫn ký tự phụ thuộc thuật toán căn chỉnh chuỗi	Thấp	Áp ngưỡng tần suất tối thiểu trước khi đưa một cặp vào bảng luật (5.5.4)

Ba mối đe dọa mức *Cao* đều thuộc về **dữ liệu**, không thuộc về mô hình hay cách cài đặt — và cả ba đều đã có hướng khắc phục xác định, chỉ thiếu nhãn hoặc thiếu một nguồn dữ liệu độc lập.

5.10. Đối chiếu với các công trình đã công bố

Ba khác biệt khiến việc đặt cạnh nhau hai con số độ chính xác của hai công trình là **không hợp lệ về phương pháp**, trừ khi cả ba đều trùng. **Bộ dữ liệu và quốc gia**: biển Trung Quốc chủ yếu một dòng, biển Brazil có bố cục và phong chữ riêng, biển Việt Nam có tỷ lệ biển hai dòng cao. **Định nghĩa chỉ số**: "*accuracy*" trong các bài được khảo sát khi thì là mức ký tự, khi là mức chuỗi, khi là toàn trình tính cả bước phát hiện. **Điều kiện ảnh**: camera tĩnh ở trạm thu phí khác hẳn ảnh chụp tự do. Bằng chứng mạnh nhất đến từ chính lĩnh vực: Laroca và cộng sự (2022) chạy **12 mô hình OCR trên 9 tập dữ liệu công khai**, độ chính xác trung bình **sụt từ 82,4% xuống 45,2%** khi đánh giá xuyên tập dữ liệu.

⚠ Hệ quả bắt buộc cho toàn mục này. Mọi con số của công trình khác đều **kèm tên bộ dữ liệu và quốc gia ngay trong bảng**, và không con số nào được dùng để kết luận rằng hệ thống của đồ án tốt hơn hay kém hơn. Chúng chỉ trả lời một câu hỏi hẹp hơn nhiều: *kết quả của đồ án có nằm trong vùng giá trị mà lĩnh vực đã ghi nhận hay không.*

Bảng 5.13. Đối chiếu với các công trình đã công bố — mọi dòng kèm bộ dữ liệu và quốc gia

Khối	Công trình	Bộ dữ liệu · quốc gia	Chỉ số công bố	Giá trị
Phát	Batra và cộng sự	Google Open Images +	mAP@0.5	87,2%

Khối	Công trình	Bộ dữ liệu · quốc gia	Chỉ số công bố	Giá trị
hiện	(2022)	biển Ấn Độ, 5.991 ảnh		
Phát hiện	Ba nghiên cứu dùng YOLO11 cho ALPR (mục 3.2)	các tập khác nhau	mAP@0.5	90,6% – 99,5%
Phát hiện	Đồ án này	corpus Việt Nam hợp nhất, 1.514 ảnh test	mAP@0.5	98,29%
Nhận dạng	Xu và cộng sự — RPnet (2018)	CCPD · Trung Quốc	accuracy end-to-end	98,5%
Nhận dạng	Laroca và cộng sự (2021)	8 tập từ 5 khu vực	recognition rate trung bình	96,9%
Nhận dạng	Xu và cộng sự — LPTR-AFLNet (2025)	biển Trung Quốc	accuracy riêng biển hai dòng	99,37%
Nhận dạng	Tran và Bui (2024)	biển Việt Nam, chạy trên Raspberry Pi 4	accuracy	95,68%
Nhận dạng	Đồ án này	2.801 biển Việt Nam có nhãn chuỗi	A6 / A7	75,12% / 55,52%

Khối phát hiện so sánh được nhiều nhất vì **mAP@0.5** có định nghĩa thống nhất; kết quả của đồ án nằm trong vùng đã công bố — điều này **không** chứng minh mô hình tốt hơn hay kém hơn công trình nào, mỗi dòng đo trên một tập khác nhau, nhưng xác nhận khối phát hiện không có bất thường. Điều kiện phải nêu kèm: tập test của đồ án **không xuyên bộ dữ liệu**, nên **98,29% lạc quan hơn** mức đạt được khi gặp nguồn ảnh hoàn toàn mới (mục 5.3.2).

Ở khối nhận dạng, khoảng cách là thật và không được lấy khác biệt bộ dữ liệu ra biện minh cho toàn bộ nó. Nhưng chẩn đoán ở 5.5.3 định vị nó rất rõ: phần thiếu hụt nằm gần như trọn ở biển hai dòng — loại biển chiếm tỷ lệ lớn ở Việt Nam nhưng tỷ lệ nhỏ trong các bộ dữ liệu Trung Quốc mà phần lớn công trình ở bảng trên dùng. Hai dòng đáng đọc kỹ nhất là **LPTR-AFLNet (99,37% riêng biển hai dòng)** và **Tran–Bui (95,68% trên biển Việt Nam)**: cả hai cho thấy vùng giá trị này **đạt được**, tức khoảng cách của đồ án không phải giới hạn của bài toán mà là giới hạn của lựa chọn kỹ thuật — đồ án dùng engine OCR **đa ngữ tổng quát chưa tinh chỉnh**, hai công trình kia dùng mô hình huấn luyện riêng cho biển số. Đây là hướng phát triển ưu tiên cao nhất ở mục 5.4.1, và mục 4.5.3 đã đo thử một bước theo hướng đó.

Bốn điều đồ án báo cáo mà khảo sát (mục 2.5.4) không tìm thấy tương đương, đều thuộc cách **báo cáo** kết quả: *tách riêng độ chính xác biển một dòng và hai dòng trên cùng hệ thống* (Bảng 5.5); *báo cáo độ chính xác toàn trình mức chuỗi bên cạnh mAP khâu phát hiện* — **98,29%** và **55,52%**, mà **con số thứ hai kém hơn hẳn con số thứ nhất**, chính là lý do khoảng trống này tồn tại vì báo cáo toàn trình thì phải công bố cả phần hỏng; *công bố số hiệu năng kèm phần cứng* (5.2); và *đóng góp thuần của khối hậu xử lý theo luật* (5.5.2).

5.11. Kết luận chương

Trả lời trực tiếp sáu câu hỏi nghiên cứu. RQ1: bộ phát hiện YOLO11n đạt **toàn bộ** chỉ tiêu, vượt mục tiêu (5.4.1). RQ2: có, chênh lệch giữa hai layout **có ý nghĩa và rất lớn — 25,45 điểm A6** — nhưng nằm ở tầng OCR (Bảng 5.5) chứ không ở tầng phát hiện (2,09 điểm, Bảng 5.2). RQ3: khối hậu xử lý đóng góp **+11,39 điểm**, sửa đúng **319** biển, làm hỏng **0** (Bảng 5.4), dồn gần trọn vào biển hai dòng (**+13,97 điểm**) nhưng không đủ tới ngưỡng; việc định vị đóng góp về từng nhóm luật **chưa đo được** và là hạng mục cần viết mã. RQ4: NFR-P1 **chỉ đạt ngưỡng tối thiểu** (●) với $p95 = 1.143,10\text{ ms}$ — thoái lui có chủ ý đổi lấy 34 biển đọc thêm; cùng nguyên nhân đó làm **NFR-P2 trượt cả sàn (2,379 FPS, sàn 3)**; nút thắt thời gian vẫn là OCR (**64,3%**) và detector (**34,0%**), không phải 93,3% / 6,7% như báo cáo cũ, nên tối ưu detector giờ có ý nghĩa thật. RQ5: bảng luật hiện hành **phần lớn không khớp** cặp nhằm thật — chỉ **2/10** cặp nhằm nhiều nhất được phủ (Bảng 5.6). RQ6: các mối đe dọa liệt kê và đánh giá ở 5.9.3, ba mối nghiêm trọng nhất ở mức "cao". Kết luận hiệu năng đã **đảo hai lần** — con số cũ 5.857 ms bị bác bỏ, rồi bậc thang thử-lại đảo ngược lần nữa (5.6.1): **kiến trúc phần mềm không còn là vấn đề; nhưng độ trễ suy luận thì vẫn là vấn đề**, bên cạnh độ chính xác OCR trên biển hai dòng.

Các giới hạn nghiêm trọng nhất (5.9.3): (i) rò rỉ tồn dư không khử được — ở Hamming 12 vẫn còn **791 cặp** gần trùng train↔test, rò rỉ ngữ nghĩa thì không đo được; (ii) tập test **không xuyên bộ dữ liệu**; (iii) mẫu số nhỏ cho chỉ số OCR (**2.801** biển có nhãn chuỗi trên 15.133 ảnh); (iv) chưa có tập vùng cắt nhỏ do chính bộ phát hiện sinh ra kèm nhãn chuỗi nên bậc siêu phân giải **chưa đo được** lợi ích. Hệ quả: **mọi chỉ số độ chính xác trong chương nên được đọc như cận trên lạc quan**; riêng $A7 = 0,5552$ thì ngược lại — **cận dưới bi quan** do giao thức đo trên ảnh crop.

Chuyển tiếp sang Chương 6. Chương này xác định bằng số liệu — không bằng phỏng đoán — ba nhóm hướng phát triển: (i) **tối ưu hoặc thay thế khối OCR** cho biển hai dòng, từ 5.5.3 và breakdown 5.6.2; (ii) **hiệu chỉnh bảng luật sửa lỗi theo ma trận nhằm lẫn đo được** (8/10 cặp nhằm nhiều nhất chưa có luật phủ), từ 5.5.4, với ràng buộc phải kiểm chứng trên tập giữ riêng để tránh khớp luật trên chính tập đánh giá; (iii) **xây dựng tập test xuyên bộ dữ liệu, chia split theo nhóm biển số, và gán nhãn chuỗi cho một phân bố hiện trường** để đo NFR-A7 và NFR-A9 đúng cách, từ 5.3.2, 5.5.5 và 5.11.3.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

Đồ án đã bàn giao một hệ thống nhận dạng biển số xe Việt Nam hoàn chỉnh, chạy đầu-cuối trên máy **không có GPU**: bộ phát hiện tự huấn luyện, khối nhận dạng ký tự, bộ luật hậu xử lý theo quy chuẩn Việt Nam, REST API, giao diện web, cơ sở dữ liệu và đóng gói Docker. Trạng thái xác minh bằng HTTP thật — 10 thao tác trên 9 đường dẫn phản hồi đúng, **1.000/1.001** kiểm thử tự động đạt, bao phủ tăng nghiệp vụ 87,7%.

Bảng 6.1. Đối chiếu chỉ tiêu cam kết ở Phase 0 với số đo trên models/best.pt

Mã	Chỉ tiêu	Mục tiêu	Đo được	
A1 · A2	mAP@0,5 · mAP@0,5:0,95 (phát hiện)	0,90 · 0,65	0,9829 · 0,7834	✓
A3	Precision · Recall (phát hiện)	0,92 · 0,90	0,9837 · 0,9714	✓
A4	1 – CER (mức ký tự)	0,95	0,9454	●
A5 · A6	Chuỗi trước · sau hậu xử lý	0,85 · 0,90	0,6373 · 0,7512	✗
A7	Toàn trình từ ảnh gốc	0,88	0,5552	✗ *
A8	Chênh lệch layout ở tầng phát hiện (điểm %)	—	2,09	—
P1	Độ trễ p95 một ảnh (ms)	≤ 800	1.143,10	●
P4 · P5 · P6	Nạp mô hình (s) · Overhead API · Truy vấn 10.000 bản ghi (ms)	≤ 15 · ≤ 50 · ≤ 500	6,41 · 19,01 · 18,71	✓
P7 · R4 · SC1	RSS (GB) · Thành công khi chạy liên tục · Yêu cầu đồng thời	≤ 2 · ≥ 99% · ≥ 5	0,806 · 100% · 10	✓

* A7 phải đọc như **cận dưới bi quan** — đo trên ảnh nằm ngoài phân bố huấn luyện của bộ phát hiện nên tỉ lệ bỏ sót bị thổi phồng (Phụ lục G.2.1).

Vạch ngăn "đạt / không đạt" trùng khít vạch ngăn giữa hai tầng: mọi chỉ tiêu phát hiện, độ tin cậy và chịu tải đều đạt; mọi chỉ tiêu độ chính xác chuỗi đầy đủ đều không đạt. Riêng NFR-P1 chỉ đạt ngưỡng tối thiểu — thoái lui **có chủ ý** đối lấy 34 biến đọc thêm.

Bốn đại lượng đo được mà khảo sát không tìm thấy tương đương trong tài liệu Việt Nam.

- Đóng góp thuần của khối hậu xử lý theo vị trí: +11,39 điểm** — sửa đúng 319 biến, làm hỏng 0 biến trên 2.801 mẫu.
- Chênh lệch giữa hai bố cục biến trên dữ liệu Việt Nam thật: 25,45 điểm** ở khối nhận dạng, so với chỉ 2,09 điểm ở khối phát hiện. Rủi ro R-04 vì vậy nằm trọn ở tầng đọc ký tự.

3. **Benchmark ba engine OCR trên 2.801 biển, cùng một tầng bao quanh:**
PaddleOCR **68,87%** so với EasyOCR 14,28% và Tesseract 10,28%. Phép đo lấp khoảng trống nghiên cứu số 4 và **bác bỏ** tài liệu công khai vốn nghiêng về EasyOCR. Nó cũng cho một kết quả trái kỳ vọng: bước tách-rời-ghép-ngang mua 34,92 điểm cho PaddleOCR nhưng chỉ 0,03 điểm cho Tesseract — **điều kiện cần, không đủ**.
4. **Bộ nhận màu nền biển đạt 97,89%** trên 1.565 ảnh có nhãn, cung cấp nguồn bằng chứng mà chuỗi ký tự không mang được: phân giải nhập nhằng giữa biển xanh nhà nước và biển trắng cá nhân khi hai chuỗi giống hệt nhau.

Ngoài các con số, đồ án để lại **một quy trình đánh giá có kiểm chứng**: mọi số liệu sinh lại được bằng một lệnh, mọi phép so sánh kèm điều kiện đo, và các kết quả âm — hai lượt tinh chỉnh bộ nhận dạng đều không thắng model gốc ở chế độ vận hành — được ghi lại thay vì bỏ đi. Phân tích đầy đủ sáu kết quả ở **Phụ lục G.1**.

6.2. Hạn chế

Bảng 6.2. Tám hạn chế của đồ án — phân tích đầy đủ ở **Phụ lục G.2**

#	Hạn chế	Mức	Hệ quả cần lưu ý
1	OCR biển hai dòng còn yếu, kéo độ chính xác toàn trình chưa đạt	Cao	A6 = 0,7512 và A7 = 0,5552 cùng dưới ngưỡng — nút thắt lớn nhất
2	Bộ dữ liệu lệch nặng về biển trắng	Cao	97,68% mẫu thuộc một lớp, nên kết luận về độ chính xác OCR chỉ áp cho biển trắng
3	Rò rỉ dữ liệu tồn dư không khử được bằng băm tri giác	Trung bình	phash chỉ bắt tương đồng bố cục sáng-tối, không bắt "cùng xe, khác ngày"
4	Tập test không xuyên bộ dữ liệu	Trung bình	mAP 0,9829 lạc quan hơn mức gặp khi triển khai với nguồn ảnh mới
5	Độ trễ chỉ đạt ngưỡng tối thiểu	Trung bình	p95 = 1.143,10 ms; đánh đổi có chủ ý lấy 34 biển đọc thêm
6	Một yêu cầu mức <i>Must</i> (FR-4.1) bị đưa ra khỏi phạm vi	Trung bình	Trang Tổng quan đã gỡ 20/07/2026 — phải nêu rõ khi bảo vệ
7	SQLite chỉ cho phép một tiến trình ghi tại một thời điểm	Thấp	Đủ cho quy mô đồ án, chặn ở triển khai đa người dùng
8	Xem trực tiếp và xử lý nền tranh chấp CPU với nhau	Thấp	Chạy video nền làm chậm luồng nhận dạng ảnh

6.3. Hướng phát triển

Bảng 6.3. Chín hướng phát triển, xếp theo mức tác động — chi tiết ở **Phụ lục G.3**

#	Hướng	Giải hạn chế	Ghi chú
1	Huấn luyện lại module nhận dạng riêng cho biển số Việt Nam	1	Hướng quan trọng nhất. Hai lượt tinh chỉnh đã thực hiện đều chưa thắng model gốc ở chế độ vận hành; Phụ lục G.3.1 nêu điều kiện để lượt sau khác kết quả
2	Thu thập dữ liệu cho các loại biển hiếm	2	Điều kiện để mở rộng kết luận ra ngoài biển trắng
3	Bổ sung nhãn chuỗi cho toàn tập	1, 2	Hiện chỉ 2.801/15.133 ảnh có nhãn chuỗi
4	Xây dựng tập test xuyên bộ dữ liệu	3, 4	Giữ nguyên một nguồn hoàn toàn không dùng để huấn luyện
5	Tăng tốc suy luận: lượng tử hoá OCR, đóng gói ONNX/OpenVINO	5	Phép so sánh runtime chưa chạy — khoản nợ ghi ở mục 5.9.2
6	Thí nghiệm cô lập biến độ phân giải · dữ liệu · số epoch	4	Ma trận E1–E3, ước tính ≈ 33 giờ CPU (Phụ lục G.3.6)
7	Bám vết đối tượng qua khung hình cho video (SORT/DeepSORT)	—	Gộp nhiều lần đọc cùng một biển thành một kết quả
8	Tách lịch chạy giữa xem trực tiếp và xử lý nền	8	Hàng đợi ưu tiên hoặc giới hạn luồng cho tác vụ nền
9	Chuyển sang PostgreSQL nếu triển khai đa người dùng	7	Chỉ cần khi vượt quy mô một tiến trình ghi

6.4. Kết luận chung

Đề tài đặt ra một bài toán có ràng buộc rõ: nhận dạng biển số xe Việt Nam, hỗ trợ **cả biển một dòng và biển hai dòng**, suy luận hoàn toàn trên CPU. Hệ thống bàn giao đáp ứng ràng buộc vận hành và đạt toàn bộ chỉ tiêu ở tầng phát hiện với biên rộng, nhưng **chưa đạt** chỉ tiêu độ chính xác ở tầng nhận dạng ký tự — và phần thiếu hụt nằm gần như trọn ở biển hai dòng.

Giá trị của đề án vì vậy không nằm ở một con số cao nhất. Nó nằm ở ba chỗ: **một hệ thống hoàn chỉnh chạy được trong đúng ràng buộc phần cứng đã tuyên bố; bốn đại lượng đo được mà tài liệu trong nước chưa công bố tách bạch**, trong đó có benchmark ba engine OCR trên chính ảnh biển số Việt Nam; và **một cách báo cáo trong đó phần chưa đạt được trình bày với cùng mức chi tiết như phần đạt** — kể cả khi điều đó có nghĩa là công bố rằng một đóng góp kỹ thuật lõi không độc lập với engine như đã kỳ vọng.

Với một hệ thống mà nút thắt đã được định vị bằng số liệu, bước tiếp theo là rõ ràng: thay module nhận dạng ký tự bằng mô hình huấn luyện riêng cho biển số Việt Nam. Ràng buộc

kiến trúc NFR-M5 — nay đã được chứng minh bằng chính phép đo ba engine — bảo đảm việc thay thế đó không chạm tới phần còn lại của hệ thống.

TÀI LIỆU THAM KHẢO

- [1] Báo Dân trí, "Việt Nam có 77 triệu xe máy, cứ 1.000 dân có 770 người sở hữu xe máy," Báo Dân trí, 2024. [Trực tuyến]. Địa chỉ: <https://dantri.com.vn/thoi-su/viet-nam-co-77-trieu-xe-may-cu-1000-dan-co-770-nguoi-so-huu-xe-may-20241104141910472.htm> (truy cập ngày 2026-07-19).
- [2] C. E. Anagnostopoulos, I. E. Anagnostopoulos, I. D. Psoroulas, V. Loumos, E. Kayafas, "License Plate Recognition From Still Images and Video Sequences: A Survey," *IEEE Transactions on Intelligent Transportation Systems*, q. 9, s. 3, tr. 377–391, 2008. doi: 10.1109/TITS.2008.922938.
- [3] S. Du, M. Ibrahim, M. Shehata, W. Badawy, "Automatic License Plate Recognition (ALPR): A State-of-the-Art Review," *IEEE Transactions on Circuits and Systems for Video Technology*, q. 23, s. 2, tr. 311–325, 2013. doi: 10.1109/TCSVT.2012.2203741.
- [4] eParking, "Nhận dạng biển số xe tự động trong bãi giữ xe thông minh," eParking, không rõ năm. [Trực tuyến]. Địa chỉ: <https://eparking.vn/nhan-dang-bien-so-xe/> (truy cập ngày 2026-07-19).
- [5] VETC, "Ô tô đi qua trạm thu phí không dừng sẽ quét biển hay quét mã thẻ," VETC, không rõ năm. [Trực tuyến]. Địa chỉ: <https://vetc.com.vn/o-to-di-qua-tram-thu-phi-khong-dung-se-quet-bien-hay-quet-ma-the-n114.html> (truy cập ngày 2026-07-19).
- [6] VisCom Solution, "VietANPR — phần mềm nhận diện biển số xe máy & xe hơi," VisCom Solution, không rõ năm. [Trực tuyến]. Địa chỉ: <https://viscomsolution.com/viet-anpr-phan-mem-nhan-dien-bien-so-xe-may-xe-hoi/> (truy cập ngày 2026-07-19).
- [7] R. Laroca, E. V. Cardoso, D. R. Lucio, V. Estevam, D. Menotti, "On the Cross-Dataset Generalization in License Plate Recognition," trong *International Conference on Computer Vision Theory and Applications (VISAPP)*, 2022. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2201.00267> (truy cập ngày 2026-07-19).
- [8] Bộ Công an, "Thông tư số 79/2024/TT-BCA quy định về cấp, thu hồi chứng nhận đăng ký xe, biển số xe cơ giới, xe máy chuyên dùng," 2024. [Trực tuyến]. Địa chỉ: <https://chinhphu.vn/?pageid=27160&docid=211945&classid=1&orggroupid=4> (truy cập ngày 2026-07-19).
- [9] Bộ Công an, "Thông tư số 13/2025/TT-BCA sửa đổi, bổ sung một số điều của Thông tư số 79/2024/TT-BCA," 2025.
- [10] Bộ Công an, "Thông tư số 51/2025/TT-BCA sửa đổi, bổ sung một số điều của Thông tư số 79/2024/TT-BCA đã được sửa đổi tại Thông tư số 13/2025/TT-BCA," 2025. [Trực tuyến]. Địa chỉ: <https://congbao.chinhphu.vn/van-ban/thong-tu-so-51-2025-tt-bca-45356.htm> (truy cập ngày 2026-07-19).

[11] Bộ Công an, "Quy chuẩn kỹ thuật quốc gia về biển số xe QCVN 08:2024/BCA," 2024. [Trực tuyến]. Địa chỉ: <https://mps.gov.vn/chinh-sach-phap-luat/bai-viet/quy-chuan-ky-thuat-quoc-gia-ve-bien-so-xe-d1-t1592> (truy cập ngày 2026-07-19).

[12] Bộ Công an, "Thông tư số 24/2023/TT-BCA quy định về cấp, thu hồi đăng ký, biển số xe cơ giới," 2023. [Trực tuyến]. Địa chỉ: <https://xaydungchinhhsach.chinhphu.vn/toan-van-thong-tu-24-2023-tt-bca-quy-dinh-ve-cap-thu-hoi-dang-ky-bien-so-xe-co-gioi-119230712221629971.htm> (truy cập ngày 2026-07-19).

[13] Bộ Công an, "Nhận diện màu sắc, seri, ký hiệu biển số xe của cơ quan, tổ chức, cá nhân từ 01/01/2025," Cổng Thông tin điện tử Bộ Công an, 2024. [Trực tuyến]. Địa chỉ: <https://bocongan.gov.vn/chinh-sach-phap-luat/bai-viet/nhan-dien-mau-sac-seri-ky-hieu-bien-so-xe-cua-co-quan-to-chuc-ca-nhan-tu-01012025-d1-t1617> (truy cập ngày 2026-07-19).

[14] Thư viện Nhà đất, "Chính thức ký hiệu biển số xe 34 tỉnh thành sau sáp nhập theo Thông tư 51/2025/TT-BCA," Thư viện Nhà đất, 2025. [Trực tuyến]. Địa chỉ: <https://thuviennhadat.vn/phap-luat/chinh-thuc-ky-hieu-bien-so-xe-34-tinh-thanh-sau-sap-nhap-theo-thong-tu-51-2025-tt-bca-690227.html> (truy cập ngày 2026-07-19).

[15] Bộ Quốc phòng, "Thông tư 169/2021/TT-BQP quy định về đăng ký, quản lý, sử dụng xe cơ giới, xe máy chuyên dùng trong Bộ Quốc phòng," 2021. [Trực tuyến]. Địa chỉ: <https://luatvietnam.vn/giao-thong/thong-tu-169-2021-tt-bqp-bo-quoc-phong-216143-d1.html> (truy cập ngày 2026-07-19).

[16] G. Jocher, J. Qiu, "Ultralytics YOLO11," Ultralytics, 2024. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/models/yolo11/> (truy cập ngày 2026-07-19).

[17] C. Cui, "PP-OCrV5: A Specialized 5M-Parameter Model Rivaling Billion-Parameter Vision-Language Models on OCR Tasks," *CVPR 2026 / arXiv:2603.24373*, 2026. [Trực tuyến]. Địa chỉ: <https://arxiv.org/html/2603.24373v1> (truy cập ngày 2026-07-19).

[18] Ultralytics, "Intel OpenVINO Export — Ultralytics Docs (ma nguồn markdown, day du bang benchmark CPU/GPU/NPU)," GitHub / Ultralytics Docs, 2026. [Trực tuyến]. Địa chỉ: https://raw.githubusercontent.com/ultralytics/ultralytics/main/docs/en/integrations/open_vino.md (truy cập ngày 2026-07-19).

[19] F. Meyer, L. Guichard, D. Coquenat, G. Gravier, Y. Soullard, B. Couasnon, "Relaxed syntax modeling in Transformers for future-proof license plate recognition," *arXiv preprint arXiv:2506.17051*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2506.17051> (truy cập ngày 2026-07-19).

[20] Z. Li, M. A. Ghaffar, "A detailed review on license plate detection and recognition methods," *Journal of Traffic and Transportation Engineering (English Edition)*, q. 13, s. 3, tr. 974–1005, 2026. doi: 10.1016/j.jtte.2024.10.007.

[21] D. H. Le, D. Mazumder, L. D. Quach, S. Banerjee, V. D. Nguyen, "Robust Vietnam's Motorcycle License Plate Detection and Recognition Using Deep Learning Model," trong

Future Data and Security Engineering (FDSE), Springer, 2023. doi: 10.1007/978-981-99-8296-7_5.

[22] S. M. Silva, C. R. Jung, "License Plate Detection and Recognition in Unconstrained Scenarios," trong *European Conference on Computer Vision (ECCV)*, 2018. doi: 10.1007/978-3-030-01258-8_36.

[23] R. Laroca, L. A. Zanolensi, G. R. Gonçalves, E. Todt, W. R. Schwartz, D. Menotti, "An efficient and layout-independent automatic license plate recognition system based on the YOLO detector," *IET Intelligent Transport Systems*, q. 15, s. 4, tr. 483–503, 2021. doi: 10.1049/itr2.12030.

[24] G. Hsu, J. Chen, Y. Chung, "Application-Oriented License Plate Recognition," *IEEE Transactions on Vehicular Technology*, q. 62, s. 2, tr. 552–561, 2013. [Trực tuyến]. Địa chỉ: https://www.researchgate.net/publication/260498098_Application-Oriented_License_Plate_Recognition (truy cập ngày 2026-07-19).

[25] R. Laroca, "UFPR-ALPR dataset — kho mã nguồn chính thức," GitHub / VRI Lab, Federal University of Parana, 2018. [Trực tuyến]. Địa chỉ: <https://github.com/raysonlaroca/ufpr-alpr-dataset> (truy cập ngày 2026-07-19).

[26] Cổng Thông tin điện tử Chính phủ, "Quy định ký hiệu biển số xe ô tô, xe máy tại các địa phương (theo Thông tư 51/2025/TT-BCA)," Xây dựng chính sách, pháp luật — Chinhphu.vn, 2025. [Trực tuyến]. Địa chỉ: <https://xaydungchinh sach.chinhphu.vn/quy-dinh-ky-hieu-bien-so-xe-o-to-xe-may-tai-cac-dia-phuong-119250704073354464.htm> (truy cập ngày 2026-07-19).

[27] Cổng Thông tin điện tử Chính phủ, "Từ 15/8, sêri biển số xe máy cấp cho xe cá nhân có 2 chữ cái," Xây dựng chính sách, pháp luật — Chinhphu.vn, 2023. [Trực tuyến]. Địa chỉ: <https://xaydungchinh sach.chinhphu.vn/tu-15-8-seri-bien-so-xe-may-cap-cho-xe-ca-nhan-co-2-chu-cai-11923082122483385.htm> (truy cập ngày 2026-07-19).

[28] Oto.com.vn, "Bỏ quy định phân biệt seri đăng ký với một số dòng xe," Oto.com.vn, 2025. [Trực tuyến]. Địa chỉ: <https://oto.com.vn/thi-truong-o-to/bo-quy-dinh-phan-biet-seri-dang-ky-voi-mot-so-dong-xe-articleid-6ehu4o0> (truy cập ngày 2026-07-19).

[29] Kho Biển Số Đẹp, "Những quy định bạn cần biết về biển số xe kể từ năm 2025," Kho Biển Số Đẹp, 2025. [Trực tuyến]. Địa chỉ: <https://khobiensodep.vn/blogs/news/nhung-quy-dinh-ban-can-biet-ve-bien-so-xe-ke-tu-nam-2025> (truy cập ngày 2026-07-19).

[30] VietNamNet, "Cách đọc ký hiệu biển số xe ngoại giao, nước ngoài ở Việt Nam," VietNamNet, 2023. [Trực tuyến]. Địa chỉ: <https://vietnamnet.vn/cach-doc-ky-hieu-bien-so-xe-ngoai-giao-nuoc-ngoai-o-viet-nam-333426.html> (truy cập ngày 2026-07-19).

[31] Công an tỉnh Lạng Sơn, "Một số quy định mới của Thông tư số 79/2024/TT-BCA quy định về cấp, thu hồi chứng nhận đăng ký xe, biển số xe cơ giới, xe máy chuyên dùng," Cổng thông tin Công an tỉnh Lạng Sơn, 2024. [Trực tuyến]. Địa chỉ: <https://congan.langson.gov.vn/9688/pho-bien-giao-duc-phap-luat/68/mot-so-quy-dinh->

[moi-cua-thong-tu-so-79-2024-tt-bca-quy-dinh-ve-cap-thu-hoi-chung-nhan-dang-ky-xe-bien-so-xe-co-gioi-xe-may-chuyen-dung/9688.aspx](#) (truy cập ngày 2026-07-19).

[32] Thư viện Pháp luật, "Quy định về màu sắc, seri biển số xe của cơ quan, tổ chức, cá nhân trong nước từ năm 2025," Thư viện Pháp luật, 2025. [Trực tuyến]. Địa chỉ: <https://thuvienphapluat.vn/banan/tin-tuc/quy-dinh-ve-mau-sac-seri-bien-so-xe-cua-co-quan-to-chuc-ca-nhan-trong-nuoc-tu-nam-2025-12612.html> (truy cập ngày 2026-07-19).

[33] "License Plate Localization Based on Edge Detection and Morphology," trong *Lecture Notes in Electrical Engineering*, Springer, 2012. doi: 10.1007/978-3-642-25899-2_92.

[34] "License plate localization based on edge-geometrical features using morphological approach," trong *IEEE International Conference*, 2013. [Trực tuyến]. Địa chỉ: <https://ieeexplore.ieee.org/document/6738937/> (truy cập ngày 2026-07-19).

[35] "Research on Characters Segmentation in One-Row and Two-Row of Vietnam License Plates," *Advanced Materials Research*, q. 479-481, 2012. [Trực tuyến]. Địa chỉ: <https://www.scientific.net/AMR.479-481.2293> (truy cập ngày 2026-07-19).

[36] mrzaizai2k, "mrzaizai2k/VIETNAMESE_LICENSE_PLATE," GitHub, 2025. [Trực tuyến]. Địa chỉ: https://github.com/mrzaizai2k/VIETNAMESE_LICENSE_PLATE (truy cập ngày 2026-07-19).

[37] R. Laroca và cộng sự, "A Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector," trong *International Joint Conference on Neural Networks (IJCNN)*, 2018. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/1802.09567> (truy cập ngày 2026-07-19).

[38] Z. Xu và cộng sự, "Towards End-to-End License Plate Detection and Recognition: A Large Dataset and Baseline," trong *European Conference on Computer Vision (ECCV)*, 2018. [Trực tuyến]. Địa chỉ: https://openaccess.thecvf.com/content_ECCV_2018/papers/Zhenbo_Xu_Towards_End-to-End_License_ECCV_2018_paper.pdf (truy cập ngày 2026-07-19).

[39] H. Li, P. Wang, C. Shen, "Toward End-to-End Car License Plate Detection and Recognition With Deep Neural Networks," *IEEE Transactions on Intelligent Transportation Systems*, q. 20, s. 3, tr. 1126–1136, 2019. doi: 10.1109/TITS.2018.2847291.

[40] S. Zherzdev, A. Gruzdev, "LPRNet: License Plate Recognition via Deep Neural Networks," *arXiv preprint arXiv:1806.10447*, 2018. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/1806.10447> (truy cập ngày 2026-07-19).

[41] L. Zhang, P. Wang, H. Li, Z. Li, C. Shen, Y. Zhang, "A Robust Attentional Framework for License Plate Recognition in the Wild," *IEEE Transactions on Intelligent Transportation Systems*, q. 22, s. 11, tr. 6967–6976, 2020. doi: 10.1109/TITS.2020.3000072.

[42] E. Shabaninia, F. Asadi-zeydabadi, H. Nezamabadi-pour, "Layout-Independent License Plate Recognition via Integrated Vision and Language Models," *arXiv preprint*

arXiv:2510.10533, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2510.10533> (truy cập ngày 2026-07-19).

[43] N. AlDahoul và cộng sự, "Advancing Vehicle Plate Recognition: Multitasking Visual Language Models with VehiclePaliGemma," *arXiv preprint arXiv:2412.14197*, 2024. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2412.14197> (truy cập ngày 2026-07-19).

[44] H. Gong, H. Liu, "LP-LLM: End-to-End Real-World Degraded License Plate Text Recognition via Large Multimodal Models," *arXiv preprint arXiv:2601.09116*, 2026. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2601.09116> (truy cập ngày 2026-07-19).

[45] Y. Wang, Z. Bian, Y. Zhou, L. Chau, "Rethinking and Designing a High-performing Automatic License Plate Recognition Approach," *IEEE Transactions on Intelligent Transportation Systems*, 2021. doi: 10.1109/TITS.2021.3087158.

[46] A. Wang và cộng sự, "YOLOv10: Real-Time End-to-End Object Detection," *arXiv preprint arXiv:2405.14458*, 2024. doi: 10.48550/arXiv.2405.14458.

[47] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, M. E. Kalfaoglu, "Ultralytics YOLO26," Ultralytics, 2025. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/models/yolo26/> (truy cập ngày 2026-07-19).

[48] "Advanced deep learning techniques for automated license plate recognition," *Scientific Reports*, 2025. [Trực tuyến]. Địa chỉ: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12639091/> (truy cập ngày 2026-07-19).

[49] G. Jocher, A. Chaurasia, J. Qiu, "Ultralytics YOLOv8," Ultralytics, 2023. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/models/yolov8/> (truy cập ngày 2026-07-19).

[50] R. Khanam, M. Hussain, "YOLOv11: An Overview of the Key Architectural Enhancements," *arXiv preprint arXiv:2410.17725*, 2024. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2410.17725> (truy cập ngày 2026-07-19).

[51] Ultralytics, "ultralytics/nn/modules/block.py — định nghĩa C2f, C3k, C3k2, C2PSA, PSABlock, Attention," GitHub, 2026. [Trực tuyến]. Địa chỉ: <https://github.com/ultralytics/ultralytics/blob/main/ultralytics/nn/modules/block.py> (truy cập ngày 2026-07-19).

[52] Ultralytics, "YOLO11 vs YOLOv8 — so sánh chính thức," Ultralytics, 2025. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/compare/yolo11-vs-yolov8/> (truy cập ngày 2026-07-19).

[53] C. Wang, I. Yeh, H. M. Liao, "YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information," *ECCV 2024 / Ultralytics Docs*, 2024. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/models/yolov9/> (truy cập ngày 2026-07-19).

- [54] Y. Tian, Q. Ye, D. Doermann, "YOLOv12: Attention-Centric Real-Time Object Detectors," *arXiv preprint arXiv:2502.12524*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2502.12524> (truy cập ngày 2026-07-19).
- [55] M. Lei và cộng sự, "YOLOv13: Real-Time Object Detection with Hypergraph-Enhanced Adaptive Visual Perception," *arXiv preprint arXiv:2506.17733*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2506.17733> (truy cập ngày 2026-07-19).
- [56] P. Batra và cộng sự, "A Novel Memory and Time-Efficient ALPR System Based on YOLOv5," *Sensors*, q. 22, s. 14, tr. 5283, 2022. doi: 10.3390/s22145283.
- [57] "Automatic License Plate Detection System with YOLOv11 Algorithm," *Journal of Applied Informatics and Computing (JAIC)*, 2025. [Trực tuyến]. Địa chỉ: <https://jurnal.polibatam.ac.id/index.php/JAIC/article/view/11484> (truy cập ngày 2026-07-19).
- [58] "Vehicle License Plate Number Detection with YOLO11," *Journal of Computer Science and Informatics Engineering (J-Cosine)*, 2025. [Trực tuyến]. Địa chỉ: <https://jcosine.if.unram.ac.id/index.php/jcosine/article/view/656> (truy cập ngày 2026-07-19).
- [59] Le Quy Don Technical University, "An efficient method to improve the accuracy of Vietnamese vehicle license plate recognition in unconstrained environment," trong *International Conference on Information and Computer Science (NICS)*, IEEE, 2021. [Trực tuyến]. Địa chỉ: <https://ieeexplore.ieee.org/document/9585279/> (truy cập ngày 2026-07-19).
- [60] Jaided AI, "JaidedAI/EasyOCR — DeepWiki (kien truc CRAFT + CRNN, kích thuoc model)," DeepWiki, 2025. [Trực tuyến]. Địa chỉ: <https://deepwiki.com/JaidedAI/EasyOCR> (truy cập ngày 2026-07-19).
- [61] "A Feasible Framework for Arbitrary-Shaped Scene Text Recognition," *arXiv preprint arXiv:1912.04561*, 2019. [Trực tuyến]. Địa chỉ: <https://arxiv.org/pdf/1912.04561> (truy cập ngày 2026-07-19).
- [62] PaddleOCR community, "PaddleOCR Issue #14109 — rec_image_shape mac dinh '3, 48, 320' tu PP-OCRv3," GitHub, không rõ năm. [Trực tuyến]. Địa chỉ: <https://github.com/PaddlePaddle/PaddleOCR/issues/14109> (truy cập ngày 2026-07-19).
- [63] "PatrolVision: Automated License Plate Recognition in the wild," *arXiv preprint arXiv:2504.10810*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/html/2504.10810v1> (truy cập ngày 2026-07-19).
- [64] we0091234, "double_plate_split_merge.py — mã tách và ghép biển hai tầng (5/12 và 1/3 + hstack)," GitHub, không rõ năm. [Trực tuyến]. Địa chỉ: https://raw.githubusercontent.com/we0091234/Chinese_license_plate_detection_recognition/main/plate_recognition/double_plate_split_merge.py (truy cập ngày 2026-07-19).

- [65] PaddlePaddle, "PaddleOCR 3.x — OCR Pipeline Usage Tutorial," PaddleOCR, không rõ năm. [Trực tuyến]. Địa chỉ: https://www.paddleocr.ai/latest/en/version3.x/pipeline_usage/OCR.html (truy cập ngày 2026-07-19).
- [66] trungdinh22, "function/helper.py — logic phân biệt biển một dòng / hai dòng bằng kiểm tra thẳng hàng (abs_tol=3) và ghép theo y_mean," GitHub, không rõ năm. [Trực tuyến]. Địa chỉ: <https://raw.githubusercontent.com/trungdinh22/License-Plate-Recognition/main/function/helper.py> (truy cập ngày 2026-07-19).
- [67] PaddlePaddle, "PaddleOCR — ứng dụng nhận dạng biển số nhẹ (CCPD, PP-OCRv3, số liệu tinh chỉnh)," PaddleOCR v2.9, không rõ năm. [Trực tuyến]. Địa chỉ: <http://www.paddleocr.ai/v2.9/applications/%E8%BD%BB%E9%87%8F%E7%BA%A7%E8%BD%A6%E7%89%8C%E8%AF%86%E5%88%AB.html> (truy cập ngày 2026-07-19).
- [68] M. Li, "TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models," *arXiv preprint arXiv:2109.10282*, 2021. [Trực tuyến]. Địa chỉ: <https://arxiv.org/pdf/2109.10282> (truy cập ngày 2026-07-19).
- [69] Roboflow, "TrOCR — Roboflow Inference Models," Roboflow, 2025. [Trực tuyến]. Địa chỉ: <https://inference-models.roboflow.com/models/trocr/> (truy cập ngày 2026-07-19).
- [70] L. Dang, V. Duong Ngoc, L. T. V. Pham Cung, "Vietnam Vehicle Number Recognition Based on an Improved CRNN with Attention Mechanism," *International Journal of Intelligent Transportation Systems Research*, 2024. doi: 10.1007/s13177-024-00402-7.
- [71] R. Laroca và cộng sự, "ICPR 2026 Competition on Low-Resolution License Plate Recognition," trong *International Conference on Pattern Recognition (ICPR)*, 2026. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2604.22506> (truy cập ngày 2026-07-19).
- [72] M. Del Castillo Velarde, G. Velarde, "Benchmarking Algorithms for Automatic License Plate Recognition," *arXiv preprint arXiv:2203.14298*, 2022. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2203.14298> (truy cập ngày 2026-07-19).
- [73] L. Tao, S. Hong, Y. Lin, Y. Chen, P. He, Z. Tie, "A Real-Time License Plate Detection and Recognition Model in Unconstrained Scenarios," *Sensors*, q. 24, s. 9, tr. 2791, 2024. doi: 10.3390/s24092791.
- [74] M. Shpir, N. Shvai, A. Nakib, "License Plate Images Generation with Diffusion Models," *arXiv preprint arXiv:2501.03374*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2501.03374> (truy cập ngày 2026-07-19).
- [75] G. Xu, P. Zuo, Z. Ke, B. Lei, "LPTR-AFLNet: Lightweight Integrated Chinese License Plate Rectification and Recognition Network," *arXiv preprint arXiv:2507.16362*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2507.16362> (truy cập ngày 2026-07-19).
- [76] L. Wojcik, G. E. Lima, V. Nascimento, E. Nascimento Jr., R. Laroca, D. Menotti, "LPLC: A Dataset for License Plate Legibility Classification," trong *Conference on Graphics*,

Patterns and Images (SIBGRAPI), 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2508.18425> (truy cập ngày 2026-07-19).

[77] Z. Ebrahimi Vargoorani, A. M. Ghoreyshi, C. Y. Suen, "Efficient License Plate Recognition via Pseudo-Labeled Supervision with Grounding DINO and YOLOv8," *arXiv preprint arXiv:2510.25032*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2510.25032> (truy cập ngày 2026-07-19).

[78] V. Nascimento, R. Laroca, R. O. Ribeiro, W. R. Schwartz, D. Menotti, "Enhancing License Plate Super-Resolution: A Layout-Aware and Character-Driven Approach," trong *Conference on Graphics, Patterns and Images (SIBGRAPI)*, 2024. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2408.15103> (truy cập ngày 2026-07-19).

[79] D. Tran-Anh, K. L. Tran, H. Vu, "License Plate Recognition Based on Multi-Angle View Model," *arXiv preprint arXiv:2309.12972*, 2023. [Trực tuyến]. Địa chỉ: <https://arxiv.org/html/2309.12972> (truy cập ngày 2026-07-19).

[80] Tran, Bui, "Implementation of a License Plate Recognition System in Vietnam Using Embedding Devices," trong *Multi-disciplinary Trends in Artificial Intelligence (MIWAI)*, Springer, 2024. doi: 10.1007/978-981-96-0695-5_19.

[81] H. Tran, G. Ma, T. Nguyen, T. Cao, "Building Vietnam's License Plate Recognition System Based on OpenALPR," *International Journal of Multidisciplinary Research and Publications (IJMRAP)*, q. 5, s. 11, tr. 133–137, 2023. [Trực tuyến]. Địa chỉ: <http://ijmrmap.com/wp-content/uploads/2023/05/IJMRAP-V5N11P102Y23.pdf> (truy cập ngày 2026-07-19).

[82] "Nghiên cứu các phiên bản YOLOv8 và YOLO-NAS trong phát hiện biển số xe," *Tạp chí Khoa học Đại học Đà Lạt*, 2024. [Trực tuyến]. Địa chỉ: <https://scholar.dlu.edu.vn/thuvienso/bitstream/DLU123456789/290338/1/100567-1297-209737-1-10-20240806.pdf> (truy cập ngày 2026-07-19).

[83] "Building a license plate recognition system for Vietnam tollbooth," trong *Proceedings of the 3rd Symposium on Information and Communication Technology (SoICT)*, ACM, 2012. doi: 10.1145/2350716.2350734.

[84] Vietnamese Association for Pattern Recognition (VAPR), "Vietnamese Bike License Plate Recognition Challenge (MAPR 2018)," 1st International Conference on Multimedia Analysis and Pattern Recognition, UIT — DHQG TP.HCM, 2018. [Trực tuyến]. Địa chỉ: <https://mapr.uit.edu.vn/2018/vietnamese-bike-license-plate-recognition> (truy cập ngày 2026-07-19).

[85] T. L. Nguyễn, X. P. Đào, T. T. U. Nguyễn, H. P. Nguyễn, "Đề xuất mô hình YOLO V5 ứng dụng trong nhận diện biển số xe," *Tạp chí Khoa học Trường Đại học Mở Hà Nội*, 2023. [Trực tuyến]. Địa chỉ: <https://vjol.info.vn/index.php/jshou/article/view/86449> (truy cập ngày 2026-07-19).

- [86] Z. Xu, "CCPD: a diverse and well-annotated dataset for license plate detection and recognition," GitHub (detectRecog / USTC), 2018. [Trực tuyến]. Địa chỉ: <https://github.com/detectRecog/CCPD> (truy cập ngày 2026-07-19).
- [87] HyperAI, "AOLP Application-Oriented License Plate Dataset," HyperAI — hyper.ai, không rõ năm. [Trực tuyến]. Địa chỉ: <https://hyper.ai/en/datasets/19309> (truy cập ngày 2026-07-19).
- [88] R. Laroca, E. V. Cardoso, D. R. Lucio, V. Estevam, D. Menotti, "RodoSol-ALPR dataset — kho mã nguồn chính thức," GitHub, 2022. [Trực tuyến]. Địa chỉ: <https://github.com/raysonlaroca/rodosol-alpr-dataset> (truy cập ngày 2026-07-19).
- [89] OpenALPR, "OpenALPR benchmarks — kho mã nguồn chính thức," GitHub, 2016. [Trực tuyến]. Địa chỉ: <https://github.com/openalpr/benchmarks> (truy cập ngày 2026-07-19).
- [90] S. Agrawal, "Global License Plate Dataset," *arXiv preprint arXiv:2405.10949*, 2024. [Trực tuyến]. Địa chỉ: <https://arxiv.org/html/2405.10949v1> (truy cập ngày 2026-07-19).
- [91] fict-labs, "VNLP — Vietnamese license plate dataset," GitHub, 2025. [Trực tuyến]. Địa chỉ: <https://github.com/fict-labs/VNLP> (truy cập ngày 2026-07-19).
- [92] "How many labeled license plates are needed?," *arXiv preprint arXiv:1808.08410*, 2018. [Trực tuyến]. Địa chỉ: <https://arxiv.org/html/1808.08410> (truy cập ngày 2026-07-19).
- [93] NNDam, "Vietnamese License Plate Generator," GitHub, 2024. [Trực tuyến]. Địa chỉ: <https://github.com/NNDam/Vietnamese-License-Plate-Generator> (truy cập ngày 2026-07-19).
- [94] Ultralytics, "Object Detection task docs — chú thích phần cứng dùng để benchmark," GitHub / Ultralytics Docs, 2026. [Trực tuyến]. Địa chỉ: <https://github.com/ultralytics/ultralytics/blob/main/docs/en/tasks/detect.md> (truy cập ngày 2026-07-19).
- [95] Sutikno, A. Sugiharto, R. Kusumaningrum, "Enhanced Automatic License Plate Detection and Recognition using CLAHE and YOLOv11 for Seat Belt Compliance Detection," *Engineering, Technology & Applied Science Research*, q. 15, s. 1, tr. 20271–20278, 2025. doi: 10.48084/etasr.9629.
- [96] Ultralytics, "Model Export with Ultralytics YOLO — danh sách hơn 20 định dạng xuất và tham số," Ultralytics, 2026. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/modes/export/> (truy cập ngày 2026-07-19).
- [97] Ultralytics, "Model Benchmarking with Ultralytics YOLO — che do benchmark tu dong tren CPU," Ultralytics, 2026. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/modes/benchmark/> (truy cập ngày 2026-07-19).

- [98] Ultralytics, "Ultralytics Licensing (AGPL-3.0 va Enterprise)," Ultralytics, 2026. [Trực tuyến]. Địa chỉ: <https://www.ultralytics.com/license> (truy cập ngày 2026-07-19).
- [99] "Comparative study of YOLO models for Oman car plate detection," *ScienceDirect*, 2026. [Trực tuyến]. Địa chỉ: <https://www.sciencedirect.com/science/article/pii/S277318632600068X> (truy cập ngày 2026-07-19).
- [100] "Optimized YOLOv8 for Automatic License Plate Recognition on Resource Constrained Devices," *Engineering, Technology & Applied Science Research*, q. 15, s. 2, 2025. doi: 10.48084/etasr.9983.
- [101] Tesseract OCR project, "Tesseract Release Notes," Tesseract OCR, 2026. [Trực tuyến]. Địa chỉ: <https://tesseract-ocr.github.io/tessdoc/ReleaseNotes.html> (truy cập ngày 2026-07-19).
- [102] PaddlePaddle, "Text Detection Module — PaddleX Documentation," PaddleX, 2026. [Trực tuyến]. Địa chỉ: https://paddlepaddle.github.io/PaddleX/3.4/en/module_usage/tutorials/ocr_modules/text_detection.html (truy cập ngày 2026-07-19).
- [103] PaddlePaddle, "Text Recognition Module — PaddleOCR/PaddleX Documentation," PaddleOCR / PaddleX, 2026. [Trực tuyến]. Địa chỉ: http://www.paddleocr.ai/main/en/version3.x/module_usage/text_recognition.html (truy cập ngày 2026-07-19).
- [104] A. Rosebrock, "Tesseract Page Segmentation Modes (PSMs) Explained: How to Improve Your OCR Accuracy," PyImageSearch, 2021. [Trực tuyến]. Địa chỉ: <https://pyimagesearch.com/2021/11/15/tesseract-page-segmentation-modes-psms-explained-how-to-improve-your-ocr-accuracy/> (truy cập ngày 2026-07-19).
- [105] PaddleOCR community, "Is there any option to whitelist or blacklist character in PaddleOCR — Discussion 7515," GitHub, 2022. [Trực tuyến]. Địa chỉ: <https://github.com/PaddlePaddle/PaddleOCR/discussions/7515> (truy cập ngày 2026-07-19).
- [106] Jaided AI, "EasyOCR API Documentation," Jaided AI, 2025. [Trực tuyến]. Địa chỉ: <https://www.jaided.ai/easyocr/documentation/> (truy cập ngày 2026-07-19).
- [107] A. Rosebrock, "Whitelisting and Blacklisting Characters with Tesseract and Python," PyImageSearch, 2021. [Trực tuyến]. Địa chỉ: <https://pyimagesearch.com/2021/09/06/whitelisting-and-blacklisting-characters-with-tesseract-and-python/> (truy cập ngày 2026-07-19).
- [108] OpenMMLab, "open-mmlab/mmodocr — GitHub," GitHub, 2023. [Trực tuyến]. Địa chỉ: <https://github.com/open-mmlab/mmodocr> (truy cập ngày 2026-07-19).

- [109] A. Kandratavicius, "ankandrew/fast-plate-ocr — GitHub," GitHub, 2026. [Trực tuyến]. Địa chỉ: <https://github.com/ankandrew/fast-plate-ocr> (truy cập ngày 2026-07-19).
- [110] Mindee, "Choosing the right model — docTR documentation," Mindee, 2026. [Trực tuyến]. Địa chỉ: https://mindee.github.io/doctr/latest/using_doctr/using_models.html (truy cập ngày 2026-07-19).
- [111] "Enhancing automated vehicle identification by integrating YOLO v8 and OCR techniques for high-precision license plate detection and recognition," *Scientific Reports*, 2024. doi: 10.1038/s41598-024-65272-1.
- [112] PaddlePaddle Team, "PP-OCRv6: From 1.5M to 34.5M Parameters, Surpassing Billion-Scale VLMs on OCR Tasks," *arXiv preprint arXiv:2606.13108*, 2026. [Trực tuyến]. Địa chỉ: <https://arxiv.org/html/2606.13108v1> (truy cập ngày 2026-07-19).
- [113] Y. Du, "PP-OCR: A Practical Ultra Lightweight OCR System," *arXiv preprint arXiv:2009.09941*, 2020. [Trực tuyến]. Địa chỉ: <https://arxiv.org/pdf/2009.09941> (truy cập ngày 2026-07-19).
- [114] Y. Du, "PP-OCRv2: Bag of Tricks for Ultra Lightweight OCR System," *arXiv preprint arXiv:2109.03144*, 2021. [Trực tuyến]. Địa chỉ: <https://arxiv.org/pdf/2109.03144> (truy cập ngày 2026-07-19).
- [115] Reddy, Shruthi, "License Plate Detection using YOLO v8 and Performance Evaluation of EasyOCR, PaddleOCR and Tesseract," trong *IEEE International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, 2024. [Trực tuyến]. Địa chỉ: <https://ieeexplore.ieee.org/document/10725878/> (truy cập ngày 2026-07-19).
- [116] Intel OpenVINO, "Precision Control — OpenVINO documentation (FP16 chuyển về FP32 trên CPU, bf16/AMX)," Intel, 2025. [Trực tuyến]. Địa chỉ: <https://docs.openvino.ai/2025/openvino-workflow/running-inference/optimize-inference/precision-control.html> (truy cập ngày 2026-07-19).
- [117] Microsoft ONNX Runtime, "Thread management — ONNX Runtime Performance Tuning (intra/inter op threads, spinning, NUMA)," Microsoft, 2025. [Trực tuyến]. Địa chỉ: <https://onnxruntime.ai/docs/performance/tune-performance/threading.html> (truy cập ngày 2026-07-19).
- [118] PaddlePaddle, "Introduction to PP-OCRv5 — PaddleOCR Documentation," PaddleOCR, 2026. [Trực tuyến]. Địa chỉ: <http://www.paddleocr.ai/main/en/version3.x/algorithm/PP-OCRv5/PP-OCRv5.html> (truy cập ngày 2026-07-19).
- [119] Ultralytics, "Model Evaluation Insights — hướng dẫn vật thể nhỏ, imgsz, rect, SAH tiling," Ultralytics, 2026. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/guides/model-evaluation-insights/> (truy cập ngày 2026-07-19).

[120] Intel OpenVINO, "Performance Hints and Thread Scheduling — OpenVINO CPU Device (LATENCY hint, hyper-threading, inference_num_threads)," Intel, 2024. [Trực tuyến]. Địa chỉ: <https://docs.openvino.ai/2024/openvino-workflow/running-inference/inference-devices-and-modes/cpu-device/performance-hint-and-thread-scheduling.html> (truy cập ngày 2026-07-19).

[121] "TransLPRNet: Lite Vision-Language Network for Single/Dual-line Chinese License Plate Recognition," *arXiv preprint arXiv:2507.17335*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2507.17335> (truy cập ngày 2026-07-19).

[122] "Advancing Multinational License Plate Recognition Through Synthetic and Real Data Fusion: A Comprehensive Evaluation," *arXiv preprint arXiv:2601.07671*, 2026. [Trực tuyến]. Địa chỉ: <https://arxiv.org/pdf/2601.07671> (truy cập ngày 2026-07-19).

[123] Microsoft ONNX Runtime, "Quantize ONNX models — ONNX Runtime (dynamic vs static, VNNI/AVX512, cảnh báo phần cứng cũ)," Microsoft, 2025. [Trực tuyến]. Địa chỉ: <https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html> (truy cập ngày 2026-07-19).

PHỤ LỤC

Phụ lục cung cấp các thông tin chi tiết được nhắc tới trong thân đồ án nhưng không đưa vào thân bài để giữ mạch đọc: quy mô và tổ chức mã nguồn, cấu hình huấn luyện đầy đủ, xuất xứ và giấy phép của từng bộ dữ liệu, hướng dẫn cài đặt, kết quả kiểm thử và đặc tả giao diện lập trình.

Mọi số liệu trong phụ lục lấy trực tiếp từ kho mã nguồn và các tệp kết quả đã được lưu, không có số nào nhập tay.

Phụ lục A. Mã nguồn

A.1. Quy mô

Số liệu đếm trên kho mã tại thời điểm nộp, không tính thư mục phụ thuộc (node_modules, môi trường ảo), dữ liệu ảnh, trọng số mô hình và các tệp sinh tự động.

Bảng A.1. Quy mô mã nguồn theo thành phần

Thành phần	Ngôn ngữ	Số tệp	Số dòng
ai/ — tầng trí tuệ nhân tạo	Python	31	16.949
scripts/ — công cụ dựng dữ liệu, đo đạc, xuất tài liệu	Python	32	17.395
tests/ — kiểm thử tự động	Python	23	9.246
backend/ — dịch vụ web và truy cập dữ liệu	Python	29	9.992
frontend/ — giao diện người dùng	TypeScript / TSX / CSS	52	10.471
ai/ — cấu hình huấn luyện và bộ dữ liệu	YAML	3	328
deployment/, docker-compose.yml	Dockerfile / YAML	5	780
Tổng		178	65.805

Tỷ trọng đáng chú ý: phần **kiểm thử và công cụ đo đạc** (tests/ + scripts/) chiếm **26.641 dòng, tức 40,5%** toàn bộ mã nguồn — nhiều hơn cả tầng AI. Đây là hệ quả trực tiếp của nguyên tắc trình bày đã nêu ở mục 5.1.2: mỗi con số công bố trong Chương 5 phải sinh ra được bằng một lệnh chạy lại được.

A.2. Tổ chức thư mục

ai/	
inference/	pipeline suy luận – detector, recognizer, hậu xử lý, hai dò
ng	
evaluation/	đo độ chính xác, hiệu năng, phân tích lỗi, kiểm rò rỉ

training/	cấu hình và notebook huấn luyện
backend/	
api/	các route FastAPI
services/	tầng nghiệp vụ – điều phối, không chứa logic AI
repositories/	truy cập cơ sở dữ liệu
models/	mô hình dữ liệu SQLAlchemy
migrations/	Alembic
frontend/	
src/pages/	ba trang: nhận dạng ảnh, nhận dạng video, lịch sử
src/components/	thành phần dùng chung
src/api/	tầng gọi API và ánh xạ kiểu dữ liệu
tests/	kiểm thử đơn vị, tích hợp, kiến trúc
scripts/	dựng bộ dữ liệu, đo đạc, dựng quyền và slide
deployment/	Dockerfile, nginx, entrypoint
models/	trọng số đã huấn luyện
docs/	tài liệu, báo cáo, quyền đồ án

Ranh giới quan trọng nhất trong cây thư mục: **ai/ không được import bất cứ thứ gì từ backend/**. Ràng buộc này là NFR-M1 và được canh giữ tự động bởi `tests/test_architecture.py` — không phải bằng quy ước mà bằng một test sẽ fail nếu ai đó vi phạm. Lý do và hệ quả trình bày ở mục 4.2.1 và 5.5.1.

Phụ lục B. Siêu tham số huấn luyện

Cấu hình đầy đủ của lượt huấn luyện sinh ra `models/best.pt` — mô hình được dùng cho mọi số liệu công bố trong Chương 5. Nguồn: `runs/final-640-v3/args.yaml`.

Bảng B.1. Siêu tham số huấn luyện YOLO11n

Nhóm	Tham số	Giá trị	Ghi chú
Mô hình	model	yolo11n.pt	Khởi tạo từ trọng số tiền huấn luyện COCO
	Số tham số	2.590.035	Biến thể nano — do ràng buộc CPU
Dữ liệu	data	yolo_v3/data.yaml	Split v3
	imgsz	640	Đúng độ phân giải mà NFR-A1/A2 đặt chỉ tiêu
	fraction	1,0	Dùng toàn bộ dữ liệu
Lịch huấn luyện	epochs	20	
	patience	20	Dừng sớm không kích hoạt
	batch	8	Giới hạn bởi RAM và tốc độ CPU

Nhóm	Tham số	Giá trị	Ghi chú
	close_mosaic	10	Tắt mosaic trong 10 epoch cuối
Tối ưu hoá	optimizer	AdamW	
	lr0 / lr1	0,001 / 0,01	Tốc độ học đầu và hệ số cuối
	cos_lr	true	Lịch cosine
	momentum	0,937	
	weight_decay	0,0005	
	warmup_epochs	3,0	
Trọng số mất mát	box / cls / dfl	8,0 / 0,5 / 1,5	
Tăng cường dữ liệu	hsv_h / hsv_s / hsv_v	0,015 / 0,7 / 0,4	
Thiết bị	device	cpu	Không có GPU CUDA (ràng buộc CON-02)

Hai giá trị đáng giải thích thêm. batch = 8 không phải lựa chọn tối ưu mà là giới hạn phần cứng; epochs = 20 là con số bị ngân sách thời gian CPU quyết định chứ không phải điểm hội tụ — chi phí và hệ quả của cả hai trình bày ở mục 4.5.1 và 3.6.

Cấu hình tinh chỉnh bộ nhận dạng ký tự (30 epoch, 6.672 mẫu, bộ ký tự 36) trình bày tại mục 4.5.3 cùng kết quả đo bốn cấu hình. **Bản giao hàng không dùng mô hình tinh chỉnh** — lý do ở cùng mục.

Phụ lục C. Bộ dữ liệu

C.1. Nguồn và giấy phép — nhánh phát hiện biển số

Bảng C.1. Bảy bộ dữ liệu đã hợp nhất, kèm giấy phép và số ảnh còn lại

#	Bộ (slug)	Nguồn	Giấy phép	Vào gộp	Còn lại
1	roboflow_school_fuh ih	Roboflow school-fuhih/vietnamese- license-plate-tptd0v1	CC BY 4.0	8.35 7	6.86 8
2	hf_vn_plates_segmen t	HuggingFace hoanglvuit/Vietnam_License_Plate_Seg ment_Datasets	⚠ chưa xác nhận	4.57 8	4.37 5
3	roboflow_traffic_ca	Roboflow traffic-camera/vietnam-	CC	3.84	3.16

#	Bộ (slug)	Nguồn	Giấy phép	Vào gộp	Còn lại
	mera	license-plate-hayn8 v4	BY 4.0	3	2
4	roboflow_eric_nguyen	Roboflow eric-nguyen-knfxn/vietnam-license-plate-curhr v1	CC BY 4.0	840	353
5	roboflow_demo_tracking	Roboflow demo-tracking/license-plate-vietnam-car v2	CC BY 4.0	236	235
6	roboflow_cuong_ta	Roboflow cuong-ta-ulxex/vietnamese-car-license-plate v1	Public Domain (người đăng tự khai)	8.25 4	140
7	roboflow_tran_ngoc_xuan_tin	Roboflow tran-ngoc-xuan-tin-k15-hcm-dpuid/vietnam-license-plate-h8t3n v1	CC BY 4.0	1.00 5	0
Tổng				27.1 13	15.1 33

Ghi công theo giấy phép. Năm bộ ở trên phát hành theo **CC BY 4.0**, bắt buộc ghi công tác giả — bảng này chính là phần ghi công đó. Một bộ được người đăng tự khai **Public Domain**, nhưng đồ án **không khẳng định** đó là Public Domain thật vì ảnh nguồn có dấu hiệu là ảnh báo chí. Một bộ trên HuggingFace **chưa xác nhận được giấy phép**; nó đóng góp 28,91% corpus nên đây là rủi ro pháp lý phải nêu chứ không phải chi tiết bỏ qua được.

Bộ thứ bảy còn lại 0 ảnh sau khử trùng lặp — toàn bộ 1.005 ảnh của nó trùng với ảnh đã có ở các bộ khác. Con số "hợp nhất từ 7 bộ" vì vậy phải đọc là **6 nguồn nguyên tố**, và điều này được nêu nhất quán ở mục 4.4.2 và 6.3.1.

C.2. Nguồn nhãn chuỗi ký tự — nhánh nhận dạng

Bảng C.2. Hai bộ nhãn mức ký tự

Bộ	Nguồn	Giấy phép	Chuỗi dùng được
roboflow_ocr_plate	Roboflow, nhãn mức ký tự	CC BY 4.0	2.650
roboflow_ocr_conversion	Roboflow, nhãn mức ký tự	CC BY 4.0	151
Tổng			2.801

Toàn bộ số liệu độ chính xác OCR trong Chương 5 đo trên 2.801 mẫu này. Giới hạn phạm vi kết luận kéo theo (97,68% mẫu là biển trắng) nêu tại Phụ lục G.2.7.

C.3. Khử trùng lặp và chia tập

Bảng C.3. Hai phép khử trùng lặp, hai mẫu số khác nhau

Phép đo	Mẫu số	Ngưỡng Hamming	Ảnh có thể loại	Tỷ lệ	Đã xoá thật
(a) Trên toàn bộ ảnh của 7 bộ vào hợp nhất	27.111	5	11.978	44,2%	Rồi
(b) Trên corpus đã gộp merged_v2 còn lại	15.133	10	7.227	47,8%	Chưa

Nguồn: datasets/reports/v2/deduplication_report.json và datasets/reports/v3/deduplication_report.json.

Hai con số 44,2% và 47,8% **không cộng được với nhau** vì mẫu số khác nhau — đây là chỗ rất dễ đọc nhầm và được phân tích riêng ở mục 5.3.1. Bài học về giới hạn của bấm tri giác — nó tóm tắt bố cục khung ảnh chứ không tóm tắt chiếc xe — trình bày ở mục 4.4.3.

C.4. Phân bố nguồn dữ liệu giữa các split

Cột source_dataset trong split_manifest.csv chứa một **tập xuất xứ** ngăn bằng |: một ảnh có thể đến từ nhiều nguồn, nên v3 gồm **16 tổ hợp xuất xứ** dựng từ đúng **6 nguồn nguyên tố**. Cộng dồn số đếm của 6 nguồn cho **33.828**, vượt xa 15.133 vì ảnh đa nguồn bị đếm nhiều lần — **không được dùng 33.828 làm mẫu số**. Toàn tập chia **15.133 = 10.592 / 3.027 / 1.514**, tức **70,0% / 20,0% / 10,0%**; bảng đầy đủ 16 tổ hợp ở split_manifest.csv và Phụ lục C. Phải phân biệt ba con số nguồn: **9 bộ đã tải về**, **7 bộ vào hợp nhất detection**, **6 nguồn nguyên tố**.

Tiêu chí đọc: tỉ lệ một nguồn trong tập test lệch quá 10 điểm phần trăm so với tỉ lệ tổng thể (10,0%) thì phải nêu tên. Hai tổ hợp vượt: roboflow_traffic_camera thuần (**2.582** ảnh) có **20,3%** rơi vào test — gấp đôi tỉ lệ tổng thể; roboflow_school_fuhih|roboflow_traffic_camera (**250** ảnh) có tới **69,6%**. Nghĩa là **tập test nghiêng về ảnh camera giao thông** — góc rộng, biển nhỏ — nên khi đọc mAP theo dải kích thước (5.4.3) phải nhớ đối tượng nhỏ trong tập test tập trung ở một nguồn. Hệ quả của việc các tổ hợp nhỏ khó chia đều, ghi nhận như yếu tố đọc kèm chứ không phải khiếm khuyết vô hiệu hoá kết quả.

Một bộ dữ liệu dư thừa hoàn toàn. roboflow_tran_ngoc_xuan_tin vào hợp nhất với **1.005 ảnh**, ra khỏi khử trùng lặp chéo bộ với **0 ảnh** — **loại 100,0% (1.005/1.005)**: cả 1.005 ảnh đều dính ít nhất một cặp gần trùng (**1.577** cặp với roboflow_school_fuhih, **1.569** với roboflow_cuong_ta, **8** với hf_vn_plates_segment, **35** cặp nội bộ); kiểm chứng độc lập:

split_manifest.csv không chứa tên bộ này lần nào. Đây là **bằng chứng định lượng** cho cảnh báo ở Phase 1 rằng các bộ Roboflow tái sử dụng ảnh của nhau rất nặng, và là lý do **không được cộng dồn expected_images để suy ra quy mô thật** (02-dataset-report.md mục 5.3.1).

Phụ lục D. Hướng dẫn cài đặt và chạy

D.1. Yêu cầu

Hạng mục	Yêu cầu
Hệ điều hành	Windows 10/11, macOS hoặc Linux
Docker	Docker Engine 24+ và Docker Compose v2
Bộ nhớ	Tối thiểu 4 GB RAM trống
Đĩa	Khoảng 6 GB cho image và dữ liệu
GPU	Không cần — toàn hệ thống chạy trên CPU

D.2. Chạy bằng Docker Compose (khuyến nghị)

```
git clone <địa chỉ kho mã>
cd vn-license-plate-recognition
docker compose up -d --build
```

Sau khi các container khởi động, mở trình duyệt tại:

Địa chỉ	Nội dung
http://localhost:5173	Giao diện người dùng
http://localhost:8000/docs	Tài liệu API (Swagger UI, tự sinh)
http://localhost:8000/health	Trạng thái hệ thống

Kiểm tra hệ thống đã nạp được mô hình:

```
curl http://localhost:8000/health
```

Trường model_loaded phải trả về true. Nếu trả về false, hệ thống vẫn chạy nhưng mọi yêu cầu nhận dạng sẽ trả lỗi thay vì trả kết quả bịa — cơ chế UnavailablePipeline, trình bày ở mục 4.7.4.

D.3. Chạy trực tiếp không dùng Docker

Tầng AI và backend

```
python -m venv backend/.venv
backend/.venv/Scripts/pip install -r backend/requirements.txt
backend/.venv/Scripts/alembic upgrade head
backend/.venv/Scripts/uvicorn backend.main:app --port 8000
```

Giao diện, ở một cửa sổ lệnh khác

```
cd frontend && npm install && npm run dev
```


Lưu ý về môi trường ảo. Đồ án dùng **ba môi trường ảo Python tách biệt**, không phải một. Lý do bắt buộc phải tách — xung đột phiên bản giữa hai framework học sâu — trình bày ở mục 4.3.2. Gộp chúng lại sẽ hỏng.

D.4. Biến môi trường đáng chú ý

Biến	Mặc định	Tác dụng
ALPR_MODEL_PATH	models/best.pt	Đường dẫn trọng số bộ phát hiện
ALPR_RECTIFY_ENABLED	true	Bật bước nắn hình biến nghiêng
ALPR_SR_RETRY_ENABLED	false	Bậc siêu phân giải — tắt mặc định , xem mục 5.5.7
ALPR_OCR_SKIP_DETECTION	false	Bỏ bước phát hiện chữ — tắt mặc định , xem mục 5.6.6
ALPR_OCR_REC_MODEL_DIR	(rỗng)	Thư mục mô hình nhận dạng tinh chỉnh; để rỗng là dùng mô hình gốc

Chi tiết đầy đủ về triển khai — kiến trúc mạng Docker, các volume, cách xử lý sự cố thường gặp và lưu ý dung lượng image — ở deployment/README.md.

Phụ lục E. Kết quả kiểm thử

E.1. Tổng hợp

Bảng E.1. Kết quả chạy bộ kiểm thử tự động

Hạng mục	Kết quả
Số test thu thập	1.001
Đạt	1.000
xfail (<i>dự kiến hỏng, có ghi lý do</i>)	1
Fail	0
Skip	0
Ngày chạy	02/08/2026

E.2. Phân nhóm

Nhóm	Kiểm chứng điều gì
Kiểm thử đơn vị	Bộ luật hậu xử lý theo vị trí, phân loại layout, chuẩn hoá chuỗi, quy tắc hiển thị
Kiểm thử tích hợp	Toàn bộ 10 endpoint qua HTTP thật, kèm cơ sở dữ liệu thật và migration
Kiểm thử kiến	Ranh giới ai/ không import backend/ (NFR-M1) — fail nếu ai đó vi phạm

Nhóm	Kiểm chứng điều gì
------	--------------------

Kiểm thử hồi quy Các ca lỗi đã từng xảy ra, mỗi ca một test để không tái diễn

Ý nghĩa của con số 0 fail cần được đọc đúng. Nó nói rằng hệ thống làm đúng những gì bộ kiểm thử kiểm; nó **không** nói rằng hệ thống đạt mọi chỉ tiêu. Ba chỉ tiêu phi chức năng hiện không đạt (NFR-A5, A6, A7) và một chỉ tiêu trượt sạt (NFR-P2) — bảng đối chiếu đầy đủ ở mục 5.7 và phân tích ở mục 5.9.2.

Báo cáo kiểm thử chi tiết theo từng nhóm: docs/reports/07-testing-report.md.

Phụ lục F. Giao diện lập trình và cấu hình triển khai

F.1. Danh sách endpoint

Bảng F.1. Mười endpoint của hệ thống

#	Phương thức	Đường dẫn	Chức năng
1	POST	/api/detect/image	Nhận dạng biến số từ một ảnh tĩnh
2	POST	/api/detect/video	Tạo tác vụ nhận dạng trên video, xử lý nền
3	POST	/api/detect/frame	Nhận dạng một khung hình — dùng cho chế độ thời gian thực
4	GET	/api/jobs/{job_id}	Trạng thái và tiến độ của một tác vụ video
5	GET	/api/history	Danh sách lịch sử, có tìm kiếm, lọc, phân trang
6	GET	/api/history/{detection_id}	Chi tiết một lần nhận dạng
7	GET	/api/history/export	Xuất lịch sử theo bộ lọc hiện hành
8	DELETE	/api/history/{detection_id}	Xoá một bản ghi
9	GET	/api/statistics	Số liệu thống kê tổng hợp theo cửa sổ thời gian
10	GET	/health	Trạng thái hệ thống và tình trạng nạp mô hình

Đặc tả đầy đủ — kiểu dữ liệu đầu vào, cấu trúc đầu ra, mã trạng thái và các quyết định thiết kế API — ở mục 4.7.3. Tài liệu OpenAPI do FastAPI **tự sinh** tại /docs và /openapi.json, nên nó không bao giờ lệch với mã nguồn.

F.2. Cấu hình Docker Compose

Bảng F.2. Thành phần trong `docker-compose.yml`

Thành phần	Loại	Vai trò
backend	dịch vụ	FastAPI + uvicorn, chạy pipeline AI trên CPU
frontend	dịch vụ	nginx:alpine — phục vụ tệp tĩnh và reverse proxy sang backend
alpr-net	mạng	Mạng nội bộ giữa hai dịch vụ
alpr-data	volume	Cơ sở dữ liệu SQLite — dữ liệu sống qua lần khởi động lại
alpr-model-cache	volume	Bộ nhớ đệm trọng số PaddleOCR — tránh tải lại mỗi lần dựng

Ngoài hai volume có tên ở trên, thư mục `./storage` (ảnh và video đã tải lên) và `./models` (trọng số bộ phát hiện, gắn **chỉ đọc**) được gắn trực tiếp từ máy chủ.

Bộ ba tệp triển khai: `deployment/docker/Dockerfile.backend` (build hai giai đoạn), `Dockerfile.frontend` (build rồi phục vụ tĩnh) và `nginx.conf`. Phân tích từng tệp ở mục 4.9.

Phụ lục G. Kết quả, hạn chế và hướng phát triển — phân tích chi tiết

Chương 6 trình bày kết luận ở dạng cô đọng: bảng chỉ tiêu, bảng hạn chế, bảng hướng phát triển. Phụ lục này giữ **nguyên văn phần phân tích** của từng mục — mức nghiêm trọng, bằng chứng số, điều kiện khắc phục.

Tách ra đây để thân bài gọn mà **không phải cắt bằng chứng**: một hạn chế nêu suông không kèm số đo thì hội đồng không kiểm được, mà kiểm được hay không mới là thứ phân biệt một lời thừa nhận với một lời nói cho có.

G.1. Các kết quả đạt được — phân tích chi tiết

G.1.1. Một hệ thống hoàn chỉnh, chạy được, xác minh bằng HTTP thật

Backend FastAPI, frontend React, pipeline AI và lớp dữ liệu SQLite/SQLAlchemy đóng gói Docker, khởi động một lệnh trên máy sạch. Kiểm chứng: 10 thao tác API phản hồi đúng qua HTTP sống; `/health` báo `model_loaded: true` với engine thật; stack Docker kiểm bằng `curl` từ **ngoài** container.

G.1.2. Bộ phát hiện đạt toàn bộ chỉ tiêu với biên rộng

Trên tập test v3 (1.514 ảnh, 1.611 đối tượng), YOLO11n [16] vượt **cả bốn** chỉ tiêu ở mức *mục tiêu* chứ không chỉ *ngưỡng tối thiểu* (Bảng 6.1), **mAP@0.5** tới **0,9829**. Chênh lệch hai layout chỉ **2,09 điểm** — nếu toàn trình kém trên biên hai dòng thì lỗi **không** ở khâu phát hiện.

Điểm yếu duy nhất (T5.4c): dải "rất nhỏ" (dưới 0,5% diện tích ảnh) chỉ đạt **0,8553**. Điều kiện đọc kèm: bài toán **một lớp**, nên **mAP@0.5** cao là bình thường.

G.1.3. Đo được đóng góp định lượng của khối hậu xử lý

Đóng góp khoa học riêng thứ nhất. Phần lớn công trình ALPR chỉ mô tả hậu xử lý định tính; đồ án đo tách bạch trên 2.801 biển có nhãn chuỗi: A5 = 0,6373, A6 = 0,7512, **A6 – A5 = +11,39 điểm phần trăm**. Bộ luật sửa đúng **319 biển**, làm hỏng **0 biển**, dồn gần trọn vào biển hai dòng (**+13,97** so với **+1,23 điểm**); cải thiện thuần một chiều chứng tỏ bộ luật đủ bảo thủ. Đóng góp bị chặn vì nút thắt ở tầng OCR: luật không với tới chuỗi sai nhiều ký tự do engine đọc hụt cả cụm.

G.1.4. Đo được rủi ro R-04 bằng số liệu Việt Nam thật

Đóng góp khoa học riêng thứ hai. Rủi ro R-04 — "pipeline OCR dựng sẵn gãy trên biển hai dòng" — vốn *định tính* từ Phase 0, nay đo được trên dữ liệu Việt Nam (T5.5c). Trên 567 biển một dòng so với 2.234 biển hai dòng, A6 đạt 0,9541 so với 0,6996 — chênh **25,45 điểm** (chênh 1 – CER và A5 lần lượt 5,81 và 38,18 điểm). Biển **một dòng về cơ bản đã giải xong** (A6 vượt mục tiêu 0,90); toàn bộ khoảng thiếu nằm ở biển **hai dòng**, chiếm **79,8%** tập có nhãn chuỗi — phản ánh 77 triệu xe máy Việt Nam [1]. Chênh lệch này **cùng bậc độ lớn** với mốc quốc tế: Laroca và cộng sự (VISAPP 2022) đo chênh **48,6 điểm** giữa biển một dòng (94,3%) và hai dòng (45,7%) trên bộ dữ liệu **RodoSol-ALPR của Brazil** [7].

Cảnh báo trích dẫn bắt buộc. Cặp số 94,3% / 45,7% và chênh 48,6 điểm đo trên **RodoSol-ALPR (Brazil)** [7], **không phải** số liệu Việt Nam — chỉ là *analogue định lượng* về độ khó của biển hai dòng. Con số 25,45 điểm mới là số đo Việt Nam của đồ án.

Giá trị học thuật: chưa nghiên cứu biển số Việt Nam công khai nào công bố hai con số một dòng / hai dòng tách bạch trên cùng một hệ thống; kết luận — biển hai dòng là *đặc tính có cấu trúc của bài toán* — đặt nền cho 6.4.1, nhất quán với dòng nghiên cứu coi tính độc lập layout là yêu cầu thiết kế [23].

G.1.5. Bộ nhận màu nền biển đạt 97,89% — một nguồn bằng chứng mà chuỗi ký tự không thể mang

Đóng góp kỹ thuật riêng thứ ba. Theo Thông tư 79/2024/TT-BCA, biển vàng kinh doanh vận tải mang **đúng cùng bố cục ký tự** với biển trắng cá nhân; biển ngoại giao lại nền trắng — chỉ *cặp* (chuỗi, màu) mới định danh được loại phương tiện. Bộ phân loại (ai/inference/plate_color.py) đọc biểu đồ HSV, trả unknown khi không chắc chắn. Đo trên bộ nguyênluanai/license-plate-color v4 (CC BY 4.0) — bộ mà bộ phân loại **chưa từng được hiệu chỉnh theo**: vàng **98,56%** (694 ảnh), trắng **97,40%** (808), xanh **96,83%** (63), **tổng 97,89% trên 1.565 ảnh** (19-color-accuracy.json); 542 ảnh bị loại là toàn bộ lớp bien_unknown — ảnh mà chính người gán nhãn cũng không đọc được màu nền.

Ràng buộc an toàn khi hợp nhất hai nguồn: màu chỉ được nâng cấp một ứng viên mà bộ luật chuỗi đã coi là hợp lý và tự đánh dấu nhập nhằng, nên **biển quân đội bị đọc nhầm**

màu thì vẫn là biển quân đội. Cùng đợt, họ biển (chín giá trị PlateKind) và chuỗi hiển thị đúng định dạng được giữ tới CSDL, chữa lỗi biển đỏ đọc đúng từng bị hiển thị "Sai định dạng biển số". Hạn chế: bộ dữ liệu đo **không chứa biển đỏ và biển ngoại giao** (6.3.8).

G.1.6. Một quy trình đánh giá có kiểm chứng — bản thân tính trung thực là một kết quả

Đóng góp cuối là **cách các con số được kiểm tra**: năm lần quy trình tự bắt lỗi của chính nó, cả năm đều được ghi lại. **(1)** Rò rỉ train↔test: split cũ có hàng nghìn cặp gần trùng vắt ranh giới; nâng ngưỡng gộp 5 → 10, chia lại thành v3 (6.3.2). **(2)** Một lập luận vòng tròn trong chính phép kiểm đó: 0 cặp vắt split ở ngưỡng Hamming 10 là **hệ quả định nghĩa**, không phải bằng chứng sạch. **(3)** phash ở cài đặt này chỉ sinh khoảng cách Hamming **chẵn**, nên ngưỡng lẻ vô nghĩa. **(4)** Bộ đo OCR **không đi qua đường mã sản phẩm**: ai/evaluation/ocr_accuracy.py không dựng ALPRPipeline, nên logic tăng điều phối vô hình với con số công bố; bản sửa tách bước cứu biển hai dòng thành hai hàm dùng chung; khoảng cách này không gây lỗi và **chưa có cơ chế tự động nào canh giữ nó**. **(5)** Một giả thuyết sửa lỗi hợp lý — đọc riêng từng nửa biển hai dòng rồi nối chuỗi — bị chính dữ liệu bác bỏ áp đảo (15-two-line-ab.json); bản sửa cuối giữ thiết kế ghép hiện hành, đo 900 biển không ca hổng nào. Ngoài ra quy trình còn **bác bỏ một con số độ trễ cũ** (6.3.4). **Một chương đánh giá không kể lại lần nào nó tự nghi ngờ chính mình là một chương đã tự tước bỏ khả năng bị kiểm chứng.**

G.2. Các hạn chế của đề án — phân tích chi tiết

G.2.1. Hạn chế lớn nhất: OCR biển hai dòng còn yếu, kéo E2E chưa đạt

Mức nghiêm trọng: cao — hạn chế trung tâm của toàn đề án. NFR-A5 = **0,6373** (thiếu 16,27 điểm so với ngưỡng 0,80); NFR-A6 = **0,7512** (thiếu 9,88 so với 0,85); NFR-A7 = **0,5552** (thiếu 26,48 so với 0,82); NFR-A4 = **0,9454**, vượt ngưỡng tối thiểu 0,92 nhưng dưới mục tiêu 0,95. **Nguyên nhân ở tầng OCR chứ không phải tầng hậu xử lý**, ba bằng chứng độc lập: *tách theo layout* (T5.5c) — toàn bộ khoảng thiếu nằm ở biển hai dòng, chiếm 79,8% tập; *phân tích lỗi* (T5.8) — 428/445 ca nhầm ký tự và **73/73** ca thiếu ký tự thuộc biển hai dòng; *đóng góp hậu xử lý bị chặn trên* (T5.5b) — ký tự bị xoá chiếm 56,8% toàn bộ lỗi, mà ký tự chưa từng được đọc ra thì **về nguyên tắc** không luật nào phục hồi được. Hướng khắc phục bắt buộc nằm ở tầng nhận dạng (6.4.1).

Cảnh báo hiệu lực. A7 = 0,5552 đo trên ảnh **crop biển số**, ngoài phân bố huấn luyện của bộ phát hiện, nên tỉ lệ bỏ sót 11,96% bị thổi phồng — **cận dưới bi quan** (5.5.5). Đo A7 đúng cách đòi hỏi tập test hiện trường có nhãn chuỗi, việc chưa làm được (6.4.3).

G.2.2. Rò rỉ dữ liệu tồn dư không khử được bằng băm tri giác

Mức nghiêm trọng: cao. Tại ngưỡng Hamming **12** vẫn còn **791** cặp train↔test gần trùng, tại ngưỡng **15** là **3.529** cặp. Nghiêm trọng hơn là rò rỉ **ngữ nghĩa** không ngưỡng phash nào bắt

được: cùng một xe ở góc khác mang cùng biển số nhưng khoảng cách Hamming lớn. *Đã áp dụng*: nâng ngưỡng gộp 5 → 10, đo rò rỉ ở nhiều ngưỡng. *Chưa áp dụng được*: chia split theo nhóm biển số — bất khả thi vì phần lớn corpus thiếu nhãn chuỗi. **Hệ quả bắt buộc nêu khi bảo vệ: mọi chỉ số ở tầng phát hiện và OCR phải được coi là cận trên lạc quan.**

G.2.3. Tập test không xuyên bộ dữ liệu, nên mAP lạc quan hơn khi triển khai

Mức nghiêm trọng: cao. Train và test lấy từ cùng sáu nguồn nguyên tố, nên thiết lập chỉ đo tổng quát hoá *trong phân bố*. Độ chính xác ALPR sụt đáng kể khi đánh giá xuyên bộ dữ liệu [7], nên **mAP@0.5 = 0,9829** gần như chắc chắn lạc quan hơn thực tế. *Giảm thiểu trong khuôn khổ đề án: không có; cách đúng ở G.3.2.*

G.2.4. Độ trễ — đạt ngưỡng tối thiểu, và mức vượt mục tiêu là một đánh đổi có chủ ý

p95 = **1.143,10 ms** — dưới sàn 1.500 ms nhưng vượt mục tiêu 800 ms 1,43 lần; trung vị **405,77 ms** (100 ảnh test v3, máy rảnh). **Thoái lui có chủ ý**: tắt bậc thang thử-lại đưa p95 về **866,3 ms** nhưng mất 34 biển đọc thêm (+0,75 điểm A6); chi phí dồn vào đuôi vì bậc thang chỉ chạy sau khi lần đọc đầu thất bại. Bật cả ba biến thể: p95 **1.514,26 ms**, vượt cả sàn; riêng bậc **siêu phân giải** chiếm hơn nửa (+319 ms p95) mà không mua được biển nào, nên tắt mặc định — số 0 ấy là **số 0 cấu trúc** (cổng chỉ mở cho vùng cắt dưới 200 px, 0/120 mẫu lộn): "chi phí đã đo, lợi ích chưa ai đo được". Một báo cáo trước ghi p95 = **5.857 ms**; con số đó đã bị **bác bỏ** vì điều kiện đo bị nhiễm (5.6.1). Ngân sách thật (T5.6b): OCR **64,3%**, phát hiện **34,0%**; hướng tối ưu đúng là **giảm số lần phải thử lại** (6.4.1).

G.2.5. Một yêu cầu mức Must (FR-4.1) đã bị đưa ra khỏi phạm vi

Mức nghiêm trọng: trung bình. Hạn chế duy nhất phát sinh từ một **quyết định** chứ không từ giới hạn kỹ thuật. Ngày 2026-07-20 giao diện thu gọn hai đợt còn **ba trang**: FR-3.1, FR-3.4 M → W; **FR-4.1 M → W**; FR-4.2 S → W; bảng MoSCoW từ 22/7/3/2 sang **21 Must / 6 Should / 3 Could / 4 Won't** trên 34 yêu cầu. **FR-4.1 là yêu cầu Must đầu tiên và duy nhất bị đưa ra khỏi phạm vi**; tiêu chí thành công số 1 ở mục 1.2.3 chỉ đúng theo bộ **21 Must** sau thay đổi. Nhưng **phần mất đi là màn hình hiển thị, không phải năng lực hệ thống**: GET /api/statistics vẫn phục vụ và vẫn có kiểm thử tích hợp; FR-4.3–FR-4.8 không đổi. *Đánh đổi*: gỡ recharts, gói tải về giảm ~730 KB → **328,8 KB (-55%)**. *Giảm thiểu*: mã hai trang giữ có chủ đích (6.4.6).

G.2.6. SQLite chỉ cho phép một tiến trình ghi tại một thời điểm

Mức nghiêm trọng: thấp trong phạm vi đề án. SQLite khoá ghi mức toàn tập. Với một người vận hành (giả định A-04) đây không phải nút thắt — hệ thống ổn định ở 10 yêu cầu đồng thời, soak 100%; đa người dùng ghi đồng thời thì giới hạn thành thực; hướng khắc phục: PostgreSQL (6.4.7).

G.2.7. Bộ dữ liệu lệch nặng về biển trắng, nên kết luận về độ chính xác OCR chỉ áp cho biển trắng

Mức nghiêm trọng: cao. Hạn chế này quy định **phạm vi hiệu lực** của mọi con số OCR trong quyển. Phân bố của **2.801 ảnh có nhãn ký tự** — tập sinh ra NFR-A4 đến A7 (17-plate-

type-audit.json): trắng **2.736 / 97,68%**; vàng **20 / 0,71%**; xanh **4 / 0,14%**; đỏ **0**; NG/QT **0**; không đọc được màu **41 / 1,46%**. Hệ quả: câu phát biểu đúng khi bảo vệ là "1 – CER = 0,9454 trên một tập gồm 97,7% biển trắng", không phải "trên biển số Việt Nam".

Cập nhật 02/08/2026 — hạn chế đã thu hẹp, nhưng chưa gỡ. Mọi con số A4–A7 trong quyển vẫn đo trên ngữ liệu 2.801 mẫu; phần dưới là nguyên liệu cho lần đo sau. **(a)** Đã gộp **521 biển hiếm** từ `nguyenluanai/license-plate-color v4 (30-rare-plate-integration.md)`: tổng ngữ liệu **2.801 → 3.322**, vàng **20 → 476**, xanh **4 → 45**, tỷ lệ biển hiếm **0,86% → 15,7%**; biển vàng chuyển sang **đánh giá được**. **(b)** Tập ảnh toàn cảnh gán nhãn 02/08 (`34-scene-level-a7.md`) **có** biển đỏ quân đội, biển xanh, ngoại giao và sê-ri LD — quá nhỏ để công bố độ chính xác theo loại, nhưng đủ để không còn nói "bằng không". **(c)** Biển đỏ và ngoại giao vẫn không có nguồn công khai đủ lớn (`17-plate-type-dataset-survey.md`) — hạn chế **thật**, không phải "chưa tới lượt".

Hai điều dễ bị gộp khi trả lời phản biện: **hệ thống có năng lực phân loại loại biển** (kiểm chứng 97,89% trên 1.565 ảnh, 6.2.5) nhưng **chưa có dữ liệu đo độ chính xác ký tự cho biển hiếm** — vàng ($n = 20$) và xanh ($n = 4$) không có ý nghĩa thống kê, đỏ và ngoại giao không đánh giá được. "**Chưa đo được**" không đồng nghĩa "**không làm được**", và không được trình bày như thể đã đo được (6.4.8).

G.2.8. Xem trực tiếp và xử lý nền tranh chấp CPU với nhau

Trang nhận dạng video chạy đồng thời xem trực tiếp (POST /api/detect/frame từng khung) và tác vụ nền trên cùng CPU không GPU. Đo cùng một ảnh: **89–97 ms**/khung khi không có tác vụ nền, **230–462 ms** khi có — chậm **2,5–5 lần**. Đây là hệ quả quyết định môi trường Phase 0 (suy luận CPU), không phải lỗi lập trình; mọi số đo độ trễ của xem trực tiếp vì thế phụ thuộc việc có tác vụ nền hay không (6.4.10).

G.3. Hướng phát triển — chi tiết kỹ thuật và chi phí

G.3.1. Huấn luyện lại module nhận dạng (rec) riêng cho biển số Việt Nam — hướng quan trọng nhất

Vì nút thắt nằm ở tầng OCR (6.3.1), hướng tác động lớn nhất là **thay hoặc huấn luyện lại riêng module rec** thay vì dùng trọng số PaddleOCR đa mục đích [17]. Bốn cách, đầu tư tăng dần: (1) fine-tune module rec theo công thức PP-OCR trên CCPD [67]; (2) mô hình rec hỗ trợ biển đa dòng từ thiết kế — TransLPRNet [121] hay LPTR-AFLNet [75]; (3) tách–ghép biển hai dòng trước khi đưa vào rec [64]; (4) mô hình chuyên biệt huấn luyện từ đầu — CRNN kèm chú ý [70] hoặc pipeline chuyên xe máy Việt Nam [21]. Nhờ NFR-M5, thay module rec không đụng mã tầng API.

Ba phép đo độc lập cùng củng cố thứ tự ưu tiên này, cả ba can thiệp ngoài mô hình nhận dạng: luật hậu xử lý **+11,39 điểm A6 · 319 biển** (6.2.3); bước cứu dòng trên **209 biển** (5.5.6);

bậc thang thử-lại **+0,75 điểm** A6 · 34 biển (5.5.7). Cộng lại chúng nâng A6 từ **0,6098** lên **0,7512** không tốn một giây GPU — nhưng vẫn thiếu **9,88 điểm**, và **dư địa đã cạn**: can thiệp mới nhất chỉ mua thêm 34 biển trên 2.801. Ký tự chèn thừa gần như biến mất (giảm 88%) trong khi ký tự **bị xoá** chiếm 56,8% toàn bộ lỗi (5.5.1): lỗi đã dịch sang "ký tự chưa từng được đọc ra", thứ không tăng nào ngoài mô hình nhận dạng phục hồi được. Mục 6.4.3 là điều kiện tiên quyết.

Hai lượt thử đã thực hiện và đều thất bại — kết quả âm cũng là kết quả. Lượt **thứ nhất** (28/07/2026): model đọc **0/7** ảnh demo so với **7/7** của model gốc, do tập huấn luyện **sai nhãn sinh ra một cách im lặng**: cơ chế "ảnh thay thế" kích hoạt khi không mở được ảnh gốc — mà datasets/raw/** nằm trong .gitignore — nên mọi nhãn bị ghép với ảnh của biển khác; cơ chế đã gỡ (25-finetune-attempt-failed.md). Lượt **thứ hai** (02/08/2026) huấn luyện thành công nhưng không được giao: qua đường ống thật nó **kém hơn model gốc** (A6 = 0,6762 so với 0,7512) — PaddleOCR đánh giá nhánh rec bằng *nguyên ảnh*, còn đường ống *phát hiện chữ trước*, nên model fine-tune đọc mảnh vụn rất kém. Bỏ bước phát hiện chữ thì nó thắng đậm (A6 = **0,8758**, hơn **12,46 điểm**), nhưng cấu hình ấy vẫn bị bác: trên bộ demo ảnh toàn cảnh qua bộ phát hiện thật, thứ tự **đảo ngược** (17/22 tụt còn 13/22), và chế độ chỉ-nhận-dạng **không thể trả chuỗi rỗng** (0/1.606 khung so với 173) nên khi bộ phát hiện bắt nhầm thì nó *bị* ra biển. Lượt một là **lỗi dữ liệu**, lượt hai là **lỗi phép đo** (31-detection-stage-ablation.md).

G.3.2. Xây dựng tập test xuyên bộ dữ liệu

Để chữa G.2.3: giữ một nguồn không dùng huấn luyện làm tập test xuyên bộ, báo cáo song song mAP trong phân bố và xuyên bộ [7]; kết hợp **chia split theo nhóm biển số** để khử cả rò rỉ ngữ nghĩa ở G.2.2.

G.3.3. Bổ sung nhãn chuỗi biển số cho toàn tập

Hiện chỉ **2.801** biển có nhãn chuỗi trong khi corpus có 15.133 ảnh; mẫu số nhỏ này đe dọa tính hợp lệ (5.9.3) và chặn việc đo NFR-A7 trên ảnh hiện trường (6.3.1). Hướng khắc phục: gán nhãn chuỗi bán tự động — hệ thống sinh nhãn nháp, người soát lại — hoặc bổ sung dữ liệu tổng hợp theo hướng hợp nhất đa nguồn [122].

G.3.4. Tăng tốc suy luận: lượng tử hoá OCR, đóng gói ONNX/OpenVINO cả hai tầng

Phân rã ngân sách (mục 5.6.2) chỉ ra việc phải làm: OCR chiếm 64,3%, phát hiện 34,2%. **Lượng tử hoá INT8 module OCR** tận dụng tập lệnh VNNI/AVX-512 trên CPU Intel [123] có đòn bẩy cao nhất. **Đóng gói cả hai tầng sang ONNX Runtime hoặc OpenVINO** [18][96]; hiện đường suy luận chạy PyTorch thuần, và **thí nghiệm so sánh backend (mục 5.6.3) là hạng mục đã chuẩn bị nhưng chưa chạy**. Mọi con số tăng tốc phải đo trên cùng cấu hình phần cứng và công bố kèm cấu hình.

G.3.5. Bám vết đối tượng qua khung hình cho video (SORT/DeepSORT)

Hệ thống hiện gộp các lần nhận dạng trùng theo **chuỗi ký tự** — gây khi OCR đọc sai cùng một biển ở các khung khác nhau. Hướng đúng là bám vết bằng SORT/DeepSORT: gộp theo ID theo dõi, ổn định trước lỗi OCR lẻ tẻ, mở đường cho **bỏ phiếu theo thời gian**.

G.3.6. Khôi phục hai màn hình đã gỡ, từ lịch sử git

Hướng chữa trực tiếp 6.3.6 và **rẻ nhất trong mục 6.4** — chỉ đòi hỏi một quyết định phạm vi. Đây là *phục hồi* chứ không phải *xây mới*: năng lực máy chủ chưa bao giờ bị gỡ (POST /api/detect/frame, GET /api/statistics, GET /health đều có kiểm thử tích hợp); mã giao diện và hợp đồng kiểu còn nguyên trong lịch sử git. Quy trình: lấy lại tệp, nối route, chạy `tsc --noEmit` và `vite build`; thư viện biểu đồ nên chọn bản nhẹ hơn recharts hoặc nạp trữ. Hướng này khôi phục một chỉ tiêu đã cam kết (FR-4.1 mức Must) chứ không nâng chất lượng nhận dạng — nếu chỉ làm được một việc thì 6.4.1 vẫn đáng làm trước.

G.3.7. Chuyển sang PostgreSQL nếu triển khai đa người dùng

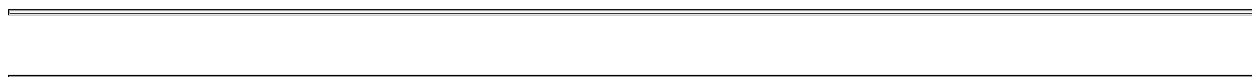
Khi có nhiều người ghi đồng thời nên chuyển sang **PostgreSQL** (khoá mức hàng). Nhờ truy cập dữ liệu đi qua SQLAlchemy 2.0 và tầng repository, việc chuyển giới hạn ở cấu hình kết nối và migration, không đụng mã nghiệp vụ hay API.

G.3.8. Thu thập dữ liệu cho các loại biển hiếm — điều kiện để mở rộng phạm vi kết luận

Hướng chữa trực tiếp 6.3.8, **không đòi hỏi thay đổi gì trong hệ thống**: năng lực phân loại đã kiểm chứng (6.2.5), thứ thiếu là dữ liệu để đo. Ba việc, mức khó tăng dần: (1) **biển vàng — đã có nguồn**: 694 ảnh của `nguyenluanai/license-plate-color`, tên tệp chứa sẵn chuỗi biển số; trở ngại là phép kéo méo về 640×640, xử lý bằng lấy ảnh gốc rồi soát nhãn; (2) **biển đỏ và ngoại giao**: nguồn công khai rất nhỏ — đủ cho tập kiểm thử, không đủ huấn luyện; cần gom bản tăng cường theo ảnh gốc và soát giấy phép tự khai; (3) **biển chuyên dùng (LD, DA, RM, HC, KT, CD, T)**: chưa có nguồn — thu thập tại chỗ hoặc ghi rõ ngoài phạm vi. Giá trị của hướng này là **mở rộng phạm vi mà các kết luận có hiệu lực**, không phải nâng độ chính xác.

G.3.9. Tách lịch chạy giữa xem trực tiếp và xử lý nền

Ba hướng cho 6.3.10: **chạy tuần tự** — hoãn tác vụ nền tới khi dừng xem trực tiếp; **giới hạn số luồng CPU của tác vụ nền**; **tách tiến trình suy luận khỏi tiến trình API** với hàng đợi có ưu tiên — đúng nhất về kiến trúc, chỉ đáng làm khi triển khai nhiều người dùng cùng 6.4.7. Mọi con số độ trễ của tính năng này bắt buộc kèm điều kiện đo.



chuyển completed; giao diện hỏi tiến độ định kỳ. pending tách khỏi processing để phân biệt tác vụ *đang xếp hàng* với tác vụ *đã treo*. **Vì sao bất đồng bộ:** một khung mất ~400 ms trên CPU (4.1.4b); video 60 giây lấy mẫu 1/5 vẫn là 360 khung ≈ 145 giây — vượt timeout của hầu hết proxy và trình duyệt, nên xử lý đồng bộ là **không khả thi** chứ không phải lựa chọn kém.

⚠ **Thay đổi phạm vi 2026-07-20:** trang Webcam đã được **gỡ khỏi giao diện web** theo quyết định thu gọn phạm vi demo. **UC-03 — Nhận dạng thời gian thực** vì vậy được hiện thực và kiểm chứng **ở tầng API** (POST /api/detect/frame); tác nhân chính trở thành một *client thời gian thực* bất kỳ gọi API — trang webcam trước đây của giao diện chính là một client như vậy. Các bước thuần giao diện (FR-3.1, FR-3.4) chuyển mức ưu tiên **M → W**.

UC-03 yêu cầu client có nguồn thu hình, mã hoá khung thành JPEG/PNG, gửi theo chu kỳ cấu hình được. Lời gọi đầu không kèm định danh nên máy chủ tạo tác vụ mới trả job_id; các lời gọi sau gửi kèm nên cả phiên quy về **một** bản ghi tác vụ, biển đã gộp trùng trong phạm vi phiên. job_id không tồn tại thì hệ thống **âm thầm mở phiên mới** để tải lại trang không làm hỏng luồng chụp. **Ràng buộc riêng:** không có GPU nên bắt buộc bỏ bớt khung kết hợp hàng đợi một khe phía client — nếu không, tốc độ chụp (~30 fps) vượt xa tốc độ xử lý (~3–5 fps), hàng đợi phình vô hạn và độ trễ tăng tuyến tính (cài đặt ở 4.8.4).

H.2. Bảng 34 yêu cầu chức năng

Đồ án đặc tả **34 yêu cầu chức năng** trong **6 nhóm**, mỗi yêu cầu có mã, mức MoSCoW và một tiêu chí chấp nhận kiểm chứng được. Phân bố: FR-1 (ảnh tĩnh) **7 Must**; FR-2 (video) **5 Must + 1 Should**; FR-3 (thời gian thực, tầng API) **3 Must + 2 Won't**; FR-4 (thống kê – lịch sử – tra cứu) **4 Must + 1 Should + 1 Could + 2 Won't**; FR-5 (quản lý dữ liệu) **2 Should + 2 Could**; FR-6 (hệ thống, vận hành) **2 Must + 2 Should**. Tổng **21 Must, 6 Should, 3 Could, 4 Won't = 34**.

FR-1: tiếp nhận, kiểm tra hợp lệ, phát hiện *tất cả* vùng biển, cắt và nhận dạng, hậu xử lý, lưu kết quả, hiển thị có bounding box. FR-1.5 quy định lưu **cả chuỗi OCR thô lẫn chuỗi đã sửa** — điều kiện cần để đo đóng góp hậu xử lý ở Chương 5 (4.7.2b). **FR-2:** thêm trích khung theo bước nhảy, **gộp trùng** (FR-2.4 — thiếu nó một video 30 giây sinh hàng nghìn bản ghi về cùng vài chiếc xe, phá hỏng thống kê FR-4), kết xuất video gắn nhãn; Should duy nhất là tiến độ phần trăm và huỷ tác vụ. **FR-3:** theo quyết định 2026-07-20, hai yêu cầu thuần giao diện FR-3.1, FR-3.4 chuyển **M → W**; FR-3.2/3.3/3.5 vẫn Must, kiểm chứng ở tầng API. **FR-4:** chỉ số tổng hợp (FR-4.1), biểu đồ theo thời gian (FR-4.2), danh sách phân trang, tìm kiếm khớp một phần, lọc, chi tiết, tải ảnh, sắp xếp; **FR-4.3 → 4.8 không đổi**. **FR-5:** xóa bản ghi kèm tệp, xuất CSV/JSON (CSV phải UTF-8 **có BOM** kéo Excel hiển thị sai tiếng Việt), dọn tệp mờ côi, xóa hàng loạt. **FR-6:** health check báo trạng thái mô hình và CSDL; log có cấu trúc; thông báo lỗi thân thiện không lộ stack trace; cấu hình qua biến môi trường.

Bốn yêu cầu mức Won't và hai đợt thu gọn phạm vi ngày 2026-07-20

Cả bốn yêu cầu Won't đều **thuần giao diện**, chuyển mức trong cùng ngày qua hai đợt: đợt 1 gỡ trang Webcam (FR-3.1, FR-3.4 **M** → **W**; năng lực còn ở POST /api/detect/frame); đợt 2 gỡ trang Tổng quan (**FR-4.1 M** → **W**, FR-4.2 S → W; năng lực còn ở GET /api/statistics và GET /health).

Phải nói thẳng: FR-4.1 là yêu cầu mức *Must* đầu tiên — và duy nhất — bị đưa ra khỏi phạm vi trong toàn bộ đồ án. Con số đếm vì vậy giảm từ 22/7/3/2 xuống **21/6/3/4**, và mục 6.3 ghi nhận đây là **hạn chế thật**. Điều **không** thay đổi: cả hai đợt chỉ gỡ **màn hình hiển thị**, không gỡ **năng lực hệ thống** — các endpoint vẫn phục vụ, vẫn trong tài liệu OpenAPI, vẫn có kiểm thử tích hợp (tests/integration/test_api_statistics.py, test_api_health.py), thiết kế API ở 4.7.3 giữ nguyên không sửa một dòng — bằng chứng thực tế cho nguyên tắc tách tầng ở 4.2. Đánh đổi đo được của đợt 2: gỡ recharts làm gói tải về giảm từ ~730 KB xuống **328,8 KB** (−55%).

Ma trận truy vết: mỗi nhóm truy vết tới giai đoạn cài đặt và hình thức kiểm chứng (FR-1: unit + integration; FR-2: integration + performance; FR-3: performance ở tầng API; FR-4: integration + UI test cho FR-4.3→4.8; FR-5: unit; FR-6: smoke + stress). Kết quả ở Chương 5.

H.3. Bảng chỉ tiêu phi chức năng

Bảy nhóm: hiệu năng (NFR-P), độ chính xác (NFR-A), tin cậy (NFR-R), khả dụng (NFR-U), bảo trì (NFR-M), bảo mật (NFR-S), tương thích – triển khai (NFR-C), mở rộng (NFR-SC).

a) Nguyên tắc nền tảng: mọi chỉ tiêu hiệu năng đều là chỉ tiêu CPU

Toàn bộ chỉ tiêu hiệu năng của đồ án là chỉ tiêu đo trên CPU.

Máy thực hiện chạy Windows 11, Python 3.13, **không có GPU CUDA** (Intel UHD 770 tích hợp, PyTorch không dùng được để tăng tốc). Huấn luyện trên GPU miễn phí Colab/Kaggle, nhưng **suy luận và buổi bảo vệ chạy trên CPU máy cá nhân**. Đây là **ràng buộc thiết kế**, không phải hạn chế tạm thời, vì bốn lẽ: nó cố định trong toàn bộ vòng đời và tại chính buổi bảo vệ; nó đổi *bậc độ lớn* của độ trễ (ở 20 ms/khung, video đồng bộ và webcam xử lý mọi khung là hợp lý — ở mốc thực tế 400 ms cả hai bất khả thi, trực tiếp sinh ra hai quyết định kiến trúc: video bất đồng bộ AD-02 và webcam bỏ khung hàng đợi một khe); nó chỉ phối chọn biến thể mô hình (n/s/m), biến thể OCR (mobile/server), kích thước ảnh và **backend suy luận** — benchmark chính thức trên CPU i7-13700H cho thấy YOLOv8n qua ONNX Runtime nhanh hơn PyTorch khoảng **3,73 lần** (104,61 → 28,02 ms) [18], lợi ích lớn nhất đúng ở phân khúc mô hình nhỏ [117]; và nó buộc phương pháp công bố chặt hơn — quy tắc CON-06: **mọi số liệu hiệu năng phải kèm model CPU, số luồng, kích thước ảnh, backend suy luận và cỡ mẫu đo**. Các chỉ tiêu độ trễ vì vậy "rộng rãi" hơn văn liệu quốc tế đo trên GPU — đó là trung thực về điều kiện đo, không phải dễ dãi.

Cảnh báo trích dẫn. Bảng benchmark nguồn có cột mAP nhưng đo trên tập coco8 chỉ **8 ảnh**, không có ý nghĩa thống kê; đề án chỉ dùng cột thời gian và cố ý lược bỏ cột độ chính xác.

b) Chỉ tiêu định lượng nhóm hiệu năng và nhóm độ chính xác

Bảng 4.1. Chỉ tiêu phi chức năng định lượng: hiệu năng (NFR-P) và độ chính xác (NFR-A)

Mã	Chỉ tiêu	Mục tiêu	Ngưỡng tối thiểu
NFR-P1	Độ trễ toàn trình một ảnh (p95)	≤ 800 ms	≤ 1500 ms
NFR-P2	Tốc độ khung hình thời gian thực (webcam — đo ở tầng API)	≥ 5 FPS hiệu dụng	≥ 3 FPS
NFR-P3	Tốc độ xử lý video	$\geq 0,3 \times$ thời gian thực	$\geq 0,15 \times$
NFR-P4	Thời gian nạp mô hình khi khởi động	≤ 15 giây	≤ 30 giây
NFR-P5	Overhead của tầng API (không tính suy luận)	≤ 50 ms	≤ 100 ms
NFR-P6	Thời gian truy vấn lịch sử (10.000 bản ghi)	≤ 500 ms	≤ 1000 ms
NFR-P7	Bộ nhớ thường trú của backend	≤ 2 GB	≤ 4 GB
NFR-A1	mAP@0.5 của bộ phát hiện	$\geq 0,90$	$\geq 0,85$
NFR-A2	mAP@0.5:0.95 của bộ phát hiện	$\geq 0,65$	$\geq 0,55$
NFR-A3	Precision / Recall phát hiện	$\geq 0,92 / \geq 0,90$	$\geq 0,88 / \geq 0,85$
NFR-A4	Độ chính xác OCR mức ký tự (1 – CER)	$\geq 0,95$	$\geq 0,92$
NFR-A5	Độ chính xác biến đầy đủ trước hậu xử lý	$\geq 0,85$	$\geq 0,80$
NFR-A6	Độ chính xác biến đầy đủ sau hậu xử lý	$\geq 0,90$	$\geq 0,85$
NFR-A7	Độ chính xác toàn trình (ảnh vào $\rightarrow$ biến đúng)	$\geq 0,88$	$\geq 0,82$

Phương pháp đo NFR-P: P1 trên 100 ảnh test, báo p50/p95/p99; P2 đo liên tục 60 giây; P3 bằng video 60 giây phải xong trong ≤ 200 giây; P4 từ khởi động đến khi /health sẵn sàng; P5

là hiệu tổng thời gian request trừ thời gian pipeline; P6 có phân trang và bộ lọc trên 10.000 bản ghi; P7 theo dõi RSS khi chạy tải liên tục.

NFR-P1 xuất phát từ **phân rã ngân sách độ trễ**: giải mã ~50 ms; phát hiện @640 px ~150 ms; cắt ~30 ms; OCR mỗi biển ~120 ms; hậu xử lý < 5 ms; ghi CSDL ~50 ms — **tổng ~405 ms cho ảnh một biển**; ngân sách 800 ms để dự phòng ảnh nhiều biển và biển động tải. Đây là **ước lượng thiết kế, không phải kết quả đo** (số đo ở Chương 5). Ngân sách lập cho runtime mặc định đã chốt ở mục 3.4 là **ONNX Runtime** — điểm đã đối so với AD-05 sơ bộ. Nếu vượt ngưỡng, thứ tự giảm tải định trước: (1) INT8 OpenVINO; (2) giảm ảnh xuống 480 px; (3) biến thể OCR nhẹ hơn — chỉ hạ chỉ tiêu **sau khi** thử hết ba phương án.

Cặp NFR-A5/A6 đặt **tách bạch** có chủ đích: hiệu số giữa chúng là đóng góp định lượng của khối hậu xử lý — đo được nhờ quyết định lưu cả chuỗi thô lẫn chuỗi sửa ở tầng dữ liệu (4.7.2b). Bổ sung: **NFR-A8** — báo cáo độ chính xác **tách riêng biển một dòng và hai dòng**, căn cứ số liệu 94,3% / 45,7% **đo trên bộ RodoSol-ALPR (Brazil)** đã dẫn ở 4.1.1, vì một con số tổng thể sẽ che giấu đúng điểm gây cần phân tích; **NFR-A9** — báo cáo theo điều kiện ảnh nếu bộ dữ liệu có nhãn phù hợp.

c) Các nhóm yêu cầu phi chức năng còn lại

NFR-R: không sập với đầu vào hỏng/độc hại (100% lỗi bị bắt); ảnh không biển trả rỗng hợp lệ HTTP 200; video thất bại không để lại rác; tỉ lệ thành công chạy liên tục một giờ $\geq 99\%$; CSDL sống sót khởi động lại. **NFR-U**: lượt nhận dạng đầu tiên ≤ 3 nhấp chuột, không cần tài liệu; thao tác > 500 ms có phản hồi trực quan; thông báo lỗi tiếng Việt nêu nguyên nhân và cách khắc phục; dùng được từ 1366×768; tương phản WCAG AA $\geq 4,5:1$. **NFR-M**: mã AI tách hoàn toàn khỏi mã API (M1); bao phủ test tầng nghiệp vụ $\geq 70\%$ (M2); type hint + docstring (M3); không hard-code đường dẫn (M4); thay bộ OCR không sửa tầng API (M5); lint tự động (M6) — M1 và M5 là **yêu cầu kiến trúc**, lý do tồn tại của tầng AI độc lập (4.2). **NFR-S**: kiểm tra magic bytes; chống path traversal bằng tên tệp UUID; giới hạn kích thước phía máy chủ; CORS không ký tự đại diện; không log dữ liệu nhạy cảm; truy vấn tham số hoá qua ORM. **NFR-C**: chạy Windows/Linux/macOS qua Docker một lệnh; **không cần GPU là chế độ mặc định**; Chrome/Edge/Firefox; cài từ máy sạch ≤ 15 phút. **NFR-SC**: ổn định ≥ 5 yêu cầu đồng thời; không suy giảm ở 100.000 bản ghi; video nền không chặn yêu cầu khác.

Giới hạn đã biết cần công bố. SQLite chỉ cho **một tiến trình ghi tại một thời điểm** — chấp nhận được ở quy mô đồ án, nhưng phải nêu trong phần Hạn chế kèm hướng khắc phục (PostgreSQL) nếu triển khai thực tế.

H.4. Lược đồ cơ sở dữ liệu và lịch sử di trú



Hình 4.6. Sơ đồ thực thể — liên kết của cơ sở dữ liệu

Hai thực thể quan hệ một–nhiều: một lần sử dụng (ảnh, video, phiên webcam) là một DetectionJob; mỗi biến tìm thấy là một DetectionHistory (không, một, hoặc nhiều bản ghi con). Sau ba lần di trú Alembic: detection_history **21 cột**, detection_job **11 cột**; đặc tả từng trường ở backend/db/models.py. Điểm chịu lực của detection_job: khoá chính **UUID** vì định danh trả cho client — số tự tăng đoán được cho phép liệt kê tác vụ người khác; status năm giá trị vòng đời, progress ràng buộc [0, 1]; error_message **chỉ dùng phía máy chủ** (có thể chứa đường dẫn, phiên bản — rò rỉ thông tin); bốn chỉ mục. detection_history có 5 chỉ mục, trong đó chỉ mục **tổ hợp** (input_type, detected_time) phục vụ truy vấn mặc định của màn hình lịch sử; đo được **p95 = 18,71 ms** trên 10.000 bản ghi so với chỉ tiêu NFR-P6 500 ms. **Tám ràng buộc CHECK** mức CSDL (ví dụ CHECK (plate_line_count IS NULL OR plate_line_count IN (1, 2))); enum lưu văn bản kèm CHECK vì SQLite không có enum. Năm quyết định dưới đây đều xuất phát từ một yêu cầu đo lường cụ thể — bỏ qua thì hậu quả là **một con số sai mà không có gì báo hiệu**.

a) Tách confidence và ocr_confidence. Hai đại lượng khác bản chất: "vùng này có phải biển số?" và "chuỗi đọc được có đúng?". Gộp một cột thì **không phân tích lỗi được nữa**; tách hai cột cho bảng chẩn đoán bốn tổ hợp (cao–thấp: định vị đúng đọc kém ⇒ cải thiện tiền xử lý/tách dòng; thấp–cao: hạ ngưỡng, huấn luyện thêm; thấp–thấp: dương tính giả). Bộ lọc min_confidence của endpoint lịch sử cũng chỉ phát biểu rõ được khi hai cột tách.

b) Lưu cả raw_ocr_text lẫn plate_number — quyết định có giá trị học thuật cao nhất trong lược đồ. Câu hỏi tất yếu từ hội đồng: khối hậu xử lý đóng góp bao nhiêu? Chỉ lưu chuỗi đã sửa thì câu trả lời là định tính; lưu cả hai thì tỉ lệ khớp của raw_ocr_text là **NFR-A5**, của plate_number là **NFR-A6**, và **hiệu số là đóng góp định lượng của hậu xử lý**. Phép đo còn tách được **sửa đúng** với **sửa hổng** (thô đúng, sửa sai) — loại thứ hai bị che khuất hoàn toàn

nếu chỉ nhìn con số tổng. Chi phí vài chục byte mỗi bản ghi; không lưu thì bằng chứng bị **xoá âm thầm ngay lúc ghi dữ liệu**.

c) source_job_id. Một ảnh có thể chứa nhiều biển (A-02) — thông thường chứ không ngoại lệ. Không có khoá nhóm thì "tổng lượt nhận dạng" chỉ đếm được bằng số dòng lịch sử: **một ảnh ba biển bị đếm thành ba lượt**, chỉ số nhân 2–3 lần, video còn nặng hơn — và lỗi **không tự bộc lộ**: con số vẫn trông hợp lý, chỉ sai theo hướng có lợi. Với khoá nhóm, ba câu hỏi tách bạch: lượt dùng đếm detection_job, biển đã đọc đếm detection_history, trung bình là tỉ số. Cột **NOT NULL** để bản ghi mờ coi thành lỗi ồn ào lúc chèn thay vì mâu thuẫn ngầm. Lập luận không mất hiệu lực khi trang Tổng quan bị gỡ: phép tính vẫn ở StatisticsService, khoá nhóm sai vẫn cho con số sai — chỉ là sai trong JSON thay vì trên màn hình.

d) plate_line_count là trường bắt buộc về nghiệp vụ. *Vai trò 1:* NFR-A8 yêu cầu báo cáo tách một dòng / hai dòng, căn cứ số liệu 94,3% / 45,7% **đo trên bộ RodoSol-ALPR (Brazil)** đã dẫn ở 4.1.1 [7]; một con số tổng (ví dụ 81,5% khi 70% tập là một dòng đạt 95% còn hai dòng chỉ 50%) che lấp hoàn toàn điểm gây trên nhóm phương tiện đa số ở Việt Nam. *Vai trò 2:* khử nhập nhằng trong chính hậu xử lý — chuỗi 29B11234 phân giải được thành 29B-112.34 (ô tô, một dòng) hoặc 29-B1 1234 (xe máy kiểu cũ, hai dòng); chỉ nhìn chuỗi thì **không cách nào phân biệt** (2.2.2c), và ràng buộc tập chữ sê-ri không gỡ được vì cả hai cách phân giải đều đặt B ở vị trí thứ nhất. Thông tin số dòng đến từ nguồn khác hẳn — **hình học bounding box** đối chiếu kích thước chuẩn QCVN 08:2024/BCA [11] — thứ bộ OCR không có và không suy ra được từ chuỗi. Cột cho phép rỗng (lý do ở e) nhưng CHECK bảo đảm chỉ 1 hoặc 2.

e) Vì sao các cột liên quan tới OCR đều cho phép giá trị rỗng. Bốn cột plate_number, raw_ocr_text, ocr_confidence, plate_line_count cho phép rỗng; bốn cột toạ độ và confidence phát hiện thì bắt buộc. Quy tắc duy nhất:

Một biển số phát hiện được vẫn đáng lưu, kể cả khi bộ OCR không đọc ra ký tự nào.

Bản ghi tồn tại vì bộ phát hiện đã tìm thấy vùng; mọi cột dẫn xuất từ OCR có thể vắng vì biển **được định vị nhưng không đọc được** là kết quả có thật, xảy ra thường xuyên (biển xa, bẩn, ngược sáng, nghiêng, đêm). Vứt bỏ các bản ghi này là **thiên lệch chọn mẫu**: chúng biến mất khỏi mẫu số và hệ thống chỉ được đánh giá trên chính những ca đã thành công. Ví dụ số: 100 biển phát hiện, 80 đọc ra chuỗi trong đó 76 đúng — giữ mọi bản ghi cho $76/100 = 76,0\%$ (năng lực toàn trình thật); vứt bỏ cho $76/80 = 95,0\%$, lệch 19 điểm. Con số 95% đúng cho câu hỏi khác ("khi đọc được thì đúng bao nhiêu?"), nhưng câu hỏi thật là xác suất trả biển đúng khi đưa ảnh vào. Lỗi này nguy hiểm vì **luôn thiên vị theo hướng có lợi** nên ít bị nghi ngờ. Kết hợp (a), các bản ghi confidence cao nhưng plate_number rỗng là tập mẫu giá trị nhất để phân tích lỗi ở Chương 5.

f) Ba cột phân loại phương tiện — di trú 0002_plate_kind_and_color. Thêm plate_kind, plate_color, plate_color_confidence — lặp lại nguyên tắc của cặp raw_ocr_text/plate_number: **thông tin đã tính ra thì phải được ghi lại** (biển quân đội đọc

đúng 0,999 không được phép chỉ còn là `is_valid_format = 0`). **Cả ba cho phép NULL, không có mặc định:** dòng ghi trước di trú **thật sự không có giá trị**; NULL nghĩa là *chưa bao giờ đo*, "unknown" nghĩa là *đã đo, không kết luận được* — **một giá trị vắng mặt phải trông như vắng mặt**. SQLite hỗ trợ ADD COLUMN trực tiếp cho cột NULL nên không ràng buộc nào bị mất âm thầm.

g) Xử lý múi giờ. SQLite không có kiểu datetime bản địa và định dạng chuỗi của SQLAlchemy **đánh rơi phần bù múi giờ** — không lỗi, không cảnh báo. Hai hệ quả: `utcnow()` - `row.created_at` ném `TypeError`, và mốc naive sang JSON **không có hậu tố Z** nên trình duyệt đọc là giờ địa phương — trên máy UTC+7 mọi mốc lệch bảy giờ, đủ hợp lý để không ai để ý và đủ sai để hỏng mọi phân tích thời gian. Sửa ở **mức kiểu**: `TypeDecorator` tên `UtcDateTime` chuẩn hoá UTC khi ghi, gắn lại UTC khi đọc; `utcnow()` phía Python thay `CURRENT_TIMESTAMP` vì bản SQLite sinh chuỗi naive độ phân giải một giây — quá thô để sắp thứ tự các phát hiện từ cùng một video.

H.5. Tám quyết định kiến trúc AD-01 ... AD-08

Bảng 4.1. Các quyết định kiến trúc AD-01 ... AD-08

Mã	Quyết định về	Lựa chọn	Lý do	Đánh đổi phải chấp nhận
AD-01	Quan hệ giữa tầng AI và tầng API	Tầng AI là package Python độc lập	NFR-M1; kiểm thử độc lập, tái dùng được trong script huấn luyện và đánh giá (mục 4.2.3)	Thêm một lớp gián tiếp giữa hai tầng
AD-02	Xử lý video	Bất đồng bộ, trả <code>job_id</code> ngay	Thời gian xử lý vượt xa timeout HTTP (NFR-SC3); xem mục 4.1.2	Giao diện phải hỏi tiến độ định kỳ; cần quản lý vòng đời tác vụ
AD-03	Thời gian thực (webcam)	Client gửi từng khung qua HTTP (POST <code>/detect/frame</code>)	Đơn giản, dễ gỡ lỗi, đủ cho mức ~5 FPS	Cần tốc độ khung hình cao hơn thì phải chuyển sang WebSocket
AD-04	Gộp trùng biến số	Theo chuỗi ký tự kết hợp cửa sổ thời gian	Đơn giản hơn nhiều so với bám vết đối tượng, đủ dùng cho FR-2.4	Kém chính xác nếu hai xe cùng biến số trong một video — thực tế không xảy ra
AD-05	Runtime suy luận	ONNX Runtime làm mặc định , OpenVINO là tối ưu bổ sung	Nhanh hơn PyTorch khoảng 3,73 lần ở phân khúc nano; một runtime duy nhất cho cả hai mô	Thêm bước xuất mô hình; phải đặt tường minh số luồng nội bộ

Mã	Quyết định về	Lựa chọn	Lý do	Đánh đổi phải chấp nhận
			hình loại bỏ xung đột framework (mục 3.4)	
AD-06	Thiết bị suy luận	Cấu hình được, mặc định cpu	CON-02 — máy phát triển không có GPU CUDA	—
AD-07	Lưu trữ ảnh và video	Tệp trên đĩa, CSDL chỉ giữ đường dẫn	Tránh phình tệp SQLite do BLOB (mục 4.7.1)	Phải giữ đồng bộ giữa tệp và bản ghi (FR-5.1, FR-5.3)
AD-08	Đặt tên tệp	Sinh từ UUID, không dùng tên gốc	NFR-S2 — chống path traversal	Phải lưu tên gốc ở một trường riêng nếu muốn hiển thị lại

H.6. Đối chiếu toàn bộ chỉ tiêu phi chức năng — bảng đầy đủ từng mã

Mục 5.7 gom kết quả theo nhóm chỉ tiêu. Bảng dưới đây liệt kê **từng mã NFR** đã đặt ra ở Phase 0 kèm sà, mục tiêu, giá trị đo được và mục trình bày — không lọc bỏ mã nào, kể cả những mã không đạt.

Bảng 5.10. Đối chiếu toàn bộ chỉ tiêu phi chức năng

Mã	Chỉ tiêu	Sàn	Mục tiêu	Đo được	KQ	Mục
P1	Độ trễ E2E một ảnh, p95	≤ 1500 ms	≤ 800 ms	1.143,10 ms (p50 405,77 ms)	●	5.6.1
P2	FPS webcam (tầng API)	≥ 3	≥ 5	2,379 (144 khung xong / 1.815 chào / 1.671 bỏ trong 60,52 s, 0 lỗi; p50 180,05 ms, p95 1.247,70 ms)	✗	5.6.4
P3	Tốc độ xử lý video	≥ 0,15×	≥ 0,3×	0,746× (0,546 s/khung; video 14,25 s xong trong 19,1 s, sà ≤ 95 s / mục tiêu ≤ 47,5 s; vid_stride 5)	✓	5.6.4

Mã	Chỉ tiêu	Sản	Mục tiêu	Đo được	KQ	Mục
P4 / P4b	Nạp mô hình / khởi động tới /health	≤ 30 s	≤ 15 s	6,41 s / 8,36 s (baseline)	✓	5.6.5
P5	Overhead API, p95	≤ 100 ms	≤ 50 ms	19,01 ms (baseline)	✓	5.6.5
P6	Truy vấn lịch sử 10.000 bản ghi, p95	≤ 1000 ms	≤ 500 ms	18,71 ms (baseline)	✓	5.6.5
P7a / P7b	RSS pipeline / RSS backend	≤ 4 GB	≤ 2 GB	0,759 / 0,806 GB (sau soak 0,726 → 0,820, +0,094 GB)	✓	5.6.5
A1	mAP@0.5 của bộ phát hiện	≥ 0,85	≥ 0,90	0,9829	✓	5.4.1
A2	mAP@0.5:0.95 của bộ phát hiện	≥ 0,55	≥ 0,65	0,7834	✓	5.4.1
A3	Precision / Recall phát hiện	≥ 0,88 / 0,85	≥ 0,92 / 0,90	0,9837 / 0,9714 (F1 0,9775; 1.514 ảnh / 1.611 đối tượng)	✓	5.4.1
A4	Độ chính xác OCR mức ký tự (1 – CER)	≥ 0,92	≥ 0,95	0,9454	●	5.5.1
A5	Chuỗi đầy đủ trước hậu xử lý	≥ 0,80	≥ 0,85	0,6373	✗	5.5.2
A6	Chuỗi đầy đủ sau hậu xử lý	≥ 0,85	≥ 0,90	0,7512	✗	5.5.2
A6 – A5	Đóng góp của khối hậu xử lý	—	—	+11,39 điểm (319 sửa đúng / 0 làm hỏng)	✓	5.5.2
A7	Độ chính xác E2E toàn trình	≥ 0,82	≥ 0,88	0,5552 (ảnh crop); 0,563 (ảnh toàn cảnh; KTC 95% [0,520 ; 0,607])	✗	5.5.5
A8	Tách theo layout một dòng / hai dòng	báo cáo tách bạch	—	phát hiện 2,09 điểm ; OCR (A6) 25,45 điểm	●	5.4.2, 5.5.3
A9	Tách theo điều kiện ảnh	báo	—	— (bộ dữ liệu	□	5.7

Mã	Chỉ tiêu	Sản	Mục tiêu	Đo được	KQ	Mục
		cáo nếu có nhãn		<i>không có nhãn điều kiện chụp)</i>		
R4	Tỉ lệ thành công khi chạy liên tục	≥ 99%	≥ 99%	100,0% (2.028 yêu cầu, soak 15 phút)	✓	5.6.5
R5	CSDL sống sót qua khởi động lại	100%	100%	0/9.031 bản ghi mất	✓	5.6.5
SC1	Số yêu cầu đồng thời xử lý ổn định	≥ 5	≥ 5	10	✓	5.6.5
M2	Độ bao phủ test tầng nghiệp vụ	≥ 70%	≥ 70%	87,7% (2026- 07-20)	✓	5.7
M6	Tuân thủ lint và định dạng tự động	sạch	sạch	black 96/96 sạch; ruff còn 83 E501	⚠	5.7
C2	Hoạt động không cần GPU	mặc định	—	có	✓	5.2.1
R1–R3, SC2– SC3	Không sập với đầu vào hỏng / độc hại · ảnh không có biến ⇒ HTTP 200 + danh sách rỗng · tác vụ video lỗi không để lại rác · ≥ 100.000 bản ghi không suy giảm hiệu năng · tác vụ video chạy nền	—	—	—	□	5.7
M1, M3– M5, S1–S6, C1, C3– C4, U1–U5	Tách mã AI khỏi mã API · type hint + docstring · không hard- code đường dẫn · thay được bộ OCR không sửa mã API · sáu chỉ tiêu bảo mật (magic bytes, path traversal, HTTP 413, CORS không dùng *, không log dữ liệu nhạy cảm, ORM tham số hoá) · ba chỉ tiêu tương thích (Windows / Linux / macOS qua docker compose up, Chrome / Edge / Firefox, cài từ đầu ≤ 15 phút) · năm chỉ tiêu khả dụng (≤ 3 click, phản hồi trực quan > 500 ms, lỗi tiếng Việt, dùng được từ 1366×768, tương phản WCAG	—	—	—	□	5.7

Mã	Chỉ tiêu	Sàn	Mục tiêu	Đo được	KQ	Mục
	AA ≥ 4,5:1)					

Phụ lục I. Chi tiết cài đặt backend, frontend và đóng gói

Năm mục dưới đây là **chi tiết cài đặt**, không phải quyết định thiết kế. Chương 4 nêu có cơ chế gì và vì sao cần nó; phần liệt kê từng lớp ngoại lệ, từng cờ cấu hình và từng dòng Dockerfile để ở đây, phục vụ người tái lập hệ thống chứ không phải người đọc mạch lập luận.

I.1. Xử lý lỗi, log có cấu trúc và request_id

Mỗi ngoại lệ mang hai mô tả cho hai độc giả: `user_message` — tiếng Việt, ngắn, có hành động, vào thân HTTP; `internal_detail` — tiếng Anh, kỹ thuật, chỉ vào log. Cây ngoại lệ: `APIError` (mang `status_code`) với `ValidationError` 400, `NotFoundError` 404, `FileTooLargeError` 413, `UnsupportedMediaTypeError` 415, `ProcessingError` 500. **Bốn bộ xử lý được đăng ký** — `APIError`, `RequestValidationError`, `StarletteHTTPException` và một bộ **bắt tất cả** cho `Exception` (không có nó, ngoại lệ ngoài dự kiến ở cấu hình debug hiển thị cả stack trace — NFR-S4). Thân lỗi dựng **từ danh sách khoá an toàn tường minh** nên trường mới không thể rò rỉ theo mặc định; `RequestValidationError` được viết lại vì thân lỗi gốc liệt kê giá trị vi phạm — tốt cho lập trình viên, sai với người dùng cuối.

Log có cấu trúc: mỗi dòng một đối tượng JSON — log video xen kẽ log tải lên đồng thời, văn bản thuần không tách lại được; với JSON, jq `'select(.request_id == ...)'` dựng lại toàn bộ câu chuyện một yêu cầu. **request_id đi trong ContextVar**, không truyền tay — mọi hàm quên chuyển tiếp sẽ âm thầm đứt vết. Middleware tôn trọng X-Request-ID từ ngoài; cả X-Request-ID lẫn X-Process-Time khai trong `expose_headers` CORS. `safe_extra()` xử lý việc logging từ chối một số tên khoá trong `extra=` và ném `KeyError` — sự cố ném bởi chính lời gọi log, đúng lúc log quan trọng nhất — bằng cách **đổi tên** khoá trùng (tiền tố `ctx_`) thay vì bỏ. Log ra stdout vì runtime container sở hữu việc thu thập.

I.2. Ba lỗi thực tế đã gặp và sửa trong quá trình cài đặt

Cả ba đều **đi qua được kiểm thử đơn vị** — test xanh không phải bằng chứng đầy đủ khi lỗi nằm ở ranh giới giữa mã và môi trường.

a) pydantic-settings JSON-decode trường list trước validator. Dòng `.env` tự nhiên nhất (`ALPR_CORS_ORIGINS=a,b`) làm dịch vụ **sập lúc khởi động** với json. `JSONDecodeError`, vì thư viện chạy `json.loads` trên giá trị thô của trường `list[str]` trước mọi validator; unit test vẫn xanh vì nguồn `init` **không** JSON-decode — test và môi trường chạy đi qua hai nhánh mã khác nhau của cùng thư viện. Sửa: `StringList = Annotated[list[str],`

NoDecode] cho ba trường danh sách, đưa giá trị thô tới `_split_list` nhận cả hai dạng; validator từ chối "*" và danh sách rỗng ngay lúc khởi động.

b) SQLite âm thầm nuốt tzinfo — cơ chế và cách sửa ở 4.7.2g. Lọt qua rà soát vì bản ghi 14:30 hiển thị 21:30 vẫn là mốc bình thường — không gì trông sai, nhưng mọi phân tích thời gian vô hiệu.

c) Log tiếng Việt làm sập console cp1252 trên Windows. Một dòng log tiếng Việt ném `UnicodeEncodeError` **bên trong cỗ máy logging** (console Windows mặc định cp1252, `JsonFormatter` đặt `ensure_ascii=False` có chủ ý) — sự cố trong lúc đang báo cáo sự cố, phá huỷ chính thông tin chẩn đoán. Container Linux dùng UTF-8 nên lỗi **chỉ xuất hiện trên Windows** và **sống sót qua toàn bộ kiểm thử trong Docker**. Sửa: `_utf8_stdout()` gọi `reconfigure(encoding="utf-8", errors="backslashreplace")` trước khi gắn handler, bọc trong `try/except`.

Điểm chung: cả ba nằm ở ranh giới mã–môi trường (nguồn cấu hình, tầng lưu trữ, bảng mã đầu ra) — lập luận cụ thể cho việc bộ kiểm thử phải gồm kiểm thử tích hợp chạy trên đường dẫn thật.

I.3. Tầng gọi API và ánh xạ kiểu dữ liệu

`services/api.ts` là **nơi duy nhất frontend biết về axios hoặc mã HTTP**: component nhận dữ liệu đã có kiểu hoặc `ApiError` chuẩn hoá. **Sáu hàm gọi API** ứng một–một với sáu endpoint, cộng hai hàm dựng URL (`exportHistoryUrl`, `fileUrl`). **Ba endpoint còn lại không còn hàm gọi phía giao diện nhưng vẫn hoạt động ở backend**: `detect/frame` do client thời gian thực gọi, `statistics` do script và kiểm thử tích hợp, `health` do Docker HEALTHCHECK — cần phân biệt **hàm gọi bị xoá với endpoint thì không**. **Không hostname viết cứng**: `origin` đọc từ biến môi trường lúc build, **mặc định rỗng** (cùng-origin); `resolveOrigin()` cắt / cuối và hậu tố `/api` — không cắt thì `/health` (chủ ý nằm ngoài tiền tố `/api`) không với tới được. Riêng `PlateLineCount` khai là 1 | 2 chữ không number — kiểu tĩnh mã hoá lại ràng buộc CHECK của CSDL ở đầu bên kia đường truyền.

I.4. Hàng đợi một khe ở client thời gian thực (trang webcam đã gỡ 2026-07-20)

Trang webcam đã gỡ khỏi frontend, nhưng lập luận thiết kế dưới đây vẫn đúng và trở thành **khuyến nghị bắt buộc cho bất kỳ client nào** gọi `POST /api/detect/frame`. Suy luận CPU ~5 FPS, nên bộ đếm giờ ngây thơ `await` từng phản hồi sẽ, ngay khi một khung mất 900 ms, khởi động yêu cầu thứ hai trước khi yêu cầu thứ nhất trở về — tồn đọng chỉ tăng và tab đứng hình. **Giải pháp: một khe duy nhất** — giữ đúng một yêu cầu đang bay; `inFlightRef` đang đặt thì khung bị **bỏ qua** (tăng `framesSkipped`) chứ không xếp hàng: bỏ một khung không tốn gì (khung sau cập nhật hơn), xếp hàng thì tốn tất cả. Giải phóng khe đặt trong `finally` — nếu trong `try`, một khung lỗi khoá vòng lặp vĩnh viễn. Hai bảo vệ kèm: **tự tạm dừng sau 5 lỗi liên tiếp**, và `AbortController` huỷ yêu cầu đang bay (huỷ chủ động không báo là lỗi).

Một job_id cho cả phiên — nếu không, ba mươi giây chụp thành ~40 lượt tải lên thay vì 1. Khoá khử trùng bỏ ký tự không phải chữ-số và viết hoa ("90C-76040" $\equiv$ "90c 76040"), nhưng là **khóa, không phải giá trị hiển thị**: một 0 do OCR đọc ra vẫn là 0 — âm thầm sửa thành 0 sẽ **che giấu đúng loại sai lầm mà chương đánh giá cần đo**.

I.5. Triển khai bằng Docker

Đóng gói phục vụ NFR-C1: môi trường chạy tái lập được, không phụ thuộc máy cá nhân.

Dockerfile.backend — **build hai giai đoạn**, với bốn quyết định: **hai tệp requirements cài thành hai lớp riêng** (web + CSDL trước, ngăn xếp ML sau) nên thay đổi một tầng không mất bộ đệm tầng kia — hệ quả trực tiếp của 4.3.2; chạy dưới người dùng không đặc quyền appuser; **ENV OMP_NUM_THREADS=4** tường minh, vì không có nó BLAS/OpenMP dùng toàn bộ nhân và hai container cạnh tranh đến mức cùng chậm; **HEALTHCHECK gọi /health** với --start-period=60s vì nạp trọng số mất vài chục giây. models/ gắn từ ngoài — **trọng số không nằm trong ảnh Docker**.

Dockerfile.frontend — **build rồi phục vụ tĩnh**. builder dùng node:20-alpine + npm ci rồi build; runtime dùng nginx:alpine chỉ chép dist/ — không Node, không node_modules, không mã nguồn. VITE_API_BASE_URL truyền lúc **build** (ARG) vì Vite nhúng biến VITE_* vào bundle khi biên dịch — hạn chế thật: frontend không đổi được origin API mà không build lại; mặc định chuỗi rỗng (same-origin) được chọn chính để tránh điều đó.

docker-compose.yml khai báo hai dịch vụ, mạng bridge riêng alpr-net, volume alpr-data (CSDL, ảnh) và alpr-model-cache cho bộ đệm mô hình PaddleOCR — không có volume này, mỗi lần down && up tải lại vài trăm MB, và không có mạng thì container không khởi động được. Chi tiết ở **Phụ lục F.2. Trạng thái kiểm chứng**: docker compose config hợp lệ; đo hiệu năng trong container thuộc Chương 5.

I.6. Cài đặt chi tiết hai adapter mô hình

Mục 4.6.2 nêu hợp đồng và phát hiện đáng kể; phần dưới giữ nguyên văn mô tả cài đặt của YoloPlateDetector và PaddleOcrRecognizer.

Gói gồm mười một mô-đun cùng __init__.py, tổng **4.852 dòng**: types.py, interfaces.py, config.py (InferenceConfig), exceptions.py (cây ALPError), plate_rules.py, normalizer.py, detector.py, recognizer.py, two_line.py, plate_color.py, pipeline.py. Ràng buộc "không import FastAPI" kiểm chứng tự động ở 4.2.3; lý do nền tảng: gói phải chạy được trong Jupyter, script benchmark và Colab.

Ba lớp trừu tượng: **BaseDetector.detect(image) $\rightarrow$ list[PlateDetection]** — đã lọc ngưỡng và NMS, mọi hộp **kẹp trong biên ảnh**, và **danh sách rỗng là kết quả bình thường**, không bao giờ là lỗi. **BaseRecognizer.recognize(plate_image) $\rightarrow$ PlateRecognition** — trả chuỗi thô kèm độ tin cậy; sửa lỗi ký tự và kiểm tra định dạng **không** thuộc trách nhiệm

của nó — chính việc tách đó làm đóng góp hậu xử lý **đo được** qua hiệu giữa `raw_ocr_text` và `plate_number`; không đọc được thì trả chuỗi rỗng, không ném ngoại lệ.

BaseNormalizer.normalize(raw_text) → tuple[str, bool] — kết quả không hợp lệ vẫn **trả về**, vì loại bỏ sẽ xóa đúng những thất bại chương đánh giá cần đếm. Hợp đồng "trả rỗng, không ném" nhất quán NFR-R2: **không tìm thấy** và **lỗi** là hai chuyện khác nhau. Hai lớp đầu có `warmup()` để chuyển chi phí nạp trọng số ra khỏi yêu cầu đầu tiên (NFR-P1).

Các kiểu dữ liệu khai báo **bất biến** (frozen dataclass) ở chỗ có thể; riêng `DetectionResult/PipelineResult` không bất biến vì chứa mảng ảnh nặng cần giải phóng sau khi lưu. `BoundingBox` lưu (x, y, width, height) khớp trực tiếp bốn cột `bbox_*`, kèm thuộc tính `aspect_ratio` phục vụ phân loại số dòng. Tên thuộc tính đặt **trùng tên cột CSDL có chủ đích** để tăng lưu trữ sao chép trường-sang-trường — một lớp biên dịch trung gian là thêm một chỗ để `confidence` và `ocr_confidence` bị hoán đổi (4.7.2a).

Bộ phát hiện — YoloPlateDetector

Adapter mỏng trên Ultralytics: không nơi nào ngoài mô-đun này chạm vào `Results` hay `tensor`. **Import trễ** (from ultralytics import YOLO trong `_load_yolo_model`) cho unit test không cần ngăn xếp ML; ngược lại **trọng số nạp ngay trong hàm khởi tạo** để tập thiếu làm hệ thống thất bại lúc khởi động kèm hướng dẫn khắc phục. Bốn chi tiết: nhận `.pt/.onnx/.torchscript` cộng **thư mục** OpenVINO (kiểm tệp `.xml`) — từ chối thư mục sẽ khiến cấu hình nhanh nhất trên CPU Intel không cấu hình được; `_resolve_plate_class_ids` giữ tất cả khi mô hình một lớp (trường hợp của đồ án), mô hình nhiều lớp chỉ giữ lớp khớp `PLATE_CLASS_ALIASES` — checkpoint COCO 80 lớp trả danh sách rỗng là đúng, không phải lỗi; `_build_clamped_bbox` kẹp về biên, hoán đổi nếu `x2 < x1`, trả `None` kèm log nếu hộp suy biến; name trả `yolo:{stem}{suffix}` vì một con số benchmark chỉ tái lập được nếu nêu đúng bộ trọng số.

Bộ nhận dạng ký tự — PaddleOcrRecognizer

Ba đặc điểm: **khởi tạo trễ và tái sử dụng** (máy OCR đặt để dựng); **ghim phiên bản** `OCR_VERSION = "PP-OCRv5"` tường minh — nâng cấp thư viện không được âm thầm đổi mô hình đứng sau benchmark đã công bố; **tắt tiền xử lý mức tài liệu** vì đầu vào đã là vùng biển cắt sẵn.

Phát hiện kỹ thuật: backend oneDNN làm sập suy luận trên PP-OCRv5. Trên đúng nền tảng mục tiêu (paddlepaddle 3.3.1, Windows, CPU), chạy mô hình phát hiện văn bản qua oneDNN kết thúc bằng `NotImplementedError: (Unimplemented) ConvertPirAttribute2RuntimeAttribute not support [pir::ArrayAttribute<pir::DoubleAttribute>]` — khiếm khuyết phía thư viện, không phải lỗi cấu hình. Xử lý: hằng số có tài liệu `DEFAULT_ENABLE_MKLDNN: Final[bool] = False`. Đây là **núm hiệu năng, không phải núm độ chính xác** — khi lỗi thượng nguồn được sửa chỉ cần lật giá trị và đo lại; nó cũng giải thích một phần NFR-P1: một con đường tăng tốc CPU hiển nhiên đang bị chặn bởi lỗi thư viện chứ không phải chưa được thử.

Lọc mảnh văn bản theo hình học. CLAHE khuếch đại nhiễu ở vùng gần đồng nhất, có thể sinh mảnh rác đọc thành chuỗi vô nghĩa với độ tin cậy cao (đã gặp mảnh cao 10 px, độ tin

cây 0,84) — **ngưỡng tin cậy không tách được**, hình học mới tách được:

MIN_FRAGMENT_HEIGHT_RATIO = 0.35 đo **so với mảnh cao nhất**, và

_drop_short_fragments trả nguyên đầu vào nếu không mảnh nào báo được hình học. **Tổng hợp độ tin cậy** dùng **trung bình có trọng số theo độ dài mảnh** — trung bình cộng cho phép mảnh một ký tự 0,99 che lấp mảnh bảy ký tự 0,40, trong khi mảnh dài mới mang danh tính biển số.

Phụ lục J. Danh mục khảo sát công trình và bộ dữ liệu

Hai bảng dưới đây là **danh mục tra cứu** của phần khảo sát ở mục 2.5. Phần luận điểm rút ra từ chúng — ba lưu ý bắt buộc khi đọc, quan sát về khoảng cách giữa số công bố và số đo lại, ba nhận xét về bộ dữ liệu — nằm trong thân bài; phần liệt kê từng công trình và từng bộ dữ liệu để ở đây.

J.1. Các công trình quốc tế tiêu biểu về ALPR (2018 – 2026)

- | # | Tác giả, năm — đóng góp, dataset và kết quả chính |
|----|---|
| 1 | Zherzdev và Gruzdev, 2018. LPRNet — segmentation-free, CTC, không RNN; biển Trung Quốc, tới 95% accuracy, 3 ms/biển trên GTX 1080 và 1,3 ms/biển trên CPU i7-6700K [40] |
| 2 | Laroca và cộng sự, 2018. Pipeline YOLO nhiều giai đoạn; SSIG (2.000 khung hình, 101 xe): 93,53% recognition rate ở 47 FPS [37] |
| 3 | Xu và cộng sự, 2018. RPnet end-to-end, dự đoán đồng thời hộp bao và chuỗi; công bố CCPD: 98,5% accuracy, trên 61 FPS [38] |
| 6 | Laroca và cộng sự, 2021. Hợp nhất detection và phân loại layout trong một mạng YOLO: 96,9% end-to-end trung bình trên 8 tập công khai từ 5 khu vực [23] |
| 7 | Wang và cộng sự, 2021. VSNet (VertexNet, SCR-Net) cascade: trên 99% trên CCPD và AOLP, 149 FPS trên GPU , giảm hơn 50% lỗi tương đối [45] |
| 8 | Laroca và cộng sự, 2022. Tổng quát hoá xuyên tập dữ liệu , 9 tập và 12 mô hình OCR; công bố RodoSol-ALPR : trung bình sụt 82,4% → 74,5% với giao thức <i>leave-one-dataset-out</i> , AOLP sụt 90,8% → 62,7% [7] |
| 9 | Batra và cộng sự, 2022. YOLOv5 học chuyển giao kết hợp EasyOCR; biển Ấn Độ (5.991 ảnh): mAP@0.5 = 87,2% , mAP@0.5:0.95 = 46,5% , Recall 82,2%, Precision 88,2%, mô hình 14 MB , detection 4,8 ms trên Nvidia T4 , toàn hệ thống 85 ms [56] |
| 10 | Del Castillo Velarde và Velarde, 2022. Benchmark độc lập LPRNet với Tesseract, 1.000 ảnh mỗi tập: LPRNet 90% trên biển thật, 89% trên biển tổng hợp; Tesseract 93% <i>chỉ</i> trên dữ liệu tổng hợp và <i>chỉ sau tiền xử lý</i> [72] |
| 11 | Tao và cộng sự, 2024. YOLOv5-PDLPR — Multi-Head Attention, giải mã song song; CCPD tổng thể 99,4% ở 159,8 FPS trên GPU , Base 99,9%, Challenge chỉ 94,1% , PKUData 95,5% [73] |
| 13 | AlDahoul và cộng sự, 2024 – 2025. VehiclePaliGemma — tinh chỉnh VLM cho biển |

#	Tác giả, năm — đóng góp, dataset và kết quả chính
	Malaysia điều kiện phức tạp: 87,6% accuracy nhưng chỉ 7 FPS trên GPU A100-80GB [43]
14	Shpir và cộng sự, 2025 . Sinh dữ liệu biển Ukraine bằng mô hình khuếch tán ; tập tổng hợp gán nhãn giả cải thiện +3% so với baseline [74]
16	Xu và cộng sự, 2025 . LPTR-AFLNet hợp nhất nắn chỉnh và nhận dạng, cả biển 1 và 2 dòng; biển Trung Quốc: 99,37% riêng trên biển 2 dòng với 2,7 triệu tham số [75]
17	Wójcik và cộng sự, 2025 . LPLC — bài toán phân loại độ đọc được; cả ba baseline (ViT, ResNet, YOLO) đều F1 dưới 80% [76]
19	Vargoorani và cộng sự, 2025 . Gán nhãn giả bằng Grounding DINO kết hợp YOLOv8: recall phát hiện 94% trên CENPARMI và 91% trên UFPR-ALPR [77]
21	Laroca và cộng sự, 2026 . ICPR 2026 LRLPR — benchmark biển độ phân giải thấp dữ liệu thật (LRLPR-26): đội vô địch chỉ 82,13% , chỉ 4/99 đội vượt mốc 80% [71]

J.2. Các bộ dữ liệu chuẩn của lĩnh vực

Bộ dữ liệu (năm, vùng)	Quy mô	Đặc điểm nổi bật và giấy phép
CCPD [86] (2018 / 2019, Trung Quốc)	Trên 250.000 ảnh (bản 2018); trên 300.000 sau 2019	Nhân nhúng trong tên tệp : tỷ lệ diện tích, độ nghiêng, hộp bao, 4 đỉnh , chỉ số ký tự, độ sáng, độ mờ. MIT
AOLP [87] (2013, Đài Loan)	2.049 ảnh (AC 681, LE 757, RP 611)	Ba kịch bản theo độ khó tăng dần. Học thuật, cấm thương mại
UFPR-ALPR [25] (2018, Brazil)	4.500 ảnh, trên 30.000 ký tự, từ 150 xe	Cả xe lẫn camera chuyển động . Học thuật, cấm phân phối lại, phải xin quyền
RodoSol-ALPR [88] (2022, Brazil)	20.000 ảnh, 4 nhóm mỗi nhóm 5.000	Camera tĩnh trạm thu phí; ngày và đêm; 2 layout; số mẫu dễ và khó bằng nhau . Xem kho chính thức
CLPD [41] (2020, Trung Quốc)	1.200 ảnh từ cả 31 tỉnh thành	Kiểm tra tổng quát hoá địa lý rộng. Xem kho chính thức
OpenALPR benchmark [89] (2016, đa quốc gia)	445 ảnh (EU 108, US 222, BR 115)	Quá nhỏ để huấn luyện; chỉ để benchmark xuyên tệp . AGPL-3.0
LPLC [76] (2025)	10.210 ảnh xe, 12.687 biển gán nhãn	Nhân che khuất cấp xe và cấp biển; 4 mức độ đọc được . Xem kho chính thức
LRLPR-26 [71] (2026, đa quốc gia)	20.000 track huấn luyện, 3.000 track kiểm thử	Benchmark đầu tiên cho biển độ phân giải thấp dữ liệu thật . Theo điều lệ cuộc thi
Global License Plate Dataset [90] (2024, 74	Trên 5.000.000 ảnh từ 74 quốc gia	Nhân đầy đủ: ký tự, mặt nạ, 4 đỉnh, thông tin xe. Không phải giấy phép

Bộ dữ liệu (năm, vùng quốc gia)	Quy mô	Đặc điểm nổi bật và giấy phép chuẩn — rủi ro pháp lý trung bình
---------------------------------	--------	---

J.3. Các công trình về nhận dạng biển số xe Việt Nam

Bảng J.3. Các công trình về nhận dạng biển số xe Việt Nam

#	Nhóm tác giả — năm — nơi công bố — phương pháp và kết quả
1	Học viện Kỹ thuật Quân sự — 2021 — MAPR 2021. Phát hiện điểm đặc trưng cho detection, encoder-decoder segmentation-free cho OCR, môi trường không ràng buộc: detection mIoU 95,01% , P_{75} 99,5%; OCR 99,28% mức chuỗi , 99,7% mức ký tự [59]
2	Trần Anh Đạt, Trần Khánh Linh, Vũ Hoài Nam — 2023 — arXiv. Mô hình đa góc nhìn kết hợp CnOCR; công bố PTITPlates (500 ảnh): F1 91,3% (baseline: YOLOv5 + OCR cơ bản 75,2%; YOLOv8 + Tesseract 82,9%; YOLOv8 + CnOCR 85,2%) [79]
3	Le, Mazumder, Quach, Banerjee, Nguyen — 2023 — FDSE 2023. Kiến trúc 3 giai đoạn toàn YOLOv8 (xe máy → biển → ký tự): mAP 93% sau 300 epoch [21]
4	Tran, Bui — 2024 — MIWAI 2024. SSD MobileNetV2 cho detection, YOLOv8-nano cho ký tự, trên Raspberry Pi 4 : 95,68% độ chính xác trung bình, 0,478 giây/ảnh [80]
5	Dang và cộng sự — 2024 — IJITSR. YOLO phát hiện xe, WPOD-NET nắn phẳng, CRNN cải tiến huấn luyện đồng thời CTC và attention: WER 0,014 trên bãi đỗ xe trong nhà [70]
6	Trần Hải và cộng sự — 2023 — IJMRAP. Tuỳ chỉnh OpenALPR cho Việt Nam, template hậu xử lý; tập kiểm thử chỉ 120 ảnh, không công bố độ chính xác cuối [81]
7	Đặng Thị Dung và cộng sự — 2024 — TNU Journal of Science and Technology. So sánh YOLOv8 và YOLO-NAS trên 1.567 ảnh: YOLO-NAS-S Accuracy 83,92% , F1 0,9125; YOLOv8n Accuracy 81,4%, F1 0,8979. Không đo FPS [82]
8	2012 — SoICT 2012. ALPR cho trạm thu phí dùng <i>peak-to-valley</i> tách ký tự trên cả biển 1 dòng và 2 dòng ; nền tảng tiền học sâu [83]
9	VAPR và Trường ĐH Công nghệ Thông tin – ĐHQG TP.HCM — 2018 — MAPR 2018 Challenge. Cuộc thi <i>Vietnamese Bike License Plate Recognition</i> ; dataset 3.000 ảnh xe máy (2.000 huấn luyện, 1.000 kiểm thử), kết quả xếp hạng không được công bố [84]
10	Nguyễn Thanh Lợi và cộng sự — 2023 — Tạp chí Khoa học Trường ĐH Mở Hà Nội. Đề xuất YOLOv5; bài chỉ ghi "độ chính xác cao", không công bố số liệu cụ thể [85]

Phụ lục K. Nhật ký phương pháp đo — ba lần cùng một loại lỗi

Mục 5.5.6 tóm tắt bài học; phụ lục này giữ **diễn biến đầy đủ** của cả ba lần công cụ đánh giá lệch khỏi đường chạy thật, vì bài học chỉ kiểm chứng được khi người đọc thấy được triệu chứng, nguyên nhân gốc và cả biện pháp **đã thất bại**.

K.1 – K.3. Ba lần đầu — công cụ đo chạy một pipeline ngắn hơn bản giao hàng

Ghi chú phương pháp đo — ba lần cùng một loại lỗi, ghi lại thay vì giấu đi. Lướt đo lại **đầu tiên** sau khi thêm bước cứu cho kết quả tự mâu thuẫn: **A6 tăng 1,75 điểm trong khi A7 đứng yên ở đúng 0,5227**. A7 bao hàm phần A6 đo, nên một biện pháp đang thực sự chạy **không thể** nâng chỉ số con mà để chỉ số bao hàm nó bất động ở từng chữ số thập phân. Nguyên nhân: trong `ai/evaluation/ocr_accuracy.py`, bước cứu mới chỉ nối vào nhánh A5/A6, còn **hàm dựng đường E2E vẫn gọi thẳng bộ chuẩn hoá rồi trả kết quả**; nối xong và đo lại, A7 mới lên 0,5295. Đó là **lần thứ hai**; lần thứ nhất ở chính nhánh A5/A6, cả hai lần triệu chứng đều là một chỉ số **đứng yên một cách vô lý** — dễ được cho qua hơn nhiều so với một con số sai.

Khi chỉ số *X* **bao hàm** chỉ số *Y* về mặt định nghĩa, mà một can thiệp làm *Y* dịch chuyển còn *X* thì không nhúc nhích, **giả thuyết đầu tiên phải là đường đo của *X* bị lệch khỏi đường chạy thật**.

Biện pháp đặt ra khi đó — viết bước cứu thành hàm tự do dùng chung — đã thất bại. Ngày 28/07/2026, **đúng loại lỗi ấy xuất hiện lần thứ ba**: bậc thang thử-lại (5.5.7), thêm vào pipeline ngày 21/07, **chưa bao giờ được kích bản đánh giá gọi ở cả hai nhánh**, nên mọi con số A4–A7 công bố từ 21/07 đến 28/07 mô tả một pipeline ngắn hơn bản giao hàng. Bậc thang **đã** được viết đúng như biện pháp quy định (`should_retry_skewed`, `retry_skewed_variants` là hàm tự do dùng chung) — biện pháp được tuân thủ đầy đủ mà lỗi vẫn tái diễn, vì hàm dùng chung chỉ bảo đảm **nếu** kích bản gọi thì gọi đúng bản cài đặt, chứ **không** bảo đảm kích bản có gọi. Nguyên nhân gốc: `measure_crops` và `_run_pipeline` **dừng lại đường xử lý thay vì gọi nó**, nên mỗi bậc mới thêm vào `ALPRPipeline._process_one` đều **mặc định rơi ra ngoài phép đo**, im lặng.

Một đường đo **liệt kê lại** các bước của hệ thống sẽ lệch khỏi hệ thống ở đúng thời điểm hệ thống thay đổi — tức đúng thời điểm phép đo cần chính xác nhất. Chia sẻ mã ở mức *hàm* không đủ; chỉ **gọi thẳng cùng một điểm vào** mới đủ.

Hai chốt chặn được thêm: kích bản ghi thẳng **trạng thái các công tắc** vào tệp kết quả, và ghi **số biến mà từng bậc cứu được** (`recovery_contribution`) — một bậc không được gọi giờ hiện ra dưới dạng số 0 có nhãn thay vì biến mất không dấu vết. Biện pháp triệt để — cho harness đầu-cuối gọi thẳng `ALPRPipeline.process` — vẫn là **việc bỏ ngỏ**, ghi ở 5.9.2.

K.4. Lần thứ tư — bỏ bước phát hiện chữ, chặn được trước khi vào bản giao

Ghi lại vì đây là lần thứ tư cùng một họ lỗi, và là lần đầu chặn được trước khi vào bản giao. Ba lần trước — siêu phân giải, công cụ đo bỏ sót bậc thang thử-lại, val acc của lượt fine-tune — đều là lỗi đã mắc rồi mới phát hiện. Lần này phép đo nói "lỗi 12,46 điểm" và chỉ vì bộ demo được chạy lại **trước** khi đổi mặc định mới lộ ra rằng nó làm hệ thống **tệ đi 4 điểm**. Quy tắc rút ra: *không đổi cấu hình mặc định dựa trên một phép đo mà đầu vào của nó khác đầu vào thật*.

Phụ lục L. Các mối đe dọa đến tính hợp lệ — phân tích chi tiết

Mục 5.9.3 liệt kê tám mối đe dọa ở dạng bảng. Phụ lục này giữ **nguyên văn phần phân tích** của từng mục: bằng chứng số, biện pháp đã áp dụng, biện pháp **chưa** áp dụng được và lý do, cùng hệ quả phải nhớ khi đọc các con số của chương.

(1) Rò rỉ dữ liệu tồn dư không khử được bằng băm tri giác. *Cao.* Bảng 5.1 cho thấy rò rỉ tồn dư **có thật, đo được**: ngoài vùng bảo vệ của ngưỡng gộp, tại Hamming **12** vẫn còn **791 cặp** và tại 15 là **3.529 cặp**; nghiêm trọng hơn, hai ảnh của **cùng một chiếc xe** ở góc khác nhau vẫn mang cùng biển số nhưng Hamming lớn — rò rỉ **ngữ nghĩa** mà không ngưỡng phash nào phát hiện được. *Đã áp dụng*: nâng ngưỡng gộp 5 → 10, đo ở nhiều ngưỡng cao hơn ngưỡng gộp. *Chưa áp dụng được*: chia split theo nhóm biển số, bất khả thi vì thiếu nhãn chuỗi cho phần lớn corpus. **Hệ quả: mọi chỉ số ở 5.4 và 5.6 phải coi là cận trên lạc quan.**

(2) Tập test không xuyên bộ dữ liệu. *Cao.* Train và test đều lấy từ **cùng sáu nguồn nguyên tố**, nên chỉ đo được tổng quát hoá *trong phân bố*, **không** đo được *xuyên phân bố* — thử quyết định khi triển khai trên camera mới, địa điểm mới, chiếu sáng mới; tài liệu đã chỉ ra độ chính xác ALPR **sụt giảm đáng kể** khi đánh giá xuyên bộ dữ liệu [7]. *Giảm thiểu*: không có — cách đúng là giữ lại một nguồn hoàn toàn không dùng để huấn luyện làm tập test xuyên bộ; **chưa thực hiện**.

(3) Mẫu số nhỏ cho các chỉ số OCR. *Cao.* Phần lớn corpus 15.133 ảnh **chỉ có nhãn hộp giới hạn, không có nhãn chuỗi ký tự**, nên A4...A7 đo trên tập con **2.801 biển**; với mẫu số nhỏ, chênh lệch vài điểm phần trăm có thể nằm trong dao động ngẫu nhiên. *Giảm thiểu*: công bố mẫu số ở mọi bảng của 5.5, **không** rút kết luận về chênh lệch nhỏ.

(4) Đo trên một cấu hình phần cứng duy nhất. *Trung bình.* Mọi số hiệu năng đo trên một máy Intel Raptor Lake 14 nhân chạy Windows 11 và **không ngoại suy** sang CPU khác kiến trúc, sang Linux, hay máy khác số nhân — đặc biệt vì ONNX Runtime và OpenVINO đều nhạy với cấu hình luồng và tập lệnh vector [117] [120]. *Giảm thiểu*: công bố cấu hình đầy đủ ở 5.2 và nhắc lại ở đầu 5.6.

(5) Một lượt huấn luyện duy nhất, không ước lượng được phương sai. *Trung bình.* Với seed=42 cố định và ngân sách khoảng 12 giờ CPU cho một lượt, mọi chỉ số là kết quả của

một lần chạy. *Giảm thiểu:* cố định seed để đảm bảo tái lập; không phát biểu so sánh nào dựa trên chênh lệch nhỏ hơn dao động giữa các seed dự kiến.

(6) Bộ dữ liệu không đạt tiêu chí Q6 về tỉ lệ đối tượng nhỏ. *Trung bình. 10,91%* số hộp có diện tích dưới 0,5% diện tích ảnh, vượt ngưỡng 10%. *Giảm thiểu:* báo cáo mAP **tách theo dải kích thước** ở 5.4.3.

(7) Nhãn layout suy ra từ tỉ lệ khung hình khi bộ dữ liệu không khai báo. *Thấp đến trung bình.* Ngưỡng 2,5 có cơ sở từ QCVN 08:2024/BCA nhưng vẫn là heuristic; biến chụp nghiêng mạnh có thể bị phân loại nhầm. *Giảm thiểu:* ưu tiên nhãn lớp tường minh khi có, ghi rõ ở chú thích Bảng 5.2 tỉ lệ ô nào suy bằng heuristic.

(8) Ma trận nhầm lẫn ký tự phụ thuộc thuật toán căn chỉnh chuỗi. *Thấp.* Khi hai chuỗi khác độ dài, việc gán cặp "ký tự thật ↔ ký tự đọc được" phụ thuộc cách căn chỉnh Levenshtein xử lý các đường đi tối ưu đồng hạng, nên với chuỗi nhiều lỗi chèn/xoá, ma trận có thể ghi nhận cặp không phản ánh nhầm lẫn thị giác thật. *Giảm thiểu:* áp ngưỡng tần suất tối thiểu trước khi đưa một cặp vào bảng luật (tiêu chí 1 ở 5.5.4).

Phụ lục M. Các bảng khảo sát công nghệ

Ba bảng khảo sát của Chương 3: đối chiếu hai mô hình phát hiện, so sánh các engine OCR ứng viên, và khảo sát ảnh hưởng của độ phân giải cùng chất lượng split. Chương 3 nêu quyết định và căn cứ; phần liệt kê đầy đủ để ở đây.

M.1. So sánh baseline-416-v1.pt với best.pt

Bảng 3.6. So sánh baseline-416-v1.pt với best.pt — ba biến thay đổi đồng thời

Hạng mục	baseline-416-v1.pt	best.pt (chính thức)	Chênh lệch
Cấu hình			
imgsz	416	640	+224 px
Bộ dữ liệu	v1 — 4.578 ảnh, 1 nguồn	v3 — 15.133 ảnh, 6 nguồn nguyên tố (hợp nhất từ 7 bộ)	×3,3
Ngưỡng gộp trùng lặp	5	10	+5
Rò rỉ train↔test (ngưỡng 10)	619 cặp	0 cặp (<i>hệ quả định nghĩa, xem T5.3b</i>)	
Số epoch	40	20	−20
Tổng thời gian huấn luyện	156 phút	≈ 712 phút	
Kết quả trên tập test tương ứng			

Hạng mục	baseline-416-v1.pt	best.pt (chính thức)	Chênh lệch
mAP@0.5	0,9933 (<i>epoch 38</i>)	0,9829	-0,0104
mAP@0.5:0.95	0,8597 (<i>epoch 38</i>)	0,7834	-0,0763
Precision	0,9822	0,9837	+0,0015
Recall	0,9810	0,9714	-0,0096
mAP biển một dòng	0,9856	0,9884	+0,0028
mAP biển hai dòng	0,9592	0,9675	+0,0083
Chênh lệch theo layout (điểm %)	2,6	2,09	-0,51
Độ trễ E2E p95 (ms)	763,75 (<i>client-side</i>)	1.143,10 (<i>in-process, có bậc thang thử-lại</i>)	—

M.2. Bảng so sánh các engine OCR ứng viên

Bảng 3.2. So sánh các engine OCR ứng viên

Tiêu chí	PaddleOCR (PP-OCRv5 mobile)	EasyOCR	Tesseract
Kiến trúc	2 giai đoạn: DB và SVTR-LCNet/CTC [17]	2 giai đoạn: CRAFT và CRNN/CTC [60]	LSTM theo dòng [101]
Kích thước mô hình	4,7 MB det + 16 MB rec ≈ 21 MB [102], [103]	Khoảng 200 MB	Khoảng 30 MB
Thời gian CPU	det 57,77 ms + rec 21,20 ms [102], [103]	Cần đo thực nghiệm	Nhanh nhất trong nhóm
Giấy phép	Apache 2.0	Apache 2.0	Apache 2.0
Hỗ trợ nhiều dòng	Tự nhiên — mỗi dòng một hộp, cần tự sắp xếp	Tự nhiên — CRAFT tách vùng	Lý thuyết có, thực tế kém [104]
Giới hạn tập ký tự khi suy luận	Không có — phải tinh chỉnh [105]	Có, tham số native [106]	Tốt nhất [107]
Độ khó triển khai Windows + CPU	Trung bình — framework riêng	Dễ nhất — chỉ cần PyTorch	Cần cài binary hệ thống

Ghi chú bắt buộc về cột thời gian CPU: số của PaddleOCR đo trên Intel Xeon Gold 6271C, FP32, trên tập nội bộ gồm ảnh tài liệu — **không phải ảnh biển số**.

M.3. Khảo sát độ phân giải và chất lượng split — chi tiết

Mục 3.6 nêu kết luận và lý do phép so sánh này không quy kết được nguyên nhân; phần dưới giữ nguyên văn số đo, phân tích ba biến và ma trận thí nghiệm đề xuất.

Khảo sát ảnh hưởng của độ phân giải và chất lượng split

Đồ án có hai mô hình đã huấn luyện: `baseline-416-v1.pt` và `best.pt`. So sánh chúng là tự nhiên nhưng **phải thận trọng về phương pháp luận** ($\triangle$ dưới bảng).

Bảng đối chiếu đầy đủ mười tám dòng chỉ số giữa hai mô hình ở **Phụ lục M.1**.

$\triangle$ Ba biến thay đổi đồng thời (imgsz, bộ dữ liệu + cách chia, số epoch) và chúng tác động **ngược chiều** nhau — không được quy kết nguyên nhân cho bất kỳ biến nào (xem mục 3.6 và Phụ lục M.3). Dòng độ trễ E2E dùng con số **client-side đã xác minh** cho **cả hai** mô hình (763,75 ms và 731,15 ms, máy rảnh, qua HTTP); con số 5.857,19 ms từng ghi cho baseline ở báo cáo Phase 7 đã bị **bác bỏ** vì nhiễm tranh chấp CPU và đo sai checkpoint (mục 5.6.1). Đo cùng phương pháp trên máy rảnh, hai mô hình cho độ trễ gần như y hệt.

Vì sao so sánh này không quy kết được nguyên nhân

So sánh này có ít nhất ba biến cùng thay đổi, và chúng tác động **ngược chiều** nhau:

Biến thay đổi	Baseline	Chính thức	Hướng ảnh hưởng dự kiến
Độ phân giải đầu vào	416	640	Tăng độ phân giải → dự kiến cải thiện , nhất là đối tượng nhỏ
Bộ dữ liệu và cách chia	v1, có rò rỉ	v3, ngưỡng gộp chặt hơn	Khử rò rỉ → dự kiến làm giảm chỉ số, vì chỉ số cũ bị thổi phồng
Số epoch	40	20	Ít epoch hơn → dự kiến làm giảm , nếu chưa hội tụ

Phát biểu duy nhất được phép là mô tả: "*cấu hình A cho X, cấu hình B cho Y, ba biến đổi đồng thời nên không tách được đóng góp từng biến.*"

Kết quả thực tế: `best.pt` cho `mAP@0.5:0.95` = 0,7834, thấp hơn baseline 0,8597 đúng 7,63 điểm (`mAP@0.5` thấp hơn 1,04 điểm) — và đây là kết quả có giá trị, không phải thụt lùi: baseline đánh giá trên split v1 **có rò rỉ (619 cặp gần trùng train↔test ở ngưỡng 10) nên mô hình *ghi nhớ* thay vì *tổng quát hoá*, con số 0,8597 bị **thổi phồng**; `best.pt` đánh giá trên split v3 đã khử trùng lặp (0 cặp) nên 0,7834 **trung thực hơn**. Nghịch lý cốt lõi khi bảo vệ: **một con số thấp hơn nhưng đo đúng có giá trị hơn một con số cao hơn đo trên tập bị rò rỉ**. Không được quy toàn bộ 7,63 điểm cho khử rò rỉ (vì imgsz và số epoch cũng đổi), và không được trình bày `best.pt` như mô hình "tệ hơn baseline": ở tầng phát hiện nó vẫn **vượt mọi ngưỡng NFR** (mục 5.4.1).**

Thí nghiệm cô lập biến — đề xuất, chưa thực hiện

Muốn quy kết nguyên nhân cho từng biến, cần một ma trận thí nghiệm cô lập:

Thí nghiệm	imgsz	Bộ dữ liệu	Mục đích	Chi phí ước tính (CPU)	Trạng thái
E1	416	v3	Cô lập ảnh hưởng của độ phân giải (so với best.pt)	≈ 5 giờ	☐ chưa chạy
E2	640	v1	Cô lập ảnh hưởng của chất lượng bộ dữ liệu	≈ 4 giờ	☐ chưa chạy
E3	640	v3, 40 epoch	Cô lập ảnh hưởng của số epoch	≈ 24 giờ	☐ chưa chạy

Ba thí nghiệm **không được thực hiện** vì tổng khoảng 33 giờ CPU vượt ngân sách còn lại; ghi nhận kèm chi phí ước tính trung thực hơn là im lặng, đồng thời là hướng phát triển cho Chương 6.

Phụ lục N. Kiến thức nền về ALPR và phát hiện đối tượng

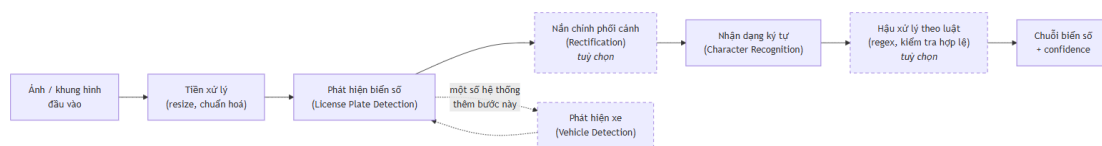
Bốn mục dưới đây là **bối cảnh lĩnh vực**: chúng không ràng buộc quyết định thiết kế nào của hệ thống. Tách khỏi Chương 2 để thân bài chỉ giữ phần lý thuyết trực tiếp chống đỡ một lựa chọn cụ thể, nhưng vẫn có mặt đầy đủ cho người đọc cần dừng lại bối cảnh.

N.1. Tổng quan bài toán ALPR: định nghĩa, ứng dụng và pipeline điển hình

Nhận dạng biển số xe tự động (*Automatic License Plate Recognition*, ALPR) là bài toán định vị biển số trong ảnh hoặc video và chuyển ký tự trên biển thành chuỗi văn bản, kèm độ tin cậy (*confidence*). Khác nhận dạng văn bản cảnh tổng quát, ALPR có ràng buộc cấu trúc mạnh — kích thước chuẩn hoá, bộ ký tự đóng, cú pháp theo luật: vừa là lợi thế cho hậu xử lý, vừa là bẫy — mô hình dễ học thuộc cú pháp tập huấn luyện rồi suy giảm khi định dạng đổi [19].

Hai khảo sát kinh điển chuẩn hoá ALPR thành ba bước: trích xuất vùng biển, phân đoạn ký tự, nhận dạng ký tự [2] [3]; bài tổng quan mới nhất giữ cách phân rã này [20]. Ba khối tùy chọn: **phát hiện phương tiện** đặt trước, đã áp dụng cho xe máy Việt Nam [21]; **nắn chỉnh phối cảnh** (*rectification*) — đóng góp cốt lõi của WPOD-NET [22]; **hậu xử lý theo luật** — Laroca và cộng sự hợp nhất phân loại layout vào detector để chọn bộ luật theo khu vực [23].

Bốn nhóm ứng dụng khác nhau ở điều kiện vận hành: bãi đỗ, kiểm soát ra vào — điều kiện **ràng buộc** (*constrained*); thu phí không dừng; giám sát, phạt nguội — điều kiện **không ràng buộc** (*unconstrained*); camera tuần tra — khó nhất vì cả camera lẫn đối tượng chuyển động. AOLP tách ba tập AC, LE, RP theo độ khó tăng dần [24]; UFPR-ALPR đặt toàn bộ dữ liệu ở tình huống cả xe lẫn camera chuyển động [25]. Riêng Việt Nam: thu phí không dừng dùng RFID làm cơ chế chính, ảnh biển số chỉ để đối soát, dự phòng [5] — ALPR là hệ thống bổ trợ.



Hình N.1. Sơ đồ pipeline ALPR điển hình (tổng hợp từ [2], [3], [20], [22], [23]; khối nét đứt là tùy chọn)

Quan hệ detection – recognition là **nhân quả một chiều, không phục hồi được**: box lệch làm ký tự bị cắt cụt vĩnh viễn; OCR sai một ký tự thì cả chuỗi sai. Vì chỉ tiêu cuối là khớp chuỗi tuyệt đối, sai số hai giai đoạn **nhân lên** — lý do mục 2.5 nhấn mạnh chỉ số end-to-end.

N.2. Lịch sử phát triển các phương pháp ALPR

Trước học sâu, ALPR dùng đặc trưng thủ công: lọc cạnh dọc **Sobel**, nhị phân hoá, **hình thái học**, chiếu ngang dọc khoanh vùng ứng viên [33] [34]; phân đoạn ký tự bằng thành phần liên thông hoặc histogram chiếu; phân lớp bằng đối sánh mẫu, mạng nơ-ron nông hoặc **SVM**. Một công trình trên biển Việt Nam phân đoạn ký tự cho **cả biển một dòng và hai dòng**, thử trên 600 biển (300 mỗi loại), đạt 98,03% với phân đoạn *peak-to-valley* theo tham số thống kê biển Việt Nam [35].

Điểm yếu cố hữu là tính giòn: mỗi ngưỡng chỉnh thủ công, hiệu năng sụt nhanh khi ánh sáng không đều, biển nghiêng. Bằng chứng: một cài đặt cổ điển công khai cho biển Việt Nam (KNN + OpenCV) phát hiện chỉ đạt **49,2% biển một dòng** (182/370) và **39,3% biển hai dòng** (924/2.349); trong số đã phát hiện, đọc đúng hoàn toàn chỉ 33,5% và 31% [36].

Lưu ý khi đọc hai con số 33,5% và 31%. Chúng tính trên số biển đã phát hiện được, không phải toàn tập kiểm thử; quy về end-to-end còn thấp hơn nhiều — ví dụ cho nguyên tắc phải đọc kỹ mẫu số trước khi so sánh (mục 2.5.1).

Nhịp thứ nhất (2016 – 2020) — pipeline học sâu hai giai đoạn: Laroca và cộng sự dùng YOLO cho từng giai đoạn, đạt **93,53% recognition rate ở 47 FPS** trên SSIG, vượt hai hệ thống thương mại đối chứng [37]; Silva và Jung giới thiệu WPOD-NET để mạng học luôn phép nắn chỉnh [22]; Xu và cộng sự công bố CCPD — bộ dữ liệu quy mô lớn đầu tiên — cùng baseline RPnet đạt **98,5% accuracy trên 61 FPS** [38]. **Nhịp thứ hai (2020 – 2026) — end-to-end, Transformer, VLM:** hợp nhất detection và recognition vào một mạng end-to-end [39]; bỏ phân đoạn ký tự, đọc thẳng cả chuỗi bằng CTC [40] hoặc attention 2D [41]; đưa mô hình ngôn ngữ–thị giác (*Vision-Language Model*, VLM) cùng LLM vào ALPR [42] [43] [44].

Bảng N.1. So sánh phương pháp xử lý ảnh cổ điển và phương pháp học sâu

Tiêu chí	Xử lý ảnh cổ điển	Học sâu
Trích đặc trưng và	Thủ công: Sobel, morphology,	Học tự động qua các tầng tích

Tiêu chí	Xử lý ảnh cổ điển	Học sâu
phân lớp ký tự	projection, contour; template matching, KNN, SVM, mạng nơ-ron nông	chập; CNN, CRNN, Transformer, VLM
Dữ liệu gán nhãn và chi phí phát triển	Thấp, chủ yếu hiệu chỉnh ngưỡng; rẻ ban đầu nhưng tăng nhanh khi mở rộng điều kiện	Cao, cần hàng nghìn tới hàng trăm nghìn ảnh; đắt ban đầu, ổn định khi mở rộng
Chi phí tính toán khi suy luận	Rất thấp, chạy được trên phần cứng yếu	Cao hơn nhiều, thường cần tối ưu để chạy trên CPU
Chịu nghiêng, mờ, thiếu sáng; khả năng giải thích	Kém, mỗi ngưỡng phải chỉnh lại theo điều kiện; bù lại quan sát được từng bước	Tốt hơn rõ rệt nếu dữ liệu đủ đa dạng; nhưng mô hình là hộp đen
Bảng chứng định lượng trên biển số Việt Nam	Phát hiện 49,2% (một dòng) / 39,3% (hai dòng) [36]	Nhiều công trình báo cáo trên 90% (mục 2.5)

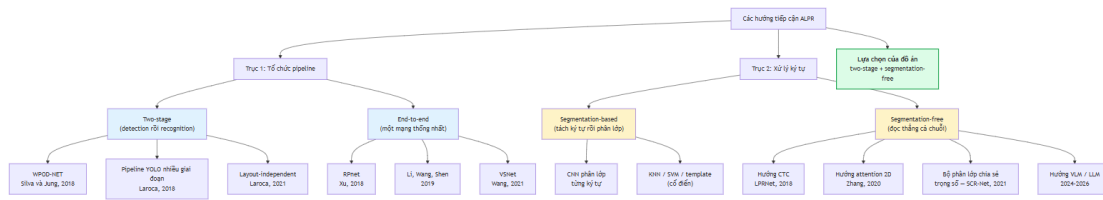
Học sâu là bắt buộc về hiệu năng, nhưng ràng buộc **suy luận trên CPU** khiến đồ án không thể chọn mô hình lớn nhất (Chương 3). Kỹ thuật cổ điển vẫn làm lớp dự phòng cho bài toán tách dòng: *peak-to-valley* [35] và biến đổi hình học OpenCV (mục 2.4.3).

N.3. Phân loại các hướng tiếp cận ALPR hiện nay

Hai trục độc lập thường bị trộn lẫn: **cách tổ chức pipeline** (two-stage / end-to-end) và **cách xử lý ký tự** (segmentation-based / segmentation-free); hệ thống two-stage hoàn toàn có thể dùng bộ nhận dạng segmentation-free.

Two-stage tách detection và recognition thành hai mô hình độc lập: tối ưu, thay thế, gỡ lỗi riêng được; nhược điểm là lỗi detection lan truyền không phục hồi, thời gian là tổng hai bước. Đại diện: WPOD-NET [22], pipeline YOLO nhiều giai đoạn [37], hệ thống độc lập layout [23]. **End-to-end** hợp nhất vào một mạng: Li, Wang và Shen định vị và nhận dạng trong **một lần lan truyền xuôi** [39]; RPnet đồng thời dự đoán hộp bao và chuỗi [38]. Nhược điểm: thay bộ nhận dạng phải huấn luyện lại toàn mạng.

Segmentation-based tách từng ký tự rồi phân lớp riêng [37]; chất lượng phân đoạn quyết định tất cả — biến mờ hoặc ký tự sát nhau khiến bước này thất bại. **Segmentation-free** đọc thẳng cả chuỗi, bốn nhánh: **CTC** — LPRNet [40]; **attention 2D** — encoder Xception [41]; **bộ phân lớp chia sẻ trọng số** — SCR-Net trong VSNet [45]; **VLM / LLM** đọc trực tiếp [42], [44]. Về đa layout: **phân loại layout tường minh** — Laroca và cộng sự hợp nhất phát hiện biển và phân loại layout vào một mạng, đạt **96,9% end-to-end recognition rate trung bình trên 8 tập công khai từ 5 khu vực** [23]; hoặc **không phụ thuộc layout** bằng VLM kết hợp tinh chỉnh hậu-OCR [42].



Hình N.2. Sơ đồ phân loại hai trục các hướng tiếp cận ALPR và định vị lựa chọn của đồ án

Đồ án theo **two-stage** với bộ nhận dạng **segmentation-free** có sẵn — hệ quả của ràng buộc cứng: phải **thay được bộ OCR mà không huấn luyện lại toàn hệ thống**, vì quyết định engine OCR phụ thuộc thực nghiệm (mục 3.3), còn end-to-end khoá cứng lựa chọn đó. Về đa layout, chọn **phân loại layout tường minh** thay vì VLM: VLM chỉ phí suy luận cao hơn nhiều bậc độ lớn, không tương thích CPU (mục 2.5.1), còn quy chuẩn Việt Nam đã cho sẵn cơ sở định lượng mạnh (mục 2.2.6).

N.4. Bài toán phát hiện đối tượng, IoU và NMS

Phát hiện đối tượng đồng thời định vị và phân loại: mô hình trả về hộp bao $B = (x, y, w, h)$, nhãn lớp và điểm tin cậy $s \in [0, 1]$; đồ án chỉ có một lớp `license_plate`. **IoU** đo chồng lấp giữa hộp dự đoán B_p và hộp thực B_{gt} :

$$\text{IoU}(B_p, B_{gt}) = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|}$$

Dự đoán là đúng (*true positive*) khi IoU vượt ngưỡng, thường 0,5; P_{75} là precision tại ngưỡng 0,75. Với biến số, **hộp bao rất dẹt** nên IoU nhạy với sai số định vị: hộp 4,7:1 lệch vài pixel chiều cao làm IoU giảm mạnh — nguyên nhân khoảng cách lớn giữa **mAP@0.5** và **mAP@0.5:0.95** (mục 2.3.3).

NMS (*Non-Maximum Suppression*) khử hộp chồng lấp: giữ hộp điểm cao nhất, loại hộp có IoU với nó vượt ngưỡng, lặp lại. Ngưỡng quá thấp xoá nhầm hai biển sát nhau, quá cao để lọt hộp trùng — với ảnh giao thông Việt Nam nhiều xe máy sát nhau, tham số này hiệu chỉnh bằng thực nghiệm (Chương 5). Hướng mới **bỏ hẳn NMS**: YOLOv10 dùng *consistent dual assignments*, sinh đúng một dự đoán mỗi đối tượng khi suy luận [46]; YOLO26 đưa NMS-free thành mặc định [47].

Phụ lục O. Tập cấu hình gốc, báo cáo đo và mã nguồn

Phụ lục này giữ **hiện vật thô** — thứ cần để tái lập chứ không cần để đọc hiểu. Phụ lục B trình bày siêu tham số dưới dạng bảng đã biên tập; mục O.1 dưới đây là **nguyên văn tập máy sinh**, vì một bảng biên tập lại không thay được tập gốc khi có người muốn chạy lại đúng lượt huấn luyện ấy.

O.1. runs/final-640-v3/args.yaml — cấu hình lượt huấn luyện sinh ra models/best.pt

Nguyên văn, không lược bỏ dòng nào. Các giá trị đáng chú ý đã bình luận ở Phụ lục B và mục 4.5.1.

```
task: detect
mode: train
model: yolo11n.pt
data: D:\DATN\datasets\processed\yolo_v3\data.yaml
epochs: 20
time: null
patience: 20
batch: 8
imgsz: 640
save: true
save_period: 10
cache: false
device: cpu
workers: 2
project: D:\DATN\runs
name: final-640-v3
exist_ok: false
pretrained: true
cls_remap: true
optimizer: AdamW
verbose: true
seed: 42
deterministic: true
single_cls: false
rect: false
cos_lr: true
close_mosaic: 10
resume: false
amp: false
fraction: 1.0
profile: false
freeze: null
multi_scale: 0.0
compile: false
overlap_mask: true
mask_ratio: 4
dropout: 0.0
val: true
split: val
save_json: false
conf: null
iou: 0.7
```

```
max_det: 300
quantize: null
dnn: false
plots: true
end2end: null
source: null
vid_stride: 1
stream_buffer: false
visualize: false
augment: false
agnostic_nms: false
classes: null
retina_masks: false
embed: null
show: false
save_frames: false
save_txt: false
save_conf: false
save_crop: false
show_labels: true
show_conf: true
show_boxes: true
line_width: null
format: torchscript
keras: false
optimize: false
dynamic: false
simplify: true
opset: null
workspace: null
nms: false
lr0: 0.001
lrf: 0.01
momentum: 0.937
weight_decay: 0.0005
warmup_epochs: 3.0
warmup_momentum: 0.8
warmup_bias_lr: 0.1
distill_model: null
dis: 6.0
box: 8.0
cls: 0.5
cls_pw: 0.0
dfl: 1.5
pose: 12.0
kobj: 1.0
rle: 1.0
angle: 1.0
nbs: 64
hsv_h: 0.015
```

```

hsv_s: 0.7
hsv_v: 0.4
degrees: 5.0
translate: 0.1
scale: 0.5
shear: 2.0
perspective: 0.0005
flipud: 0.0
fliplr: 0.0
bgr: 0.0
mosaic: 1.0
mixup: 0.0
cutmix: 0.0
copy_paste: 0.0
copy_paste_mode: flip
auto_augment: randaugment
erasing: 0.4
cfg: null
tracker: tracktrack.yaml
save_dir: D:\DATN\runs\final-640-v3

```

O.2. Danh mục báo cáo đo dạng JSON

Toàn bộ số liệu công bố trong quyển sinh ra từ các tệp dưới đây, nằm ở docs/reports/.
Danh mục **không chép nội dung tệp**: gộp lại chúng dài hàng chục nghìn dòng, chép vào thì quyển phình mà vẫn không ai đọc. Thay vào đó mỗi dòng ghi **câu hỏi tệp đó trả lời và mục nào trong quyển dùng nó**, để một con số bất kỳ đều truy ngược được về tệp sinh ra nó.

Bảng O.1. Báo cáo đo dạng JSON và mục sử dụng

Tệp trong docs/reports/	Trả lời câu hỏi gì	Dùng ở mục
03-evaluation-ch5-best-test.json	mAP, Precision, Recall của best.pt trên tập test v3	5.4.1
07-leak-check-t10.json	Số cặp ảnh gần trùng train↔test theo từng ngưỡng Hamming	5.3.2
17-plate-type-audit.json	Phân bố màu nền của 2.801 mẫu có nhãn chuỗi — căn cứ cảnh báo 97,68% biển trắng	5.3, 6.2
04-ocr-accuracy.json	A4–A7 lượt đo cơ sở	5.5
16-ocr-accuracy-rescued.json	A4–A7 sau khi thêm bước cứu dòng trên	5.5.6
28-ocr-accuracy-finetuned.json	A4–A7 của bộ nhận dạng đã tinh chỉnh	4.5.3
29-reconly-ablation.json	Bốn cấu hình det+rec ↔ chỉ-rec, hai model	4.5.3,

Tệp trong docs/reports/	Trả lời câu hỏi gì	Dùng ở mục
		5.6.6
15-two-line-ab.json	A/B ghép-rời-đọc ↔ đọc-từng-nửa, 200 biến hai dòng	5.5.6
15-two-line-fallback-700.json	A/B bước cứu dòng trên, mẫu 700 biến	5.5.6
15-two-line-fallback.json	A/B bước cứu dòng trên, mẫu 200 biến	5.5.6
15-two-line-rescue-ladder.json	Chi phí và lợi ích từng bậc của bậc thang thử-lại	5.5.7
15-fragment-height-ab.json	Ngưỡng lọc mảnh văn bản theo hình học	4.6.3
19-color-accuracy.json	Độ chính xác bộ nhận màu nền trên 1.565 ảnh ngoài hiệu chỉnh	4.6.7
07-benchmark-p1-resolved.json	Độ trễ đầu-cuối p50/p95 và phân rã theo bước	5.6.1, 5.6.2
07-api-overhead.json	Overhead của tầng API so với gọi pipeline trực tiếp	5.6.5
07-stress-load.json	Chịu tải đồng thời và tỉ lệ thành công khi chạy liên tục	5.6.5
07-stress-db.json	Thời gian truy vấn lịch sử trên 10.000 bản ghi	5.6.5
07-benchmark-optimized.json	(chưa chạy) So sánh PyTorch ↔ ONNX Runtime ↔ OpenVINO	5.6.3

Thư mục còn **23 tệp JSON khác** thuộc các lượt đo trung gian đã bị lược sau thay thế; chúng được giữ lại trong kho để đối chiếu lịch sử chứ không được trích dẫn trong quyển. Nguyên tắc áp dụng xuyên suốt: **một số liệu chỉ được đưa vào quyển khi tệp sinh ra nó còn trong kho và chạy lại được.**

O.3. Vị trí mã nguồn của các đóng góp kỹ thuật

Quyển không chép mã nguồn thành trang giấy — mã đầy đủ nằm trong kho, và một bản chép trên giấy sẽ lệch khỏi kho ngay lần sửa đầu tiên. Bảng dưới đây trở tới đúng tệp và đúng hàm của từng đóng góp, kèm mục đã phân tích thiết kế của nó.

Bảng O.2. Vị trí mã nguồn của các đóng góp kỹ thuật

Tệp	Thành phần	Phân tích ở mục
ai/inference/two_line.py	estimate_line_count,	4.6.4

Tệp	Thành phần	Phân tích ở mục
	split_two_line, merge_two_line, rescue_two_line_upper — thuật toán tách-rời-ghép-ngang và bước cứu dòng trên	
ai/inference/plate_rules.py	POSITION_MASKS, TO_DIGIT, TO_LETTER, PROVINCE_CODES — bộ luật hậu xử lý ràng buộc theo vị trí	4.6.5
ai/inference/plate_color.py	classify_plate_color, refine_kind_with_color — nhận màu nền và phép hợp nhất chuỗi-màu	4.6.7
ai/evaluation/benchmark_engines.py	Tăng bao quanh dùng chung cho ba engine OCR	3.3.3
scripts/dataset/dedupe.py	Khử trùng lặp bằng băm tri giác đa chỉ mục	4.4.2

Quy mô mã nguồn và tổ chức thư mục toàn dự án ở **Phụ lục A**; hướng dẫn dựng lại môi trường và chạy ở **Phụ lục D**.

Phụ lục P. Phương pháp nghiên cứu và các phân tích chi tiết của Chương 5

Thân bài giữ bảng số và đoạn đọc kết quả; phụ lục này giữ **nguyên văn phần phân tích**: từng cặp ký tự bị nhầm và vì sao bảng luật không phủ nó, từng bậc của bậc thang thử-lại cùng chi phí đo được, từng loại lỗi và diễn biến qua các lượt đo. Tách ra đây để thân bài thanh thoát mà **không phải cắt bằng chứng**.

P.1. Phương pháp nghiên cứu

Nghiên cứu lý thuyết

(a) Khảo sát tài liệu có hệ thống theo bốn trục — ALPR, các thể hệ YOLO, engine OCR, bộ dữ liệu biển số công khai — cho **232 mục tài liệu tham khảo** trong references.bib, kèm **bản đồ trích dẫn**. **(b) Đối chiếu văn bản pháp quy gốc** — chính cách này phát hiện TT 24/2023/TT-BCA đã hết hiệu lực. **(c) Kiểm chứng đối kháng nguồn trích dẫn**: mỗi số liệu được truy về nguồn gốc, **loại bỏ hoặc gắn nhãn cảnh báo** nếu không tái lập được; đã phát hiện và sửa **25 lỗi**, trong đó **3 lỗi mức nghiêm trọng**. **Mệnh đề bị bác bỏ**: giả thuyết "biển số Việt Nam loại trừ 6 chữ cái I J O Q R W" **sai** — tập loại trừ đúng chỉ gồm **5 chữ** (I J O Q W), R **hợp lệ** ở vị trí seri thứ hai của biển xe mô tô; hệ quả: charset OCR dùng **đủ A-Z + 0-9**,

ràng buộc hợp lệ áp ở **tầng hậu xử lý** (mục 1.6.3). **Số liệu giữ nhưng gắn cảnh báo:** benchmark trên CPU Intel Core i7-13700H cho thấy **ONNX Runtime nhanh gấp khoảng 3,73 lần PyTorch** ở phân khúc nano (104,61 ms → 28,02 ms, imgsz 640, FP32) [18] — giữ làm căn cứ giảm độ trễ, nhưng **cột mAP kèm bảng gốc bị loại bỏ có chủ ý** vì đo trên coco8.yaml, tập chỉ **8 ảnh, không có ý nghĩa thống kê**.

Nghiên cứu thực nghiệm

(a) Kiến trúc phân tầng với ràng buộc cứng về tách biệt trách nhiệm (mục 1.2.2). **(b) Huấn luyện có kiểm soát:** chia train/val/test **có kiểm soát rò rỉ dữ liệu** (loại ảnh trùng lặp trước khi chia); đánh giá trên **tập test độc lập**. **(c) Đo đạc và công bố** theo một nguyên tắc bắt buộc:

Mọi số liệu hiệu năng công bố đều phải kèm: model CPU, số luồng, kích thước ảnh đầu vào (imgsz), backend suy luận (PyTorch / ONNX / OpenVINO), và cỡ mẫu đo.

Công bố FPS không kèm cấu hình phần cứng là **lỗi phương pháp luận**; nguyên tắc này cũng cấm so số liệu đo trên phần cứng khác nhau và so trực tiếp mAP@0.5 với mAP@0.5:0.95. **(d) Đánh giá tách bạch:** trước ↔ sau hậu xử lý (NFR-A5 ↔ NFR-A6); một dòng ↔ hai dòng (NFR-A8); theo điều kiện ảnh (NFR-A9).

Quy trình phát triển theo giai đoạn

Đề tài thực hiện theo **12 giai đoạn (Phase 0 – Phase 11)**, tổng công sức ước lượng **77 ngày-người**; mỗi giai đoạn kết thúc bằng **điểm chốt M0 – M11** có điều kiện thông qua tường minh, **không tự động chuyển giai đoạn**. Đường găng gần như tuyến tính; **ba giai đoạn nặng nhất — Dataset (10), Model Training (12), OCR (8 ngày-người) — chiếm 42% tổng công sức**, cũng là ba mắt xích rủi ro nhất: **P2 → P3** (dữ liệu quyết định **trần** độ chính xác); **P3 → P4** (box lệch ⇒ vùng cắt lệch ⇒ OCR sai; dấu hiệu: mAP@0.5:0.95 thấp dù mAP@0.5 cao); **P4 với biến hai dòng** — rủi ro đã định lượng ở mục 1.1.3.

Trạng thái tại thời điểm viết: Phase 0 và Phase 1 hoàn thành, chốt M0, M1; backend FastAPI xác minh bằng yêu cầu HTTP thật (10 endpoint); frontend build sạch. Mô hình chính thức (YOLO11n, imgsz=640, split v3, 20 epoch) đạt **mAP@0.5 = 0,9829** và **mAP@0.5:0.95 = 0,7834**; NFR-A4/A5/A6/A7 và NFR-P1 **đã đo**. models/baseline-416-v1.pt chỉ còn là **mô hình đối chứng**, không đóng góp con số nào vào kết quả công bố: imgsz=416 trong khi chỉ tiêu đặt ở 640, và split v1 có rò rỉ train↔test.

P.2. Ma trận nhầm lẫn ký tự — phân tích chi tiết

Mục này trả lời RQ5: **thay tri thức suy đoán bằng tri thức đo được**. Bảng luật hiện hành trong ai/inference/plate_rules.py gồm TO_DIGIT = {0→0, Q→0, D→0, I→1, J→1, L→1, Z→2, A→4, S→5, G→6, T→7, B→8} và TO_LETTER = {0→D, 1→L, 2→Z, 3→B, 4→A, 5→S, 6→G, 7→T, 8→B}. Docstring thừa nhận nguồn gốc: "This table is derived from glyph-shape

reasoning, not from measurement", và đánh dấu một số cặp (đặc biệt L→1) là **phỏng đoán yếu**. Ma trận 36×36 (10 chữ số + 26 chữ cái) đo trên các cặp ký tự đã căn chỉnh là bằng chứng thực nghiệm để chuyển giả thuyết đó thành tri thức. Hai hình minh hoạ — ma trận 36×36 thang log(1+n) (04-ocr-confusion-matrix.png) và biểu đồ cột 15 cặp bị nhầm nhiều nhất (04-ocr-top-confusions.png) — **chưa sinh**.

Bảng 5.6. Các cặp ký tự bị nhầm nhiều nhất, đối chiếu bảng luật hiện hành

Hạng	Ký tự thật → bị đọc thành	Số lần	Tỉ lệ trong tổng lỗi thay thế	Bảng luật có phủ không?
1	L → 1	90	10,44%	có (TO_DIGIT) — đúng chiều
2	E → F	73	8,47%	không
3	4 → L	53	6,15%	không
4	U → 1	38	4,41%	không
5	D → 0	34	3,94%	có (TO_DIGIT) — đúng chiều
6	Z → 7	32	3,71%	không
7	2 → 7	26	3,02%	không
8	X → Y	21	2,44%	không
9	B → R	20	2,32%	không
10	9 → 0	19	2,20%	không

Ma trận 36×36, mẫu số 2.801 biến có nhãn chuỗi, tổng lỗi thay thế $S = 862$ (cột tỉ lệ lấy S làm mẫu số).

RQ5 được trả lời theo hướng ít ai ngờ: bảng luật hiện hành phần lớn *không khớp* các cặp nhầm thật. Chỉ **2/10** cặp nhầm nhiều nhất được phủ; **8 cặp còn lại chưa có luật nào phủ** — đáng chú ý E→F, 4→L, U→1, đều là cặp **suy đoán hình dạng không dự đoán được**, phát sinh từ đặc thù phong chữ biển số và điều kiện ảnh thật. Ngược lại, bảy cặp có trong bảng luật lại có **số lần quan sát bằng 0** và thuộc diện *xem xét loại*: D→0, J→1, A→4, T→7, B→8 (TO_DIGIT) và 2→Z, 3→B (TO_LETTER).

Hai chiều của cùng một cặp glyph. L → 1 (ký tự thật L bị đọc thành chữ số 1) quan sát **90 lần**; nhưng chiều ghi trong TO_DIGIT là "khi engine đọc ra L tại vị trí chữ số thì đổi L → 1" — chiều này chỉ khớp **2 lần**. Bất đối xứng ấy đúng chứ không phải lỗi: bảng luật sửa ký tự *engine đọc ra*, ma trận nhầm lẫn đếm ký tự *thật bị đọc sai*. Đề xuất hiệu chỉnh đầy đủ ở khoá doi_chieu_bang_luat.proposed_updates trong 05-results.json.

0 → 0 hợp lệ tại vị trí chữ số, nhưng 0 → 0 **không bao giờ** hợp lệ vì 0 không phải chữ cái sê-ri hợp pháp; loại cả O và Q thì ứng viên đồng hình duy nhất ở vị trí chữ cái là D, nên chiều đúng là 0 → 0 tại vị trí chữ số và 0 → D tại vị trí chữ cái. Ma trận chỉ đếm tần suất; chuyển từ tần suất sang luật vẫn cần **ràng buộc miền** từ quy chuẩn biển số. **Tiêu chí chấp nhận một**

cấp vào bảng luật đã hiệu chỉnh (định trước để tránh chọn theo kết quả): (1) tần suất vượt một ngưỡng thống kê tối thiểu; (2) chiều ánh xạ **tương thích với ràng buộc vị trí** của định dạng biến số Việt Nam; (3) áp vào toàn tập cho **đóng góp thuần không âm**. Không thoả cả ba thì loại, **kể cả khi nghe có vẻ hợp lý về hình dạng chữ**.

P.3. Bậc thang thử-lại biến nghiêng/méo — chi phí, lợi ích, và một quyết định tắt mặc định

Chế độ thất bại thứ hai: **biến bị nghiêng hoặc méo phối cảnh**, khiến tỉ lệ khung hình lệch đủ để bộ phân loại bố cục xếp nhầm, hoặc khiến ký tự dính vào nhau. Bậc thang thử-lại dùng ba biến thể theo thứ tự rẻ trước: **nắn hình trong mặt phẳng, giãn dọc chống méo phối cảnh, siêu phân giải** cho vùng cắt quá nhỏ. **Điểm mấu chốt là cổng kích hoạt, không phải các biến thể**: hình học từng được đo ở dạng *luôn bật* và kết quả là **mất** — 42 lần đọc hợp lệ tụt xuống 40, vì một hình chữ nhật khớp sai trên vùng cắt nhỏ và mờ sẽ cắt ký tự của một biến vốn đang đọc tốt. Đặt nó **sau cổng "lần đọc đầu đã thất bại"** đảo ngược kinh tế học: đường đi của biến đọc đúng **không bị chạm tới về mặt cấu trúc**, mọi ca cứu được là lãi ròng.

Bóc tách chi phí – lợi ích từng bậc (độ chính xác trên 2.801 biến có nhãn chuỗi; độ trễ trên 100 ảnh hiện trường của tập test v3, máy rảnh; nguồn 27-ocr-accuracy-with-ladder.json và ba lượt ai.evaluation.benchmark_system bóc tách qua ALPR_RECTIFY_ENABLED / ALPR_SR_RETRY_ENABLED; phân tích đầy đủ ở docs/reports/27-retry-ladder-cost-benefit.md): **tắt hẳn bậc thang** — 1 – CER 0,9416, A6 0,7437, **0 biến được cứu**, p95 **866,3 ms**, p99 1.101,1 ms; **nắn hình / giãn dọc (cấu hình giao hàng)** — 1 – CER **0,9454**, A6 **0,7512**, **34 biến được cứu**, p95 **1.110,4 ms**, p99 1.349,0 ms; **thêm siêu phân giải** — 1 – CER 0,9454, A6 0,7512, vẫn **34 biến**, p95 **1.428,7 ms**, p99 **2.730,4 ms**.

Bậc thang gần như miễn phí ở trường hợp thường và rất đắt ở đuôi: trung vị chỉ tăng 3,7% (405,8 ms) trong khi p99 tăng 148%. **Nắn hình / giãn dọc: giữ** — mua 34 biến (+0,75 điểm A6, +0,38 điểm A4) với giá +244 ms ở p95. **Siêu phân giải: tắt mặc định** — mua **0 biến** với giá **+319 ms ở p95 và +1.381 ms ở p99**, và một mình nó đẩy NFR-P1 vượt ngưỡng tối thiểu 1.500 ms. **Vì sao số 0 đó không phải bằng chứng nó vô dụng**: cổng của bậc này chỉ mở cho vùng cắt có cạnh dài ≤ 200 px, mà đo trên **120 mẫu** ngẫu nhiên của tập có nhãn chuỗi, cạnh dài sau bước khôi phục tỉ lệ khung hình có **giá trị nhỏ nhất 565 px, trung vị 868 px — 0/120 mẫu lọt cổng**. Ngưỡng này **không thể kích hoạt nên không thể đo** bậc siêu phân giải; số 0 là **số 0 cấu trúc**, không phải kết quả âm. Suy ra: toàn bộ 34 biến cứu được đều là công của nắn hình / giãn dọc, và quyết định tắt **không** dựa trên "đã đo và thấy vô dụng".

Chi phí đã đo được và lớn; lợi ích **chưa ai đo được** trên bất kỳ tập đại diện nào; trong khi NFR-P1 là yêu cầu mức *Must* và riêng bậc này làm nó vượt ngưỡng. Một lợi ích chưa định lượng không đủ để biện minh cho một vi phạm đã định lượng.

Bảng chứng duy nhất hiện có cho bậc siêu phân giải vẫn là **một vùng cắt demo được chọn tay trong bốn ca thử**. Mã, kiểm thử, công tắc và báo cáo **giữ nguyên**;

ALPR_SR_RETRY_ENABLED=true là bật lại. Muốn đo cho tử tế cần một tập **vùng cắt nhỏ do chính bộ phát hiện sinh ra, có nhãn chuỗi** — đồ án không có, hạng mục bỏ ngỏ ghi ở 5.11.

P.4. Phân tích lỗi — sáu loại lỗi, diễn biến và ca điển hình

Sáu loại lỗi **đầy đủ và loại trừ lẫn nhau**, mỗi ca sai gán đúng một loại theo thứ tự ưu tiên: **E1 bỏ sót biển** (ảnh có biển nhưng không hộp nào khớp), **E2 phát hiện nhầm** (hộp ở vùng không phải biển), **E3 nhầm ký tự** (đúng độ dài, sai ký tự), **E4 thiếu ký tự**, **E5 thừa ký tự**, **E6 sai thứ tự** (đủ ký tự nhưng sắp sai, hầu như chỉ ở biển hai dòng do ghép nhầm chiều). E6 đáng chú ý riêng vì nó **chỉ tồn tại do bài toán có biển hai dòng** và là loại lỗi hậu xử lý sửa được triệt để nếu logic ghép dòng đúng.

Bảng 5.11. Tần suất từng loại lỗi

Mã	Loại lỗi	Số ca	Tỉ lệ trong tổng ca sai	Tỉ lệ toàn tập đánh giá	Một dòng	Hai dòng
E1	Bỏ sót biển	335	—	—	—	—
E2	Phát hiện nhầm	(chưa đo)	—	—	—	—
E3	Nhầm ký tự	445	63,85%	15,89%	17	428
E4	Thiếu ký tự	73	10,47%	2,61%	0	73
E5	Thừa ký tự	18	2,58%	0,64%	5	13
E6	Sai thứ tự	0	0,00%	0,00%	0	0
Tổng số ca sai		697	100%	24,88%	—	—
Tổng ca đánh giá (mẫu số)		2.801	n/a	100%	—	—

Mẫu số của E1 khác mẫu số của E3–E6. E1 lấy từ lượt đo E2E ở 5.5.5 trên 2.801 mẫu; tỉ lệ E1 trên mẫu số riêng của nó là **11,96%**, nên hai cột tỉ lệ **cố ý để trống ở dòng E1**. E2 để (chưa đo). **Hai loại lỗi ngoài khung E1–E6:** empty_read = **10** (OCR trả chuỗi rỗng) và mixed = **151** (một biển vừa thiếu vừa thừa vừa nhầm ký tự) — có trong cài đặt nhưng không có mã E riêng; ghi nhận để tránh ảo giác "các mã E cộng lại đủ 100% số ca sai" (E3+E4+E5+E6 = **536**, phần còn lại tới 697 là 151 ca mixed và 10 ca empty_read). **Nguồn:** khoá by_line_count.*.error_classes của 05-results.json — cùng lượt 28/07 với 5.5, tức **đã có** cả bước cứu dòng trên lẫn bậc thang thử-lại.

So với lượt 20/07, phân bố lỗi đã đổi hình rõ rệt — tổng ca sai **916 → 697**, giảm **219** ca. Giảm mạnh nhất là mixed (277 → 151) và E5 (95 → 18), tức các chuỗi **hỏng về cấu trúc** đã xử lý phần lớn; ngược lại E3 **tăng** (399 → 445) — hiện tượng **phân loại lại** chứ không phải thoái lui, vì một biển trước cho chuỗi sai độ dài (rơi vào mixed hoặc E4) nay cho chuỗi đúng độ dài nhưng sai một ký tự. **Cấu trúc lỗi xác nhận chẩn đoán ở 4.7.1:** gần như toàn bộ lỗi ký

tự dồn về biển hai dòng (E3 428/445, E4 **73/73** — **tuyệt đối, không một ca nào thuộc biển một dòng**, E5 13/18), biển một dòng chỉ sinh **22 ca** trên cả ba loại; khớp với chênh lệch 25,45 điểm A6 ở 5.5.3. E6 = **0** trên toàn tập: logic ghép hai dòng hoạt động đúng, không ca nào ghép nhầm chiều.

Bước cứu dòng trên để lại dấu vết đo được ngay trong bảng này. So với lượt đo trước khi có bước cứu, **E4 giảm mạnh nhất: 217 → 134 ca**, trong khi E3 tăng 376 → 399 và mixed tăng 266 → 277: bước cứu nhằm đúng chế độ "mất hẳn dòng trên" nên rút bớt quần thể E4, còn những ca cứu được một phần **chuyển sang** E3 hoặc mixed thay vì biến mất — kiểm chứng chéo độc lập, vì nếu bước cứu chỉ "làm số đẹp lên" thì phân bố sẽ co lại đồng đều chứ không dịch chuyển có hướng. Dù vậy, E4 (**73 ca**) và $D = 1.272$ ký tự bị xoá vẫn cùng trở về chế độ thất bại còn lại: OCR đọc **hụt** ký tự trên biển hai dòng.

Bốn hình minh hoạ ca điển hình **chưa sinh**: E1 biển bị bỏ sót kèm kích thước box tương đối và điều kiện ảnh (05-error-e1-missed.png); E3 nhầm ký tự với vùng cắt, chuỗi thô, chuỗi sau hậu xử lý và nhãn thật (05-error-e3-substitution.png); E6 sai thứ tự trên biển hai dòng (05-error-e6-order.png); và ca mà **hậu xử lý làm hỏng** một chuỗi vốn đã đúng — nếu tồn tại ca như vậy, **bắt buộc phải trưng ra**, vì nó là bằng chứng phản biện đối với đóng góp công bố ở 5.5.2; một chương đánh giá chỉ trưng ra các ca hệ thống làm tốt là một chương đã tự loại bỏ khả năng bị kiểm chứng.

